# APLIKASI KELAYAKAN ASET LABORATORIUM KOMPUTER BERBASIS SIMPLE ADDITIVE WEIGHTING DAN STRAIGHT LINE UNTUK PENGAMBILAN KEPUTUSAN

**SKRIPSI**

Karya Tulis sebagai syarat memperoleh
Gelar Sarjana Komputer dari Fakutas Teknologi Informasi
Universitas Bale Bandung


Disusun oleh:
**ANDHIKA MAULANA SHIDDIQ**
**NPM. 301220024**



PROGRAM STRATA 1
PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS BALE BANDUNG
BANDUNG

2026

**LEMBAR PERSETUJUAN PENGUJI**

# APLIKASI KELAYAKAN ASET LABORATORIUM KOMPUTER BERBASIS SIMPLE ADDITIVE WEIGHTING DAN STRAIGHT LINE UNTUK PENGAMBILAN KEPUTUSAN

Disusun oleh:
ANDHIKA MAULANA SHIDDIQ
NPM. 301220024

Telah diterima dan disetujui untuk memenuhi persyaratan mencapai gelar

**SARJANA KOMPUTER**

Pada

**PROGRAM STUDI TEKNIK INFORMATIKA**
**FAKULTAS TEKNOLOGI INFORMASI**
**UNIVERSITAS BALE BANDUNG**

Baleendah, Agustus 2026

Disetujui oleh:

| Penguji 1 | Penguji 2 |
|---|---|
| Mohammad Bayu Anggara, S.Kom., M.Kom. | Cecep Suwanda, S.Si., M.Kom. |
| NIK. 04104823002 | NIK. 3273110210820006 |

**LEMBAR PERSETUJUAN PEMBIMBING**

# APLIKASI KELAYAKAN ASET LABORATORIUM KOMPUTER BERBASIS SIMPLE ADDITIVE WEIGHTING DAN STRAIGHT LINE UNTUK PENGAMBILAN KEPUTUSAN

Disusun oleh:
ANDHIKA MAULANA SHIDDIQ
NPM. 301220024

Telah diterima dan disetujui untuk memenuhi persyaratan mencapai gelar

**SARJANA KOMPUTER**

Pada

**PROGRAM STUDI TEKNIK INFORMATIKA**
**FAKULTAS TEKNOLOGI INFORMASI**
**UNIVERSITAS BALE BANDUNG**

Baleendah, Agustus 2026

Disetujui oleh:

| Pembimbing Utama | Pembimbing Pendamping |
|---|---|
| Yudi Herdiana, S.T., M.T. | Yusuf Muharam, S.Kom., M.Kom. |
| NIK. 04104808008 | NIK. 04104820003 |

**LEMBAR PENGESAHAN PROGRAM STUDI**

# APLIKASI KELAYAKAN ASET LABORATORIUM KOMPUTER BERBASIS SIMPLE ADDITIVE WEIGHTING DAN STRAIGHT LINE UNTUK PENGAMBILAN KEPUTUSAN

Disusun oleh:
ANDHIKA MAULANA SHIDDIQ
NPM. 301220024

SKRIPSI ini telah diterima dan disetujui untuk memenuhi persyaratan mencapai gelar

**SARJANA KOMPUTER**

Pada

**PROGRAM STUDI TEKNIK INFORMATIKA**
**FAKULTAS TEKNOLOGI INFORMASI**
**UNIVERSITAS BALE BANDUNG**

Baleendah, Agustus 2026

Disetujui oleh:

| Mengetahui, | Mengesahkan, |
|---|---|
| Dekan, | Ketua Program Studi |
| Yudi Herdiana, S.T., M.T. | Yusuf Muharam, S.Kom., M.Kom. |
| NIK. 04104808008 | NIK. 04104820003 |

# HALAMAN PERNYATAAN

Yang bertanda tangan dibawah ini, saya:

| | |
|---|---|
| Nama | : Andhika Maulana Shiddiq |
| NPM | : 301220024 |
| Program Studi | : Teknik Informatika |
| Fakultas | : Fakultas Teknologi Informasi |
| Judul | : Aplikasi Kelayakan Aset Laboratorium Komputer Berbasis Simple Additive Weighting Dan Straight Line Untuk Pengambilan Keputusan |

Dalam rangka pengembangan kajian ini, penulis dengan tulus mengungkapkan bahwa penulisan skripsi ini merujuk pada hasil penelitian yang telah dilakukan, serta merupakan hasil pemikiran dan pemaparan yang bersifat orisinal dari penulis sendiri, baik dalam konteks penulisan laporan maupun pelaksanaan aktivitas pemrograman yang dicantumkan sebagai bagian integral dari penelitian ini. Setiap kontribusi atau karya dari pihak lain akan diidentifikasi dengan jelas sesuai dengan pedoman penulisan karya ilmiah yang berlaku, dengan penekanan pada penggunaan referensi yang akurat dan sistematis.

Baleendah, Agustus 2026

Andhika Maulana Shiddiq

NPM: 301220024

# ABSTRAK

Pengelolaan aset di Laboratorium Komputer Fakultas Teknologi Informasi Universitas Bale Bandung sebelumnya masih mengandalkan pencatatan terpisah berbasis spreadsheet dan dokumen fisik. Pola pengelolaan ini memicu risiko tinggi terjadinya data ganda, sulitnya melacak riwayat aset, pengawasan kelayakan yang minim dokumentasi, serta keputusan peremajaan fasilitas yang bersifat reaktif akibat baru dilakukan saat perangkat mati total. Untuk mengatasi kendala operasional tersebut, penelitian ini bertujuan melakukan rancang bangun aplikasi manajemen aset laboratorium komputer berbasis web yang terpusat. Aplikasi ini dilengkapi dengan kodifikasi 27 digit otomatis, pelabelan siber-fisik menggunakan QR Code, serta modul evaluasi dan pemeringkatan prioritas penggantian aset guna memberikan rekomendasi pembaruan fasilitas secara terukur, otomatis, dan objektif. Metode pembuatan aplikasi mengadopsi kerangka *Software Development Life Cycle* (SDLC) model *Waterfall*. Pada lapisan logika komputasi (*backend*), aplikasi mengintegrasikan metode *Straight Line* untuk menghitung depresiasi finansial dan sisa umur ekonomis aset, yang selanjutnya disuntikkan ke dalam matriks *Simple Additive Weighting* (SAW) bersama parameter kondisi fisik dan biaya perawatan untuk menghasilkan urutan prioritas penggantian secara matematis. Perangkat lunak ini dibangun menggunakan microframework Flask berbasis bahasa pemrograman Python dengan SQLite sebagai basis data relasionalnya. Hasil dari penelitian ini adalah terwujudnya aplikasi manajemen aset berbasis web yang mampu mengeliminasi tumpang tindih data dan mempermudah pelacakan aset secara terpusat melalui pemindaian stiker QR Code. Pengujian fungsional menggunakan metode Black-Box Testing dengan teknik *Equivalence Partitioning* (EP) dan *Boundary Value Analysis* (BVA) menunjukkan bahwa seluruh fitur aplikasi beroperasi 100% sesuai rancangan tanpa adanya galat fungsional. Lebih lanjut, hasil *User Acceptance Testing* (UAT) dari para pengguna akhir (Kepala dan Asisten Laboratorium) memperoleh skor rata-rata sebesar 87,50%, yang mengonfirmasi bahwa aplikasi ini sangat layak dan sangat diterima untuk mendukung tata kelola fasilitas laboratorium.

Kata Kunci: Aplikasi Manajemen Aset, *Straight Line*, *Simple Additive Weighting* (SAW), Python Flask, SQLite, *Black Box Testing*, *User Acceptance Testing*.

## *ABSTRACT*

*Asset management at the Computer Laboratory of the Faculty of Information Technology, Universitas Bale Bandung, previously relied on separate spreadsheet-based logging and physical documents. This management pattern triggered a high risk of duplicate data, difficulties in tracking asset history, minimally documented feasibility monitoring, and reactive facility renewal decisions made only when devices suffered total failure. To overcome these operational constraints, this study aims to design and construct a centralized web-based computer laboratory asset management application. The application is equipped with automatic 27-digit codification, cyber-physical labeling using QR Codes, and an evaluation and priority ranking module for asset replacement to provide measurable, automated, and objective facility renewal recommendations. The application construction method adopts the Waterfall model of the Software Development Life Cycle (SDLC) framework. On the backend computational logic layer, the application integrates the Straight Line method to calculate the remaining economic life of assets, which is then injected into the Simple Additive Weighting (SAW) matrix alongside physical condition parameters and maintenance costs to generate a mathematical replacement priority sequence. This software was built using the Python-based microframework Flask with SQLite as its relational database. The result of this research is a web-based asset management application capable of eliminating data redundancy and facilitating centralized asset tracking through QR Code scanning. Functional testing using the Black-Box Testing method with Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) techniques demonstrates that all application features operate 100% according to design specifications without any functional errors. Furthermore, the User Acceptance Testing (UAT) conducted with end-users (the Head of the Laboratory and Assistants) achieved an average score of 87.50%, confirming that the application is Highly Feasible and exceptionally well-received in supporting transparent, accurate, and structured laboratory facility governance.*



*Keywords: Asset Management Application, Straight Line, Simple Additive Weighting (SAW), Python Flask, SQLite, User Acceptance Testing.*

# KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan ke hadirat Allah Subhanahu wa Ta'ala, karena atas limpahan rahmat serta karunia-Nya, penyusunan laporan Skripsi ini dapat diselesaikan dengan baik. Laporan akademis ini disusun sebagai salah satu syarat wajib dalam rangka menyelesaikan tahapan Skripsi pada Fakultas Teknologi Informasi, Universitas Bale Bandung. Adapun kegiatan penelitian ini dilaksanakan di lingkungan Laboratorium Komputer Fakultas Teknologi Informasi, dengan fokus utama pada perancangan dan pembuatan aplikasi pengelolaan data aset laboratorium berbasis web.

Penulis menyadari sepenuhnya bahwa seluruh proses penyusunan laporan skripsi ini tidak terlepas dari bantuan, dukungan positif, serta bimbingan yang sangat berharga dari berbagai pihak. Oleh karena itu, dengan penuh rasa hormat, penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Kedua orang tua tercinta, Bapak Rudi Kusdiana dan Ibu Rusmiati, yang senantiasa memberikan doa, cinta, serta dukungan baik moril maupun materiel yang tak terhingga.
2. Bapak Dr. Ir. H. M. Ibrahim Danuwikarsa, M.S, selaku Rektor Universitas Bale Bandung.
3. Bapak Yudi Herdiana, S.T., M.T., selaku Dekan Fakultas Teknologi Informasi Universitas Bale Bandung, juga selaku dosen pembimbing yang telah banyak meluangkan waktu untuk memberikan arahan, bimbingan, dan masukan selama pelaksanaan dan penyusunan laporan ini.
4. Bapak Yusuf Muharam, S.Kom., M.Kom., selaku Ketua Program Studi Teknik Informatika Universitas Bale Bandung, juga selaku dosen pembimbing yang telah banyak meluangkan waktu untuk memberikan arahan, bimbingan, dan masukan selama pelaksanaan dan penyusunan laporan ini.
5. Seluruh pihak pendukung, baik keluarga besar, sahabat, maupun rekan-rekan mahasiswa seperjuangan yang senantiasa memberikan motivasi dan bantuan yang tidak dapat penulis sebutkan satu per satu.

6. Bapak Sutiyono S.T., M.Kom., selaku pembimbing lapangan sekaligus Kepala Laboratorium Komputer Fakultas Teknologi Informasi Universitas Bale Bandung yang telah memfasilitasi dan mendukung jalannya penelitian ini.

Baleendah, Agustus 2026

Penulis

# DAFTAR ISI

**ABSTRAK** ..... **vi**
***ABSTRACT*** ..... **vii**
**KATA PENGANTAR** ..... **viii**
**DAFTAR ISI** ..... **x**
**DAFTAR GAMBAR** ..... **xiii**
**DAFTAR TABEL** ..... **xv**
**DAFTAR LAMPIRAN** ..... **xvi**
**BAB I PENDAHULUAN** ..... **1**
1.1 Latar Belakang ..... 1
1.2 Rumusan Masalah ..... 3
1.3 Batasan Masalah ..... 4
1.4 Tujuan Penelitian ..... 4
1.5 Metodologi Penelitian ..... 5
1.6 Sistematika Penulisan ..... 6
**BAB II TINJAUAN PUSTAKA** ..... **8**
2.1. Landasan Teori ..... 8
2.2. Dasar Teori ..... 12
2.2.1 Manajemen Aset dan Tata Kelola Laboratorium ..... 13
2.2.2 Manajemen Pemeliharaan Aset ..... 14
2.2.3 Sistem Pendukung Keputusan ..... 16
2.2.4 Metode Straight Line ..... 17
2.2.5 Metode *Simple Additive Weighting* ..... 19
2.2.6 Sistem Informasi Berbasis Web ..... 20
2.2.7 Bahasa Pemrograman Python ..... 22
2.2.8 Framework Flask ..... 23
2.2.9 Database SQLite ..... 24
2.2.10 Teknologi Identifikasi Quick Response Code ..... 25
2.2.11 Metode Pengembangan SDLC ..... 26

2.2.12 Metode Pengembangan Sistem Waterfall ..... 28
2.2.13 Object-Oriented Design and Programming..... 30
2.2.14 Pemodelan Sistem Visual Menggunakan UML ..... 31
2.2.15 Autentikasi, Otorisasi, dan Role-Based Access Control ..... 35
2.2.16 *Black-Box Testing*..... 38
2.2.17 *User Acceptance Testing* ..... 39
**BAB III METODOLOGI ..... 42**
3.1 Kerangka Pikir ..... 42
3.2 Deskripsi ..... 43
**BAB IV ANALISIS DAN PERANCANGAN ..... 49**
4.1 Analisis..... 49
4.1.1 Analisis Masalah ..... 49
4.1.2 Analisis Software ..... 50
4.1.3 Analisis Pengguna..... 51
4.1.4 User Interface..... 52
4.1.5 Fitur-Fitur..... 53
4.1.6 Analisis Data ..... 56
4.1.7 Analisis Biaya ..... 58
4.2 Perancangan ..... 59
4.2.1 Unified Modeling Language ..... 59
4.2.2 Struktur Tabel..... 80
4.2.3 Desain..... 86
**BAB V IMPLEMENTASI DAN PENGUJIAN ..... 98**
5.1 Implementasi..... 98
5.1.1 Listing Program..... 98
5.1.2 Implementasi Sistem ..... 109
5.1.3 Spesifikasi Sistem .....110
5.1.4 Instalasi Sistem .....112
5.1.5 Menjalankan Sistem.....114
5.2 Pengujian..... 124
5.2.1 Black-Box Testing..... 124

5.2.2 Validasi Perhitungan Matematis ..... 128
5.2.3 *User Acceptance Testing* ..... 133
**BAB VI KESIMPULAN DAN SARAN ..... 137**
6.1 Kesimpulan ..... 137
6.2 Saran ..... 138
**DAFTAR PUSTAKA ..... 139**
**LAMPIRAN ..... 147**
**RIWAYAT HIDUP ..... 160**

# DAFTAR GAMBAR

Gambar 2. 1 Arsitektur Alur Manajemen Pemeliharaan Aset ..... 15
Gambar 2. 2 Struktur QR Code ..... 25
Gambar 2. 3 Tahapan Fase Software Development Life Cycle ..... 27
Gambar 2. 4 Model Waterfall Sommerville ..... 29
Gambar 2. 5 Arsitektur Autentikasi dan Otorisasi Berbasis RBAC ..... 36
Gambar 3. 1 Kerangka Pikir ..... 42
Gambar 4. 1 Pohon Navigasi Aplikasi ..... 54
Gambar 4. 2 Use Case Diagram Admin ..... 59
Gambar 4. 3 Use Case Diagram User ..... 60
Gambar 4. 4 Activity Diagram Login ..... 63
Gambar 4. 5 Activity Diagram Pengelolaan Data Master ..... 64
Gambar 4. 6 Activity Diagram Pengelolaan Data Aset ..... 65
Gambar 4. 7 Activity Diagram Pemetaan Denah ..... 66
Gambar 4. 8 Activity Diagram Pengelolaan Data Perbaikan Aset ..... 67
Gambar 4. 9 Activity Diagram Prioritas Peremajaan Aset ..... 68
Gambar 4. 10 Activity Diagram Pengelolaan Data Pengguna ..... 69
Gambar 4. 11 Activity Diagram Logout ..... 70
Gambar 4. 12 Sequence Diagram Login Sistem ..... 71
Gambar 4. 13 Sequence Diagram Pengelolaan Data Master ..... 72
Gambar 4. 14 Sequence Diagram Pengelolaan Data Aset ..... 73
Gambar 4. 15 Sequence Diagram Pemetaan Denah Laboratorium ..... 74
Gambar 4. 16 Sequence Diagram Pengelolaan Data Perbaikan Aset ..... 75
Gambar 4. 17 Sequence Diagram Prioritas Penggantian Aset ..... 76
Gambar 4. 18 Sequence Diagram Pengelolaan Data Pengguna ..... 77
Gambar 4. 19 Sequence Diagram Logout Sistem ..... 78
Gambar 4. 20 Class Diagram ..... 79
Gambar 4. 21 Rancangan Antarmuka Halaman Login ..... 87
Gambar 4. 22 Rancangan Antarmuka Dashboard ..... 88
Gambar 4. 23 Rancangan Antarmuka Halaman Data Master ..... 88
Gambar 4. 24 Rancangan Antarmuka Tabel Daftar Aset ..... 89

Gambar 4. 25 Rancangan Antarmuka Form Tambah Aset ..... 90
Gambar 4. 26 Rancangan Antarmuka Form Edit Aset ..... 90
Gambar 4. 27 Rancangan Antarmuka Detail Aset ..... 91
Gambar 4. 28 Rancangan Antarmuka Pemetaan Denah 2D Laboratorium ..... 92
Gambar 4. 29 Rancangan Antarmuka Perbaikan Aset ..... 92
Gambar 4. 30 Rancangan Antarmuka Prioritas Penggantian Aset Metode SAW . 93
Gambar 4. 31 Rancangan Antarmuka Manajemen Pengguna & Hak Akses ..... 94
Gambar 4. 32 Rancangan Stiker Label Aset ..... 94
Gambar 4. 33 Rancangan Laporan Data Aset ..... 95
Gambar 4. 34 Rancangan Laporan Rekomendasi Penggantian Aset ..... 96
Gambar 5. 1 Halaman Login .....114
Gambar 5. 2 Halaman Dashboard untuk Admin .....115
Gambar 5. 4 Halaman Data Master .....115
Gambar 5. 5 Halaman Data Aset .....116
Gambar 5. 6 Halaman Tambah Aset .....117
Gambar 5. 7 Halaman Edit Aset .....117
Gambar 5. 8 Halaman Detail Aset .....118
Gambar 5. 9 Halaman Pemetaan Denah .....119
Gambar 5. 10 Halaman Perbaikan Aset .....119
Gambar 5. 11 Halaman Prioritas Penggantian Aset ..... 120
Gambar 5. 12 Halaman Manajemen Pengguna ..... 121
Gambar 5. 13 Stiker Label Aset ..... 121
Gambar 5. 14 Laporan Data Aset ..... 122
Gambar 5. 15 Laporan Prioritas Penggantian Aset ..... 123

# DAFTAR TABEL

Tabel 2. 1 Tabel Literature Review ..... 10
Tabel 2. 2 Simbol Use Case Diagram ..... 32
Tabel 2. 3 Simbol Activity Diagram ..... 33
Tabel 2. 4 Simbol Class Diagram..... 34
Tabel 2. 5 Simbol Sequence Diagram ..... 35
Tabel 2. 6 Kategori Interpretasi Kelayakan UAT ..... 40
Tabel 4. 1 Kebutuhan Perangkat Lunak (*Software*) Minimum ..... 51
Tabel 4. 2 Analisis Estimasi Biaya Pengembangan Sistem..... 58
Tabel 4. 3 Deskripsi Skenario Use Case ..... 61
Tabel 4. 4 Ringkasan Spesifikasi dan Relasi Class Diagram ..... 79
Tabel 4. 5 Spesifikasi Tabel Pengguna (tb_user) ..... 80
Tabel 4. 6 Spesifikasi Tabel Lokasi Laboratorium (tb_location) ..... 81
Tabel 4. 7 Spesifikasi Tabel Kategori Aset (tb_category) ..... 82
Tabel 4. 8 Spesifikasi Tabel Program Studi (tb_prodi) ..... 82
Tabel 4. 9 Spesifikasi Tabel Kepala Laboratorium (tb_kepala_lab) ..... 82
Tabel 4. 10 Spesifikasi Tabel Kode Aset Lokasi (tb_kode_lokasi)..... 83
Tabel 4. 11 Spesifikasi Tabel Kode Barang (tb_kode_barang) ..... 83
Tabel 4. 12 Spesifikasi Tabel Data Aset Inventaris (tb_asset) ..... 84
Tabel 4. 13 Spesifikasi Tabel Perbaikan Aset (tb_perbaikan)..... 85
Tabel 4. 14 Spesifikasi Tabel Bobot Kriteria Evaluasi (tb_bobot_kriteria) ..... 86
Tabel 5. 1 Spesifikasi Minimum Perangkat Keras (Hardware).....110
Tabel 5. 2 Spesifikasi Minimum Perangkat Lunak (Software) .....111
Tabel 5. 3 Matriks Uji Black-Box dengan Teknik Equivalence Partitioning..... 125
Tabel 5. 4 Rekapitulasi Hasil Eksekusi Pengujian Fungsional ..... 126
Tabel 5. 5 Data Sampel Alternatif Aset Laboratorium..... 129
Tabel 5. 6 Matriks Keputusan Mentah X (ukuran 3 x 3) ..... 131
Tabel 5. 7 Matriks Normalisasi (R)..... 131
Tabel 5. 8 Validasi Akuntansi Penyusutan ..... 132
Tabel 5. 9 Validasi Algoritma SAW ..... 132
Tabel 5. 10 Distribusi Jawaban Responden Pengujian UAT..... 133
Tabel 5. 11 Rekapitulasi Persentase Hasil Kelayakan UAT per Aspek..... 135

# DAFTAR LAMPIRAN

Lampiran 1: Pedoman Wawancara ..... 147
Lampiran 2: Term of Reference ..... 149
Lampiran 3: Dokumentasi ..... 150
Lampiran 4: Kode Program ..... 152

# BAB I

# PENDAHULUAN

## 1.1 Latar Belakang

Era digitalisasi kini mendorong transformasi tata kelola aset institusi menjadi jauh lebih efisien melalui penerapan sistem informasi berbasis web. Pendekatan digital ini terbukti mampu mempercepat pencarian barang hingga 60%, menekan risiko kehilangan arsip hingga 80%, dan memangkas waktu pengolahan data dari berjam-jam menjadi hitungan detik (Dilla & Jupron, 2026; Erik dkk., 2025) . Dengan adanya sentralisasi data, pengelola dapat meminimalisasi kesalahan manusia *(human error)*, mencegah duplikasi informasi, dan merangkai rekam jejak pelacakan yang terintegrasi (Putra, 2025; Triansyah dkk., 2026). Praktik ini sekaligus menggantikan kerepotan pencatatan konvensional berbasis *spreadsheet* terpisah, sehingga pendataan tidak perlu diulang setiap masa evaluasi dan keamanan arsip jauh lebih terjamin (Andrianto dkk., 2022; Augusto dkk., 2026). Lebih dari sekadar media inventarisasi, inovasi sistem saat ini juga memungkinkan integrasi fitur evaluasi cerdas agar kebijakan peremajaan fasilitas tidak lagi diambil secara subjektif atau sekadar reaktif (Irianto dkk., 2026; Sorongan dkk., 2025). Melalui fungsi kalkulasi kelayakan yang akurat tersebut, kondisi setiap perangkat dapat dinilai secara objektif untuk memastikan keputusan perbaikan atau penggantian aset dilakukan di waktu yang tepat demi mencegah pemborosan anggaran (Sudarman dkk., 2024; Taryono dkk., 2024a).

Kegiatan penelitian ini dilaksanakan di lingkungan Fakultas Teknologi Informasi (FTI) Universitas Bale Bandung, dengan menitikberatkan fokus pada sistem pengelolaan fasilitas praktikum akademik mahasiswa. Secara spesifik, FTI UNIBBA saat ini mengelola operasional dua ruang praktikum utama, yakni Laboratorium Multimedia dan Laboratorium Basis Data. Berdasarkan hasil pendataan inventaris fasilitas, kedua laboratorium tersebut masing-masing menampung kapasitas sebanyak 20 unit komputer. Keberadaan puluhan unit perangkat keras ini menjadikan optimalisasi tata kelola sarana dan prasarana

sebagai aspek operasional yang sangat krusial guna mendukung kelancaran seluruh kegiatan belajar mengajar di fakultas.

Berdasarkan identifikasi masalah di lapangan, pengelolaan Laboratorium Komputer FTI UNIBBA menghadapi tiga kendala teknis yang saling berkaitan. Pertama, pendataan inventaris masih menggunakan *spreadsheet* terpisah yang menyebabkan terjadinya data ganda dan terputusnya riwayat pelacakan aset. Hal ini berdampak pada lambatnya pencarian informasi, sehingga pengelola sering kali membuat catatan baru akibat kesulitan menemukan arsip lama pada penyimpanan yang tidak terpusat. Kedua, ketiadaan sistem yang terintegrasi menjadikan pengawasan kelayakan aset minim dokumentasi. Pelaporan status perangkat hanya dapat dilakukan secara insidental satu tahun sekali, sehingga kondisi fasilitas laboratorium tidak terpantau secara rutin di luar jadwal evaluasi tersebut. Ketiga, pola pembaruan fasilitas berjalan sangat reaktif. Akibat ketiadaan instrumen pengolahan data kelayakan yang akurat, keputusan peremajaan atau penggantian unit komputer baru dieksekusi oleh Kepala Laboratorium ketika perangkat sudah dalam kondisi mati total. Padahal, secara teknis batas umur ekonomis perangkat tersebut sesungguhnya telah terlampaui.

Berbagai penelitian terdahulu menunjukkan bahwa permasalahan tata kelola aset dapat diatasi secara efektif melalui sistem informasi yang memadukan perhitungan akuntansi dan algoritma keputusan. Secara khusus, kombinasi antara metode *Straight Line* dan *Simple Additive Weighting* (SAW) telah terbukti mampu memberikan penilaian yang objektif untuk menentukan kelayakan peremajaan fasilitas (Afandi & Nasution, 2023; Siahaan dkk., 2021). Berpijak pada literatur tersebut, penelitian ini menawarkan pengembangan sistem dengan dua titik kebaruan (*novelty*). Pertama, perhitungan sisa umur ekonomis dari metode *Straight Line* tidak sekadar ditampilkan sebagai informasi pasif, tetapi langsung diproses secara otomatis di sisi *backend* sebagai kriteria biaya (*cost* atau C1) di dalam matriks keputusan SAW. Kedua, seluruh logika sistem ini secara spesifik dibangun menggunakan *microframework* Python Flask dan basis data SQLite. Pilihan teknologi ini diterapkan karena kemampuannya yang sangat ringan untuk mengeksekusi kalkulasi matriks secara lokal, sehingga laboratorium tidak perlu menyiapkan spesifikasi peladen (*server*) yang rumit (Dilla & Jupron, 2026;

Ramadhan dkk., 2026). Terakhir, untuk memastikan rancang bangun perangkat lunak ini berjalan rapi dan sesuai dengan kebutuhan awal, proses pengembangannya dipandu secara berurutan menggunakan metode *Software Development Life Cycle* (SDLC) model *Waterfall* (Putra, 2025) .

Pembaruan sistem tata kelola aset di Laboratorium Komputer FTI UNIBBA merupakan langkah nyata untuk mencegah pemborosan anggaran institusi dan menjamin kelancaran praktikum mahasiswa. Secara praktis, luaran (*output*) penelitian ini adalah aplikasi manajemen aset yang cerdas secara analitik sekaligus efisien secara arsitektur. Titik kebaruan (*novelty*) dari sistem ini secara utuh terletak pada peleburan perhitungan penyusutan *Straight Line* sebagai kriteria biaya langsung ke dalam algoritma *Simple Additive Weighting* (SAW), yang seluruh pemrosesannya dibangun secara spesifik menggunakan *microframework* Flask dan basis data SQLite. Penerapan inovasi ini menargetkan dampak lanjutan (*outcome*) berupa terbangunnya mekanisme tata kelola laboratorium yang jauh lebih transparan. Melalui sistem tersebut, Kepala Laboratorium akan selalu memiliki acuan data yang pasti dan objektif saat harus mengeksekusi kebijakan peremajaan atau penghapusan fasilitas. Untuk mewujudkan seluruh tujuan itu, gagasan dan hasil perancangan dituangkan ke dalam naskah skripsi yang berjudul: **"Aplikasi Kelayakan Aset Laboratorium Komputer Berbasis Simple Additive Weighting Dan Straight Line Untuk Pengambilan Keputusan"**.

## 1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah:

1. Bagaimana mengatasi risiko timbulnya data ganda serta lambatnya pencarian riwayat pelacakan aset akibat penggunaan *spreadsheet* yang terpisah?
2. Bagaimana mengatasi kendala pengawasan kelayakan aset laboratorium yang selama ini berjalan lambat dan minim dokumentasi agar terpantau secara rutin?
3. Bagaimana mengatasi pola pembaruan fasilitas laboratorium yang reaktif melalui penyusunan rekomendasi prioritas penggantian aset?

## 1.3 Batasan Masalah

1. Pendekatan metode yang diterapkan pada tahapan modul evaluasi sistem ini dibatasi secara spesifik pada integrasi metode *Straight Line* guna menghitung sisa umur ekonomis aset, serta algoritma *Simple Additive Weighting* (SAW) untuk menghasilkan daftar peringkat prioritas penggantian perangkat.
2. Objek aset yang dievaluasi dalam penelitian ini dibatasi secara spesifik pada seluruh perangkat keras yang ada di Laboratorium Komputer Fakultas Teknologi Informasi, Universitas Bale Bandung.
3. enggunaan alat dan teknologi perangkat lunak dalam tahapan rancang bangun sistem aplikasi ini dibatasi secara spesifik pada implementasi *microframework* Flask berbasis bahasa pemrograman Python, serta didukung oleh SQLite sebagai pengelola basis data relasionalnya.
4. Tahapan implementasi dan pengujian aplikasi ini dibatasi secara ketat hanya pada lingkungan *localhost* (server lokal). Dengan demikian, ruang lingkupnya tidak mencakup tahapan *deployment* menuju layanan *hosting* publik, instalasi *server production*, maupun konfigurasi jaringan multi-pengguna berskala luas.

## 1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Merancang dan membangun aplikasi manajemen aset berbasis web yang terpusat untuk menggantikan pengelolaan *spreadsheet* yang terpisah, guna menghilangkan risiko data ganda dan mempercepat pencarian riwayat aset?
2. Merancang bagian modul pengawasan dan pemantauan kelayakan aset pada aplikasi tersebut agar pelaporan status serta pengawasan kesehatan aset dapat dilakukan secara rutin dan terdokumentasi.
3. Merancang modul evaluasi dan pemeringkatan prioritas penggantian aset pada aplikasi menggunakan kombinasi metode *Straight Line* dan *Simple Additive Weighting* guna menggantikan pola pembaruan fasilitas yang selama ini bersifat reaktif.

## 1.5 Metodologi Penelitian

a. Metode Pengumpulan Data

Pengumpulan data dan analisis masalah dalam penelitian ini dilakukan melalui tiga pendekatan berikut:

1. Observasi: Pengamatan dilakukan secara langsung terhadap lingkungan Laboratorium Komputer FTI UNIBBA, serta observasi terhadap fungsionalitas manajemen aset yang telah berjalan sebelumnya. Observasi ini bertujuan untuk memahami batasan sistem yang sudah ada saat ini.
2. Wawancara: Proses ini dilakukan melalui wawancara mendalam (*in-depth interview*) bersama Kepala Laboratorium Komputer FTI UNIBBA. Tujuannya adalah untuk menggali data kebutuhan pengguna secara nyata di lapangan dan menentukan bobot kepentingan kriteria yang akan dijadikan parameter utama dalam Sistem Pendukung Keputusan (SPK).
3. Studi Pustaka: Pengumpulan dan pengkajian referensi dari buku, jurnal ilmiah, serta penelitian terdahulu yang relevan.

b. Metode Pengembangan Sistem

Metode pengembangan yang digunakan dalam penelitian ini adalah *Software Development Life Cycle* (SDLC) model *Waterfall*. Pendekatan ini diterapkan secara linier untuk membangun model komputasi SPK penggantian aset dan mengimplementasikannya ke dalam aplikasi berbasis *web* melalui lima tahapan berurutan:

1. Analisis Kebutuhan: Mengidentifikasi permasalahan operasional laboratorium (data ganda, pencatatan manual *spreadsheet*, kelayakan yang tidak terdokumentasi). Hasil akhirnya adalah mendefinisikan kebutuhan fungsional sistem secara keseluruhan, seperti modul login, modul pencatatan aset berbasis database, modul log perbaikan, dan modul hasil keputusan.
2. Desain Sistem: Merancang arsitektur visual aplikasi menggunakan 4 diagram UML (*Use Case*, *Activity*, *Class*, *Sequence*) untuk merepresentasikan seluruh interaksi aktor terhadap semua modul. Selain

itu, dilakukan perancangan skema relasi database SQLite (ERD) dan desain antarmuka pengguna (*User Interface*).

3. Implementasi: Melakukan proses pengkodean (*coding*) menggunakan Python Flask dan SQLite biner lokal untuk merealisasikan database relasional, antarmuka HTML/CSS, modul QR Code, pencatatan log perbaikan, dan baris kode algoritma pendukung keputusan.
4. Pengujian: Melakukan pengujian fungsionalitas seluruh fitur aplikasi menggunakan *Black-Box Testing* (menguji halaman login, CRUD data master, pemindaian QR Code, pengisian log perbaikan, dan cetak laporan) serta pengujian penerimaan pengguna (*User Acceptance Testing*).

## 1.6 Sistematika Penulisan

Sistematika penulisan menyajikan gambaran terstruktur mengenai susunan dan alur pembahasan yang terdapat dalam laporan skripsi ini secara keseluruhan. Mengacu pada pedoman penyusunan skripsi fakultas, laporan ini disusun dengan sistematika sebagai berikut:

**BAB I PENDAHULUAN**

Bab pembuka ini memaparkan kerangka umum penelitian, yang secara terstruktur mencakup latar belakang permasalahan, rumusan masalah, batasan masalah untuk menyempitkan ruang lingkup kajian, tujuan penelitian yang hendak dicapai, metodologi penelitian yang diterapkan, serta uraian sistematika penulisan.

**BAB II TINJAUAN PUSTAKA**

Bab ini menyajikan kajian teoretis yang melandasi penelitian dan dibagi menjadi dua poin pokok. Bagian pertama adalah landasan teori yang memaparkan kajian dari minimal tiga penelitian terdahulu yang relevan sebagai bentuk abstraksi kelayakan topik. Bagian kedua adalah dasar teori yang menguraikan kaidah-kaidah konseptual serta asumsi keilmuan yang digunakan sebagai pijakan utama untuk memecahkan permasalahan.

**BAB III METODOLOGI**

Bab ini menguraikan pendekatan dan prosedur pemecahan masalah yang meliputi pemetaan kerangka pikir dan deskripsi. Kerangka pikir berfungsi memberikan gambaran pola hubungan konsep yang digunakan untuk

menyelesaikan masalah, sementara deskripsi berfungsi memaparkan rincian langkah-langkah operasional yang ditempuh dari awal hingga akhir.

**BAB IV ANALISIS DAN PERANCANGAN**

Bab ini menjabarkan secara mendalam proses pembongkaran masalah hingga penyusunan arsitektur perangkat lunak. Pada tahap analisis, penjabaran difokuskan pada analisis masalah, analisis *software* (kebutuhan minimum peranti lunak), analisis pengguna, perancangan antarmuka pengguna (*user interface*), spesifikasi fitur-fitur, analisis alur data, hingga estimasi analisis biaya pengembangan. Pada tahap perancangan, kajian memuat pemodelan alur kerja sistem (UML), rancangan basis data (*Entity Relational Diagram*/ERD), tahapan normalisasi, struktur tabel, hingga rincian desain tampilan sistem.

**BAB V IMPLEMENTASI DAN PENGUJIAN**

Bab ini membahas hasil nyata atas perwujudan sistem berdasarkan rancangan yang telah dibuat. Uraian mencakup penjabaran kode (*listing*) program per modul, waktu dan lokasi implementasi sistem, spesifikasi minimum dari perangkat keras maupun lunak, rincian tata cara instalasi sistem (aplikasi dan basis data), tata cara menjalankan modul program, beserta hasil pelaksanaan pengujian kelayakan operasional sistem.

**BAB VI KESIMPULAN DAN SARAN**

Bab penutup ini menyajikan kesimpulan yang merumuskan temuan jawaban atas tujuan penelitian, serta memaparkan rekomendasi berupa saran-saran aplikatif yang bermanfaat bagi pengembangan keilmuan maupun penyempurnaan sistem di masa mendatang.

# BAB II

# TINJAUAN PUSTAKA

## 2.1. Landasan Teori

Penelitian Dilla & Jupron (2026) mengembangkan Sistem Pendukung Keputusan (SPK) berbasis web untuk menyeleksi penerima bantuan sosial menggunakan metode *Simple Additive Weighting* (SAW). Keduanya memanfaatkan *tech stack* yang serupa dengan kajian ini, yakni Python Flask dan SQLite. Hal menarik dari riset mereka adalah penerapan *inverse logic* yang otomatis mengubah kriteria biaya (*cost*) menjadi keuntungan (*benefit*), sehingga mengefisienkan penulisan kode normalisasi. Selain itu, proses pengolahan data dipercepat hingga memakan waktu 0,85 detik berkat *batch processing* dari pustaka Pandas. Perbedaan utamanya, literatur tersebut menilai kelayakan warga prasejahtera lewat indikator sosial ekonomi. Sebaliknya, studi saat ini berfokus pada evaluasi aset laboratorium komputer dengan memasukkan nilai depresiasi *Straight Line* sebagai parameter masukan dinamis.

Selanjutnya, Triansyah dkk. (2026) membangun aplikasi pemantauan laboratorium berbasis CodeIgniter 4 dengan metodologi Scrum. Kesamaannya dengan proyek ini terletak pada objek manajemen aset akademik serta ketersediaan fitur pelaporan kerusakan. Titik berat optimasi mereka ada pada klasifikasi insiden (perangkat keras, *malware*, dan jaringan) untuk mempercepat diagnosis teknisi, yang disajikan lewat dasbor visual interaktif. Pengelolaan datanya juga diperketat melalui *Primary Key* unik pada setiap unit komputer demi mencegah duplikasi dan merekam riwayat aktivitas. Namun, sistem acuan tersebut hanya mencakup operasional administratif berbasis PHP. Di sisi lain, riset yang sedang kita kembangkan memakai Python Flask dan dilengkapi lapisan logika buatan yang memadukan penyusutan linier serta SAW untuk menyusun prioritas penggantian barang.

Studi ketiga oleh Putra (2025) menciptakan platform manajemen aset memanfaatkan teknologi *QR Code* dan pendekatan *Waterfall*. Kajian ini memiliki kesamaan dalam metode pengembangan, lokasi penelitian, serta penggunaan

instrumen label kode pindai. Inovasi Putra menonjol pada pemindaian waktu nyata (*real-time*) guna menekan risiko kesalahan manusia saat identifikasi fisik perangkat. Manajemen pengawasannya pun ditata rapi berdasarkan hak akses ketat bagi Admin IT, Penanggung Jawab, dan pengguna umum. Kendati demikian, pencatatan di sana masih bersifat pasif. Berbeda dengan penelitian kita yang menanamkan perhitungan analitik di sisi *backend* untuk menaksir sisa nilai buku secara langsung sebelum diproses lebih lanjut oleh algoritma perankingan.

Taryono dkk. (2024b) merancang perangkat lunak tata kelola barang kampus melalui metode RAD yang juga memadukan *Straight Line* dan SAW. Persamaan kajian tersebut dengan proyek ini adalah pemakaian dua metode komputasi yang identik. Mereka mengoptimalkan kalkulasi melalui perkalian aditif antara skor kinerja dan bobot kriteria terkait, serta menstrukturkan opsi ke dalam matriks baku bersandarkan spesifikasi, kualitas, hingga harga beli. Perbedaan krusialnya, acuan sebelumnya memisahkan kedua fungsi tersebut penyusutan hanya dipakai untuk tabel pasif, sementara SAW khusus digunakan dalam penyeleksian barang baru. Sebaliknya, riset ini meleburkan angka depresiasi langsung ke dalam matriks SAW agar dapat mengevaluasi kelayakan perangkat yang sudah lama beroperasi.

Publikasi Siahaan dkk. (2021) mengusulkan Sistem Pendukung Keputusan (SPK) penghapusan inventaris berbasis web yang secara teknis mengombinasikan metode penyusutan *Straight Line* dengan algoritma SAW. Konsep penggabungan kedua algoritma dalam menentukan tingkat kelaikan dan prioritas peremajaan aset tersebut pada dasarnya sejalan dengan pendekatan yang kita gunakan. Penelitian tersebut menitikberatkan optimasi pada transformasi nilai akumulasi depresiasi akuntansi untuk dijadikan sebagai penentu bobot kriteria biaya. Selain itu, data barang diklasifikasikan berdasarkan kondisi fisik, lama waktu pemakaian, serta besaran nilai susut guna menghasilkan tiga opsi rekomendasi: dimusnahkan, dijual, atau disumbangkan. Letak perbedaan utamanya, platform mereka dirancang spesifik bagi kebutuhan tata kelola perusahaan umum (PT. Andaru Arti Agung). Sementara itu, fokus penelitian kita lebih spesifik mengarah pada pengelolaan perangkat keras laboratorium institusi pendidikan, yang didukung secara teknis oleh fleksibilitas *microframework* Flask dan basis data lokal SQLite yang ringan.

Tabel 2. 1 Tabel Literature Review

| No | Nama & Tahun | Judul | Metode | Hasil / Kesimpulan |
|---|---|---|---|---|
| 1 | **Jovanka Surya Dilla, dkk. (2026)** | Sistem Pendukung Keputusan Kelayakan Penerima Bantuan Sosial Menggunakan Metode SAW Berbasis Web | Python, Flask, SQLite, Pandas, & *Simple Additive Weighting* (SAW) | Membuktikan efisiensi komputasi *backend* Python Flask dan pustaka Pandas dalam memproses data massal hanya dalam **0,85 detik** (dari manual 28,5 jam), serta kestabilan database SQLite lokal. |
| 2 | **Triansyah, dkk. (2026)** | Optimalisasi Manajemen Aset dan Pelaporan Kerusakan Laboratorium Komputer Melalui Sistem Monitoring Berbasis Web | SDLC *Scrum*, Web-based, & CodeIgniter 4 | Menunjukkan bahwa sistem monitoring inventaris laboratorium berbasis web secara signifikan meningkatkan efisiensi administrasi, pelacakan kerusakan secara *real-time*, dan keandalan integritas data. |
| 3 | **Putra (2025)** | Rancang Bangun Sistem Informasi Manajemen dan Monitoring Aset IT Laboratorium Komputer Berbasis Web | SDLC *Waterfall*, Web-based, & QR Code | Menemukan bahwa implementasi QR Code pada inventaris IT mampu meminimalkan kesalahan pencatatan manual (*human error*) dan mempercepat pelacakan kondisi perangkat secara *real-time*. |
| 4 | **Taryono, dkk. (2024)** | Perancangan Sistem Informasi Manajemen Aset Kampus STMIK Kuwera Berbasis Web Menggunakan Metode Simple Additive Weighting (SAW) Dan Straight Line | SDLC RAD, QR Code, *Straight Line*, & SAW | Berhasil membangun sistem manajemen aset kampus terintegrasi yang menyatukan pencatatan administrasi, pengadaan, dan pelaporan nilai penyusutan aset. |
| 5 | **Siahaan, dkk. (2021)** | Sistem Pendukung Keputusan Penghapusan Aset Menggunakan Metode Simple Additive Weighting Berbasis Web | *Simple Additive Weighting* (SAW), *Straight Line*, & Web | Membuktikan bahwa peleburan perhitungan penyusutan akuntansi *Straight Line* ke dalam kriteria *cost* pada matriks keputusan SAW menghasilkan rekomendasi |

| | | | | |
|---|---|---|---|---|
| | | Pada PT. Andaru Arti Agung | | kelayakan aset yang sangat objektif dan akurat. |

Sumber: Diolah oleh Penulis (2026)

Guna mempertegas kontribusi ilmiah dan memosisikan penelitian skripsi ini di antara kajian terdahulu, dilakukan analisis kesenjangan (*research gap*) mendalam terhadap lima literatur acuan utama termutakhir (2021–2026). Kebaruan utama (*primary novelty*) dari penelitian ini terletak pada sintesis algoritmik dinamis yang meleburkan perhitungan depresiasi akuntansi metode *Straight Line* langsung ke dalam matriks keputusan *Simple Additive Weighting* (SAW) secara otomatis di sisi *backend*. Pendekatan integratif ini secara fundamental berbeda dengan penelitian Taryono dkk. (2024) yang memisahkan fungsi komputasi *Straight Line* sekadar untuk pelaporan pasif dan SAW untuk pemilihan vendor baru, maupun penelitian Siahaan dkk. (2021) yang berfokus pada perusahaan umum tanpa otomatisasi penarikan data basis data lokal pada arsitektur server mikro. Selain itu, sistem ini mengatasi keterbatasan aplikasi manajemen laboratorium milik Triansyah dkk. (2026) dan Putra (2025) yang belum memiliki lapisan logika keputusan cerdas (*logic tier*), serta memperluas penerapan SAW oleh Dilla dkk. (2026) dari domain sosial tanpa penyusutan fisik ke dalam evaluasi aset laboratorium komputer.

Secara teknis, mekanisme kebaruan tersebut beroperasi secara *real-time* di latar belakang melalui *backend* Python Flask. Sistem secara otomatis mengekstrak variabel harga perolehan, nilai residu, dan umur ekonomis dari basis data SQLite untuk mengeksekusi perhitungan *Straight Line*, yang kemudian menghasilkan nilai sisa umur ekonomis perangkat. Nilai ini diumpankan secara dinamis sebagai parameter kriteria biaya (*cost* / C1) utama dengan bobot tertinggi dalam matriks SAW, di mana rasio sisa umur yang semakin kecil menandakan prioritas penggantian yang semakin mendesak. Bahkan, ketika umur ekonomis perangkat telah habis (c1 = 0.0), *backend* secara otomatis menetapkan rasio normalisasi darurat $r_{c1} = 1.0$. Nilai C1 tersebut lantas disilangkan secara proporsional dengan skor kerusakan fisik (C2) dan akumulasi biaya perawatan historis dari kueri agregasi SQL SUM (C3), sehingga menghasilkan nilai preferensi (Vi) secara langsung (*live*) yang secara objektif memadukan nilai sisa teoretis akuntansi, kondisi fisik riil, serta beban finansial pemeliharaan.

Kebaruan kedua berfokus pada efisiensi arsitektur *web lightweight* melalui pengemasan sistem keputusan multikriteria menggunakan kombinasi *microframework* Python Flask dan basis data relasional SQLite biner lokal yang tertanam (*self-contained*). Berbeda dengan mayoritas sistem terdahulu (seperti Taryono dkk., 2024; Putra, 2025; Triansyah dkk., 2026) yang mengandalkan arsitektur PHP (CodeIgniter/Laravel) dan MySQL dengan instalasi server terpisah yang kompleks, penelitian ini mengimplementasikan logika matematika SAW dan *Straight Line* secara terpusat di dalam blok fungsi perutean (*routing*) Flask. Dengan memproses seluruh transaksi data SPK secara langsung di dalam memori sementara (*in-memory processing*), sistem tidak membebani indeks kueri basis data secara berulang. Karakteristik SQLite yang *serverless* dan *zero-configuration* memastikan aplikasi dapat dijalankan secara instan dan mandiri di lingkungan lokal laboratorium tanpa membebani memori komputer klien.

Kebaruan ketiga terwujud dalam transformasi tata kelola laboratorium berbasis siklus hidup aset terpadu, khususnya pada fase peremajaan atau penghapusan (*disposal*). Aplikasi inventaris laboratorium konvensional umumnya hanya memfasilitasi fase pencatatan (*acquire*) dan pemantauan harian (*operate*), sehingga keputusan peremajaan aset kerap bersifat reaktif dan spekulatif. Melalui integrasi teknologi pemindaian *QR Code* 27 digit, modul pencatatan log biaya perbaikan dinamis, dan modul SPK terautomasi, setiap pemindaian pada komputer fisik akan langsung memanggil data historis untuk diolah oleh model SPK. Transformasi digital ini berhasil mengatasi masalah operasional di FTI UNIBBA secara sistematis dengan meniadakan risiko data ganda akibat pengarsipan *spreadsheet* terpisah, mempercepat pengawasan rutin, serta menggeser pola keputusan dari reaktif-subjektif menjadi rekomendasi peremajaan preventif yang objektif dan transparan sebelum masa manfaat ekonomis perangkat keras benar-benar habis.

## 2.2. Dasar Teori

Bagian dasar teori ini menguraikan berbagai konsep, prinsip, asumsi, serta data sekunder yang bersumber dari berbagai literatur ilmiah untuk dimanfaatkan sebagai pijakan utama dalam menganalisis dan menyelesaikan permasalahan

penelitian. Rangkaian landasan konseptual yang dibahas pada bagian ini mencakup teori mengenai tata kelola aset laboratorium, konsep Sistem Pendukung Keputusan, pendekatan matematis penyusutan *Straight Line* dan matriks keputusan *Simple Additive Weighting* (SAW), kerangka kerja arsitektur sistem informasi berbasis web, hingga prosedur rekayasa dan pengujian perangkat lunak. Konsep-konsep dasar inilah yang kemudian dipadukan secara komprehensif sebagai panduan teoritis dalam proses perancangan dan pembangunan aplikasi.

### 2.2.1 Manajemen Aset dan Tata Kelola Laboratorium

Berdasarkan pedoman standar internasional ISO 55000, aset merupakan entitas fisik yang memiliki nilai potensial maupun aktual bagi suatu organisasi (Farinha dkk., 2023; Jung & Kim, 2021). Manajemen aset didefinisikan sebagai pendekatan sistematis untuk memelihara infrastruktur guna mencapai tingkat layanan optimal dengan biaya siklus hidup (*life cycle cost*) yang paling efisien. Ruang lingkup tata kelola ini sangatlah komprehensif, mencakup perencanaan strategis, pengambilan keputusan, manajemen siklus hidup, hingga pengoperasian sistem informasi aset itu sendiri (Jung & Kim, 2021).

Dalam implementasi operasional laboratorium yang modern, tata kelola aset beroperasi secara berkelanjutan melalui empat tahapan utama siklus hidup, yaitu akuisisi (*acquire*), operasi (*operate*), pemeliharaan (*maintain*), dan penghapusan atau penggantian (*dispose*) (Jung & Kim, 2021). Integrasi sistem informasi pada tahapan-tahapan tersebut sangat krusial untuk mengumpulkan data kondisi secara sistematis, yang mana hal ini berimplikasi langsung pada konsep keberlanjutan (*sustainability*) dan ekonomi sirkular institusi. Pemanfaatan masa pakai suatu instrumen harus dimaksimalkan untuk menekan tingkat pemborosan serta mencegah pengadaan sumber daya baru yang prematur (Farinha dkk., 2023). Oleh karena itu, fase *dispose* menjadi titik analitik yang paling kritis di mana manajemen harus dapat mengambil keputusan yang akurat untuk menghentikan operasional perangkat apabila telah melampaui umur ekonomis dan kelayakan fungsinya.

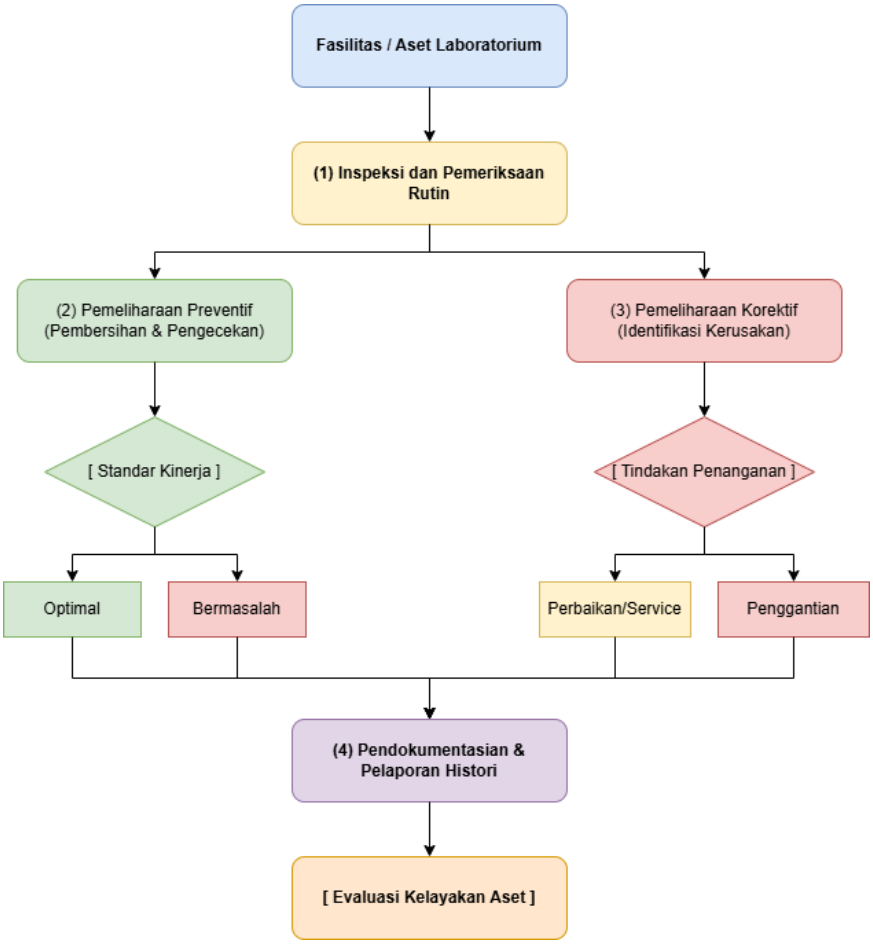
Konsep tata kelola dan siklus hidup ini merupakan pijakan teoritis yang mendasari pengembangan Aplikasi Manajemen Aset Laboratorium Komputer di Fakultas Teknologi Informasi Universitas Bale Bandung. Teori tersebut

memvalidasi bahwa sistem informasi tidak boleh hanya sekadar menjadi alat pencatatan administratif (fase *acquire* dan *operate*), melainkan harus memiliki kapabilitas analitik dalam pengambilan keputusan. Pengintegrasian metode SAW dan *Straight Line* di dalam aplikasi berbasis web ini secara khusus menargetkan penyelesaian masalah pada fase keempat siklus aset (*dispose*). Dengan adanya evaluasi nilai penyusutan yang terukur, aplikasi ini memfasilitasi institusi untuk mengambil keputusan penggantian perangkat secara objektif, menghindari pemborosan anggaran, serta menjaga kualitas fasilitas praktikum mahasiswa agar senantiasa berada dalam kondisi yang prima.

### 2.2.2 Manajemen Pemeliharaan Aset

Manajemen pemeliharaan aset merupakan serangkaian kegiatan yang direncanakan dan dilaksanakan secara sistematis dengan tujuan untuk menjaga, memulihkan, serta mengoptimalkan kondisi suatu fasilitas agar tetap mampu beroperasi sesuai dengan standar kinerja yang telah ditetapkan. Proses ini tidak hanya berfokus pada penanganan kerusakan ketika terjadi, tetapi juga mencakup berbagai langkah pencegahan untuk memperpanjang masa pakai aset dan mempertahankan tingkat keandalannya. (Nurzanah & Yun, 2023) menegaskan bahwa pelaksanaan pemeliharaan secara rutin, khususnya pada fasilitas berbentuk perangkat keras seperti komputer di lingkungan laboratorium, memiliki peranan yang sangat penting dalam meminimalisasi risiko kerusakan komponen secara tiba-tiba yang berpotensi mengganggu kelancaran kegiatan operasional, terutama proses belajar mengajar. Dalam praktiknya, tata kelola pemeliharaan aset mencakup berbagai tindakan teknis yang saling berkaitan, mulai dari pemeliharaan preventif sebagai upaya pencegahan dini, pemeliharaan korektif (*servis*) untuk mengatasi gangguan atau anomali fungsi, perbaikan menyeluruh (*repairment*) terhadap komponen yang mengalami kerusakan, hingga tahapan penggantian suku cadang (*replacement*) apabila suatu komponen telah mencapai batas kelayakan dan tidak lagi dapat dipertahankan untuk mendukung operasional secara optimal.

Secara prosedural, mekanisme operasional pemeliharaan ini dapat direpresentasikan ke dalam alur kerja sistematis seperti pada gambar berikut:




Gambar 2. 1 Arsitektur Alur Manajemen Pemeliharaan Aset

Sumber: (Nurzanah & Yun, 2023)

Berdasarkan gambar di atas, tata kerja dari teori manajemen pemeliharaan ini sangat bergantung pada tahapan inspeksi berkelanjutan yang diakhiri dengan pendokumentasian riwayat (histori) kerusakan secara terstruktur. Proses ini berawal ketika teknisi melakukan pemeriksaan langsung terhadap unit perangkat keras untuk mengevaluasi standar kinerjanya. Apabila kondisi aset normal, prosedur diarahkan pada pemeliharaan preventif rutin untuk mempertahankan masa pakai. Sebaliknya, apabila ditemukan penurunan fungsi atau cacat komponen, teknisi akan melakukan tindakan pemeliharaan korektif dengan mengambil keputusan taktis untuk memperbaiki komponen tersebut (*repairment*) atau menggantinya dengan unit baru (*replacement*). Setiap aktivitas intervensi yang telah diselesaikan di lapangan kemudian bermuara pada satu tahapan yang diwajibkan, yakni mendokumentasikan rincian kegiatan ke dalam daftar periksa untuk dilaporkan menjadi rekam jejak (*track record*) perbaikan aset.

Konsep pencatatan rekam jejak dalam prosedur pemeliharaan ini memiliki korelasi yang sangat kuat dengan perancangan fitur fungsional pada Sistem Pendukung Keputusan yang dikembangkan di dalam penelitian ini. Berdasarkan analisis kebutuhan aplikasi manajemen aset yang dibangun, sistem dirancang untuk memberikan kapabilitas khusus bagi aktor Admin untuk mengelola histori dan nominal biaya data perbaikan aset secara terpusat. Implementasi dari teori pemeliharaan tersebut diwujudkan dengan menyediakan entitas tabel relasional di dalam basis data SQLite yang bertugas merekam setiap jejak aktivitas servis yang dikeluarkan untuk masing-masing unit keras. Keberadaan data riwayat pemeliharaan yang terdigitalisasi ini pada akhirnya berperan sangat krusial dalam mendukung objektivitas komputasi algoritma *Simple Additive Weighting* (SAW). Ketika matriks algoritma SAW merekomendasikan sebuah komputer untuk segera diremajakan akibat sisa umur ekonomisnya yang telah menurun drastis (Kriteria C1) dan kondisi fisiknya yang memburuk (Kriteria C2), tingginya frekuensi kerusakan dan akumulasi biaya pemeliharaan dari data historis ini akan menjadi bukti justifikasi empiris yang memvalidasi bahwa aset bersangkutan memang sudah sangat membebani anggaran dan harus segera diganti.

### 2.2.3 Sistem Pendukung Keputusan

Sistem Pendukung Keputusan secara fundamental dirancang untuk memfasilitasi pengambil keputusan dalam memecahkan permasalahan multikriteria yang kompleks. Dalam perkembangannya, meluasnya penetrasi internet telah mengubah tren pengembangan SPK menuju arsitektur perangkat lunak berbasis web (*web-based DSS*). Arsitektur ini memberikan keuntungan yang signifikan dibandingkan sistem tradisional berbasis *desktop*, di antaranya adalah aksesibilitas komputasi tanpa batas ruang dan waktu melalui peramban (*web browser*), serta dihilangkannya keharusan instalasi aplikasi secara lokal pada perangkat keras pengguna (Sharabiani & Mousavi, 2023).

Secara konseptual, operasional SPK berbasis web berjalan melalui skema komunikasi klien-server (*client-server*). Alur pemrosesan ini diinisiasi oleh pengguna (*user*) yang mengirimkan permintaan melalui peramban web yang terhubung dengan jaringan internet menuju pusat pengendali utama atau *DSS*

*Server* (Sharabiani & Mousavi, 2023). Server ini bertugas merender antarmuka sekaligus mengeksekusi logika algoritma pengambilan keputusan secara terpusat. Selama proses komputasi berlangsung, *DSS Server* melakukan pertukaran informasi secara dua arah dengan basis data (*database*) relasional guna menarik maupun menyimpan parameter kriteria tanpa memicu redundansi rekaman data (Sharabiani & Mousavi, 2023). Hasil kalkulasi matang dari server tersebut kemudian ditransmisikan kembali melalui internet untuk divisualisasikan menjadi rekomendasi interaktif pada layar pengguna.

Konsep arsitektur SPK berbasis web ini menjadi landasan teori yang krusial bagi pengembangan kerangka sistem dalam penelitian ini. Pada konteks pengelolaan aset di Laboratorium Komputer Fakultas Teknologi Informasi Universitas Bale Bandung, skema *client-server* tersebut diimplementasikan secara langsung menggunakan *framework* Flask berbasis Python yang bertindak sebagai *DSS Server*, serta SQLite yang beroperasi sebagai *database* relasionalnya. Melalui adopsi kerangka kerja ini, pengelola laboratorium selaku pengguna dapat dengan mudah mengakses modul evaluasi kelayakan perangkat. Ketika data dimasukkan, Flask akan menarik histori aset dari SQLite, memprosesnya secara sistem di latar belakang (*backend*) menggunakan integrasi algoritma penyusutan *Straight Line* dan matriks perankingan SAW, untuk kemudian menyajikan laporan prioritas penggantian aset yang sangat objektif langsung ke peramban pengguna.

### 2.2.4 Metode Straight Line

Metode *Straight Line* atau Garis Lurus adalah teknik penyusutan dalam akuntansi yang paling banyak digunakan untuk membagi penurunan nilai aset secara rata dan stabil selama masa pakainya. Perhitungan ini bertumpu pada tiga hal pokok: harga beli awal, nilai sisa (residu), dan perkiraan umur ekonomis. Asumsi dasarnya adalah manfaat sebuah aset akan terus menyusut seiring berjalannya waktu karena faktor usia dan tingkat keusangan. Di era digital saat ini, memadukan formula ini secara otomatis ke dalam sistem inventaris menjadi langkah yang sangat menguntungkan. Selain mencegah rentannya kesalahan pencatatan manual menggunakan *spreadsheet*, cara ini juga membuat pengelolaan operasional menjadi lebih transparan dengan adanya visualisasi data penyusutan

yang selalu akurat (Purnama dkk., 2024; Sadewa dkk., 2024; Saputra & Rasiban, 2024).

Untuk tahapan operasionalnya, nilai penyusutan dengan metode *Straight Line* dihitung dengan cara mencari selisih antara harga perolehan awal dan estimasi nilai sisa aset, lalu hasilnya dibagi rata sesuai dengan taksiran umur ekonomisnya. Bentuk matematis dari perhitungan beban penyusutan periodik ini dirumuskan secara formal sebagai berikut:

Beban Penyusutan = (Harga Perolehan-Nilai Residu) / (Umur Ekonomis)

Pada rumus tersebut, komponen 'Harga Perolehan' mencakup seluruh dana yang dikeluarkan sejak awal pengadaan aset hingga alat siap beroperasi. Sementara itu, 'Nilai Residu' adalah perkiraan nilai jual aset saat purna pakai, dan 'Umur Ekonomis' merupakan target lamanya durasi pemanfaatan aset tersebut. Walaupun instansi sektor publik di Indonesia pernah menggunakan kebijakan peniadaan nilai sisa (dianggap nol) agar pencatatan lebih praktis, standar akuntansi modern seperti PSAP Nomor 10 dan PMK Nomor 22/PMK.05/2022 kini mewajibkan perhitungan nilai residu. Tujuannya adalah untuk memastikan nilai buku aset disajikan secara wajar dan mencegah munculnya perhitungan biaya perawatan yang keliru (Mairuhu & Tinangon, 2014; Suryanovi, 2023).

Terkait dengan penelitian ini, yaitu pada Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA, metode *Straight Line* tidak sekadar dipakai sebagai catatan nilai buku yang pasif. Formula ini dihidupkan secara dinamis di sisi *backend* menggunakan bahasa pemrograman Python dan *framework* Flask. Logika perhitungannya ditanamkan ke dalam metode kelas terkait sisa umur ekonomis pada basis data relasional SQLite untuk melacak sisa masa manfaat setiap perangkat keras secara *real-time*, berdasarkan selisih antara tahun berjalan dan tahun pengadaan. Angka sisa umur ekonomis tersebut kemudian diumpankan langsung sebagai Kriteria Biaya (*Cost Criteria* / C1) ke dalam Sistem Pendukung Keputusan (SPK) berbasis *Simple Additive Weighting* (SAW), dan bahkan diberi persentase bobot paling tinggi. Melalui integrasi ini, ketika sebuah perangkat komputer makin menipis sisa usianya, nilai prioritas penggantiannya akan melonjak naik. Hasilnya, pengelola laboratorium di FTI UNIBBA bisa mendapatkan rekomendasi

peremajaan fasilitas yang sangat objektif, terbuka, dan tentunya divalidasi oleh perhitungan ilmiah akuntansi.

### 2.2.5 Metode *Simple Additive Weighting*

*Simple Additive Weighting* (SAW) dikenal luas sebagai salah satu metode pengambilan keputusan multikriteria (*Multi-Attribute Decision Making*) yang sangat populer. Konsep dasar metode ini bertumpu pada pencarian jumlah terbobot dari nilai kinerja setiap alternatif di seluruh atribut penilaian. Dalam penerapannya, pengambil keputusan (*decision maker*) wajib menentukan bobot preferensi untuk masing-masing kriteria secara proporsional, sejalan dengan seberapa penting kriteria tersebut. Dengan memadukan seluruh nilai kriteria dan bobotnya, SAW mampu menyajikan hasil pemeringkatan alternatif yang tidak hanya objektif dan logis, tetapi juga cepat dan mudah dipahami oleh pengguna (Dari dkk., 2024).

Secara operasional, metode SAW dijalankan melalui serangkaian tahapan sistematis: mulai dari penyusunan matriks keputusan awal, normalisasi skala kriteria, hingga kalkulasi vektor preferensi akhir. Proses ini diawali dengan membentuk Matriks Keputusan (X) berukuran m x n, yang merepresentasikan nilai kecocokan alternatif (i) terhadap kriteria (j), sebagaimana ditunjukkan pada persamaan:

$$X = \begin{bmatrix} x11 & x12 & \dots & x1n \\ x21 & x22 & \dots & x2n \\ \vdots & \vdots & \ddots & \vdots \\ xm1 & xm2 & \dots & xmn \end{bmatrix}$$

Setelah itu, matriks tersebut diubah menjadi Matriks Normalisasi (R) agar skala penilaian antar-kriteria menjadi seragam. Formula normalisasi ini dibedakan bergantung pada sifat kriterianya, yakni keuntungan (*benefit*) atau biaya (*cost*), seperti yang dijabarkan pada persamaan berikut:

$$r_{ij} = \begin{cases} \left( x_{ij} / \max_i x_{ij} \right), & \text{jika j adalah kriteria keuntungan (benefit)} \\ \left( \min_i x_{ij} / x_{ij} \right), & \text{jika j adalah kriteria biaya (cost)} \end{cases}$$

Dengan $r_{ij}$ sebagai nilai rating yang telah dinormalisasi. Selanjutnya, nilai preferensi akhir (Vi) bagi tiap alternatif dihitung dengan mengalikan nilai normalisasi tersebut dengan vektor bobot kriteria ($w_j$), merujuk pada persamaan berikut:

$$V_i = \Sigma\, w_j\, r_{ij}$$

Pada tahap akhir ini, alternatif dengan nilai Vi tertinggi menandakan bahwa opsi tersebut menduduki prioritas teratas dan dianggap sebagai pilihan yang paling optimal (Rozakh dkk., 2025).

Dalam konteks pengembangan Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA, metode SAW difungsikan sebagai tulang punggung pada lapisan logika (*logic tier*) di sisi *backend* Flask. Algoritma ini bekerja secara dinamis untuk memproses normalisasi dan pembobotan kriteria inventaris. Secara mekanis, data sisa umur ekonomis perangkat keras yang sebelumnya dihitung menggunakan metode *Straight Line* ditarik secara otomatis dari basis data SQLite. Nilai tersebut kemudian diposisikan sebagai kriteria biaya (*cost criteria* / C1) yang mengemban bobot preferensi tertinggi, untuk selanjutnya disilangkan dengan kriteria skor kerusakan fisik (C2) dan riwayat biaya perawatan (C3). Melalui mekanisme pemrosesan dalam memori (*in-memory processing*) oleh Python Flask, hasil akhir berupa nilai preferensi (Vi) akan diurutkan secara menurun (*descending*). Proses ini pada akhirnya menyajikan daftar rekomendasi yang objektif terkait perangkat keras mana yang kondisinya paling kritis dan mendesak untuk segera diganti melalui dasbor web pengelola laboratorium.

### 2.2.6 Sistem Informasi Berbasis Web

Sistem informasi berbasis web secara fundamental beroperasi menggunakan model komputasi arsitektur *client-server*, di mana perangkat klien (*client*) meminta sumber daya dari komputer pusat (*server*) melalui jaringan internet. Untuk mengakomodasi pemrosesan yang kompleks dan meminimalisasi celah keamanan, aplikasi web modern pada umumnya mengadopsi model Arsitektur Tiga Tingkat (*Three-Tier Architecture*). Pendekatan ini secara logis memisahkan fungsionalitas sistem ke dalam modul presentasi antarmuka, pemrosesan logika bisnis, dan manajemen penyimpanan data. Pemisahan secara

terstruktur ini terbukti mampu meningkatkan skalabilitas dan kemudahan pemeliharaan sistem, sekaligus memberikan lapisan keamanan ekstra karena pengguna tidak dapat memanipulasi basis data secara langsung, melainkan harus melewati protokol validasi dari *server* aplikasi terlebih dahulu (Nyabuto, 2024).

Secara operasional, alur komunikasi data pada arsitektur *Three-Tier* berjalan secara dua arah melalui tiga lapisan utama yang saling terintegrasi (Nyabuto, 2024). Lapisan pertama adalah *Client Tier* yang memposisikan diri di tingkat paling luar sebagai antarmuka untuk menerima instruksi dan menampilkan informasi kepada pengguna melalui peramban (*web browser*). Instruksi tersebut kemudian ditransmisikan menuju *Logic Tier* atau lapisan aplikasi di tingkat menengah. Lapisan inilah yang menjadi pusat komputasi untuk mengeksekusi logika pemrosesan, menjalankan algoritma, dan mengekstraksi data yang dibutuhkan dari *Data Tier*. Sebagai lapisan paling dasar, *Data Tier* beroperasi secara eksklusif dalam memelihara dan mengelola rekaman informasi menggunakan sistem manajemen basis data (*Database Server*). Setelah proses komputasi di lapisan logika selesai, hasil akhirnya dikirimkan kembali ke lapisan klien untuk divisualisasikan.

Dalam konteks penelitian ini, konsep arsitektur *Three-Tier* diadopsi secara utuh sebagai kerangka dasar rancang bangun (*blueprint*) bagi pengembangan Aplikasi Manajemen Aset Laboratorium Komputer. Pada lapisan pertama, antarmuka web yang diakses secara interaktif oleh pengelola laboratorium melalui peramban bertindak sebagai *Client Tier*. Selanjutnya, *framework* Flask berbasis Python diposisikan secara strategis sebagai *Logic Tier* (*Application Server*). Lapisan ini bertanggung jawab penuh untuk mengeksekusi seluruh logika bisnis, termasuk integrasi komputasi penyusutan *Straight Line* dan pemeringkatan SAW secara terpusat di *backend*. Sementara itu, basis data SQLite memegang peranan krusial sebagai *Data Tier* guna menyimpan seluruh riwayat pendataan aset serta parameter kriteria secara relasional dan terstruktur. Pemisahan arsitektur menjadi tiga lapisan independen ini menjamin bahwa beban komputasi Sistem Pendukung Keputusan yang kompleks dikelola secara optimal oleh *server*, sehingga pengelola laboratorium selaku pengguna senantiasa mendapatkan pengalaman antarmuka yang responsif, ringan, dan terukur.

### 2.2.7 Bahasa Pemrograman Python

Bahasa pemrograman Python merupakan salah satu bahasa pemrograman tingkat tinggi yang sangat populer dan serbaguna dalam pengembangan perangkat lunak maupun analisis data. Menurut Wilyani dkk. (2024), Python sangat digemari oleh para pengembang karena memiliki sintaksis yang sederhana dan menyerupai struktur bahasa manusia, sehingga menjadikannya sangat fleksibel dan mudah dipahami baik oleh pemula maupun profesional. Berkat kombinasi antara kejelasan sintaksis dan ketangguhan fungsionalnya, Python secara konsisten menjadi pilihan utama untuk memecahkan berbagai masalah komputasi yang kompleks, mulai dari kecerdasan buatan, pemrosesan data intensif, hingga pengembangan aplikasi arsitektur berbasis web.

Secara operasional, cara kerja bahasa pemrograman Python berpusat pada pengeksekusian baris-baris instruksi kode secara terstruktur untuk menyelesaikan suatu permasalahan logika. Menurut Saharuddin & Prihatmono (2022), Python memproses kode program secara baris per baris menggunakan metode *interpreter*, yang bertugas menerjemahkan logika algoritma, deklarasi variabel, manipulasi tipe data, serta struktur kondisi (seperti pemisalan *if/else* dan perulangan) menjadi instruksi komputasi secara langsung tanpa proses kompilasi (*compile*) yang panjang. Fleksibilitas pemrosesan kode ini sangat tinggi, di mana pemrograman Python bahkan dapat ditulis dan dieksekusi secara instan melalui platform komputasi awan (seperti *Google Colaboratory*) tanpa menuntut prasyarat instalasi perangkat lunak tambahan yang rumit di sisi klien (Wilyani dkk., 2024). Melalui mekanisme *interpreter* yang efisien inilah, skrip Python mampu menerima data masukan, menjalankan kalkulasi matematis yang diinstruksikan, dan mengembalikan hasil keluaran data secara dinamis dan *real-time*.

Penggunaan bahasa pemrograman Python ini memiliki peran yang sangat fundamental sebagai tulang punggung komputasi (*Logic Tier*) pada aplikasi manajemen aset laboratorium komputer yang dibangun di dalam penelitian ini. Sesuai dengan batasan masalah yang ditetapkan, skrip Python secara eksklusif digunakan sebagai bahasa utama untuk membangun peladen aplikasi melalui *microframework* Flask. Python bertugas mengeksekusi seluruh logika bisnis dan algoritma komputasi di sisi *backend* Sistem Pendukung Keputusan. Alur kerjanya

beroperasi secara dinamis ketika instruksi Python mengekstraksi data parameter aset seperti harga perolehan, nilai residu, dan umur ekonomis dari basis data relasional SQLite. Setelah data terkumpul, *interpreter* Python memproses kalkulasi sisa umur ekonomis menggunakan formula *Straight Line*, untuk kemudian menyuntikkan hasil tersebut ke dalam matriks algoritma *Simple Additive Weighting* (SAW). Melalui keandalan struktur bahasa Python dalam mengolah array dan normalisasi matriks komputasi inilah, aplikasi mampu melakukan pemeringkatan secara cepat dan merender luaran akhir berupa daftar rekomendasi prioritas penggantian perangkat keras langsung ke layar peramban pengelola laboratorium.

### 2.2.8 Framework Flask

*Framework* Flask merupakan kerangka kerja aplikasi web berskala mikro (*micro web framework*) berbasis Python yang dirancang untuk mempermudah pengembangan sistem. Flask menawarkan arsitektur *open-source* yang sangat ringan, cepat, dan minim dependensi eksternal, sehingga mampu beroperasi secara fleksibel di berbagai *platform* (Evan dkk., 2023). Sebagai pengelola lapisan logika bisnis, kerangka kerja ini beroperasi sebagai jembatan penghubung dinamis antara antarmuka pengguna (*frontend*), logika program (*backend*), dan basis data melalui pemanfaatan dua komponen esensialnya, yakni sistem perutean (*routing*) dan mesin (Suraya & Sholeh, 2022).

Secara operasional, alur kerja Flask diinisiasi ketika pengguna berinteraksi melalui peramban. Permintaan tersebut ditangkap oleh sistem *routing* untuk dipetakan dan diarahkan secara presisi menuju fungsi Python spesifik di dalam *server*. Pada ruang komputasi inilah sistem mengeksekusi pemanggilan data, validasi logika, hingga kalkulasi algoritma. Setelah pemrosesan *backend* rampung, mesin *template* mengambil alih tugas untuk merajut (*rendering*) hasil data tersebut secara dinamis ke dalam kerangka dokumen HTML, yang kemudian ditransmisikan kembali melintasi jaringan internet sebagai halaman web interaktif kepada pengguna (Suraya & Sholeh, 2022).

Pada penelitian ini, Flask difungsikan sebagai *Application Server* penggerak Sistem Pendukung Keputusan. Logika algoritma SAW dan formula *Straight Line* diintegrasikan terpusat pada *routing* Python. Flask mandiri mengekstraksi

parameter aset dari SQLite, mengeksekusi normalisasi dan pembobotan matriks, hingga merender perankingan akhir ke antarmuka. Sifat mikro Flask memastikan komputasi rekomendasi berjalan responsif tanpa membebani memori klien.

### 2.2.9 Database SQLite

SQLite merupakan pustaka perangkat lunak berskala mikro yang menyediakan sistem manajemen basis data relasional (RDBMS) yang tertanam secara mandiri (*self-contained*) dan tanpa peladen (*serverless*). Berbeda dengan arsitektur *client-server* tradisional, SQLite tidak memerlukan perantara protokol jaringan, melainkan berjalan langsung di dalam ruang proses aplikasi itu sendiri. Karakteristik ini memungkinkannya untuk menyimpan seluruh struktur dan rekaman basis data ke dalam satu berkas (*single-file*) lintas *platform* secara utuh, tanpa memerlukan tahapan instalasi, manajemen jaringan, maupun konfigurasi peladen yang rumit (Gaffney dkk., 2022).

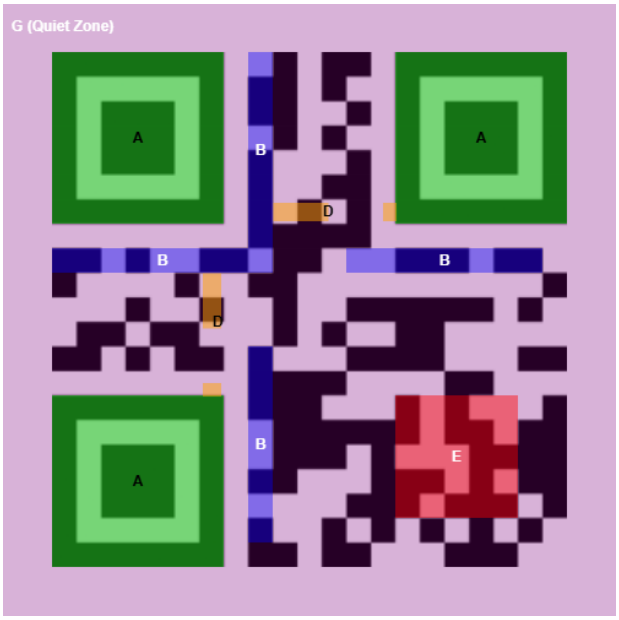
Secara operasional, alur pemrosesan data diinisiasi ketika aplikasi mengirimkan instruksi kueri SQL yang kemudian diterjemahkan (*parsing*) secara langsung oleh mesin *bytecode*. Ketiadaan jembatan jaringan memfasilitasi operasi tersebut untuk langsung mengeksekusi pembacaan atau modifikasi pada berkas tunggal basis data secara independen. Untuk menjamin keamanan informasi, SQLite menerapkan mekanisme komitmen atomik (*atomic commit*) dan pencatatan transaksional guna memastikan seluruh perubahan tersimpan permanen serta sistem terhindar dari kerusakan data apabila terjadi kegagalan daya secara tiba-tiba (Gaffney dkk., 2022).

Dalam pengembangan sistem pada penelitian ini, SQLite dimanfaatkan secara penuh sebagai lapisan penyimpan data (*Data Tier*) utama pada aplikasi manajemen aset Laboratorium Komputer. Penggunaan basis data ini sangat selaras dengan implementasi *framework* Flask di sisi *backend* karena keduanya mengusung konsep kerangka kerja berskala mikro yang ringan. Seluruh data mentah yang menjadi parameter Sistem Pendukung Keputusan meliputi daftar aset, tahun perolehan, nilai residu, hingga hasil perankingan akhir metode pembobotan SAW, disimpan secara relasional di dalam satu berkas biner SQLite. Pendekatan nirkonfigurasi ini memastikan aplikasi dapat dijalankan secara langsung dan

terisolasi di lingkungan lokal komputer institusi tanpa membebani pengelola dengan pemeliharaan peladen basis data terpisah.

### 2.2.10 Teknologi Identifikasi Quick Response Code

*Quick Response* (QR) Code merupakan evolusi teknologi kode batang dua dimensi (2D) berbentuk matriks terstruktur sel persegi yang dikembangkan pertama kali oleh Denso Corporation di Jepang pada tahun 1994 untuk pelacakan kendaraan di bagian manufaktur. Berbeda dengan kode batang satu dimensi (1D) tradisional yang hanya mampu menyimpan data secara horizontal dalam kapasitas terbatas, QR Code yang telah distandardisasi secara global dalam ISO/IEC 18004 ini memiliki kemampuan untuk menyimpan informasi secara dua arah (horizontal dan vertikal) sehingga menyajikan volume penyimpanan data yang jauh lebih besar. Karakteristik unggul dari simbol matriks ini meliputi kemampuan pengenalan berkecepatan tinggi (*high-speed recognition*), ketahanan terhadap kerusakan data melalui koreksi kesalahan (*error correction*) terintegrasi, serta kemampuan pemindaian omnidireksional yang memungkinkan kode dibaca secara instan dari segala arah sudut 360 derajat kas(Firliani & Bakti, 2022; Kastella & Ujianto, 2025; Solihin & Adli, 2023).




Gambar 2. 2 Struktur QR Code

Sumber: (Solihin & Adli, 2023)

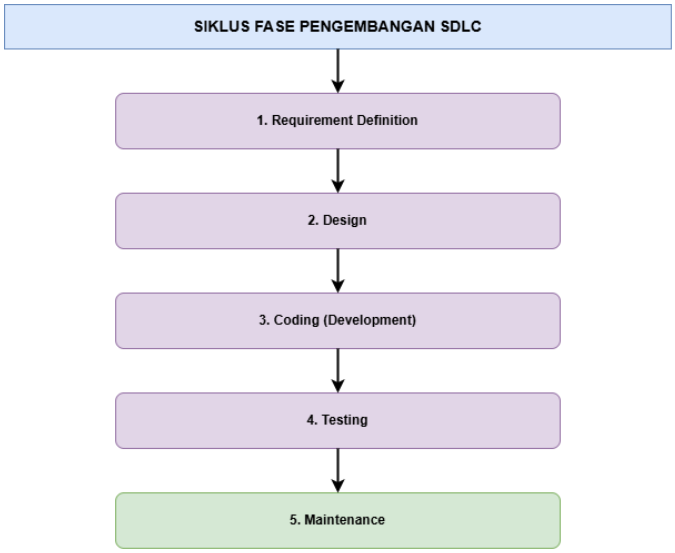
Dalam memetakan fungsionalitas keterbacaannya, arsitektur fisik QR Code secara terstruktur dibagi menjadi pola fungsi (*function patterns*) dan area pengkodean data (*encoding region*), sebagaimana divisualisasikan pada gambar di atas. Pola fungsi terdiri atas *Finder Pattern* (tiga kotak besar di setiap pojok kecuali kanan bawah untuk mendeteksi posisi), *Alignment Pattern* (untuk memperbaiki distorsi kemiringan gambar non-linier), *Timing Pattern* (garis putus-putus hitam-putih bergantian untuk menyinkronkan koordinat pusat sel), serta *Quiet Zone* (ruang kosong di sekeliling kode sebagai pembatas agar sensor mudah mendeteksi simbol). Sementara itu, area pengkodean mencakup *Format Information* (menyimpan data tingkat koreksi kesalahan dan pola masker), *Version Information* (informasi dimensi modul), serta area *Data* dan *Error Correction* tempat disisipkannya bit pesan asli berdampingan dengan kode koreksi kesalahan polinomial berbasis algoritma Reed-Solomon untuk menjaga integritas data saat fisik stiker mengalami kerusakan (Firliani & Bakti, 2022; Kastella & Ujianto, 2025; Solihin & Adli, 2023).

Dalam arsitektur pengembangan Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA, teknologi QR Code ini diimplementasikan secara dinamis sebagai jembatan siber-ke-fisik yang menghubungkan stiker fisik komputer dengan basis data relasional SQLite di sisi peladen. Pada tahap implementasi sistem, penambahan aset baru secara otomatis memicu kontroler Flask untuk melakukan penggabungan data master (konkatensi) menjadi 27 digit Kode Aset unik, yang kemudian dirender menjadi gambar stiker berformat .png menggunakan pustaka qrcode Python di direktori lokal peladen. Saat dilakukan pemindaian stiker aset di lapangan menggunakan kamera *smartphone* pengelola, aplikasi memanfaatkan pustaka ZXing (Zebra Crossing) untuk mendekode matriks piksel tersebut kembali menjadi string karakter secara *real-time*, sehingga pengelola dapat langsung mengakses riwayat pemeliharaan (*maintenance record*), mengubah informasi aset, serta memasukkan skor kondisi fisik perangkat keras tanpa risiko kesalahan pencatatan manual (*human error*).

### 2.2.11 Metode Pengembangan SDLC

*Software Development Life Cycle* (SDLC) merupakan kerangka kerja konseptual yang memegang peranan krusial sebagai pedoman baku dalam

mengarahkan, mengelola, serta mengendalikan keseluruhan siklus rekayasa perangkat lunak. Penerapan metodologi ini secara menyeluruh bertujuan untuk menjamin terciptanya produk aplikasi akhir yang memiliki mutu tinggi, tingkat keandalan yang optimal, efisiensi alokasi anggaran pembiayaan, serta kepatuhan terhadap target jadwal pengerjaan yang telah direncanakan sejak awal perancangan (Pargaonkar, 2023). Kehadiran kerangka kerja ini meminimalisasi risiko deviasi fungsional maupun pembengkakan sumber daya selama proses rekayasa berlangsung. Dalam tataran implementasi dan operasionalisasinya, SDLC berfungsi sebagai pendekatan metodologis yang mengorganisasi, menstrukturkan, dan menstandarisasi setiap tahapan aktivitas rekayasa yang diperlukan sepanjang siklus hidup sistem. Rangkaian proses tersebut berjalan secara teratur dan berkesinambungan, yang diinisiasi dari fase identifikasi serta konsepsi permasalahan awal, dilanjutkan dengan analisis kebutuhan fungsional dan non-fungsional, perancangan arsitektur antarmuka serta pemodelan basis data, translasi logika ke dalam baris kode program (implementasi), verifikasi fungsionalitas menyeluruh (pengujian), pemeliharaan operasional (*maintenance*), hingga akhirnya memasuki tahap penghentian atau dekomisioning sistem. Melalui pendekatan siklus bertahap tersebut, setiap fase dalam pengembangan perangkat lunak dapat dieksekusi secara sistematis, terukur, terarah, dan saling terintegrasi secara utuh sesuai dengan kaidah serta fase siklus yang telah ditetapkan (Abass, 2023).




Gambar 2. 3 Tahapan Fase Software Development Life Cycle

Sumber: (Abass, 2023)

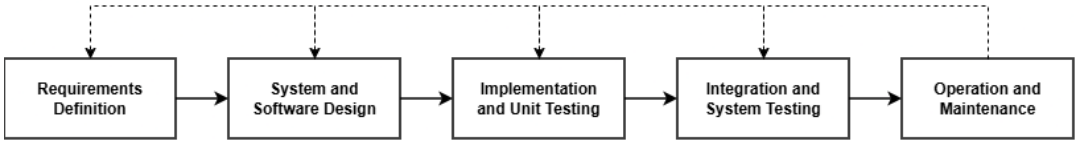
Berdasarkan Gambar 2.4, alur kerja SDLC beroperasi melalui serangkaian tahapan siklus yang saling berkesinambungan. Fase pertama diinisiasi dengan *Requirement Definition* (definisi kebutuhan), yakni tahap pemecahan masalah untuk mengidentifikasi dan mengumpulkan apa saja spesifikasi yang diperlukan oleh pengguna. Setelah itu, alur berlanjut ke fase *Design* untuk merancang arsitektur dan antarmuka sistem. Tahap inti berikutnya adalah *Coding* atau *Development*, di mana rancangan solusi tersebut direalisasikan secara aktual menjadi kode program. Sistem yang telah selesai diprogram kemudian wajib melewati tahapan *Testing* untuk divalidasi kualitasnya. Tahap mutlak yang terakhir adalah *Maintenance* (pemeliharaan) dan penerapan, yang berfokus pada peluncuran perangkat lunak sekaligus perawatan jangka panjangnya (Abass, 2023).

Dalam konteks metodologi di skripsi ini, kerangka kerja SDLC difungsikan sebagai peta jalan (*roadmap*) metodologis mutlak yang memastikan proses pembangunan aplikasi manajemen aset laboratorium berjalan secara terstruktur dan tidak acak. Berdasarkan konsep teori ini, penelitian ini secara spesifik mengadopsi salah satu model linier dari SDLC, yaitu model *Waterfall*. Pendekatan SDLC ini secara nyata mengarahkan lima tahapan penelitian ini: diawali dengan tahapan analisis kebutuhan parameter (sisa umur ekonomis dan kondisi fisik), desain arsitektur logika dan *database* SQLite, implementasi algoritma (pengkodean *Straight Line* dan SAW menggunakan *framework* Flask Python), pengujian validitas keluaran menggunakan teknik *Black Box*, hingga tahapan pemeliharaan untuk menjaga stabilitas performa modul Sistem Pendukung Keputusan tersebut di lapangan.

### 2.2.12 Metode Pengembangan Sistem Waterfall

Metode *Waterfall* atau yang sering juga dikenal sebagai *linear sequential model* (model sekuensial linier) atau *classic life cycle* (alur hidup klasik) merupakan pendekatan dalam rekayasa perangkat lunak yang proses pengembangannya berjalan secara sistematis, teratur, dan berurutan dari awal hingga akhir layaknya aliran air terjun. Ciri khas utama dari metode ini adalah alur kerjanya yang sangat kaku dan linier, di mana setiap pengerjaan harus bergerak searah mulai dari fase perencanaan sampai pemeliharaan. Dengan pendekatan sekuensial ini, setiap

aktivitas pengembangan wajib didokumentasikan secara utuh dan memiliki target fungsional yang baku. Selain itu, metode ini tidak mengizinkan adanya pengerjaan yang saling tumpang tindih (*overlapping*) ataupun proses mengulang kembali ke tahapan yang sudah terlewati (*backtracking*) selama siklus rekayasa sistem berlangsung (Adi Kurniyanti & Murdiani, 2022; Ekaputra dkk., 2020; Ningsih & Nurfauziah, 2023).




Gambar 2. 4 Model Waterfall Sommerville

Sumber:(Ratnasari dkk., 2022)

Secara fungsional, jika merujuk pada bagan arsitektur pada gambar di atas, siklus hidup rekayasa perangkat lunak menurut model Ian Sommerville dibagi ke dalam lima tahapan yang berurutan. Tahap pertama adalah Analisis dan Definisi Kebutuhan (*Requirements Definition*), yang berfokus pada pengumpulan seluruh spesifikasi, batasan, dan layanan sistem lewat komunikasi intensif dengan pengguna. Langkah kedua, Perancangan Sistem dan Perangkat Lunak (*System and Software Design*), bertujuan untuk mengubah rincian kebutuhan tersebut menjadi desain arsitektur logika, antarmuka, serta struktur basis data relasional. Selanjutnya, tahap ketiga yakni Implementasi dan Pengujian Unit (*Implementation and Unit Testing*), merupakan proses untuk mewujudkan cetak biru desain menjadi barisan kode program yang diuji fungsi unitnya secara langsung. Pada tahap keempat, Integrasi dan Pengujian Sistem (*Integration and System Testing*), seluruh unit program digabungkan dan diverifikasi ketahanannya untuk memastikan tidak ada kecacatan atau *bug*. Terakhir, tahap Operasi dan Pemeliharaan (*Operation and Maintenance*) adalah fase di mana sistem resmi dipasang di lingkungan operasional nyata, disertai dengan langkah perawatan jika muncul kendala teknis saat digunakan (Adi Kurniyanti & Murdiani, 2022; Ekaputra dkk., 2020; Ratnasari dkk., 2022).

Terkait dengan penerapannya dalam penelitian ini, perancangan Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA menerapkan kelima

tahapan *Waterfall* tersebut secara disiplin untuk menjaga kestabilan dan keakuratan logika pada Sistem Pendukung Keputusan. Kedisiplinan metode ini memastikan bahwa penulisan kode untuk formula penyusutan *Straight Line* dan algoritma perangkingan *Simple Additive Weighting* (SAW) di *backend* Python Flask baru akan dieksekusi setelah semua relasi data master dan diagram urutan UML selesai dirancang matang di fase desain. Struktur yang bertahap ini sangat efektif untuk menyelaraskan parameter masukan, seperti sisa umur ekonomis dan bobot kriteria, serta menjaganya dari risiko ketidakcocokan informasi antara memori aplikasi dan *database* SQLite. Hasilnya, Kepala Laboratorium bisa mendapatkan rekomendasi peremajaan komputer yang sangat presisi, objektif, dan dapat diandalkan.

### 2.2.13 Object-Oriented Design and Programming

Pendekatan desain dan pemrograman berorientasi objek (*Object-Oriented*) merupakan sebuah paradigma dan metode yang digunakan untuk memecahkan masalah dalam pengembangan perangkat lunak dengan memodelkan sistem berdasarkan konsep dunia nyata. Fitrani & Puspitaningrum (2023) mendefinisikan bahwa pedoman utama dalam pembentukan suatu objek pada pendekatan ini didasarkan pada kombinasi antara perilaku (*behavior*) dari suatu entitas dengan struktur datanya. Dalam praktiknya, desain dan pemrograman berorientasi objek ini dibangun di atas prinsip-prinsip fundamental untuk memastikan bahwa struktur perangkat lunak memiliki tingkat kejelasan, fleksibilitas, dan kemudahan pemeliharaan (*maintainability*) yang tinggi saat sistem semakin kompleks. Ramachandrappa (2024) menegaskan bahwa prinsip-prinsip desain berorientasi objek ini sangat krusial untuk menciptakan komponen sistem yang tangguh, modular, serta mampu beradaptasi dengan perubahan kebutuhan di masa depan tanpa merusak integritas sistem yang sudah ada.

Secara konseptual, cara kerja dari metode desain dan pemrograman berorientasi objek ini berpusat pada pemisahan dan pengelompokan fungsionalitas sistem ke dalam komponen-komponen mandiri yang disebut objek atau kelas. Proses ini dimulai dengan mengidentifikasi entitas-entitas logis dari kebutuhan operasional sistem, yang kemudian diterjemahkan ke dalam kelas-kelas dengan tanggung jawab spesifik. Setiap kelas dirancang dan diisolasi agar hanya memiliki

satu alasan operasional untuk berubah, sehingga modifikasi yang dilakukan pada satu komponen tidak akan memicu kesalahan fatal pada komponen lainnya. Selain itu, mekanisme pemrograman ini sangat bergantung pada konsep abstraksi dan dekupel (*decoupling*), di mana detail teknis tingkat rendah dari setiap komponen disembunyikan dan antarmuka komunikasi antar-objek dikelola melalui abstraksi hubungan yang terstruktur. Melalui pendekatan terdesentralisasi ini, objek-objek dapat saling berinteraksi secara dinamis, dapat mewarisi sifat dari kelas induknya untuk menjamin fleksibilitas operasional, serta dapat dimodifikasi perilakunya tanpa harus membongkar keseluruhan basis kode utama yang telah berjalan.

Konsep pemodelan dan pemrograman berorientasi objek ini memiliki hubungan yang sangat mendasar dengan tahap perancangan pada penelitian Aplikasi Manajemen Aset Laboratorium Komputer yang dibangun. Sesuai dengan batasan dan metodologi yang telah ditetapkan, arsitektur komputasi (logic tier) dari aplikasi ini menggunakan microframework Flask berbasis bahasa pemrograman Python, yang pada hakikatnya mendukung penuh penerapan paradigma Object-Oriented. Sebelum seluruh kebutuhan fungsional operasional laboratorium dan algoritma komputasi untuk evaluasi kelayakan (Straight Line dan Simple Additive Weighting) dikonversi menjadi barisan kode, pendekatan berorientasi objek ini secara mutlak digunakan sebagai landasan berpikir untuk memetakan arsitektur rancang bangun. Pemetaan analitis tersebut kemudian diejawantahkan secara visual melalui serangkaian cetak biru Unified Modeling Language (UML), yang memecah proses panjang manajemen aset menjadi sekumpulan interaksi antar-objek mandiri (seperti objek aset, objek penyusutan, dan objek pengguna). Penerapan konsep ini memastikan bahwa arsitektur fungsionalitas yang dibangun di sisi peladen bersifat sangat modular, yang pada akhirnya memberikan kemudahan luar biasa dalam hal pelacakan galat, efisiensi penulisan struktur data, serta ekstensibilitas untuk pemeliharaan fasilitas di masa mendatang.

### 2.2.14 Pemodelan Sistem Visual Menggunakan UML

*Unified Modeling Language* (UML) merupakan bahasa standar yang secara luas digunakan untuk menentukan, memvisualisasikan, membangun, dan mendokumentasikan cetak biru (*blueprint*) dari sebuah perangkat lunak.

Pendekatan pemodelan visual ini sangat penting dalam rekayasa perangkat lunak untuk memastikan bahwa arsitektur sistem maupun proses bisnis dapat dirancang secara komprehensif sebelum memasuki tahapan pengkodean. Dalam pengembangan aplikasi ini, pemodelan sistem diuraikan menggunakan beberapa diagram utama (Pitrawati & Sanjaya, 2021).

a. Use Case Diagram

*Use Case Diagram* merupakan pemodelan yang menggambarkan perilaku (*behavior*) operasional dari sistem informasi yang akan dibangun. Diagram ini difungsikan untuk memetakan secara jelas fungsionalitas apa saja yang tersedia di dalam sebuah sistem, sekaligus mengidentifikasi aktor (pengguna) mana saja yang memiliki hak akses untuk menggunakan fungsionalitas tersebut.

Tabel 2. 2 Simbol Use Case Diagram

| **Nama Simbol** | **Bentuk Visual** | **Keterangan / Fungsi** |
|---|---|---|
| Actor |  | Individu, perangkat, atau sistem eksternal yang memicu interaksi atau menerima informasi dari aplikasi. |
| Use Case |  | Mengilustrasikan fitur atau layanan spesifik yang disediakan oleh perangkat lunak bagi penggunanya. |
| Association |  | Tanda visual yang mengindikasikan adanya jalur komunikasi atau relasi antara aktor dengan suatu fitur sistem. |

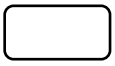
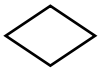
Sumber: (Pitrawati & Sanjaya, 2021)

Berdasarkan tabel Simbol Use Case Diagram tersebut, alur interaksi dalam *Use Case Diagram* memperlihatkan hubungan langsung antara pengguna fisik (*Actor*) dengan fitur sistem (*Use Case*) melalui garis relasi (*Association*). Dalam pengembangan Aplikasi Manajemen Aset Laboratorium Komputer ini, diagram *use case* digunakan secara esensial untuk mendesain hak akses interaktif pengguna (seperti Kepala Laboratorium) dalam mengelola data aset, memasukkan kriteria penilaian kelayakan fisik perangkat, serta mengakses menu eksekusi evaluasi prioritas penggantian aset.

b. Activity Diagram

*Activity Diagram* adalah pemodelan yang menggambarkan alur kerja (*workflow*) operasional atau urutan aktivitas dari sebuah sistem dan proses bisnis. Perancangan diagram ini berfokus pada pemetaan langkah-langkah komputasi yang dapat dilakukan oleh sistem, bukan rincian tindakan dari aktornya.

Tabel 2. 3 Simbol Activity Diagram

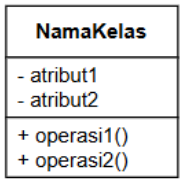
| **Nama Simbol** | **Bentuk Visual** | **Keterangan / Fungsi** |
|---|---|---|
| Initial State |  | Simbol penanda titik mula atau langkah pertama dari serangkaian eksekusi dalam sistem. |
| Activity |  | Menggambarkan suatu tindakan operasional spesifik atau langkah komputasi yang sedang diproses. |
| Decision |  | Simbol percabangan logika yang memecah alur proses berdasarkan pemenuhan kondisi atau syarat tertentu. |
| Final State |  | Titik terminasi yang menunjukkan bahwa seluruh rangkaian proses komputasi pada diagram tersebut telah selesai. |

Sumber: (Pitrawati & Sanjaya, 2021)

Berdasarkan Tabel Simbol Activity Diagram tersebut, *Activity Diagram* menguraikan algoritma sistem yang berawal dari *Initial State*, mengalir melalui berbagai proses komputasi (*Activity*), melewati seleksi logika bersyarat (*Decision*), dan berujung pada *Final State*. Di dalam skripsi ini, diagram aktivitas difungsikan secara krusial untuk merancang secara visual alur kerja algoritma di *backend* Flask. Model ini akan memetakan bagaimana sistem menghitung sisa umur ekonomis melalui formula *Straight Line*, lalu memproses nilai tersebut menjadi matriks, melakukan normalisasi, hingga mengeluarkan perankingan akhir menggunakan logika *Simple Additive Weighting* (SAW).

c. Class Diagram

Tabel 2. 4 Simbol Class Diagram

| Nama Simbol | Bentuk Visual | Keterangan / Fungsi |
|---|---|---|
| Association |  | Garis yang mengindikasikan adanya keterkaitan struktural atau relasi logis di antara berbagai entitas tabel. |
| Class |  | Entitas penyusun struktur basis data yang memuat identitas kelas, karakteristik data (atribut), serta operasi logika (metode) di dalamnya. |

Sumber: (Pitrawati & Sanjaya, 2021)

*Class Diagram* memvisualisasikan struktur statis dari sebuah sistem perangkat lunak dari segi pendefinisian kelas-kelas (*classes*) pembangun sistem tersebut. Setiap kelas dalam pemodelan ini menyimpan struktur informasi fundamental berupa nama kelas, daftar atribut atau entitas data, serta metode (*method*) atau operasi yang dimilikinya.

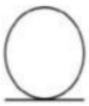
Berdasarkan Tabel Simbol Class Diagram tersebut, *Class Diagram* digunakan untuk menggambarkan struktur basis data melalui hubungan antar kelas yang terhubung dengan relasi (*Association*). Dalam pengembangan sistem di Fakultas Teknologi Informasi Universitas Bale Bandung, diagram ini berperan sebagai rancangan struktur tabel pada basis data SQLite, yang mendefinisikan entitas aset, relasi data inventaris dan penyusutan, serta penyimpanan hasil evaluasi kelayakan secara terstruktur.

d. Sequence Diagram

*Sequence Diagram* atau Diagram Urutan adalah salah satu diagram perilaku dinamis pada *Unified Modeling Language* (UML) yang berfungsi untuk memperlihatkan bagaimana objek-objek di dalam sistem saling berinteraksi dari waktu ke waktu. Melalui diagram ini, kita bisa melihat kolaborasi antar-objek secara nyata lewat rangkaian pesan (*message*) yang saling dikirimkan pada sebuah garis waktu vertikal. Hal ini sangat membantu dalam memvisualisasikan eksekusi

dari suatu skenario fungsionalitas tertentu di dalam sistem (Narulita dkk., 2024; Sutiyono & Salman Ramadan, 2025).

Tabel 2. 5 Simbol Sequence Diagram

| Nama Simbol | Bentuk Visual | Keterangan / Fungsi |
|---|---|---|
| **Lifeline (Garis Hidup)** |  | Garis waktu vertikal yang menunjukkan rentang hidup atau keberadaan sebuah objek selama proses interaksi terjadi. |
| **Boundary Class (Antarmuka)** |  | Mewakili elemen antarmuka pengguna (seperti halaman web atau *User Interface*) yang menjadi penghubung komunikasi langsung antara aktor dan sistem internal. |
| **Control Class (Pengendali)** |  | Objek yang memegang kendali atas alur logika sistem, pemrosesan algoritma, verifikasi keamanan, maupun fungsi perutean (*routing*) di sisi *backend* (peladen). |
| **Entity Class (Entitas)** |  | Mewakili struktur basis data atau tabel persisten (seperti SQLite) yang bertugas untuk mengelola dan menyimpan rekaman data. |
| **Activation Bar** |  | Menandakan rentang waktu saat sebuah objek sedang aktif memproses suatu operasi, metode, atau menjalankan logika internalnya. |

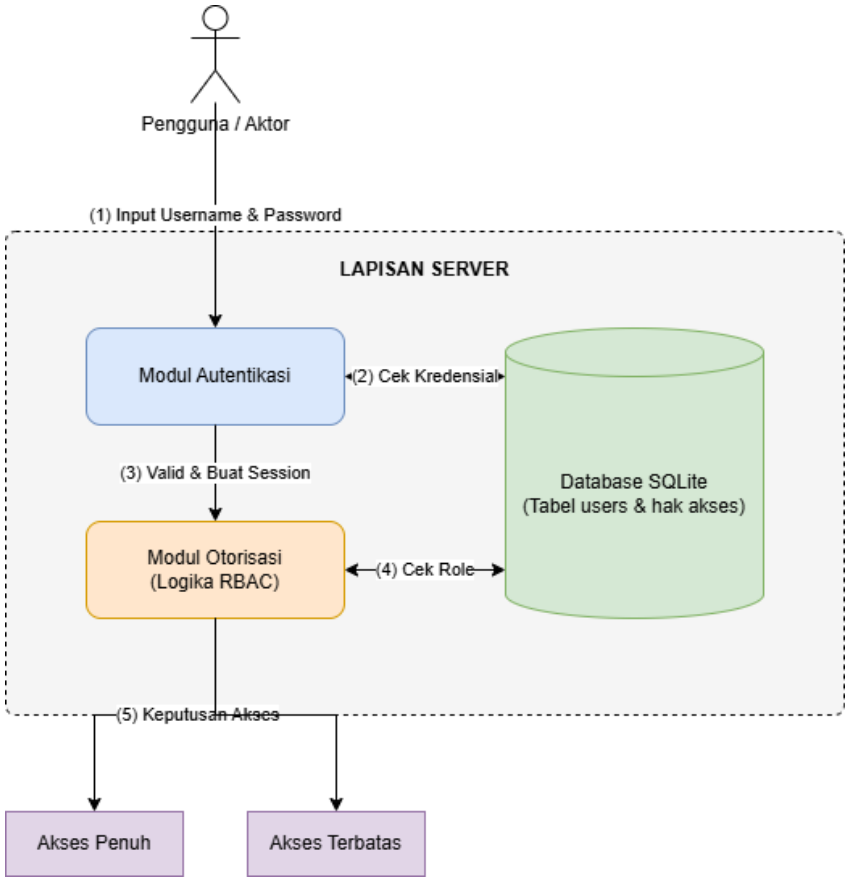
Sumber: (Rasiban dkk., 2024)

### 2.2.15 Autentikasi, Otorisasi, dan Role-Based Access Control

Keamanan sistem informasi pada aplikasi berbasis web berpusat pada perlindungan data dan sumber daya dari ancaman akses yang tidak sah melalui mekanisme autentikasi, otorisasi, dan *Role-Based Access Control* (RBAC). Autentikasi merupakan proses verifikasi identitas pengguna sebelum diberikan hak akses ke dalam sistem, yang bertindak sebagai garis pertahanan pertama untuk mencegah intervensi dari pihak yang tidak berwenang. Setelah proses autentikasi berhasil dilakukan, sistem akan menjalankan proses otorisasi untuk menentukan hak akses pengguna terhadap sumber daya sistem, yang memastikan bahwa setiap individu hanya dapat mengakses fitur atau data yang sesuai dengan batasan kewenangannya (Evan Adicandra dkk., 2026). Pengelolaan hak akses ini dapat dioptimalkan menggunakan model RBAC, yaitu sebuah pendekatan keamanan yang membatasi akses pengguna berdasarkan peran (*role*) mereka di dalam suatu

organisasi, sehingga administrator tidak perlu mengatur izin akses secara individual untuk setiap pengguna(Christopher dkk., 2026). Model RBAC juga menerapkan prinsip hak istimewa terkecil (*least privilege*), di mana pengguna hanya diberikan akses minimal yang benar-benar mereka perlukan untuk menjalankan tugas operasionalnya (Sahyudi & Susanto, 2025).

Secara struktural, mekanisme alur keamanan pada sistem ini beroperasi melalui pemisahan fungsi operasional yang sangat jelas antara tahapan verifikasi identitas (otentikasi) dan proses penentuan hak akses pengguna (otorisasi). Konsep arsitektur pemisahan kendali keamanan ini dapat dipahami dan divisualisasikan secara lebih komprehensif serta terperinci pada representasi gambar berikut.




Gambar 2. 5 Arsitektur Autentikasi dan Otorisasi Berbasis RBAC

Sumber: (Christopher dkk., 2026)

Berdasarkan Gambar Arsitektur Autentikasi dan Otorisasi Berbasis RBAC di atas, kerangka arsitektur tersebut mengadaptasi alur komponen keamanan dari Christopher dkk. (2026) yang disederhanakan dan disesuaikan dengan alur logika

keamanan milik Adicandra dkk. (2026). Alur sistem bermula ketika pengguna mengirimkan kredensial berupa nama pengguna dan kata sandi melalui antarmuka web. Kredensial tersebut kemudian diterima oleh modul autentikasi untuk dicocokkan dengan rekaman identitas pengguna yang tersimpan di dalam basis data secara aman. Apabila identitas pengguna terbukti valid, sistem akan mengeluarkan sesi (*session*) yang sah, dan alur pemrosesan secara otomatis dilanjutkan ke modul otorisasi yang menjalankan logika RBAC. Modul otorisasi ini kemudian berinteraksi kembali dengan basis data untuk mengekstrak atribut peran dari pengguna bersangkutan. Pada tahap akhir, sistem akan membuat keputusan akses secara terpusat di sisi peladen (*backend*), yang secara dinamis akan memisahkan rute dan tampilan fitur mana yang boleh diakses secara penuh serta fungsionalitas mana yang harus dibatasi.

Konsep autentikasi dan RBAC ini menjadi landasan teori yang sangat krusial bagi pengembangan Aplikasi Manajemen Aset Laboratorium Komputer berbasis *framework* Flask yang diusulkan. Dalam penelitian ini, aplikasi dirancang untuk melibatkan dua aktor utama dengan batasan hak operasional yang jauh berbeda, yaitu Admin selaku pengelola laboratorium dan Publik sebagai peninjau (*viewer*). Melalui implementasi metode RBAC dan pemenuhan prinsip *least privilege*, pemisahan hak akses antara kedua entitas tersebut dapat dibenarkan secara sistematis. Modul autentikasi (*login/logout*) diterapkan secara khusus untuk menjaga kerahasiaan logika sistem, di mana aktor Admin yang berhasil memverifikasi identitasnya akan diotorisasi untuk mendapatkan akses penuh ke seluruh fungsionalitas manajerial, mulai dari penambahan data aset, pengaturan nilai penyusutan, pencetakan kode QR, hingga eksekusi kalkulasi algoritma *Simple Additive Weighting* (SAW) untuk evaluasi kelayakan perangkat keras. Sebaliknya, bagi aktor Publik, sistem menerapkan pembatasan akses secara ketat dengan hanya memberikan izin (*permission*) untuk melihat visualisasi metrik evaluasi pada antarmuka *dashboard* tanpa memiliki kapabilitas untuk memodifikasi data apa pun. Penerapan struktur keamanan berbasis peran ini secara efektif menjamin bahwa integritas data inventaris laboratorium tetap terlindungi sekaligus mencegah terjadinya eksploitasi fungsionalitas pengambilan keputusan oleh pihak-pihak yang tidak berkepentingan.

### 2.2.16 *Black-Box Testing*

Metode *Black-Box Testing* (Pengujian Kotak Hitam) merupakan sebuah pendekatan pengujian perangkat lunak yang murni menilai fungsi dan perilaku luar sistem, tanpa perlu membongkar atau melihat kerumitan kode program di dalamnya. Dalam hal ini, penguji bertindak layaknya pengguna biasa (*end-user*) untuk memastikan bahwa setiap masukan (*input*) yang diberikan dapat diproses dengan benar hingga menghasilkan keluaran (*output*) yang valid dan sesuai harapan. Sepanjang siklus pengembangan perangkat lunak (*SDLC*), metode ini menjadi tahapan penjaminan mutu (*quality assurance*) yang sangat penting untuk menemukan potensi kegagalan. Kegagalan tersebut bisa berupa cacat pada antarmuka (*interface errors*), kendala integrasi basis data, hingga masalah pada proses inisialisasi dan terminasi program sebelum akhirnya didistribusikan secara resmi (Akhirina dkk., 2018; Arfani dkk., 2021; Sholeh dkk., 2021).

Agar proses pengujian berjalan efektif tanpa harus menumpuk kasus uji yang berulang-ulang, *Black-Box Testing* umumnya mengandalkan teknik *Equivalence Partitioning* (EP). Teknik ini bekerja dengan cara memilah data masukan menjadi dua kelompok utama, yakni data yang valid (memenuhi aturan) dan data yang tidak valid (melanggar aturan). Pendekatan ini juga sering kali diperkuat oleh teknik *Boundary Value Analysis* (BVA) yang secara khusus membidik nilai-nilai ekstrem di ambang batas masukan, mengingat di titik-titik perbatasan inilah eror sistem paling sering muncul. Setelah rangkaian pengujian selesai, tingkat keberhasilannya akan diukur secara kuantitatif melalui persentase kelulusan kasus uji (*test case*) dengan persamaan matematis berikut ini:

$$N = SP/SM \times 100\%$$

Merujuk pada persamaan tersebut, nilai keberhasilan (N) didapatkan dengan membandingkan jumlah kasus uji yang sukses dijalankan dengan tepat (SP) melawan total keseluruhan kasus uji yang telah dirancang di dalam matriks pengujian (SM) (Arfani dkk., 2021; Sholeh dkk., 2021; Wicaksono, 2025).

Terkait dengan penerapannya pada penelitian ini, metode *Black-Box Testing* diaplikasikan pada Bab V sebagai alat validasi utama untuk menguji keandalan fungsionalitas Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA.

Metode ini diandalkan untuk memverifikasi kinerja *backend* Python Flask dan basis data SQLite secara *real-time* tanpa harus menginspeksi kerumitan struktur kodenya. Secara praktis, perpaduan teknik EP dan BVA dipakai untuk memperketat pencegahan eror (*error handling*) pada kolom formulir, seperti membuat sistem otomatis menolak input angka negatif untuk Harga Perolehan dan Umur Ekonomis dari kalkulasi *Straight Line*, serta memblokir penyimpanan bobot kriteria SAW jika total akumulasinya tidak pas di angka mutlak 1,00. Berkat evaluasi fungsional yang menyeluruh ini, peleburan nilai sisa umur dari algoritma *Straight Line* ke dalam matriks keputusan *Simple Additive Weighting* (SAW) dipastikan dapat berjalan lancar dan bebas dari kecacatan sistem (*bug*), sehingga pada akhirnya mampu menyajikan rekomendasi peremajaan perangkat yang objektif dan transparan bagi pengelola di FTI UNIBBA.

### 2.2.17 *User Acceptance Testing*

*User Acceptance Testing* (UAT), yang sering juga dianggap sebagai bagian dari pengujian alfa (*alpha testing*), adalah tahap penilaian terakhir dalam pembuatan perangkat lunak yang melibatkan pengguna akhir secara langsung. Tujuannya adalah untuk memastikan bahwa sistem benar-benar siap dan layak beroperasi di lapangan sebelum resmi diluncurkan. Sebagai instrumen untuk mengukur fungsi dan tingkat penerimaan pengguna, UAT menjembatani kesesuaian antara fitur teknis buatan pengembang dengan kebutuhan bisnis, alur kerja, dan harapan nyata dari para pemangku kepentingan. Dengan mengajak pengguna untuk langsung mencoba antarmuka dan interaksi aplikasi, metode ini tidak hanya mengesahkan kelayakan sistem, tetapi juga sangat jitu untuk menemukan masalah navigasi, ketidaknyamanan tampilan, atau fitur yang kurang pas sebelum perangkat lunak tersebut dirilis ke publik (Juliartha dkk., 2022; Nurfadillah dkk., 2026; Risma dkk., 2026).

Secara teknis, pengujian UAT ini menerapkan metode deskriptif kuantitatif melalui penyebaran kuesioner terstruktur. Penilaian didasarkan pada skala Likert lima tingkat (bobot 5 untuk Sangat Setuju hingga 1 untuk Tidak Setuju). Kuesioner ini mengukur aspek kemudahan penggunaan, kepuasan, kelengkapan fungsi, dan

kinerja sistem. Guna memperoleh kesimpulan kelayakan yang objektif, tingkat penerimaan pengguna dihitung secara matematis menggunakan persamaan berikut:

$$\text{Persentase UAT} = (\text{Skor Aktual} / \text{Skor Ideal}) \times 100\%$$

$$\text{Skor Aktual} = \Sigma (f \times w)$$

$$\text{Skor Ideal} = R \times W_{max} \times Q$$

Keterangan Variabel:

a. Presentase UAT: Nilai persentase akhir yang menunjukkan tingkat kelayakan penerimaan sistem.
b. Skor Aktual : Total nilai gabungan dari jawaban seluruh responden.
c. Skor Ideal : Nilai tertinggi yang bisa dicapai jika semua responden memilih opsi Sangat Setuju.
d. f: Jumlah responden yang memilih opsi jawaban tertentu.
e. w: Bobot nilai pada skala Likert (dari rentang 1 sampai 5).
f. R: Total keseluruhan responden yang ikut serta dalam pengujian.
g. $W_{max}$: Bobot angka paling tinggi dalam skala Likert (yaitu 5).
h. Q: Jumlah keseluruhan pertanyaan yang ada di kuesioner.

Setelah persentase kelayakan berhasil dihitung, angka tersebut akan dicocokkan dengan kategori interval untuk menentukan seberapa layak sistem tersebut digunakan (Aliyah dkk., 2024; Risma dkk., 2026). Rincian dari kategori interpretasi ini dapat dilihat pada Tabel 2.7 berikut:

Tabel 2. 6 Kategori Interpretasi Kelayakan UAT

| No. | Persentase Kelayakan | Kategori Kelayakan Akademis |
|---|---|---|
| 1 | 81% – 100% | Sangat Layak / Sangat Diterima |
| 2 | 61% – 80% | Layak / Diterima |
| 3 | 41% – 60% | Cukup Layak |
| 4 | 21% – 40% | Kurang Layak |
| 5 | 0% – 20% | Tidak Layak |

Sumber: (Aliyah dkk., 2024; Risma dkk., 2026)

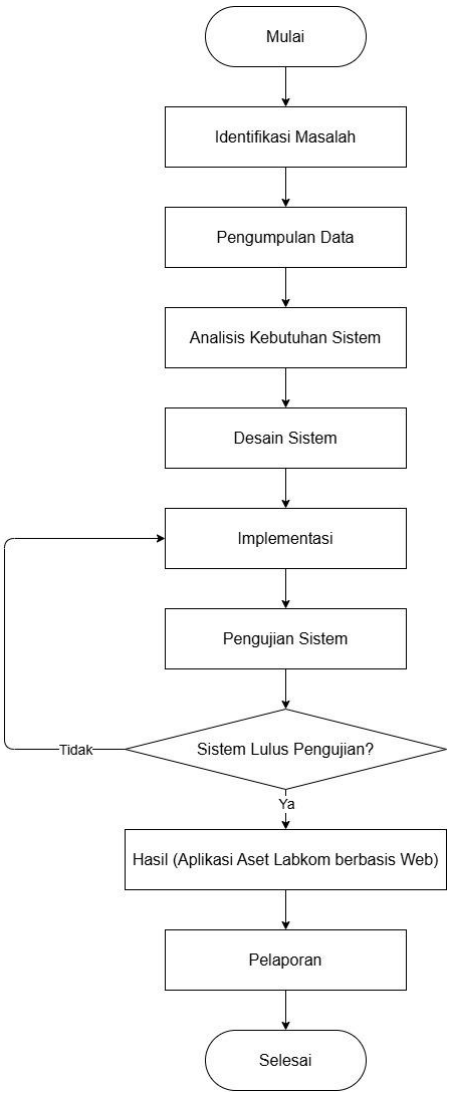
Terkait dengan penelitian ini, pengujian kelayakan menggunakan instrumen UAT diterapkan pada Bab V untuk melihat sejauh mana Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA bisa diterima oleh Kepala Laboratorium dan para staf pengelola selaku pengguna utamanya. Skenario pengujiannya dirancang secara berurutan untuk menilai seberapa simpel tata letak antarmukanya, seberapa jelas arah navigasinya, bagaimana respons fungsional *controller* Flask, hingga kestabilan grafik visual dan fitur cetak laporannya. Langkah ini menjadi sangat krusial untuk membuktikan bahwa hasil perhitungan penyusutan alat menggunakan metode *Straight Line* dan urutan rekomendasi keputusan dari algoritma *Simple Additive Weighting* (SAW) benar-benar tersaji dengan tepat, informatif, dan transparan. Dengan begitu, sistem ini dapat divalidasi keandalannya dalam meringankan beban tata kelola inventaris secara nyata di lingkungan FTI UNIBBA.

# BAB III

# METODOLOGI

## 3.1 Kerangka Pikir

Agar penyelesaian masalah lebih terarah, penelitian ini dilaksanakan melalui langkah-langkah operasional yang terstruktur. Rincian dari setiap tahapan yang ditempuh tersebut digambarkan melalui alur kerangka pikir pada Gambar 3.1 berikut.




Gambar 3. 1 Kerangka Pikir

Sumber: Diolah oleh Penulis (2026)

## 3.2 Deskripsi

Bagian deskripsi ini memaparkan secara rinci setiap langkah operasional yang ditempuh sesuai dengan alur kerangka pikir yang telah ditetapkan sebelumnya pada Gambar 3.1. Pelaksanaan penelitian ini dieksekusi secara sistematis melalui serangkaian tahapan yang saling berkesinambungan. Rangkaian metodologis tersebut diawali dengan tahapan identifikasi masalah dan pengumpulan data di lapangan, yang dilanjutkan dengan proses analisis kebutuhan dan perancangan desain sistem. Rancangan konseptual tersebut kemudian direalisasikan menjadi perangkat lunak melalui tahapan implementasi dan komputasi algoritma, sebelum akhirnya divalidasi kelayakannya melalui tahap pengujian sistem. Apabila program telah dinyatakan lulus uji, seluruh siklus penelitian ini bermuara pada pemaparan hasil akhir dan penyusunan pelaporan.Bagian deskripsi ini memaparkan secara rinci setiap langkah operasional yang ditempuh sesuai dengan alur kerangka pikir yang telah ditetapkan sebelumnya pada Gambar 3.1. Pelaksanaan penelitian ini dieksekusi secara sistematis melalui serangkaian tahapan yang saling berkesinambungan. Rangkaian metodologis tersebut diawali dengan tahapan identifikasi masalah dan pengumpulan data di lapangan, yang dilanjutkan dengan proses analisis kebutuhan dan perancangan desain sistem. Rancangan konseptual tersebut kemudian direalisasikan menjadi perangkat lunak melalui tahapan implementasi dan komputasi algoritma, sebelum akhirnya divalidasi kelayakannya melalui tahap pengujian sistem. Apabila program telah dinyatakan lulus uji, seluruh siklus penelitian ini bermuara pada pemaparan hasil akhir dan penyusunan pelaporan.

### 3.2.1 Identifikasi Masalah

Tahap awal ini difokuskan untuk membedah akar masalah dalam pengelolaan inventaris di Laboratorium Komputer FTI UNIBBA, yang bermuara pada tiga kendala utama yang saling berkaitan. Kendala tersebut mencakup pencatatan manual via *spreadsheet* terpisah yang kerap memicu duplikasi data akibat hilangnya riwayat aset, ketiadaan buku log servis berkala yang membuat kondisi fisik komputer sulit terpantau secara rutin, hingga pola peremajaan fasilitas yang masih bersifat reaktif atau baru diganti setelah perangkat rusak total karena

ketiadaan alat hitung sisa umur ekonomis. Sebagai solusi atas kendala-kendala tersebut, tahapan inisiasi ini menghasilkan rumusan batasan kebutuhan sistem yang menetapkan rancang bangun aplikasi web terintegrasi sebagai jalan keluar untuk menyelesaikan seluruh permasalahan tersebut secara menyeluruh.

### 3.2.2 Pengumpulan Data

a. Observasi

Mengamati fisik unit komputer di Laboratorium Multimedia dan Laboratorium Basis Data FTI UNIBBA. Observasi ini bertujuan untuk memetakan kondisi fisik riil perangkat keras dan pencatatan manual yang saat ini berjalan.

b. Wawancara

Dilakukan secara terstruktur bersama Kepala Laboratorium Komputer FTI UNIBBA. Wawancara ini bertujuan menggali alur birokrasi perbaikan barang, prosedur pelaporan, serta estimasi umur ekonomis perangkat keras laboratorium.

c. Studi Literatur

Mengkaji lima jurnal acuan termutakhir (2021–2026) untuk memetakan arsitektur *microframework* Python Flask, manajemen pangkalan data biner SQLite, serta implementasi logika penggabungan metode akuntansi *Straight Line* dan perangkingan SAW.

### 3.2.3 Analisis Kebutuhan Sistem

Data lapangan yang telah dikumpulkan kemudian dianalisis untuk menyusun spesifikasi kebutuhan perangkat lunak (*Software Requirement Specification*) yang terbagi ke dalam dua aspek utama:

a. Analisis Kebutuhan Fungsional

1. Modul Pencatatan: Sistem mampu mengelola seluruh operasi CRUD (*Create, Read, Update, Delete*) data inventaris secara terpusat, sekaligus mengotomatisasi pembuatan stiker label *QR Code* berbasis kodifikasi unik 27 digit untuk setiap perangkat keras.
2. Modul Perbaikan: Sistem menyediakan formulir pencatatan riwayat servis, pelacakan tanggal pemeliharaan, penghitungan akumulasi biaya perawatan secara otomatis, serta pembaruan skor kondisi fisik perangkat.

3. Modul Evaluasi: Sistem mampu menghitung sisa umur ekonomis perangkat secara otomatis menggunakan metode *Straight Line*, lalu mengintegrasikannya bersama riwayat biaya perawatan ke dalam algoritma *Simple Additive Weighting* (SAW) guna menghasilkan urutan prioritas peremajaan komputer yang objektif.

b. Analisis Kebutuhan Non-Fungsional

Sistem dirancang menggunakan arsitektur *Three-Tier* untuk menjaga performa tetap ringan dan responsif. Bagian antarmuka (*Client Tier*) dibangun menggunakan Tailwind CSS, pemrosesan logika sistem (*Logic Tier*) ditangani oleh Python Flask, sedangkan pengelolaan data (*Data Tier*) mengandalkan basis data relasional SQLite yang bersifat *serverless* dan praktis tanpa memerlukan konfigurasi rumit.

### 3.2.4 Desain Sistem

Tahap ini menetapkan spesifikasi rancangan arsitektur perangkat lunak (*software blueprint*) secara visual, struktural, dan algoritmik menggunakan pendekatan *Object-Oriented*. Spesifikasi desain ini dibagi ke dalam dua komponen utama:

a. Pemodelan Diagram UML (*Unified Modeling Language*)

Pemodelan ini menetapkan arsitektur visual sistem melalui empat diagram inti:

1. Use Case Diagram: Menetapkan spesifikasi hak akses untuk dua aktor. Admin (Pengelola Laboratorium) dirancang memiliki hak akses penuh untuk mengeksekusi otentikasi (*Login/Logout*), Kelola Data Master, Kelola Data Aset, *Generate* & Cetak QR Code, Kelola Histori Perbaikan, Penyesuaian Bobot Kriteria SAW, Eksekusi Kalkulasi SPK, dan Cetak Laporan. Sementara itu, Publik (*Viewer*) dirancang hanya memiliki hak akses terbatas untuk memantau visualisasi metrik pada dasbor.
2. Activity Diagram: Menetapkan spesifikasi alur operasional (*workflow*) komputasi sistem, yang meliputi rancangan: alur otentikasi Admin, alur pengelolaan aset dan *generate* QR Code, alur pencatatan histori servis, serta alur proses kalkulasi SAW (dari ekstraksi kriteria, normalisasi, hingga visualisasi perankingan).

3. Sequence Diagram: Menetapkan spesifikasi alur komunikasi pertukaran pesan antara antarmuka pengguna (*UI*), *Controller/Routing* Flask, dan Model SQLite. Skema komunikasi yang dirancang mencakup: eksekusi *generate* QR Code, pemindaian aset untuk histori perbaikan, dan proses integrasi SPK (mulai dari penarikan parameter penyusutan/bobot di SQLite, kalkulasi matriks di *backend*, hingga pengembalian tabel peringkat ke layar).
4. Class Diagram: Menetapkan spesifikasi struktur entitas relasional untuk basis data SQLite. Tabel yang dirancang meliputi: users, assets, categories, locations, prodi, kepala_lab, kode_lokasi, kode_barang, perbaikan_aset, dan bobot_saw.

**b. Desain Algoritma Komputasi:**

Tahap ini menetapkan spesifikasi alur logika matematika di sisi *backend* sebagai mesin penggerak SPK:

1. Logika Straight Line: Sistem dirancang mengeksekusi kalkulasi penyusutan secara terpusat berdasarkan variabel Harga Perolehan, Nilai Residu, dan Umur Ekonomis untuk menghasilkan nilai parameter Sisa Umur Ekonomis perangkat keras di setiap periode.
2. Logika Perankingan SAW: Sistem dirancang mengeksekusi pembentukan matriks keputusan dengan memproses hasil *Straight Line* sebagai Kriteria Biaya (*Cost*). Sistem kemudian diinstruksikan untuk melakukan normalisasi matriks, lalu mengagregasikannya dengan bobot kepentingan (Wj) per kategori aset guna menghasilkan nilai preferensi akhir (Vi) sebagai dasar pemeringkatan.

### 3.2.5 Implementasi

Tahap ini menetapkan spesifikasi instruksi eksekusi pengkodean (*coding*) untuk mewujudkan rancangan arsitektur sistem menjadi perangkat lunak fungsional. Eksekusi implementasi ini dibagi ke dalam tiga spesifikasi utama:

a. Spesifikasi Lingkungan Pengembangan

Eksekusi pengkodean mewajibkan penggunaan lingkungan kerja (*environment*) yang meliputi: bahasa pemrograman Python, microframework Flask

sebagai peladen aplikasi (application server), sistem manajemen basis data *embedded* SQLite, serta lingkungan pengembangan terpadu (IDE) Visual Studio Code untuk penulisan skrip.

b. Eksekusi Pengkodean Modul

Pembuatan sistem dieksekusi secara terstruktur melalui tiga lapisan arsitektur utama (*Three-Tier*):

1. Lapisan Basis Data (*Data Tier*): Mengeksekusi penulisan kueri SQL pada SQLite untuk merealisasikan tabel-tabel relasional (sesuai rancangan *Class Diagram*), yang bertugas merekam identitas inventaris, nilai residu, histori perbaikan, dan bobot kriteria.
2. Lapisan Logika (*Logic Tier*): Mengeksekusi penulisan skrip Python pada *backend* Flask untuk mengatur *routing* akses pengguna. Lapisan ini juga mengeksekusi penulisan fungsi komputasi (*def*) khusus untuk menghitung sisa umur ekonomis menggunakan metode *Straight Line* dan menyuntikkan nilai tersebut secara dinamis ke dalam matriks perankingan *Simple Additive Weighting* (SAW).
3. Lapisan Antarmuka (*Client Tier*): Mengeksekusi penulisan kode bahasa *markup* (HTML/CSS) dan manipulasi *template engine* bawaan Flask untuk membangun tata letak dasbor agar responsif dalam menerima formulir *input* dari pengguna dan merender tabel rekomendasi dari *backend*.

c. Hasil Kompilasi

Tahap implementasi ini diakhiri dengan proses kompilasi seluruh baris kode menjadi satu kesatuan perangkat lunak berbasis web. Sistem ini dikonfigurasi agar dapat dijalankan secara mandiri pada peladen lokal (*localhost*) institusi untuk selanjutnya memasuki fase pengujian teknis.

### 3.2.6 Pengujian Sistem

Aplikasi yang telah selesai dibangun kemudian melalui serangkaian tahap pengujian untuk memastikan kelayakan operasional serta kualitas fungsionalitasnya:

a. Pengujian Fungsional (*Black-Box Testing*): Menguji fungsionalitas fitur menggunakan teknik *Equivalence Partitioning* guna memastikan sistem

merespons dengan benar terhadap *input* data yang valid maupun data tidak valid (seperti membiarkan kolom wajib tetap kosong).

b. Pengujian Nilai Batas (*Boundary Value Analysis*): Memeriksa ketahanan sistem dalam menangani galat (*error handling*) ketika menerima nilai masukan pada batas ekstrem.
c. Uji Penerimaan Pengguna (*User Acceptance Testing* / UAT): Menguji pengalaman pengguna dan kemudahan antarmuka secara langsung bersama Kepala Laboratorium Komputer FTI UNIBBA melalui kuesioner berskala Likert untuk mengukur tingkat kepuasan serta penerimaan akhir terhadap sistem.

### 3.2.7 Hasil

Tahap ini menghasilkan luaran akhir berupa aplikasi web manajemen aset laboratorium komputer yang andal, responsif, dan siap digunakan secara langsung. Fitur utamanya berpusat pada sebuah *dashboard* dinamis yang mengintegrasikan pendataan inventaris berbasis pangkalan data terpusat, pencatatan riwayat kerusakan perangkat, serta penyajian laporan rekomendasi prioritas peremajaan komputer yang transparan dan objektif guna mendukung efisiensi anggaran institusi.

### 3.2.8 Pelaporan

Tahap ini menetapkan instruksi pendokumentasian akhir dari seluruh hasil eksekusi rekayasa perangkat lunak, validasi komputasi algoritma, hingga rekapitulasi data pengujian ke dalam format karya tulis ilmiah yang utuh. Pelaporan dieksekusi secara terstruktur mulai dari Bab I hingga Bab VI dengan mematuhi parameter dan sistematika baku yang tertuang pada Pedoman Skripsi Mahasiswa Strata 1 Tahun 2021 Fakultas Teknologi Informasi Universitas Bale Bandung.

# BAB IV

# ANALISIS DAN PERANCANGAN

## 4.1 Analisis

Tahap analisis merupakan langkah fundamental dalam siklus rekayasa perangkat lunak (*Software Development Life Cycle*). Pada tahapan ini, dilakukan identifikasi secara komprehensif terhadap sistem tata kelola inventaris yang sedang berjalan di Laboratorium Komputer FTI UNIBBA guna menemukan akar permasalahan (*bottleneck*) fungsionalitas dan operasional. Hasil dari penemuan tersebut kemudian diekstraksi menjadi spesifikasi kebutuhan sistem perangkat lunak yang utuh. Analisis ini diuraikan ke dalam beberapa aspek penilaian strategis, mulai dari evaluasi kendala proses bisnis, pendefinisian perangkat komputasi pendukung, pemetaan peran pengguna, hingga perumusan model kalkulasi untuk membangun Aplikasi Manajemen Aset.

### 4.1.1 Analisis Masalah

Pengelolaan aset di Laboratorium Komputer Fakultas Teknologi Informasi Universitas Bale Bandung (FTI UNIBBA), khususnya pada Laboratorium Multimedia dan Laboratorium Basis Data, mencakup interaksi harian antara pengelola, pencatatan inventaris, serta ratusan unit perangkat keras penunjang kegiatan praktikum. Berdasarkan hasil observasi langsung dan wawancara mendalam bersama Kepala Laboratorium Komputer FTI UNIBBA, ditemukan tiga titik kendala utama (*bottleneck*) dalam tata kelola berjalan yang menghambat efisiensi operasional secara menyeluruh:

a. Tingginya Risiko Data Ganda

Pendataan inventaris saat ini masih bertumpu pada lembar sebar (*spreadsheet*) konvensional dengan berkas-berkas terpisah yang belum terhubung ke dalam basis data terpusat. Kondisi ini membuat pelacakan riwayat dan spesifikasi unit tertentu memakan waktu lama. Akibat susunan arsip yang terfragmentasi, pengelola sering kali memilih membuat lembar kerja baru dari awal

ketimbang mencari dokumen lama yang terselip. Pola kerja tersebut pada akhirnya memicu risiko duplikasi pencatatan (*data redundancy*) serta menimbulkan ketidaksinkronan data spesifikasi komputer di lapangan.

b. Tidak ada Log Perawatan Berkala dan Pengawasan Pasif

Sistem yang berjalan belum menyediakan sarana terstruktur untuk mencatat histori pemeliharaan (*maintenance logs*) perangkat. Ketiadaan rekaman digital mengenai tindakan servis, tanggal perbaikan, serta akumulasi biaya perawatan harian membuat pemantauan kelayakan fisik berjalan sangat lambat. Kondisi perangkat keras umumnya hanya direkap secara insidental setahun sekali, sehingga penurunan performa dan kerusakan teknis komputer di luar jadwal evaluasi tahunan luput dari pengawasan berkala.

c. Pola Penggantian Aset yang Reaktif dan Spekulatif

Kelemahan paling mendasar dari sistem yang ada adalah belum tersedianya modul analitik untuk mengukur tingkat kelayakan aset secara objektif. Pengelola belum memiliki instrumen hitung yang pasti mengenai sisa umur ekonomis maupun persentase keausan fisik perangkat. Akibatnya, keputusan peremajaan atau penghapusan (*disposal*) unit masih bersifat spekulatif dan reaktif komputer baru diusulkan untuk diganti ketika sudah rusak total atau mati. Pendekatan ini berisiko mengganggu kelancaran praktikum mahasiswa sekaligus memicu pemborosan anggaran institusi karena ketiadaan skala prioritas peremajaan yang terencana.

### 4.1.2 Analisis Software

Tahapan analisis perangkat lunak (*software*) secara komprehensif menguraikan spesifikasi kebutuhan minimum perangkat lunak pendukung yang diperlukan untuk merancang, membangun, serta menjalankan operasional sistem aplikasi manajemen aset. Sejalan dengan batasan masalah dan rancangan metodologi penelitian yang telah ditetapkan, pemilihan spesifikasi teknis ini disesuaikan dengan arsitektur pengembangan aplikasi berbasis web yang secara khusus mengandalkan bahasa pemrograman Python serta *microframework* Flask. Sebagai panduan implementasi sistem, rincian kebutuhan perangkat lunak minimum untuk keseluruhan proses pembuatan aplikasi tersebut dijabarkan secara terstruktur pada tabel berikut:

Tabel 4. 1 Kebutuhan Perangkat Lunak (*Software*) Minimum

| Kategori | Spesifikasi Perangkat Lunak |
|---|---|
| **Sistem Operasi** | Windows 10 / 11 (64-bit), Linux, atau macOS |
| **Bahasa Pemrograman** | Python (versi 3.8 atau yang lebih baru) |
| ***Framework*** | Flask (*Microframework*) |
| **Sistem Basis Data** | SQLite |
| **Lingkungan Pengembangan (IDE)** | Visual Studio Code |
| ***Web Browser*** | Google Chrome, Mozilla Firefox, atau Microsoft Edge |
| **Pustaka Tambahan (*Library*)** | *QR Code Generator* (seperti pustaka qrcode pada Python) dan pustaka standar pendukung komputasi matematis |

Sumber: Diolah oleh Penulis (2026)

Meskipun pengembangan berfokus pada spesifikasi perangkat lunak, implementasi operasional sistem ini juga mensyaratkan spesifikasi perangkat keras (*hardware*) minimum. Aplikasi ini didesain untuk berjalan pada PC *Server* Lokal di laboratorium dengan spesifikasi memori standar (minimal RAM 4GB) untuk mengeksekusi komputasi Flask dan *database* SQLite secara lancar. Selain itu, pada sisi pengguna (*client*), sistem mengharuskan tersedianya perangkat tambahan berupa pencetak label (*printer*) dan perangkat bergerak (*smartphone*) yang memiliki fitur kamera untuk keperluan pemindaian *QR Code* aset di lapangan.

### 4.1.3 Analisis Pengguna

Analisis pengguna bertujuan untuk mengidentifikasi siapa saja entitas yang akan berinteraksi langsung dengan Aplikasi Manajemen Aset Laboratorium Komputer ini, serta bagaimana karakteristik mereka memengaruhi desain sistem. Berdasarkan perancangan keamanan sistem menggunakan model *Role-Based Access Control* (RBAC), pengguna aplikasi ini secara spesifik dibagi menjadi dua peran (*role*) utama:

1. Admin (Kepala Laboratorium dan Staf Pengelola): Merupakan pengguna operasional utama yang diasumsikan memiliki literasi komputer yang baik dan bertanggung jawab penuh atas tata kelola aset fisik di laboratorium. Pengguna ini membutuhkan hak akses menyeluruh (*full access*) untuk melakukan autentikasi, memanipulasi data inventaris (tambah, ubah,

hapus), mencetak *QR Code*, mencatat histori servis, menyesuaikan bobot kriteria, serta mengeksekusi perhitungan Sistem Pendukung Keputusan (SPK) menggunakan metode *Straight Line* dan SAW.

2. User**:** Merupakan entitas pengguna dengan hak akses yang sangat dibatasi (*restricted access*). Pengguna pada peran ini (misalnya pimpinan rektorat atau pimpinan fakultas) hanya bertindak sebagai peninjau yang memerlukan informasi dan cetak laporan.

### 4.1.4 User Interface

Desain antarmuka pengguna (*User Interface*) pada Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA memegang peranan yang sangat penting dalam menjamin kelancaran interaksi antara pengguna dan sistem. Perancangan antarmuka ini berfokus pada penyediaan visualisasi data yang bersih, intuitif, dan ergonomis agar data analisis yang kompleks dapat dipahami dengan mudah oleh pengelola. Strategi perancangan *User Interface* dalam aplikasi ini dibagi menjadi beberapa aspek utama, yaitu:

a. Desain Responsif dan Aksesibilitas Multi-perangkat: Antarmuka aplikasi dirancang menggunakan arsitektur berbasis *web* yang responsif. Tampilan halaman akan menyesuaikan tata letaknya secara otomatis ketika diakses melalui *desktop* maupun *smartphone* untuk mendukung fungsionalitas pemindaian *QR Code* di area laboratorium secara praktis.
b. Pemisahan Tata Letak Berdasarkan Peran Pengguna (*Role-Based Layout*): Aktor Admin akan dihadapkan pada antarmuka dasbor penuh yang dilengkapi dengan menu navigasi *sidebar* untuk mengelola data master, menginput parameter, mencatat histori perbaikan, dan mengeksekusi kalkulasi. Aktor Publik disajikan tampilan antarmuka dasbor yang bersih tanpa menu manajerial.
c. Dasbor Visualisasi Analitik Interaktif: Antarmuka halaman utama (dasbor) dirancang sebagai pusat informasi eksekutif (*Executive Information System*). Antarmuka ini menyajikan visualisasi data yang dinamis menggunakan elemen grafik (*chart*) seperti grafik tren penambahan aset per bulan, distribusi kondisi aset (Baik/Rusak), serta ringkasan segmentasi sisa

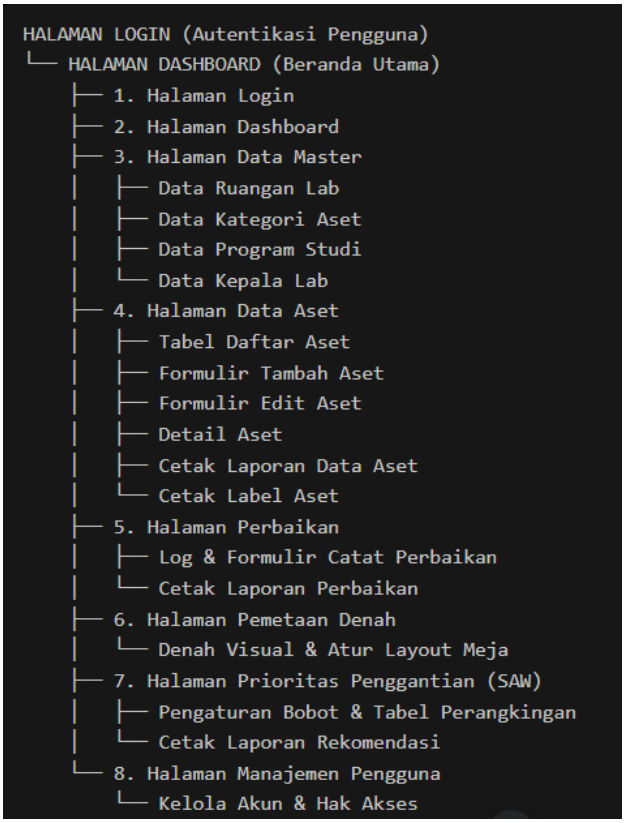
umur ekonomis. Visualisasi ini dirancang agar pengguna Publik dapat menangkap status aset laboratorium secara seketika hanya dalam satu pandangan mata tanpa perlu mengolah data mentah

d. Visualisasi Komputasi yang Intuitif: Perhitungan penyusutan akuntansi metode *Straight Line* dan perangkingan SAW ditransformasikan secara visual ke dalam dokumen tabel rekomendasi prioritas kelayakan yang terstruktur agar mudah diinterpretasikan oleh Kepala Laboratorium selaku pengambil keputusan.
e. Tipografi dan Elemen Navigasi: Penggunaan elemen teks diatur dengan meminimalkan kalimat yang terlalu panjang pada menu navigasi utama guna menghindari kejenuhan visual. Setiap formulir masukan data dilengkapi dengan petunjuk instruksi yang jelas agar pengelola terhindar dari *human error* saat memasukkan data kriteria.

### 4.1.5 Fitur-Fitur

Sistem Aplikasi Manajemen Aset Laboratorium Komputer secara khusus dirancang untuk menyediakan seperangkat fungsi operasional guna mendukung siklus pengelolaan inventaris secara terintegrasi. Rancangan fitur-fitur ini disusun secara sistematis berdasarkan hasil analisis kebutuhan pengguna, yang ruang lingkupnya mencakup pemrosesan data master, pendaftaran dan pengodean aset, pemantauan kondisi fisik, pemeliharaan perangkat, hingga penerapan analisis pendukung keputusan untuk penentuan rekomendasi penggantian unit.

Guna mempermudah pengguna maupun pengembang dalam memahami rancangan alur perpindahan antarmuka serta struktur hierarki dari keseluruhan layanan yang tersedia di dalam sistem, seluruh fitur aplikasi telah dikelompokkan secara komprehensif ke dalam sebuah bentuk bagan pohon navigasi (*navigation tree*). Pemetaan fungsional ini secara spesifik membagi antarmuka sistem menjadi 8 halaman utama yang diklasifikasikan berdasarkan dua area utama, yakni modul otentikasi akses dan lingkungan operasional sistem yang telah terotentikasi. Adapun gambaran visual dari struktur hierarki fungsi aplikasi tersebut dapat diamati secara lebih terperinci pada gambar berikut:




Gambar 4. 1 Pohon Navigasi Aplikasi

Sumber: Diolah oleh Penulis (2026)

Berdasarkan rancangan pohon navigasi aplikasi di atas, struktur alur menu dan antarmuka sistem dibagi menjadi 8 modul utama dengan rincian fungsi operasional sebagai berikut:

1. Halaman Login

Gerbang awal pintu masuk sistem yang berfungsi memverifikasi kredensial pengguna (*username* dan *password*) serta memvalidasi hak akses akun (*Role-Based Access Control*) sebelum mengarahkan pengguna ke dalam lingkungan aplikasi.

2. Halaman Dashboard

Beranda utama eksekutif yang menyajikan ringkasan statistik inventaris, indikator metrik keuangan, grafik sebaran kondisi fisik perangkat (Baik, Rusak Ringan, dan Rusak Berat), serta informasi distribusi alokasi unit perangkat di setiap ruangan laboratorium.

3. Halaman Data Master

Modul pengelolaan data referensi dasar laboratorium yang menjadi fondasi pengkodean dan pengelompokan inventaris, yang terdiri dari empat sub-menu:

a. Data Ruangan Lab: Mengelola identitas ruangan laboratorium, kode lokasi, dan kapasitas daya tampung meja.
b. Data Kategori Aset: Mengelola pengelompokan jenis perangkat komputer dan periferal laboratorium.
c. Data Program Studi: Mengelola data referensi program studi akademik yang menaungi fasilitas laboratorium.
d. Data Kepala Lab: Mengelola data pimpinan laboratorium yang berwenang dalam pengesahan dokumen laporan resmi.

4. Halaman Data Aset Modul utama pengelolaan katalog inventaris perangkat komputer laboratorium, yang mencakup enam fungsi utama:
   a. Tabel Daftar Aset: Menampilkan seluruh katalog data barang yang dilengkapi dengan fitur pencarian cepat, filter kriteria, dan indikator status perangkat.
   b. Formulir Tambah Aset: Formulir entri untuk mendaftarkan dan merekam unit perangkat laboratorium baru ke dalam basis data.
   c. Formulir Edit Aset: Formulir untuk memperbarui informasi spesifikasi teknis, kondisi fisik, maupun penempatan lokasi perangkat.
   d. Detail Aset: Menampilkan rincian spesifikasi barang, riwayat pemeliharaan, serta kalkulasi penyusutan nilai buku perangkat.
   e. Cetak Laporan Data Aset: Fitur untuk mengekspor dan mencetak dokumen rekapitulasi katalog inventaris ke dalam format PDF resmi.
   f. Cetak Label Aset: Fitur untuk mencetak stiker label kode inventaris dan kode QR siap tempel pada fisik perangkat.
5. Halaman Perbaikan

Modul pengelolaan pemeliharaan dan perawatan teknis perangkat laboratorium, yang mencakup dua fungsi utama:

a. Log & Formulir Catat Perbaikan: Menampilkan riwayat transaksi perbaikan sekaligus formulir pencatatan tindakan servis baru untuk mengelompokkan dan memperbarui akumulasi biaya pemeliharaan.
b. Cetak Laporan Perbaikan: Fitur untuk mengekspor dan mencetak dokumen rekapitulasi riwayat pemeliharaan serta total pengeluaran biaya servis ke dalam format PDF.

6. Halaman Pemetaan Denah Modul visualisasi spasial posisi meja komputer di dalam ruangan laboratorium:
    a. Denah Visual & Atur Layout Meja: Menampilkan peta denah interaktif beserta indikator warna status kondisi meja (Hijau = Normal, Kuning = Perawatan, Merah = Rusak Berat) serta fitur penyusunan dan pengubahan posisi tata letak meja.
7. Halaman Prioritas Penggantian (metode SAW)

    Modul untuk menentukan rekomendasi peremajaan perangkat komputer laboratorium:

    a. Pengaturan Bobot & Tabel Perangkingan: Formulir penentuan nilai bobot kriteria preferensi dan tabel hasil perhitungan skor akhir perankingan berdasarkan metode Simple Additive Weighting.
    b. Cetak Laporan Rekomendasi: Fitur untuk mengekspor dan mencetak dokumen PDF laporan hasil rekomendasi prioritas penggantian komputer untuk disahkan oleh pimpinan.
8. Halaman Manajemen

    Pengguna Modul otorisasi untuk mengelola akun pengguna (*username*, *password*, nama lengkap, dan peran) serta mengonfigurasi matriks hak akses dinamis dalam membatasi visibilitas menu maupun tombol aksi di dalam aplikasi.

### 4.1.6 Analisis Data

Analisis aliran data ini memetakan bagaimana informasi bergerak dan dikelola di dalam sistem secara menyeluruh di Laboratorium Komputer FTI UNIBBA. Sistem ini tidak hanya berfokus pada kalkulasi rekomendasi semata, melainkan mengawal seluruh siklus hidup perangkat keras—mulai dari registrasi awal, kodifikasi otomatis, pelabelan fisik, pencatatan histori servis, hingga evaluasi kelayakan peremajaan alat. Aliran data tersebut tersusun secara sistematis ke dalam tiga tahapan utama: data masukan (*input*), proses pengolahan (*process*), dan sajian informasi akhir (*output*).

a. Data Masukan (*Input*)

Untuk menjalankan berbagai fitur aplikasi, administrator dan Kepala Laboratorium memasukkan sejumlah data terstruktur ke dalam basis data SQLite, yang meliputi:

1. Data Master Inventaris: Informasi dasar penunjang operasional, seperti data program studi, kategori barang, lokasi ruangan laboratorium, merek, spesifikasi teknis komputer, serta tanggal pengadaan perangkat.
2. Parameter Kodifikasi: Rangkaian elemen identitas mulai dari kode yayasan, rektorat, fakultas, prodi, ruangan, hingga nomor urut barang yang nantinya dirakit menjadi kode unik aset.
3. Data Finansial (Depresiasi): Rincian nilai moneter seperti harga perolehan awal, estimasi nilai sisa (residu), dan estimasi masa pakai (umur ekonomis) untuk dasar hitungan penyusutan.
4. Data Riwayat Perawatan (*Maintenance*): Catatan log servis harian yang mencakup deskripsi kerusakan fisik, tanggal pengerjaan, nominal biaya perbaikan, serta penilaian skor kondisi perangkat (skala 1–3).
5. Bobot Penilaian: Nilai bobot kepentingan untuk setiap kriteria evaluasi yang ditentukan secara fleksibel oleh Kepala Laboratorium selaku penentu kebijakan.

b. Pengolahan Data (Process)

Setelah data tersimpan, logika komputasi pada *backend* Flask akan memprosesnya secara otomatis melalui beberapa tahapan kerja:

1. Perakitan Kode & Pembuatan *QR Code*: Sistem menggabungkan seluruh komponen kodifikasi menjadi satu kesatuan 27 digit Kode Aset unik, lalu otomatis membangkitkan gambar stiker *QR Code* (.png) menggunakan pustaka qrcode Python.
2. Pemindaian & Penerjemahan *QR Code*: Melalui pustaka ZXing (*Zebra Crossing*), sistem membaca hasil pindai kamera ponsel secara langsung dan mengonversinya kembali menjadi kode aset 27 digit untuk membuka detail profil barang secara instan.
3. Manajemen Basis Data (CRUD): *Backend* mengelola transaksi penambahan, pembaruan, penampilan, dan penghapusan data master maupun log servis secara aman dan konsisten.
4. Kalkulasi Analitik Kelayakan: Sistem menghitung depresiasi nilai aset dengan metode *Straight Line* guna memperoleh sisa umur ekonomis. Nilai

sisa umur ini kemudian disandingkan bersama skor kerusakan dan akumulasi biaya perawatan ke dalam matriks keputusan metode SAW, dinormalisasi berdasarkan sifat kriterianya (*cost* dan *benefit*), lalu dikalikan dengan bobot preferensi pengambil keputusan.

c. Informasi Keluaran (Output)

Tahap akhir dari alur data ini adalah menyajikan hasil olahan komputasi ke antarmuka pengguna secara informatif dan siap pakai:

1. Berkas Stiker *QR Code*: Gambar *QR Code* berformat .png yang siap diunduh dan dicetak untuk dijadikan label fisik pada bodi komputer laboratorium.
2. Dasbor: Tampilan status kondisi perangkat secara *real-time* yang dilengkapi tabel rekapitulasi riwayat perbaikan fisik secara terperinci.
3. Tabel Rekomendasi Prioritas: Tampilan hasil pemeringkatan preferensi akhir ($V_i$) secara transparan pada halaman web, memudahkan pengelola melihat komputer mana yang paling mendesak untuk diganti.
4. Laporan Resmi Siap Cetak (PDF): Dokumen rekapitulasi resmi berformat PDF yang merangkum data inventaris, log pengeluaran servis, dan rekomendasi peremajaan aset untuk kebutuhan arsip birokrasi institusi.

### 4.1.7 Analisis Biaya

Tabel 4. 2 Analisis Estimasi Biaya Pengembangan Sistem

| No. | Komponen Biaya | Keterangan / Spesifikasi | Estimasi Biaya (Rp) |
|---|---|---|---|
| 1 | Perangkat Keras & Operasional | | |
| | a. Laptop | Laptop pribadi pengerjaan penelitian | 6.000.000 |
| | b. Listrik dan Kuota | Listrik dan Kuota selama pengerjaan penelitian | 500.000 |
| 2 | Perangkat Lunak | | |
| | a. Lisensi Sistem Operasi | Lisensi Windows 11 bawaan laptop | 0 |
| | b. *Environment & Database* | Python, Flask, SQLite, VS Code (*Open Source*) | 0 |
| | Total Estimasi Biaya | | 6.500.000 |

Sumber: Diolah oleh Penulis (2026)

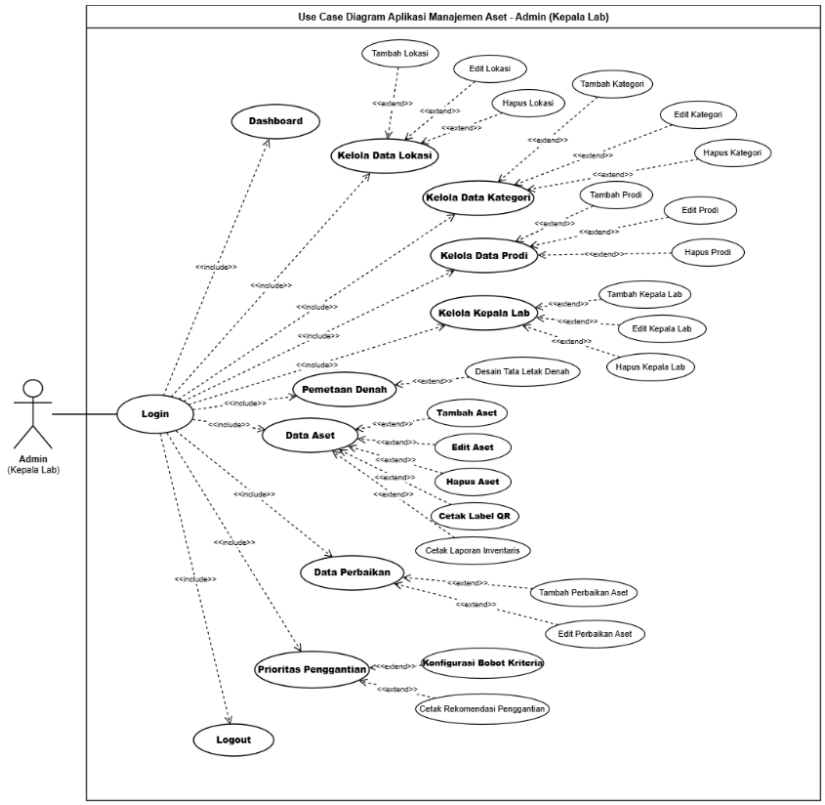
## 4.2 Perancangan

Tahap perancangan merupakan proses penerjemahan spesifikasi kebutuhan perangkat lunak ke dalam bentuk rancang bangun arsitektur sistem. Mengingat aplikasi ini dibangun menggunakan bahasa pemrograman Python berorientasi objek, maka pemodelan sistem secara visual didokumentasikan menggunakan *Unified Modeling Language* (UML).

### 4.2.1 Unified Modeling Language

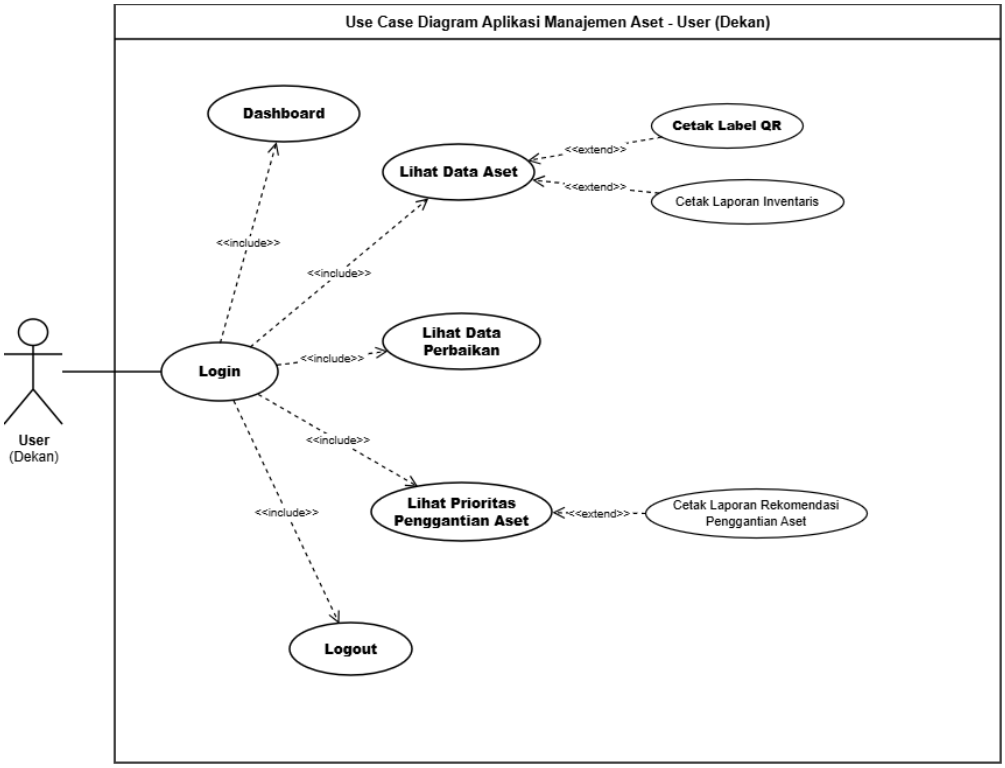
*Unified Modeling Language* (UML) menyediakan standar bahasa pemodelan visual untuk mendefinisikan, menggambarkan, dan mendokumentasikan artefak dari sistem perangkat lunak. Pemodelan UML pada Aplikasi Manajemen Aset Laboratorium Komputer ini mencakup *Use Case Diagram*, *Activity Diagram*, dan *Sequence Diagram*.

A. Use Case Diagram




Gambar 4. 2 Use Case Diagram Admin

Sumber: Diolah oleh Penulis (2026)




Gambar 4. 3 Use Case Diagram User

Sumber: Diolah oleh Penulis (2026)

*Use Case Diagram* memetakan interaksi antara aktor di luar sistem dengan fungsi-fungsi utama yang disediakan di dalam batasan sistem (*system boundary*). Dalam perancangan ini, seluruh fungsionalitas sistem disatukan ke dalam satu system boundary tunggal guna memperlihatkan pemisahan hak akses antara dua aktor utama, yaitu Admin yakni kepala laboratorium dan User yang kita misalkan di sini adalah seorang dekan fakultas.

Berdasarkan gambar Use Case Diagram tersebut, sistem membatasi interaksi pengguna melalui dua peran utama dengan rincian kewenangan sebagai berikut:

1. Admin (Staf / Kepala Laboratorium): Bertindak sebagai pengelola operasional utama dengan hak akses penuh (*full access*). Admin berwenang mengelola data master, menginput dan memperbarui katalog aset, mencatat riwayat pemeliharaan, mengonfigurasi bobot kriteria SAW, mengelola data pengguna, mengatur tata letak denah 2D, serta mencetak laporan dan label *QR Code*.

2. User (Dekan): Bertindak sebagai pemantau (*monitoring access*). Dekan memiliki hak akses terbatas untuk meninjau informasi dasbor, melihat riwayat perbaikan, meninjau hasil perangkingan prioritas penggantian (SAW), serta mencetak laporan rekapitulasi tanpa memiliki hak untuk mengubah atau merusak data master.

Untuk memperjelas batasan fungsionalitas, kondisi prasyarat (*pre-condition*), serta hasil akhir (*post-condition*) dari setiap interaksi, rincian *use case* dirangkum dalam tabel berikut

Tabel 4. 3 Deskripsi Skenario Use Case

| **ID Use Case** | **Nama Use Case** | **Aktor** | **Kondisi Awal (*Pre-Condition*)** | **Kondisi Akhir (*Post-Condition*) / Output** |
|---|---|---|---|---|
| **UC-01** | Melakukan Autentikasi (*Login*) | Admin, Dekan | Pengguna membuka halaman login aplikasi | Pengguna terautentikasi dan masuk ke dasbor sesuai hak aksesnya |
| **UC-02** | Mengelola Data Master | Admin | Berhasil *login* sebagai Admin | Data lokasi, kategori aset, prodi, dan kepala lab tersimpan atau diperbarui |
| **UC-03** | Mengelola Data Aset Inventaris | Admin | Berhasil *login* sebagai Admin | Data katalog, spesifikasi teknis, dan nilai depresiasi aset terdaftar, terubah, atau terhapus |
| **UC-04** | Memantau Denah 2D Laboratorium | Admin | Berhasil *login* | Tampilan peta denah 2D beserta indikator kondisi meja dan spesifikasi aset ditayangkan |
| **UC-05** | Mengelola Riwayat Perbaikan | Admin, Dekan | Berhasil *login* | Catatan servis dan akumulasi biaya pemeliharaan terekam (Admin) atau ditinjau (User) |
| **UC-06** | Memproses Prioritas Penggantian (SAW) | Admin, Dekan | Berhasil *login* | Bobot kriteria terkonfigurasi (Admin) dan hasil perangkingan preferensi SAW ditayangkan |

| ID Use Case | Nama Use Case | Aktor | Kondisi Awal (*Pre-Condition*) | Kondisi Akhir (*Post-Condition*) / Output |
|---|---|---|---|---|
| **UC-07** | Mengelola Data Pengguna | Admin | Berhasil *login* sebagai Admin | Akun pengguna baru terdaftar, terubah, atau matriks hak akses terkonfigurasi |
| **UC-08** | Mencetak Laporan dan Label QR | Admin, Dekan | Berhasil *login* | Dokumen PDF laporan rekapitulasi atau stiker label *QR Code* berhasil diekspor/dicetak |
| **UC-09** | Melakukan Keluar Sistem (*Logout*) | Admin, Dekan | Pengguna berada dalam sesi aktif aplikasi | Sesi otentikasi dihapus dan antarmuka kembali ke halaman *login* |

Sumber: Diolah oleh Penulis (2026)

Setiap *use case* pada Tabel 4.4 mewakili satu tujuan fungsional yang berdiri sendiri secara atomik. Otentikasi *login* (UC-01) bertindak sebagai gerbang prasyarat akses (*pre-condition*) bagi seluruh *use case* manajerial, sehingga pemodelan tidak lagi menghubungkan garis <<include>> secara berulang dari setiap *use case* ke *login* guna menghindari kerumitan visual pada diagram.

Selanjutnya, alur kerja operasional (*workflow*) serta interaksi antar-objek secara kronologis dari masing-masing *use case* ini diuraikan secara dinamis pada bagian *Activity Diagram* dan *Sequence Diagram*.

B. Activity Diagram

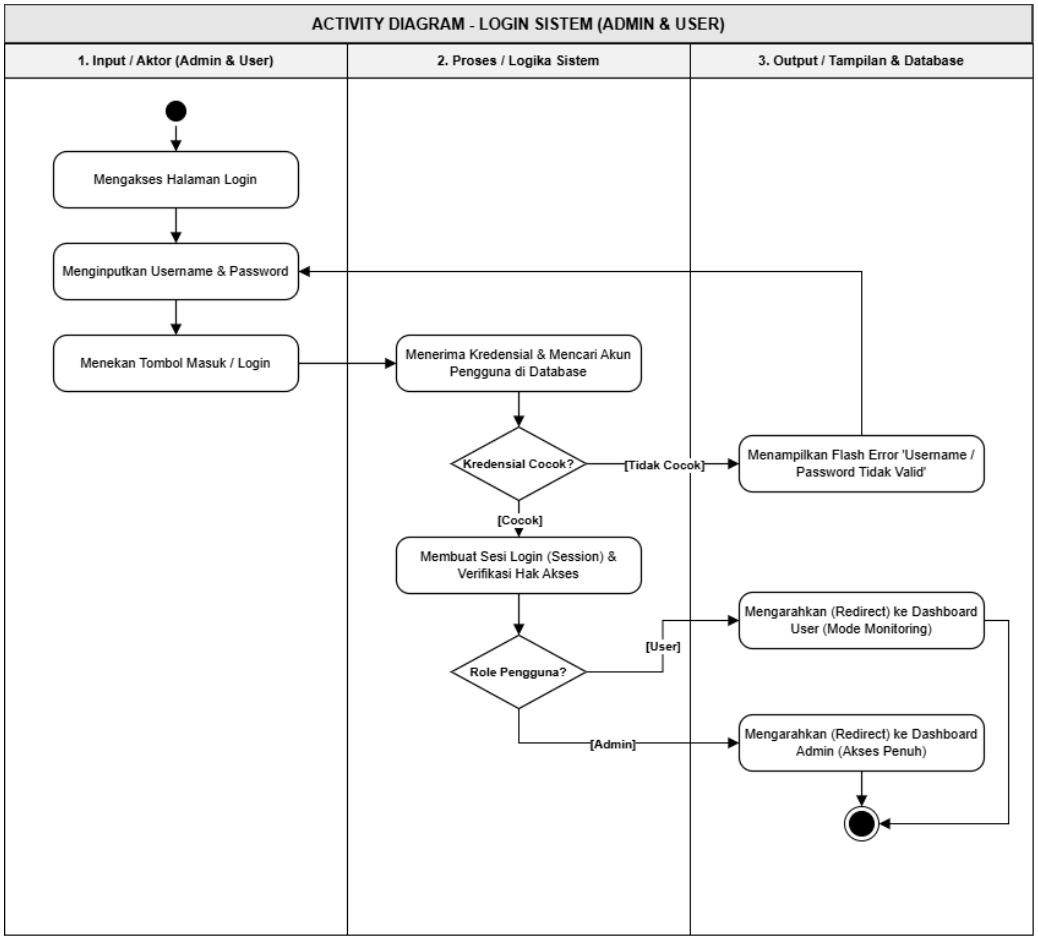
*Activity Diagram* menggambarkan alur kerja (*workflow*) fungsional sistem secara dinamis, mulai dari titik awal (*initial node*), urutan aktivitas internal, kondisi percabangan (*decision node*), hingga titik akhir (*activity final node*). Pemodelan *Activity Diagram* pada aplikasi ini dibagi ke dalam 3 *swimlane* utama untuk memperjelas pemisahan tanggung jawab (*separation of concerns*), yaitu:

1. Pengguna (Aktor): Menggambarkan aksi fisik atau masukan dari Admin maupun Dekan pada antarmuka.
2. Sistem (Aplikasi Flask): Menggambarkan logika pemrosesan, validasi input, serta eksekusi fungsi pada *backend*.

3. Penyimpanan (Database / Antarmuka): Menggambarkan reaksi operasi data (*query*) pada basis data PostgreSQL/MySQL serta umpan balik visual (*rendering*) pada layar antarmuka.

Rincian pemodelan *Activity Diagram* untuk seluruh modul fungsional utama diuraikan sebagai berikut:

1. Activity Diagram Login Sistem



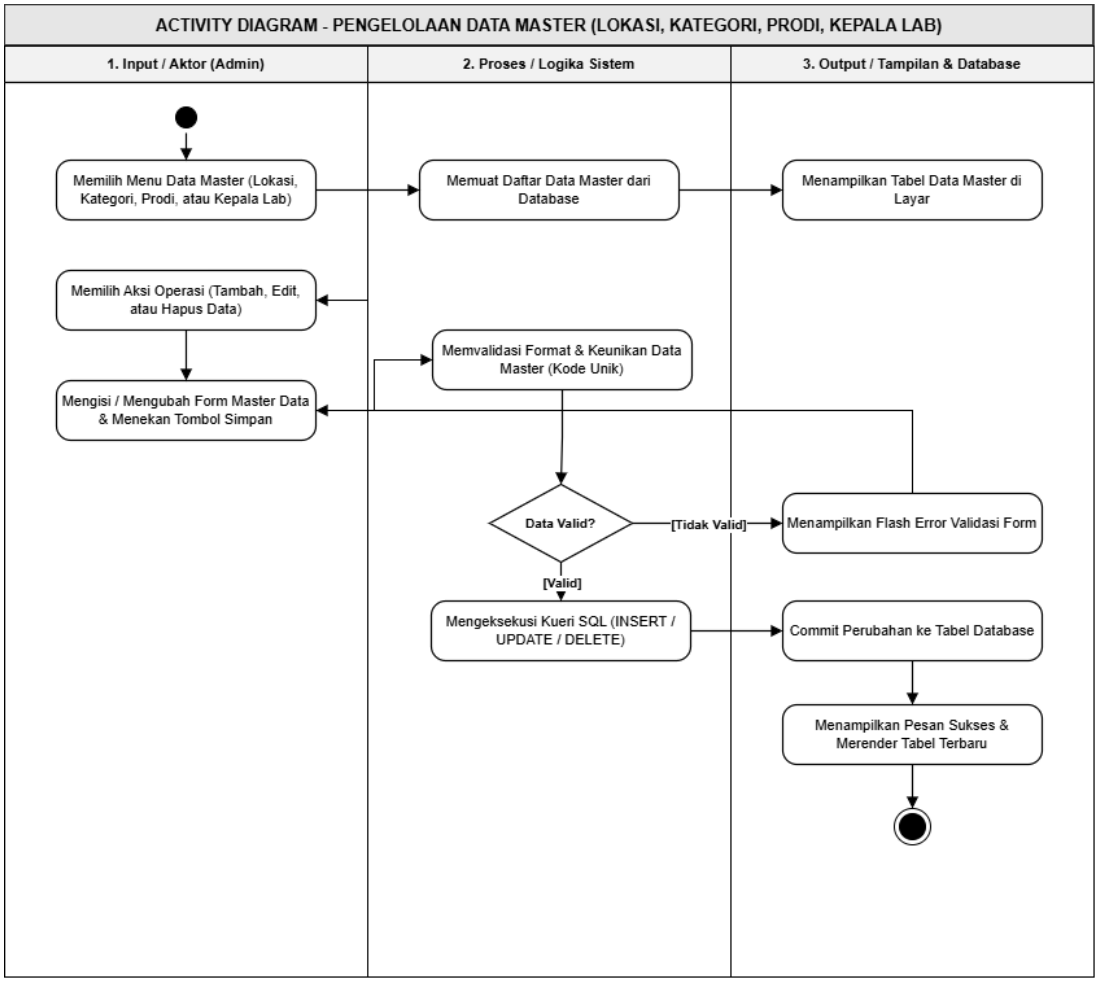

Gambar 4. 4 Activity Diagram Login

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini memetakan alur otentikasi pengguna saat pertama kali mengakses aplikasi. Alur kerja diawali ketika pengguna memasukkan nama pengguna (*username*) dan kata sandi (*password*). Sistem memvalidasi kredensial tersebut ke basis data tb_user. Jika data cocok, sistem membangkitkan sesi (*session*) dan mengarahkan antarmuka ke dasbor utama sesuai peran (*role*) aktor. Sebaliknya,

jika kredensial tidak valid, sistem menampilkan notifikasi kesalahan (*flash message*) dan mengembalikan pengguna ke formulir *login*.

2. Activity Diagram Pengelolaan Data Master

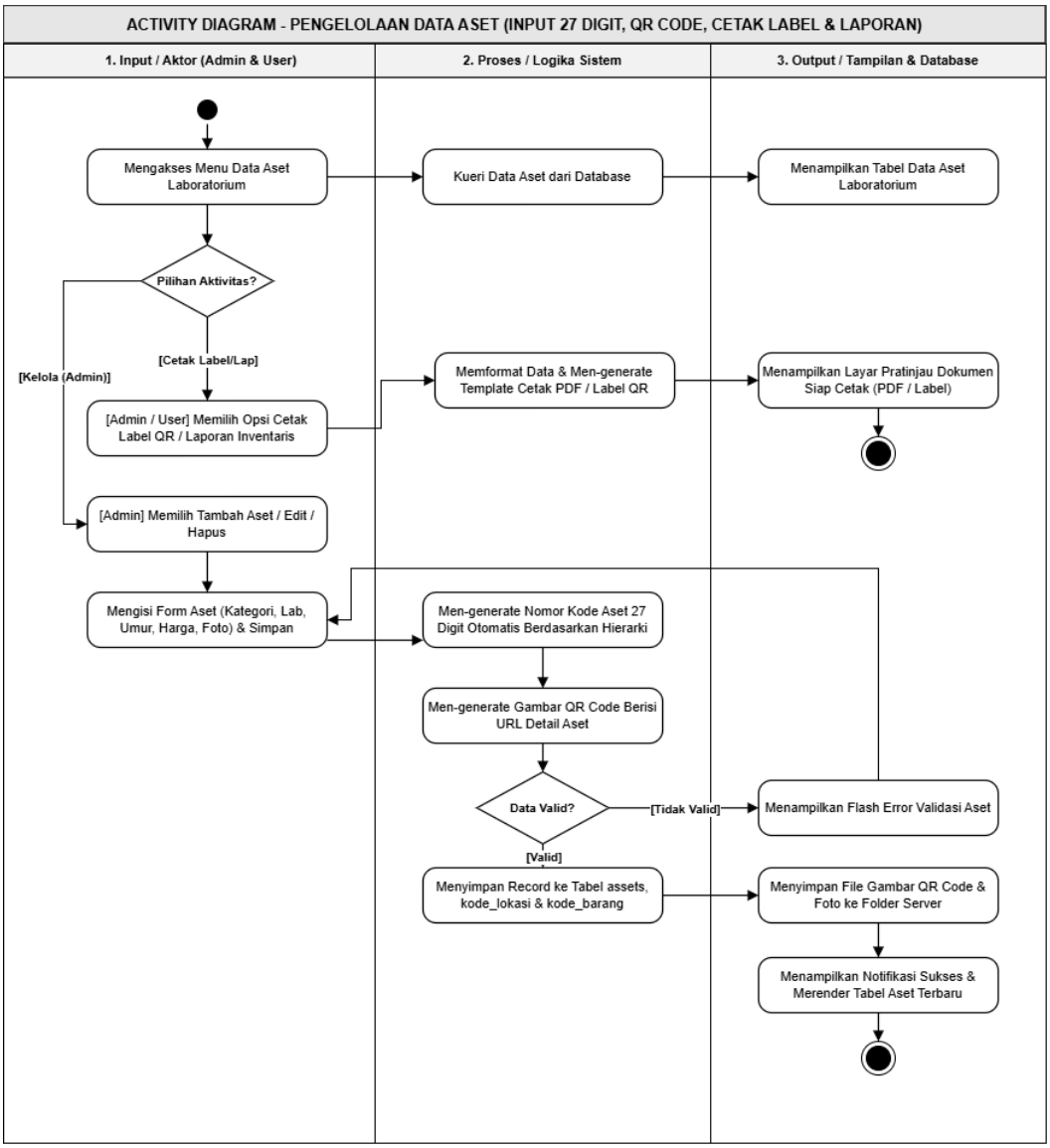



Gambar 4. 5 Activity Diagram Pengelolaan Data Master

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini menggambarkan alur kerja Admin dalam mengelola entitas data master (lokasi ruangan, kategori aset, program studi, dan data kepala laboratorium). Admin memilih menu master, kemudian sistem menampilkan daftar data terdaftar. Admin dapat memilih tindakan tambah, ubah, atau hapus data. Sistem memvalidasi keunikan serta kelengkapan isian formulir, lalu mengeksekusi perintah SQL (INSERT, UPDATE, atau DELETE) pada tabel basis data terkait. Setelah operasi *database* berhasil, antarmuka diperbarui dan menampilkan notifikasi status transaksi.

3. Activity Diagram Pengelolaan Data Aset



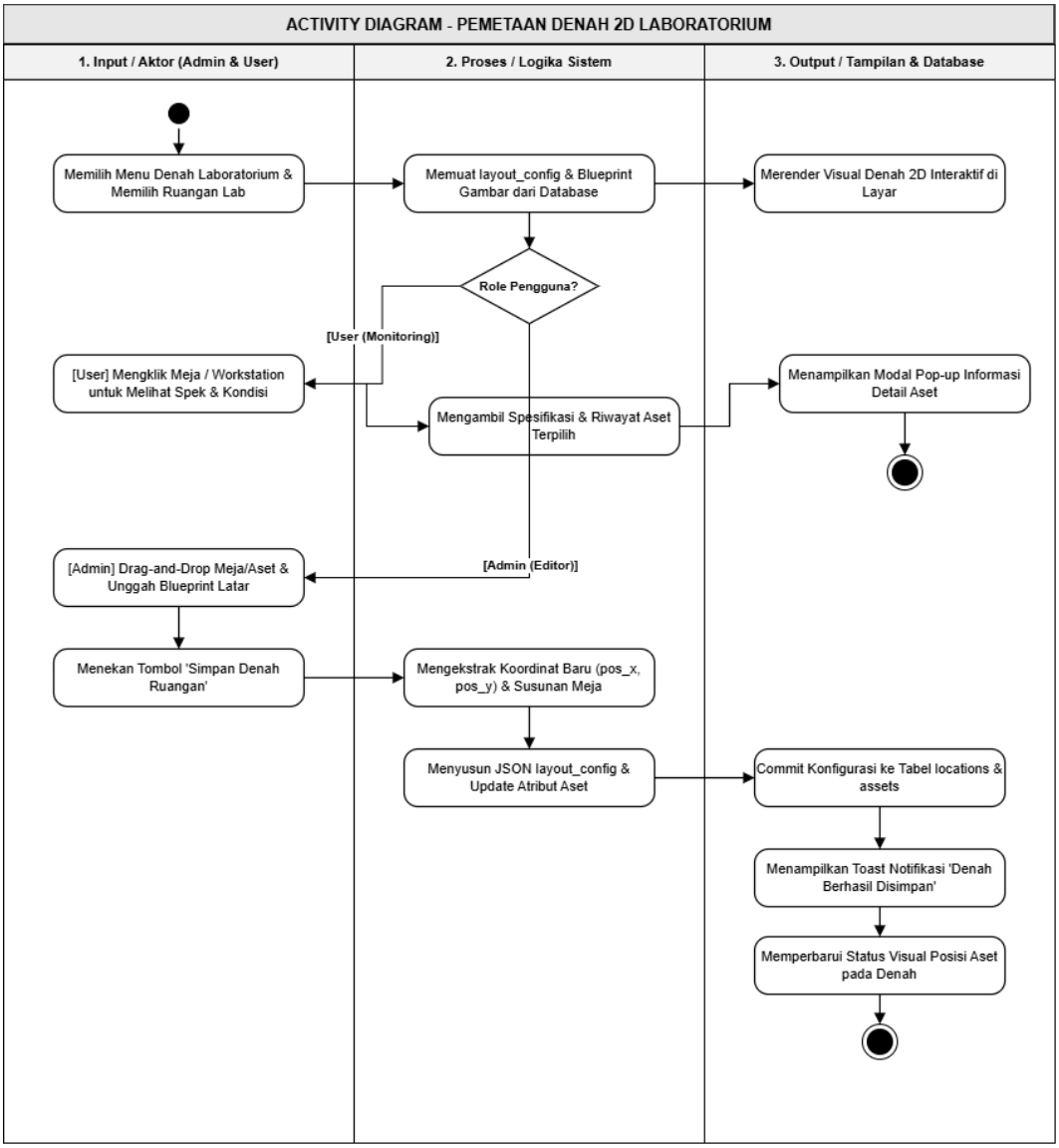

Gambar 4. 6 Activity Diagram Pengelolaan Data Aset

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini memetakan alur pendataan dan pemeliharaan katalog aset oleh Admin. Alur dimulai ketika Admin menginput spesifikasi teknis, harga perolehan, tanggal beli, dan estimasi umur ekonomis. Sistem secara otomatis merakit kodifikasi 27-digit unik dan merender berkas *QR Code* berbasis perpustakaan Python (qrcode). Selain itu, diagram ini mencakup alur pencetakan label *QR Code* dan lembar rekapitulasi inventaris ke format PDF. Operasi simpan

atau perbaikan data diteruskan ke tabel tb_asset, dan gambar *QR Code* disimpan ke direktori server.

4. Activity Diagram Pemetaan Denah 2D Laboratorium



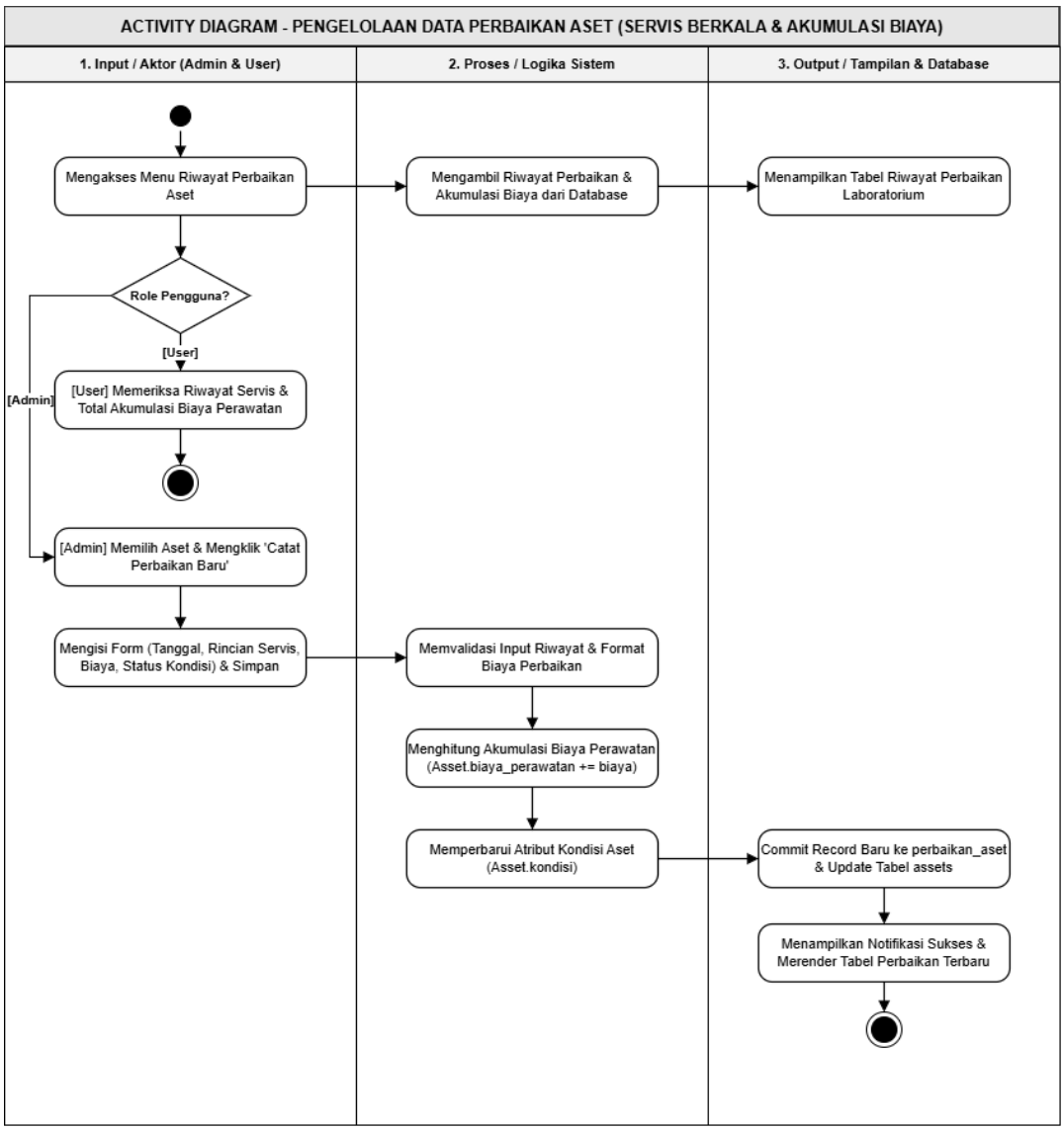

Gambar 4. 7 Activity Diagram Pemetaan Denah

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini memetakan alur pemantauan dan pengorganisasian tata letak fisik perangkat komputer secara visual. Pengguna (Admin atau User) membuka modul denah 2D, lalu sistem memuat tata letak posisi koordinat ($pos_x$, $pos_y$) dari tabel tb_location. Dekan dapat mengklik titik unit meja untuk menayangkan jendela *pop-up* spesifikasi teknis dan kondisi aset. Sementara itu,

Admin memiliki wewenang untuk mengubah tata letak unit menggunakan aksi *drag-and-drop*. Saat posisi meja digeser, sistem memperbarui nilai koordinat baru ke basis data secara otomatis.

5. Activity Diagram Pengelolaan Data Perbaikan Aset



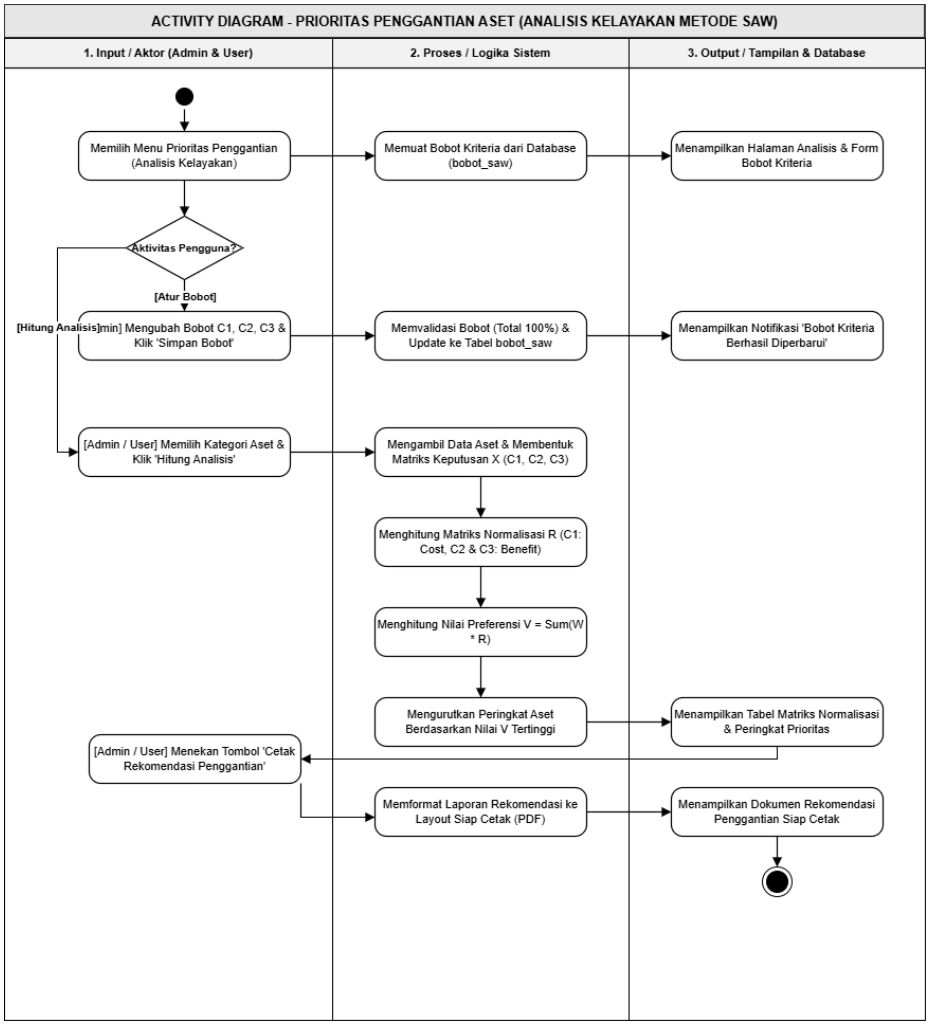

Gambar 4. 8 Activity Diagram Pengelolaan Data Perbaikan Aset

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini mencakup pencatatan tindakan pemeliharaan harian aset oleh Admin serta peninjauan log servis oleh User. Admin menginput tanggal perbaikan, rincian teknis kerusakan, serta nominal biaya servis. Sistem melakukan kalkulasi penjumlahan akumulasi biaya pemeliharaan harian (C3) pada aset terkait,

sekaligus pembaruan status kondisi fisik barang pada tabel tb_asset. Seluruh rincian transaksi disimpan pada tabel tb_perbaikan, lalu ditayangkan pada antarmuka tabel log perbaikan.

6. Activity Diagram Prioritas Penggantian Aset



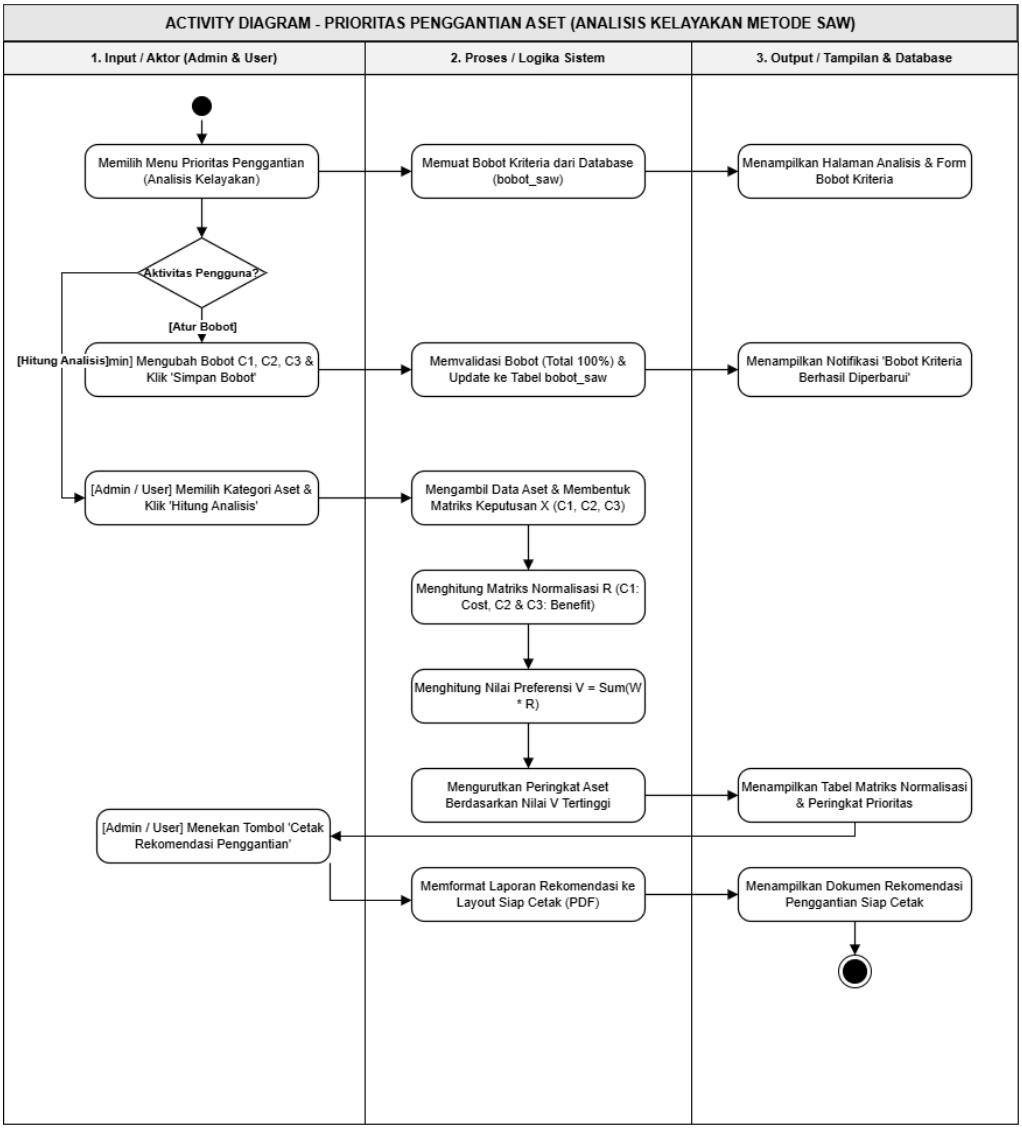

Gambar 4. 9 Activity Diagram Prioritas Peremajaan Aset

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini menggambarkan proses komputasi analitis dalam menentukan urutan prioritas peremajaan komputer. Admin dapat mengonfigurasi bobot kriteria desimal melalui antarmuka. Saat proses komputasi dipicu, sistem mengambil parameter data nilai sisa umur ekonomis (C1), tingkat kerusakan fisik (C2), dan akumulasi biaya pemeliharaan (C3). Sistem membentuk matriks keputusan X, mengeksekusi kalkulasi normalisasi matriks R menggunakan kriteria

*cost* dan *benefit*, serta menghitung nilai preferensi Vi dengan metode pembobotan terbobot (SAW). Hasil perangkingan kemudian ditayangkan pada tabel matriks dan dapat diekspor ke dokumen PDF rekomendasi.

7. Activity Diagram Pengelolaan Data Pengguna



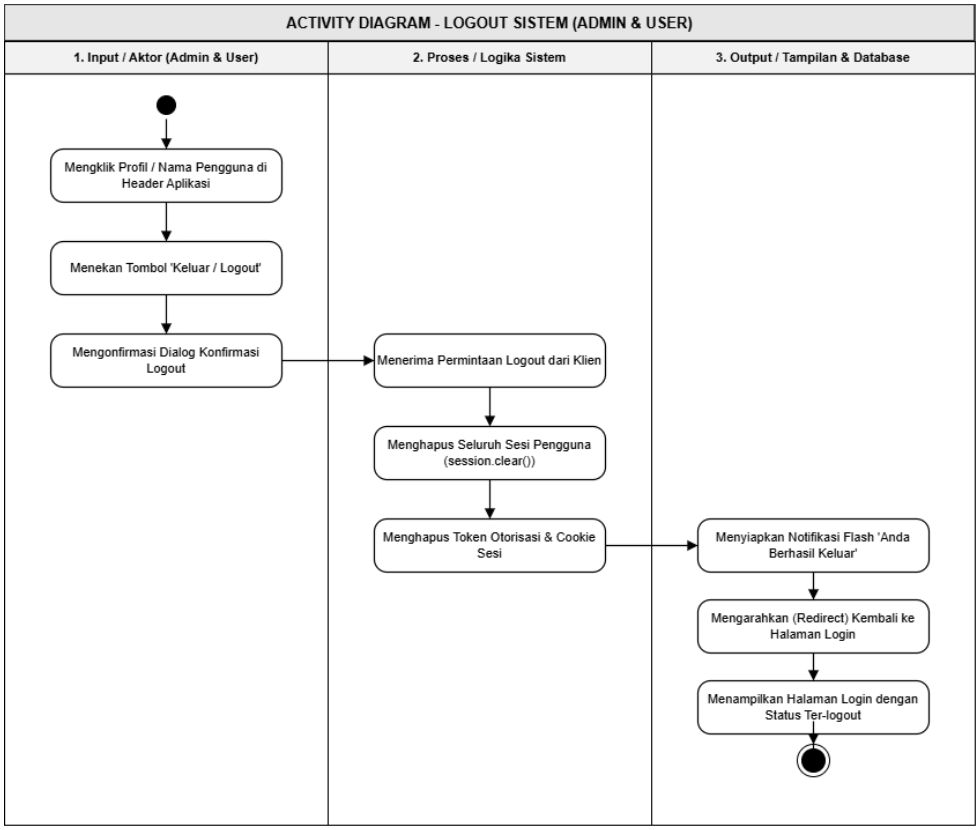

Gambar 4. 10 Activity Diagram Pengelolaan Data Pengguna

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini memetakan alur manajemen akun dan konfigurasi hak akses pengguna oleh Admin. Admin memasukkan data kredensial baru, menentukan peran (*Admin* atau *Dekan*), serta mengatur matriks izin akses berbasis *checkbox*. Pada bagian *backend*, sistem melakukan enkripsi kata sandi (*hashing*),

mengonsolidasi hak akses ke dalam format JSON, lalu mengeksekusi query simpan pada tabel tb_user. Sistem mengembalikan umpan balik visual bahwa pengaturan pengguna telah diperbarui.

8. Activity Diagram Logout Sistem




Gambar 4. 11 Activity Diagram Logout

Sumber: Diolah oleh Penulis (2026)

*Activity Diagram* ini memetakan alur pengakhiran sesi akses pengguna. Pengguna mengklik tombol *Logout* pada navigasi atas, kemudian sistem memproses pembersihan variabel sesi aktif (session.clear()) serta menghapus *token* otentikasi. Setelah sesi terhapus, sistem menampilkan notifikasi bahwa pengguna telah berhasil keluar dan melakukan pengalihan (*redirect*) antarmuka kembali ke halaman *login*.

C. Sequence Diagram

*Sequence Diagram* (Diagram Urutan) menggambarkan interaksi antar-objek dan komponen sistem berdasarkan urutan waktu (*chronological order*) dalam

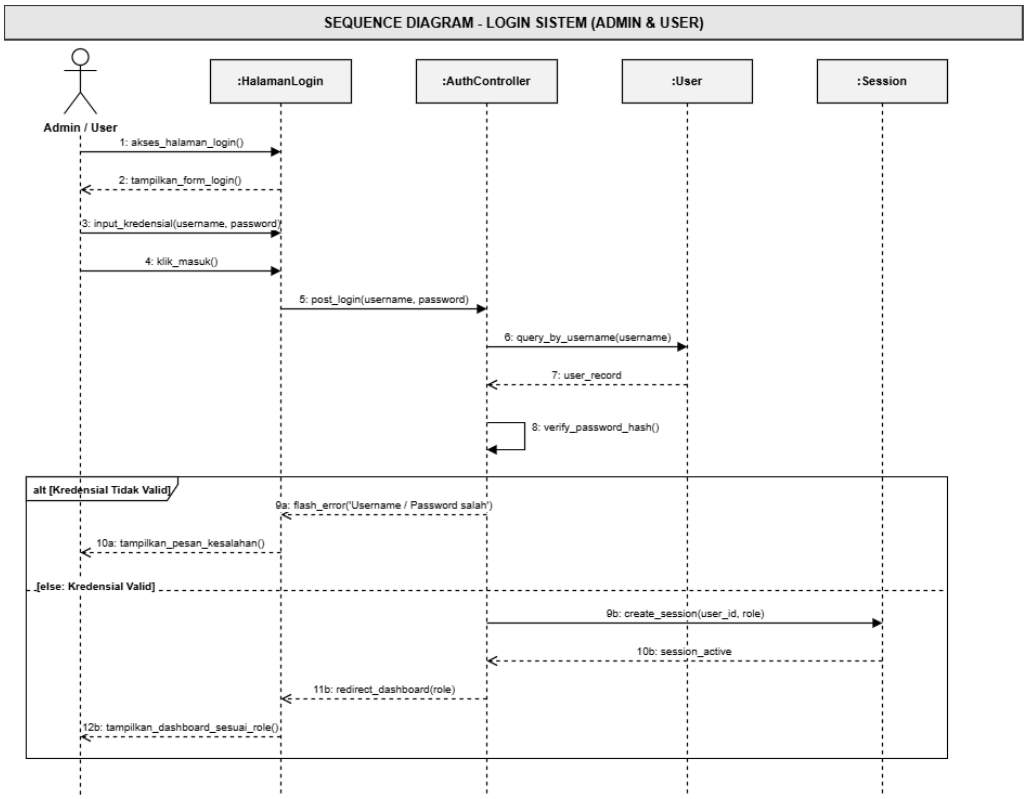
mengeksekusi skenario fungsionalitas aplikasi. Pemodelan ini diturunkan langsung dari alur kerja pada *Activity Diagram* untuk memperlihatkan bagaimana pesan (*message*) dikirimkan antar-objek pada arsitektur berbasis *web*.

Arsitektur objek pada setiap *sequence diagram* dibangun menggunakan tiga komponen utama berdasarkan pola desain berorientasi objek (*Model-View-Controller* / MVC):

1. Boundary (Antarmuka / View): Komponen halaman web atau formulir HTML tempat aktor berinteraksi dan mengirimkan aksi.
2. Control (Pengontrol / Controller): Komponen skrip Flask (.py) yang menangani logika bisnis, validasi input, serta pemrosesan algoritma.
3. Entity / Service (Model / Database): Entitas basis data atau layanan sistem yang mengelola kueri penyimpanan data dan manipulasi rekaman.

Rincian pemodelan *Sequence Diagram* untuk seluruh modul fungsional sistem diuraikan sebagai berikut:

1. Sequence Diagram Login Sistem



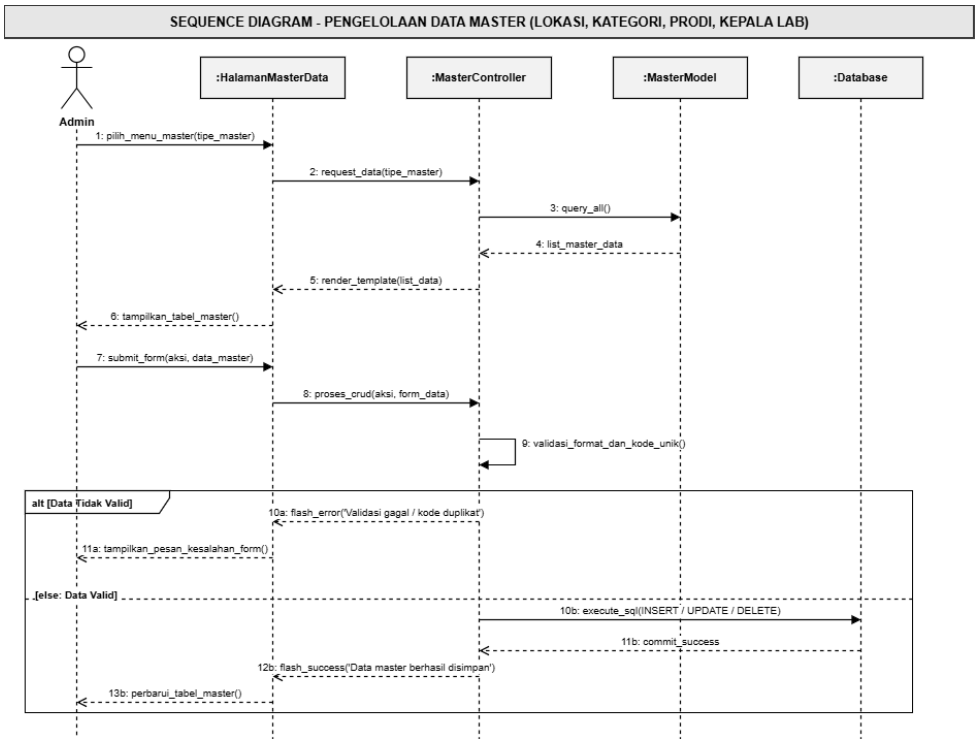

Gambar 4. 12 Sequence Diagram Login Sistem

Sumber: Diolah oleh Penulis (2026)

Interaksi kronologis diawali saat pengguna (Admin atau Dekan) mengakses :HalamanLogin dan menekan tombol masuk, yang memicu perintah post_login(username, password) ke pengontrol :AuthController. Pengontrol mengeksekusi kueri query_by_username() ke entitas :User dan memverifikasi kata sandi melalui fungsi internal verify_password_hash().

Pada fragmen percabangan alt [Kredensial Tidak Valid], pengontrol mengembalikan pesan flash_error() untuk menayangkan notifikasi kesalahan pada antarmuka :HalamanLogin. Sebaliknya, pada kondisi [else: Kredensial Valid], pengontrol memanggil perintah create_session() ke objek :Session dan mengeksekusi redirect_dashboard(role) untuk mengarahkan pengguna ke antarmuka dasbor sesuai hak aksesnya.

2. Sequence Diagram Pengelolaan Data Master



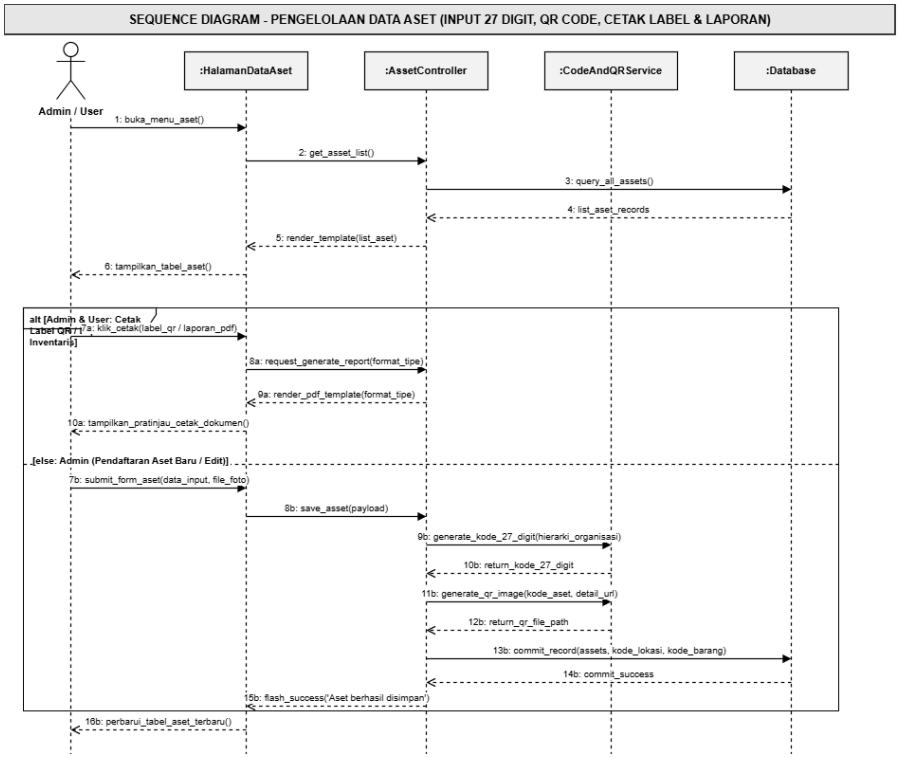

Gambar 4. 13 Sequence Diagram Pengelolaan Data Master

Sumber: Diolah oleh Penulis (2026)

Alur interaksi diawali saat Admin memilih kategori master data pada :HalamanMasterData, yang meneruskan pesan request_data() ke pengontrol :MasterController. Pengontrol memanggil fungsi query_all() ke entitas :MasterModel dan menayangkan daftar rekaman ke antarmuka.

Saat Admin mengirimkan formulir penambahan atau pengubahan data via submit_form(), :MasterController memproses kueri dan melakukan pengujian internal validasi_format_dan_kode_unik()[3]. Pada kondisi alt [Data Tidak Valid], pengontrol mengembalikan pesan flash_error(). Pada kondisi [else: Data Valid], pengontrol mengeksekusi perintah SQL (INSERT, UPDATE, atau DELETE) langsung ke objek :Database, menerima konfirmasi commit_success, serta merefresh tampilan tabel data master terbaru pada antarmuka.

3. Sequence Diagram Pengelolaan Data Aset




Gambar 4. 14 Sequence Diagram Pengelolaan Data Aset

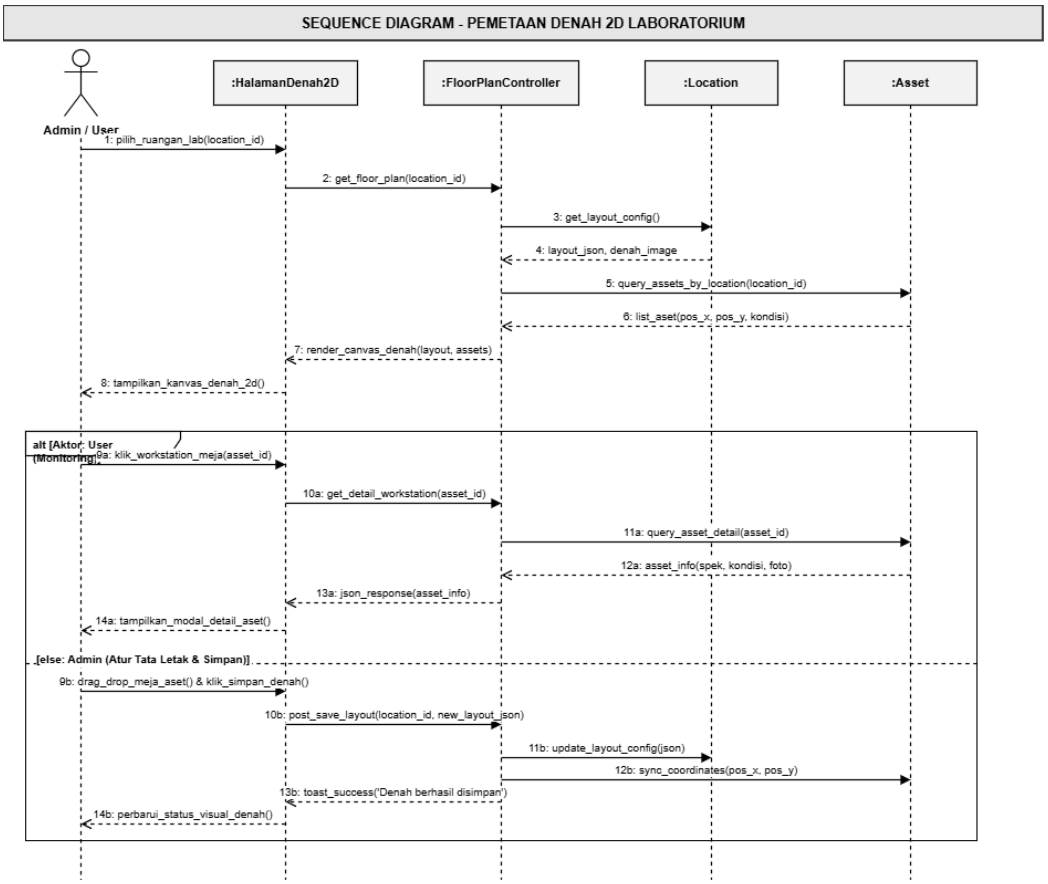
Sumber: Diolah oleh Penulis (2026)

Interaksi diawali dengan pembukaan menu katalog pada :HalamanDataAset yang meminta daftar rekaman dari pengontrol :AssetController melalui kueri query_all_assets() ke entitas :Database.

Pada percabangan alt [Admin & Dekan: Cetak Label QR / Laporan PDF], pemanggilan aksi klik_cetak() diteruskan ke :AssetController untuk memuat templat PDF via render_pdf_template() dan menampilkan layar pratinjau dokumen. Pada percabangan [else: Admin (Input / Edit Aset)], formulir dikirim via

save_asset() ke :AssetController. Pengontrol kemudian memanggil layanan :CodeAndQRService melalui fungsi generate_kode_27_digit() dan generate_qr_image(), sebelum melakukan simpan rekaman aset beserta gambar *QR Code* ke :Database.

4. Sequence Diagram Pemetaan Denah 2D Laboratorium




Gambar 4. 15 Sequence Diagram Pemetaan Denah Laboratorium

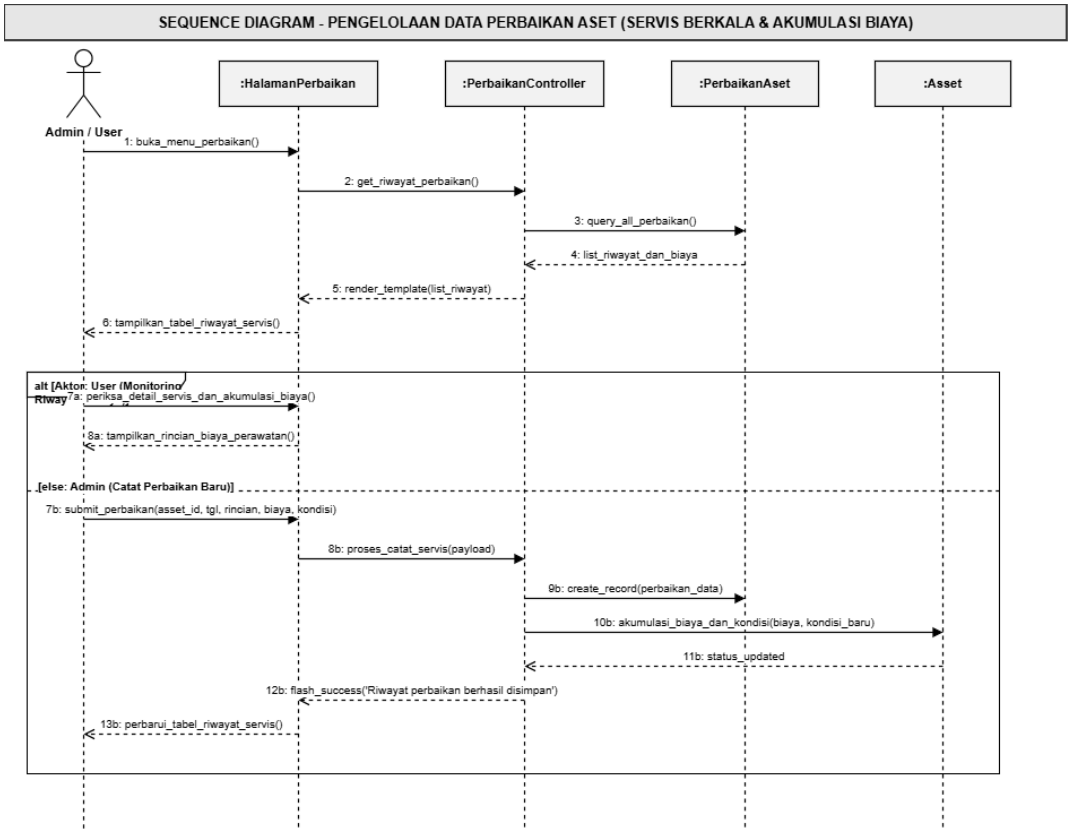
Sumber: Diolah oleh Penulis (2026)

Interaksi diawali saat pengguna memilih lokasi ruangan pada :HalamanDenah2D, yang memicu pengontrol :FloorPlanController untuk mengambil konfigurasi denah dari entitas :Location dan koordinat posisi unit dari entitas :Asset melalui kueri query_assets_by_location().

Pada percabangan alt [Aktor: Dekan (Monitoring)], pengklikan meja memanggil get_detail_workstation() untuk dikuerikan via query_asset_detail() ke entitas :Asset guna menampilkan jendela *pop-up* spesifikasi. Pada percabangan [else: Admin (Atur Layout)], aksi *drag-and-drop* posisi meja diteruskan via

post_save_layout() untuk memperbarui struktur JSON pada entitas :Location serta menyinkronkan koordinat ($pos_x$, $pos_y$) pada entitas :Asset.

5. Sequence Diagram Pengelolaan Data Perbaikan Aset




Gambar 4. 16 Sequence Diagram Pengelolaan Data Perbaikan Aset

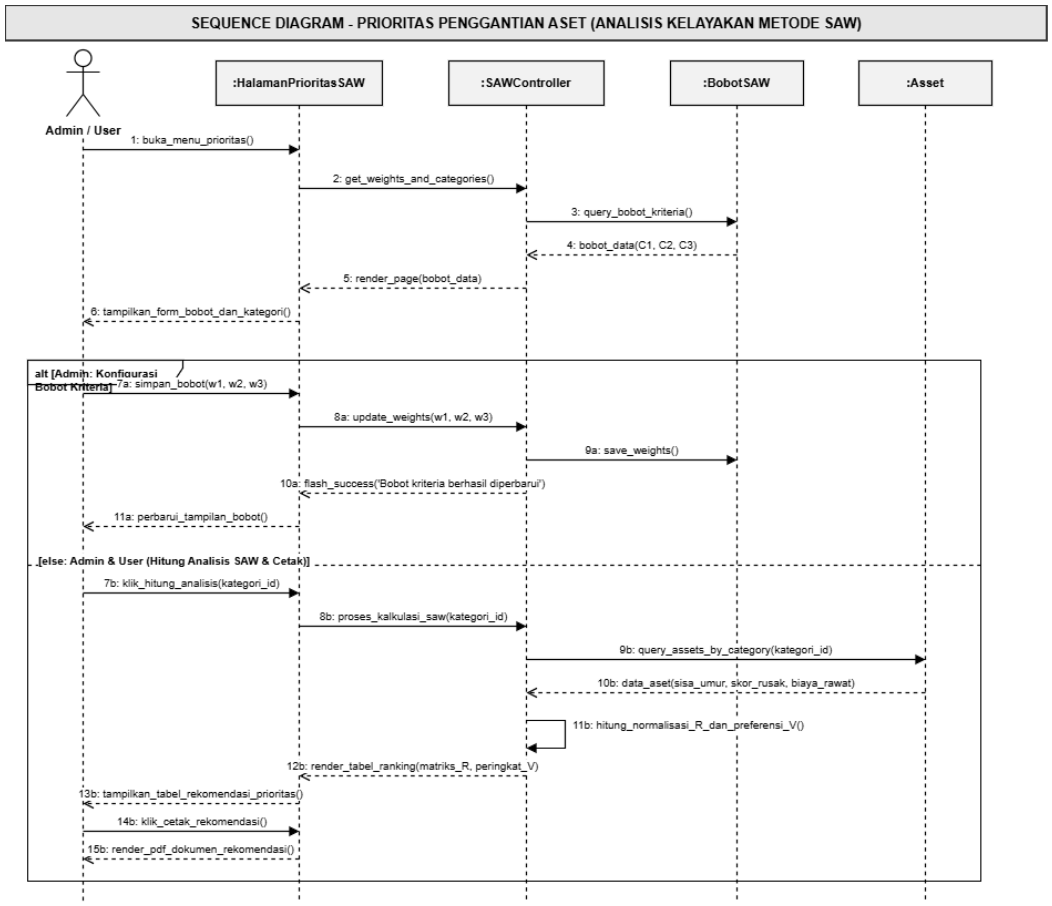
Sumber: Diolah oleh Penulis (2026)

Alur interaksi diawali saat pengguna membuka antarmuka :HalamanPerbaikan yang meminta log transaksi melalui get_riwayat_perbaikan() ke pengontrol :PerbaikanController untuk dikuerikan via query_all_perbaikan() ke entitas :PerbaikanAset.

Pada kondisi alt [Aktor: Dekan (Monitoring)], pengguna hanya mengeksekusi periksa_detail_servis() untuk menayangkan rincian log tanpa mengubah data. Pada kondisi [else: Admin (Catat Perbaikan Baru)], formulir perbaikan dikirim ke :PerbaikanController untuk mencatat transaksi baru via create_record() pada entitas :PerbaikanAset, sekaligus memperbarui akumulasi

biaya perawatan (C3) dan status kondisi fisik pada entitas :Asset melalui pesan akumulasi_biaya_dan_kondisi().

6. Sequence Diagram Prioritas Penggantian Aset




Gambar 4. 17 Sequence Diagram Prioritas Penggantian Aset

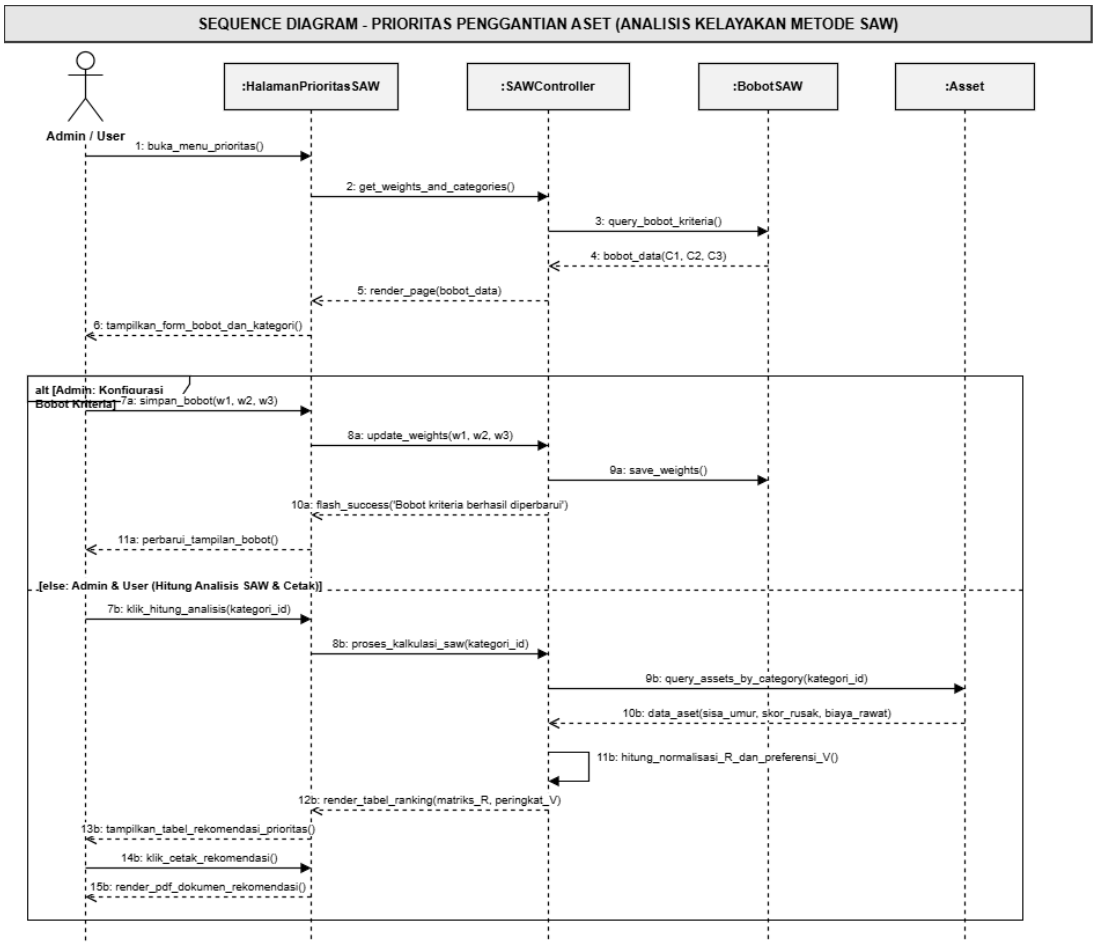
Sumber: Diolah oleh Penulis (2026)

Alur interaksi diawali saat pengguna membuka antarmuka :HalamanPrioritas yang meminta data bobot kriteria dari entitas :BobotKriteria melalui pengontrol :AnalyticalController.

Pada percabangan alt [Admin: Konfigurasi Bobot], Admin menginputkan nilai bobot desimal baru (w1, w2, w3) untuk disimpan ke entitas :BobotKriteria via save_weights(). Pada percabangan [else: Admin & User (Kalkulasi & Cetak Recomendasi)], pemanggilan klik_hitung_analisis() memicu :AnalyticalController untuk mengambil variabel aset dari entitas :Asset, mengeksekusi kalkulasi internal hitung_normalisasi_R_dan_preferensi_V(), menayangkan tabel pemeringkatan

pada antarmuka, serta merender dokumen PDF laporan rekomendasi saat tombol cetak ditekan.

7. Sequence Diagram Pengelolaan Data Pengguna




Gambar 4. 18 Sequence Diagram Pengelolaan Data Pengguna

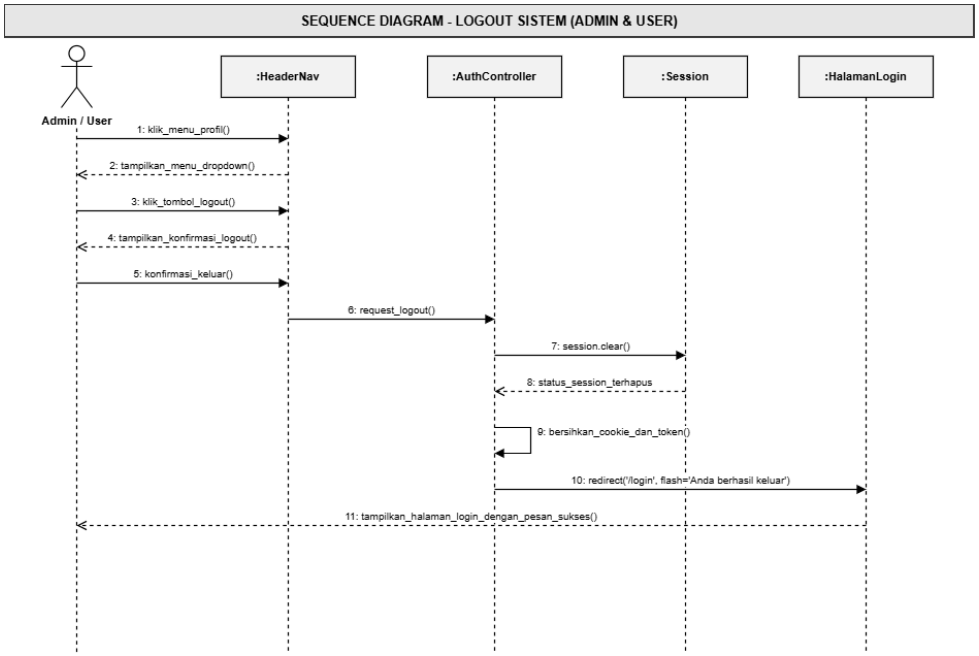
Sumber: Diolah oleh Penulis (2026)

Interaksi diawali saat Admin membuka :HalamanManagementUser yang memicu :UserController untuk mengambil daftar akun via query_all_users() dari entitas :UserModel.

Saat Admin menambah atau memperbarui akun pengguna beserta matriks hak aksesnya, masukan dikirimkan melalui submit_user_form() ke :UserController. Pengontrol melakukan enkripsi kata sandi via hash_password() dan merakit serialisasi matriks perizinan ke format JSON via serialize_permissions(). Selanjutnya, perintah simpan dikirimkan ke entitas :UserModel untuk memicu

kueri INSERT atau UPDATE pada tabel :Database, diikuti pengembalian konfirmasi keberhasilan ke antarmuka.

8. Sequence Diagram Logout Sistem




Gambar 4. 19 Sequence Diagram Logout Sistem

Sumber: Diolah oleh Penulis (2026)

Interaksi kronologis diawali dari antarmuka :HeaderNav saat pengguna menekan menu profil dan mengonfirmasi tindakan keluar melalui pesan konfirmasi_keluar().

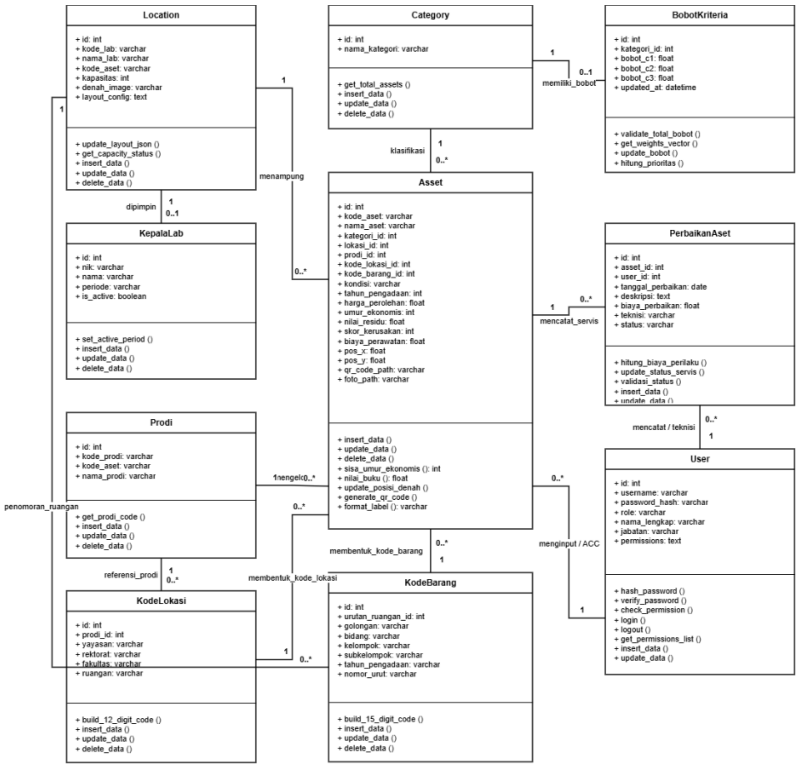
Permintaan diteruskan via request_logout() ke pengontrol :AuthController, yang kemudian memanggil fungsi session.clear() ke komponen :Session untuk menghapus data sesi pengguna di *server*. Setelah menerima konfirmasi status_session_terhapus, :AuthController menjalankan proses internal bersihkan_cookie_dan_token() dan mengarahkan antarmuka kembali ke :HalamanLogin beserta penayangan notifikasi *flash* keberhasilan keluar sistem.

D. Class Diagram

*Class Diagram* menggambarkan struktur statis sistem dengan memetakan kelas-kelas pembentuk perangkat lunak, struktur atribut beserta tipe datanya, metode operasional (*methods/operations*), serta hubungan relasional antar-objek.

Dalam perancangan sistem berbasis berorientasi objek (*Object-Oriented Software Engineering*), *Class Diagram* ini bertindak sebagai rancangan arsitektur data utama yang secara utuh mewakili struktur dan keterhubungan basis data relasional.

Struktur statis aplikasi ini dibangun oleh 10 kelas utama yang saling terinterkoneksi melalui hubungan asosiasi dan kardinalitas relasi, dengan entitas Asset bertindak sebagai kelas pusat (*central entity*).




Gambar 4. 20 Class Diagram

Sumber: Diolah oleh Penulis (2026)

Rincian spesifikasi kelas, entitas relasional, serta fungsi operasional pembentuk *Class Diagram* dirangkum pada table berikut.

Tabel 4. 4 Ringkasan Spesifikasi dan Relasi Class Diagram

| **Nama Kelas** | **Atribut Utama (*Primary Key & Foreign Key*)** | **Metode Operasional (*Methods*)** | **Relasi & Kardinalitas** |
|---|---|---|---|
| **User** | id (PK), username, password_hash, role, permissions | hash_password(), verify_password(), check_permission() | Berasosiasi ke PerbaikanAset (1 ke 0..*) |
| **Location** | id (PK), nama_lab, kode_lab, layout_config | update_layout_json(), get_capacity_status() | Berasosiasi ke Asset (1 ke 0..*) |

| Nama Kelas | Atribut Utama (*Primary Key & Foreign Key*) | Metode Operasional (*Methods*) | Relasi & Kardinalitas |
|---|---|---|---|
| | | | dan KodeBarang (1 ke 0..*) |
| **Category** | id (PK), nama_kategori | get_total_assets() | Berasosiasi ke Asset (1 ke 0..*) dan BobotKriteria (1 ke 0..1) |
| **Prodi** | id (PK), kode_prodi, nama_prodi | get_prodi_code() | Berasosiasi ke Asset (1 ke 0..*) dan KodeLokasi (1 ke 0..*) |
| **KepalaLab** | id (PK), nik, nama, periode, is_active | set_active_period() | Entitas penanggung jawab pengesahan laporan PDF |
| **KodeLokasi** | id (PK), prodi_id (FK), yayasan, rektorat, fakultas, ruangan | build_12_digit_code() | Berasosiasi ke Asset (1 ke 0..*) |
| **KodeBarang** | id (PK), urutan_ruangan_id (FK), golongan, tahun_pengadaan | build_15_digit_code() | Berasosiasi ke Asset (1 ke 0..*) |
| **Asset** | id (PK), kategori_id (FK), lokasi_id (FK), prodi_id (FK), kode_lokasi_id (FK), kode_barang_id (FK), harga_perolehan, umur_ekonomis, skor_kerusakan, biaya_perawatan, pos_x, pos_y | sisa_umur_ekonomis(), nilai_buku(), update_posisi_denah(), generate_qr_code() | Entitas pusat, berasosiasi ke PerbaikanAset (1 ke 0..*) |
| **PerbaikanAset** | id (PK), asset_id (FK), tanggal_perbaikan, biaya_perbaikan, status | hitung_biaya_perilaku(), update_status_servis() | Berasosiasi dari Asset (1 ke 0..*) dan User (1 ke 0..*) |
| **BobotKriteria** | id (PK), kategori_id (FK), bobot_c1, bobot_c2, bobot_c3 | validate_total_bobot(), get_weights_vector() | Berasosiasi dari Category (1 ke 0..1) |

Sumber: Diolah oleh Penulis (2026)

### 4.2.2 Struktur Tabel

a. Tabel Pengguna (tb_user)

Tabel 4. 5 Spesifikasi Tabel Pengguna (tb_user)

| No | Nama Field | Type Field | Size / Length | Keterangan / Key Status |
|---|---|---|---|---|
| 1 | id | INT | 11 | Primary Key (PK), Auto Increment |
| 2 | username | VARCHAR | 64 | Unique, Not Null, Nama akun login |
| 3 | password_hash | VARCHAR | 255 | Not Null, Enkripsi kata sandi (Bcrypt) |

| | | | | |
|---|---|---|---|---|
| 4 | role | VARCHAR | 50 | Not Null, Peran otorisasi (admin, user, dekan) |
| 5 | nama_lengkap | VARCHAR | 100 | Atribut biasa, Nama pemilik akun |
| 6 | jabatan | VARCHAR | 100 | Atribut biasa, Posisi fungsional pengguna |
| 7 | permissions | TEXT | - | Atribut biasa, JSON daftar izin aksi dinamis |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_user terdiri dari 7 field yang berfungsi untuk mencatat seluruh kredensial dan batas wewenang pengguna dalam mengoperasikan aplikasi. Spesifikasi penyimpanan data pengguna sistem dimodelkan pada tabel tb_user. Tabel ini berfungsi untuk mengelola kredensial autentikasi, peran hak akses (*role-based access control*), identitas pimpinan/staf, serta izin fungsional dinamis (*dynamic permissions*) dengan kunci utama (*primary key*) id.

b. Tabel Lokasi Laboratorium (tb_location)

Tabel 4. 6 Spesifikasi Tabel Lokasi Laboratorium (tb_location)

| **No** | **Nama Field** | **Type Field** | **Size / Length** | **Keterangan / Key Status** |
|---|---|---|---|---|
| 1 | id | INT | 11 | **Primary Key (PK)**, Auto Increment |
| 2 | nama_lab | VARCHAR | 50 | Not Null, Nama resmi ruangan lab |
| 3 | kode_lab | VARCHAR | 20 | Unique, Not Null, Singkatan kode ruangan |
| 4 | kode_aset | VARCHAR | 20 | Atribut biasa, Kode angka segmen 27 digit |
| 5 | kapasitas | INT | 11 | Atribut biasa, Daya tampung unit komputer |
| 6 | denah_image | VARCHAR | 255 | Atribut biasa, Path file gambar latar denah |
| 7 | layout_config | TEXT | - | Atribut biasa, Konfigurasi JSON tata letak 2D |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_location terdiri dari 7 field yang berfungsi untuk mengelola master ruangan laboratorium beserta struktur JSON perenderan denah 2D interaktif. Spesifikasi master ruangan laboratorium komputer dikelola pada tabel tb_location. Tabel ini berfungsi untuk menyimpan master ruangan, daya tampung unit komputer, gambar latar denah, serta konfigurasi format JSON tata letak kanvas visual 2D dengan kunci utama (*primary key*) id.

c. Tabel Kategori Aset (tb_category)

Tabel 4. 7 Spesifikasi Tabel Kategori Aset (tb_category)

| No | Nama Field | Type Field | Size / Length | Keterangan / Key Status |
|---|---|---|---|---|
| 1 | id | INT | 11 | Primary Key (PK), Auto Increment |
| 2 | nama_kategori | VARCHAR | 50 | Unique, Not Null, Nama jenis pengelompokan |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_category terdiri dari 2 field yang berfungsi sebagai entitas master klasifikasi jenis barang inventaris. Spesifikasi klasifikasi jenis aset laboratorium disimpan pada tabel tb_category. Tabel ini berfungsi untuk mengelompokkan jenis barang inventaris (seperti PC Komputer, Laptop, Proyektor, AC, dan Switch/Hub) dengan kunci utama (*primary key*) id.

d. Tabel Program Studi (tb_prodi)

Tabel 4. 8 Spesifikasi Tabel Program Studi (tb_prodi)

| No | Nama Field | Type Field | Size / Length | Keterangan / Key Status |
|---|---|---|---|---|
| 1 | id | INT | 11 | Primary Key (PK), Auto Increment |
| 2 | kode_prodi | VARCHAR | 20 | Unique, Not Null, Inisial singkatan prodi |
| 3 | kode_aset | VARCHAR | 20 | Not Null, Kode angka prodi pada 27 digit |
| 4 | nama_prodi | VARCHAR | 100 | Not Null, Nama lengkap program studi |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_prodi terdiri dari 4 field yang berfungsi untuk mencatat identitas program studi pengelola laboratorium. Spesifikasi data program studi pemilik fasilitas dikelola pada tabel *tb_prodi*. Tabel ini berfungsi untuk mengelola master program studi yang bertanggung jawab atas pengoperasian dan kepemilikan laboratorium dengan kunci utama (*primary key*) id.

e. Tabel Kepala Laboratorium (tb_kepala_lab)

Tabel 4. 9 Spesifikasi Tabel Kepala Laboratorium (tb_kepala_lab)

| No | Nama Field | Type Field | Size / Length | Keterangan / Key Status |
|---|---|---|---|---|
| 1 | id | INT | 11 | Primary Key (PK), Auto Increment |
| 2 | nik | VARCHAR | 20 | Unique, Not Null, NIK / NIDN Kepala Lab |
| 3 | nama | VARCHAR | 100 | Not Null, Nama lengkap dan gelar akademik |
| 4 | periode | VARCHAR | 50 | Not Null, Rentang tahun periode dinas |

| | | | | |
|---|---|---|---|---|
| 5 | is_active | BOOLEAN | 1 | Not Null, Status aktif masa dinas |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_kepala_lab terdiri dari 5 field yang berfungsi untuk mengelola data identitas dan periode dinas pimpinan laboratorium. Spesifikasi pimpinan dan penanggung jawab laboratorium disimpan pada tabel tb_kepala_lab. Tabel ini berfungsi untuk mencatat identitas dosen/kepala lab, nomor induk, beserta status aktif periode masa jabatan dengan kunci utama (*primary key*) id.

f. Tabel Kode Aset Lokasi (tb_kode_lokasi)

Tabel 4. 10 Spesifikasi Tabel Kode Aset Lokasi (tb_kode_lokasi)

| No | Nama Field | Type Field | Size / Length | Keterangan / Key Status |
|---|---|---|---|---|
| 1 | id | INT | 11 | Primary Key (PK), Auto Increment |
| 2 | yayasan | VARCHAR | 4 | Atribut biasa, Kode identitas yayasan |
| 3 | rektorat | VARCHAR | 2 | Atribut biasa, Kode identitas rektorat |
| 4 | fakultas | VARCHAR | 2 | Atribut biasa, Kode identitas fakultas |
| 5 | prodi_id | INT | 11 | Foreign Key (FK), Relasi ke tb_prodi(id) |
| 6 | ruangan | VARCHAR | 2 | Atribut biasa, Kode nomor ruangan lab |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_kode_lokasi terdiri dari 6 *field* yang berfungsi menyusun segmen hierarki lokasi untuk pembentukan 12 digit pertama kode registrasi aset. Spesifikasi pembentukan segmentasi entitas tempat disimpan pada tabel tb_kode_lokasi. Tabel pendukung ini berfungsi untuk menyusun komponen pengelompokan hierarki instansi (Yayasan, Rektorat, Fakultas, Prodi, Ruangan) guna membentuk 12 digit pertama pada kode aset dengan kunci utama (*primary key*) id dan kunci tamu (*foreign key*) prodi_id yang terhubung ke tb_prodi(id).

g. Tabel Kode Barang (tb_kode_barang)

Tabel 4. 11 Spesifikasi Tabel Kode Barang (tb_kode_barang)

| No | Nama Field | Type Field | Size / Length | Keterangan / Key Status |
|---|---|---|---|---|
| 1 | id | INT | 11 | Primary Key (PK), Auto Increment |
| 2 | golongan | VARCHAR | 2 | Atribut biasa, Kode golongan barang |

| | | | | |
|---|---|---|---|---|
| 3 | bidang | VARCHAR | 2 | Atribut biasa, Kode bidang spesifik |
| 4 | kelompok | VARCHAR | 2 | Atribut biasa, Kode kelompok barang |
| 5 | subkelompok | VARCHAR | 2 | Atribut biasa, Kode subkelompok rincian |
| 6 | tahun_pengadaan | VARCHAR | 4 | Atribut biasa, Digit tahun pengadaan |
| 7 | urutan_ruangan_id | INT | 11 | Foreign Key (FK), Relasi ke tb_location(id) |
| 8 | nomor_urut | VARCHAR | 2 | Atribut biasa, Nomor urut unit spesimen |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_kode_barang terdiri dari 8 *field* yang menyusun segmen klasifikasi barang guna membentuk 15 digit kedua pada kode registrasi aset. Spesifikasi klasifikasi barang milik institusi disimpan pada tabel tb_kode_barang. Tabel pendukung ini berfungsi untuk menyusun segmentasi klasifikasi jenis barang (Golongan, Bidang, Kelompok, Subkelompok, Tahun, Ruangan, Nomor Urut) guna membentuk 15 digit kedua kode registrasi aset dengan kunci utama (*primary key*) id dan kunci tamu (*foreign key*) urutan_ruangan_id yang terhubung ke tb_location(id).

h. Tabel Data Aset Inventaris (tb_asset)

Tabel 4. 12 Spesifikasi Tabel Data Aset Inventaris (tb_asset)

| **No** | **Nama Field** | **Type Field** | **Size / Length** | **Keterangan / Key Status** |
|---|---|---|---|---|
| 1 | id | INT | 11 | Primary Key (PK), Auto Increment |
| 2 | kode_aset | VARCHAR | 50 | Unique, Not Null, Kode registrasi 27 digit |
| 3 | nama_aset | VARCHAR | 100 | Not Null, Nama barang inventaris |
| 4 | kategori_id | INT | 11 | Foreign Key (FK), Relasi ke tb_category(id) |
| 5 | lokasi_id | INT | 11 | Foreign Key (FK), Relasi ke tb_location(id) |
| 6 | prodi_id | INT | 11 | Foreign Key (FK), Relasi ke tb_prodi(id) |
| 7 | kode_lokasi_id | INT | 11 | Foreign Key (FK), Relasi ke tb_kode_lokasi(id) |
| 8 | kode_barang_id | INT | 11 | Foreign Key (FK), Relasi ke tb_kode_barang(id) |
| 9 | deskripsi | TEXT | - | Atribut biasa, Catatan spesifikasi teknis |
| 10 | kondisi | VARCHAR | 20 | Atribut biasa, Status fisik barang |
| 11 | tahun_pengadaan | INT | 4 | Atribut biasa, Tahun pengadaan aset |

| | | | | |
|---|---|---|---|---|
| 12 | harga_perolehan | DECIMAL | 15,2 | Atribut biasa, Harga awal pembelian |
| 13 | umur_ekonomis | INT | 4 | Atribut biasa, Masa manfaat dalam tahun |
| 14 | nilai_residu | DECIMAL | 15,2 | Atribut biasa, Perkiraan nilai sisa aset |
| 15 | skor_kerusakan | INT | 2 | Atribut biasa, Skor tingkat kerusakan fisik (Kriteria C2) |
| 16 | biaya_perawatan | DECIMAL | 15,2 | Atribut biasa, Akumulasi biaya servis (Kriteria C3) |
| 17 | pos_x | FLOAT | - | Atribut biasa, Koordinat horizontal denah (%) |
| 18 | pos_y | FLOAT | - | Atribut biasa, Koordinat vertikal denah (%) |
| 19 | qr_code_path | VARCHAR | 255 | Atribut biasa, Path file gambar QR Code |
| 20 | foto_path | VARCHAR | 255 | Atribut biasa, Path file dokumentasi foto |
| 21 | created_at | DATETIME | - | Atribut biasa, Timestamp pencatatan data |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_asset terdiri dari 21 *field* yang menyimpan seluruh data teknis inventaris, posisi spasial denah, serta variabel pendukung komputasi evaluasi kelayakan aset. Spesifikasi entitas utama inventaris dikelola pada tabel tb_asset yang berfungsi untuk menyimpan rincian teknis barang, identitas kode 27 digit, koordinat denah 2D, nilai depresiasi ekonomis, serta parameter kriteria evaluasi. Tabel ini memiliki *primary key* id serta terhubung secara relasional melalui lima *foreign key*, yaitu kategori_id, lokasi_id, prodi_id, kode_lokasi_id, dan kode_barang_id.

i. Tabel Perbaikan Aset (tb_perbaikan)

Tabel 4. 13 Spesifikasi Tabel Perbaikan Aset (tb_perbaikan)

| **No** | **Nama Field** | **Type Field** | **Size / Length** | **Keterangan / Key Status** |
|---|---|---|---|---|
| 1 | id | INT | 11 | **Primary Key (PK)**, Auto Increment |
| 2 | asset_id | INT | 11 | **Foreign Key (FK)**, Relasi ke tb_asset(id) |
| 3 | tanggal_perbaikan | DATE | - | Not Null, Tanggal servis dilakukan |
| 4 | deskripsi | TEXT | - | Not Null, Uraian perbaikan/komponen |
| 5 | biaya_perbaikan | DECIMAL | 15,2 | Not Null, Nominal biaya servis per transaksi |
| 6 | teknisi | VARCHAR | 100 | Atribut biasa, Nama teknisi/vendor |

| | | | | |
|---|---|---|---|---|
| 7 | status | VARCHAR | 30 | Atribut biasa, Status pengerjaan servis |
| 8 | created_at | DATETIME | - | Atribut biasa, Timestamp pencatatan laporan |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_perbaikan terdiri dari 8 field yang mengelola riwayat pemeliharaan berkala serta biaya perawatan aset. Spesifikasi pemeliharaan dan servis berkala disimpan pada tabel tb_perbaikan. Tabel ini berfungsi untuk mengelola rekaman riwayat servis, rincian perbaikan, nama teknisi, serta biaya perawatan unit aset dengan kunci utama (*primary key*) id dan kunci tamu (*foreign key*) asset_id yang terhubung ke tb_asset(id).

j. Tabel Bobot Kriteria Evaluasi (tb_bobot_kriteria)

Tabel 4. 14 Spesifikasi Tabel Bobot Kriteria Evaluasi (tb_bobot_kriteria)

| No | Nama Field | Type Field | Size / Length | Keterangan / Key Status |
|---|---|---|---|---|
| 1 | id | INT | 11 | **Primary Key (PK)**, Auto Increment |
| 2 | kategori_id | INT | 11 | **Foreign Key (FK)**, Relasi ke tb_category(id) |
| 3 | bobot_c1 | DECIMAL | 4,2 | Not Null, Bobot C1: Sisa Umur Ekonomis (*Cost*) |
| 4 | bobot_c2 | DECIMAL | 4,2 | Not Null, Bobot C2: Tingkat Kerusakan (*Benefit*) |
| 5 | bobot_c3 | DECIMAL | 4,2 | Not Null, Bobot C3: Akumulasi Biaya (*Benefit*) |
| 6 | updated_at | DATETIME | - | Atribut biasa, Timestamp pembaruan bobot |

Sumber: Diolah oleh Penulis (2026)

Tabel tb_bobot_kriteria terdiri dari 6 *field* yang menyimpan parameter bobot preferensi kriteria untuk kalkulasi prioritas peremajaan aset. Spesifikasi konfigurasi pembobotan keputusan disimpan pada tabel tb_bobot_kriteria. Tabel ini berfungsi untuk menyimpan parameter nilai bobot preferensi kriteria evaluasi (C1, C2, C3) dengan kunci utama (*primary key*) id serta kunci tamu (*foreign key*) kategori_id yang terhubung ke tb_category(id).

### 4.2.3 Desain

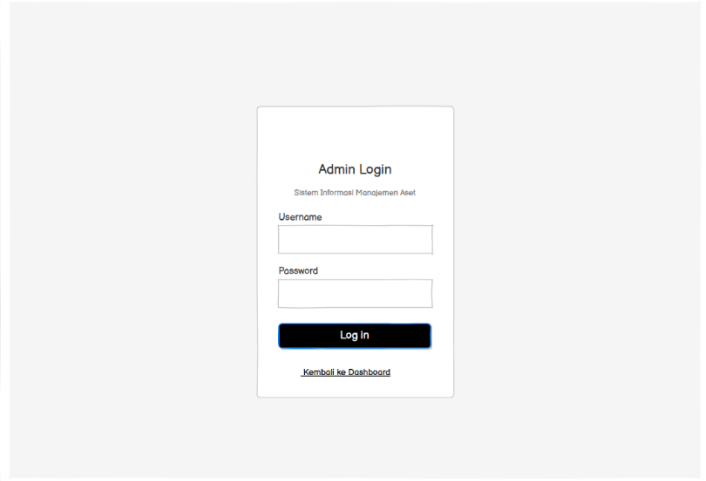
Perancangan antarmuka (*user interface wireframe*) mendefinisikan tata letak visual, struktur navigasi, serta interaksi pengguna pada aplikasi. Pada bagian

ini, perancangan antarmuka dibedakan secara tegas berdasarkan mekanisme *Role-Based Access Control* (RBAC) antara dua peran pengguna utama:

1. Admin (Kepala Laboratorium): Memiliki hak akses penuh (*full control*) yang dilengkapi dengan tombol-tombol aksi operasional, seperti tambah, sunting, hapus data, *drag-and-drop* tata letak denah, serta pengatur bobot evaluasi.
2. User (Dekan): Memiliki hak akses khusus pemantauan (*executive monitoring access*) yang menampilkan antarmuka *read-only*, visualisasi statistik eksekutif, serta fasilitas pencetakan dokumen PDF tanpa tombol pengubah data master.

Selain antarmuka layar web (*input design*), bagian ini juga memuat perancangan dokumen keluaran (*output design*) yang dihasilkan oleh sistem berupa berkas PDF resmi dan cetakan stiker label *QR Code*.

1. **Perancangan Antarmuka Login**



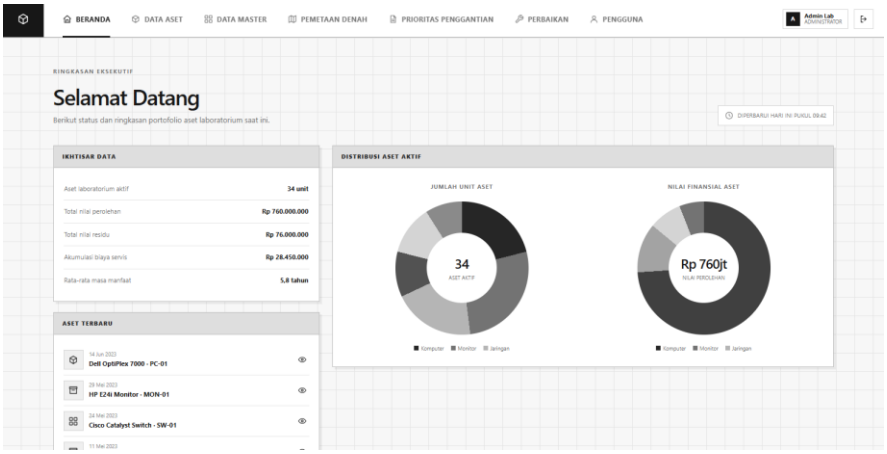

Gambar 4. 21 Rancangan Antarmuka Halaman Login

Sumber: Diolah oleh Penulis (2026)

Halaman login menggunakan tata letak kartu terpusat (*centered card*) di tengah layar dengan identitas sistem di bagian atas. Form menyediakan dua kolom masukan wajib, yaitu username dan password bertipe sandi (*masked*). Tombol aksi 'Masuk ke Sistem' di bagian bawah berfungsi mengirimkan kredensial melalui metode HTTP POST. Jika masukan salah, sistem memunculkan notifikasi

peringatan merah di atas form, sedangkan autentikasi yang berhasil akan mengarahkan pengguna ke halaman dasbor utama.

2. Rancangan Antarmuka Dasbor Utama

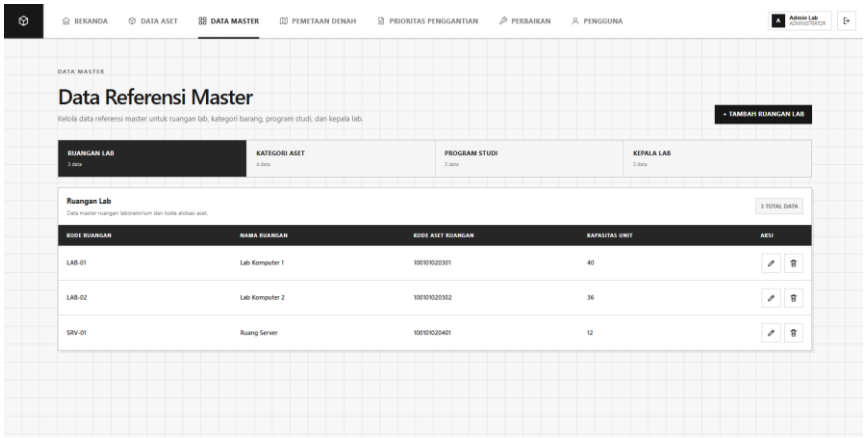



Gambar 4. 22 Rancangan Antarmuka Dashboard

Sumber: Diolah oleh Penulis (2026)

Dasbor Admin menerapkan navigasi bilah atas (*topbar*) yang memuat tujuh menu utama, tombol utilitas *backup*/*restore* database, serta info akun aktif. Area konten terbagi dua kolom: panel kiri menampilkan grafik donat distribusi aset per kategori dan nilai nominal, sedangkan panel kanan memuat ringkasan indikator finansial (total unit, nilai perolehan, depresiasi, dan biaya perbaikan) serta feed aset terbaru. Tersedia tombol navigasi pintas 'Lihat Semua' dan tautan baris yang mengarahkan langsung ke detail aset.

3. Rancangan Antarmuka Data Master

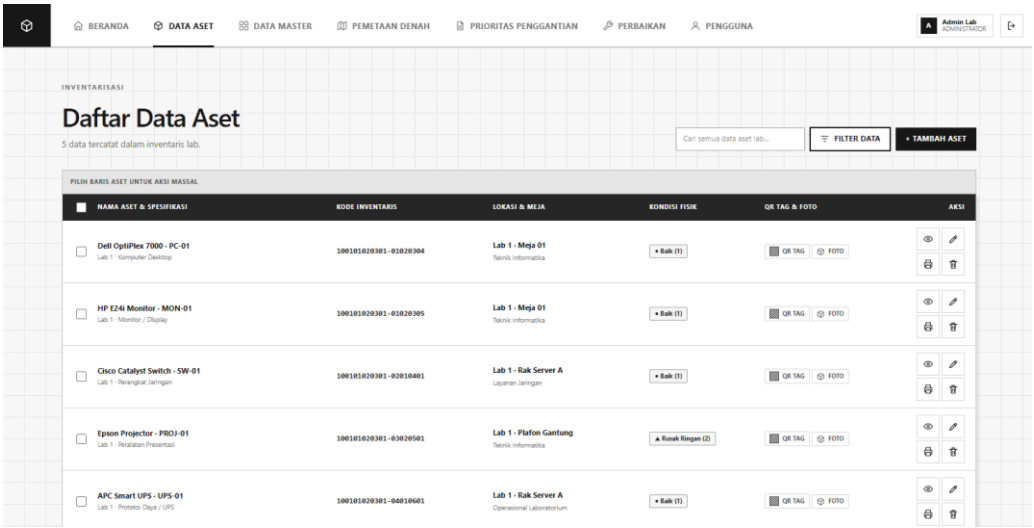



Gambar 4. 23 Rancangan Antarmuka Halaman Data Master

Sumber: Diolah oleh Penulis (2026)

Pada antarmuka aplikasi sistem ini, halaman data master dirancang menggunakan struktur tabulasi horizontal yang diposisikan tepat di bawah bilah navigasi utama untuk mengelola empat entitas referensi, yakni: Ruangan Laboratorium, Kategori, Program Studi (Prodi), dan Kepala Laboratorium. Form input data akan ditampilkan melalui jendela modal interaktif pada saat proses penambahan atau pembaruan (edit) berlangsung, yang secara spesifik mencakup isian kode, nama, kapasitas, serta nomor urut ruangan. Untuk mempermudah alur operasional pengguna, tombol aksi 'Tambah Data' ditempatkan pada bagian atas tabel, sementara tombol fungsi edit dan hapus disematkan secara langsung di setiap baris data yang terkait. Sebagai hasil akhir, sistem akan menyajikan output berupa tabel referensi terstruktur yang senantiasa dilengkapi dengan fitur dialog konfirmasi keamanan sebelum suatu tindakan penghapusan dieksekusi.

4. Rancangan Antarmuka Data Aset

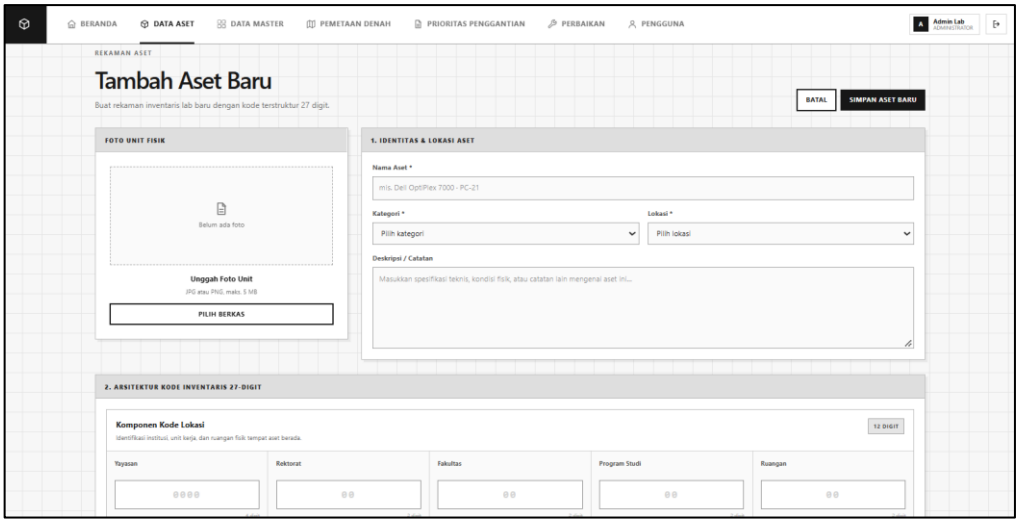



Gambar 4. 24 Rancangan Antarmuka Tabel Daftar Aset

Sumber: Diolah oleh Penulis (2026)

Katalog aset menyusun bilah pencarian dan filter (ruangan dan kategori) di bagian atas, tombol cetak dan tambah aset di kanan atas, tabel inventaris di tengah, serta paginasi di kaki tabel. Filter memungkinkan pencarian instan berdasarkan nama atau kode aset. Aksi yang tersedia meliputi tombol 'Cetak Semua QR', 'Cetak Laporan PDF', 'Tambah Aset', serta checkbox di tiap baris. Tabel menampilkan kode 27 digit, nama barang, lokasi lab, dan badge status kondisi berkode warna.

5. Rancangan Antarmuka Formulir Tambah Aset

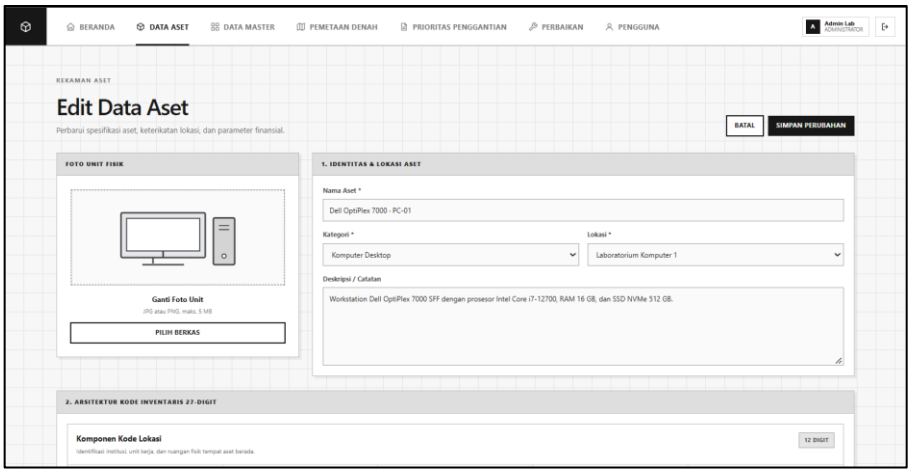



Gambar 4. 25 Rancangan Antarmuka Form Tambah Aset

Sumber: Diolah oleh Penulis (2026)

Form pendaftaran aset terbagi dalam empat kluster: informasi umum, 11 segmen pembentuk kode 27 digit, parameter finansial/kondisi, dan unggah foto fisik. Pengguna mengisikan teks nama, memilih *dropdown master*, menginputkan angka segmen kode, harga perolehan, umur ekonomis, nilai residu, serta skor kerusakan fisik. Terdapat panel pratinjau kode 27 digit secara *live-preview*. Tombol aksi 'Simpan Data Aset & Terbitkan QR Code' memvalidasi masukan dan merekam data, berdampingan dengan tombol 'Batal'.

6. Rancangan Antarmuka Formulir Edit Aset

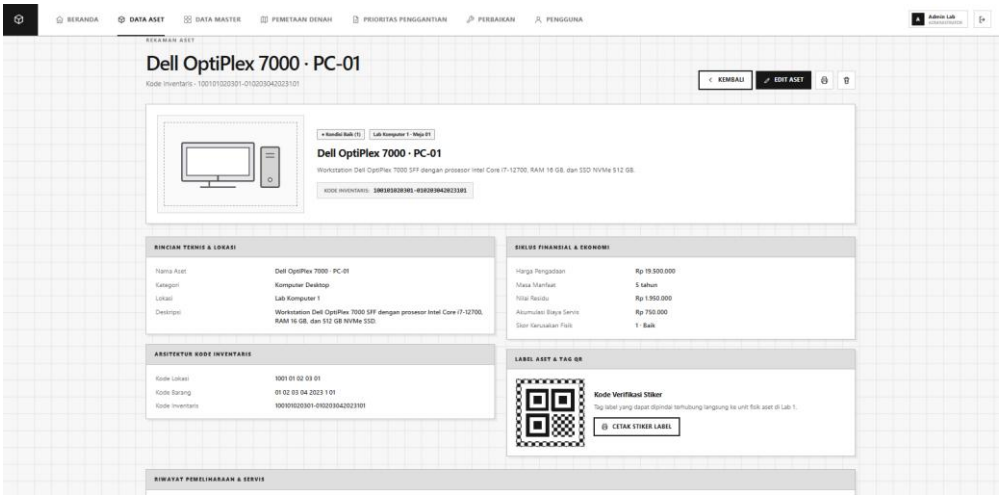



Gambar 4. 26 Rancangan Antarmuka Form Edit Aset

Sumber: Diolah oleh Penulis (2026)

Pada antarmuka sistem, form sunting data aset menyajikan spanduk peringatan terkait dampak modifikasi kode pada bagian atas halaman, lengkap bersama serangkaian kolom formulir yang terisi otomatis berdasarkan data awal di database. Melalui form interaktif ini, pengguna memiliki fleksibilitas untuk memperbarui informasi nama barang, rincian lokasi penempatan, skor evaluasi kerusakan fisik terkini, segmen kode klasifikasi, besaran nilai finansial, hingga melakukan pembaruan berkas foto fisik perangkat. Ketika tombol tindakan 'Perbarui Data Aset' ditekan, sistem secara langsung mengeksekusi penyimpanan seluruh perubahan ke dalam database sekaligus memicu proses regenerasi berkas QR Code baru, sebelum akhirnya sebuah notifikasi kilat hijau muncul memberi konfirmasi keberhasilan transaksi saat pengguna dialihkan kembali menuju halaman katalog utama.

7. Rancangan Antarmuka Lembar Detail Aset

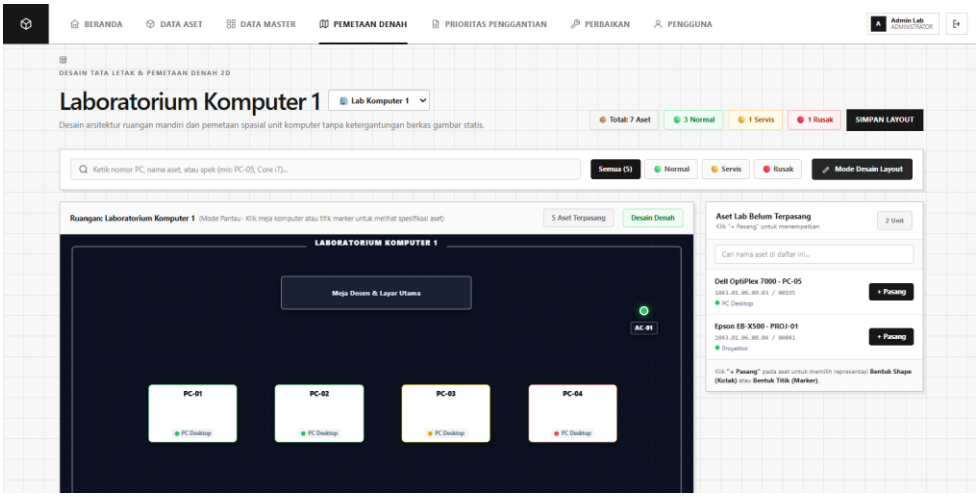



Gambar 4. 27 Rancangan Antarmuka Detail Aset

Sumber: Diolah oleh Penulis (2026)

Halaman detail menyusun informasi secara bertingkat: header nama dan kode aset, panel ganda foto fisik dan gambar QR Code, panel depresiasi garis lurus (harga perolehan, sisa umur, dan nilai buku), serta tabel riwayat perbaikan teknis. Tombol aksi meliputi 'Cetak Stiker QR' di kanan atas, tombol '+ Catat Servis Baru' di atas tabel riwayat, dan navigasi kembali. Tampilan memberikan profil lengkap kesiapan operasional serta akumulasi biaya perawatan yang terintegrasi dengan kriteria penilaian peremajaan.

8. Rancangan Antarmuka Pemetaan Denah 2D

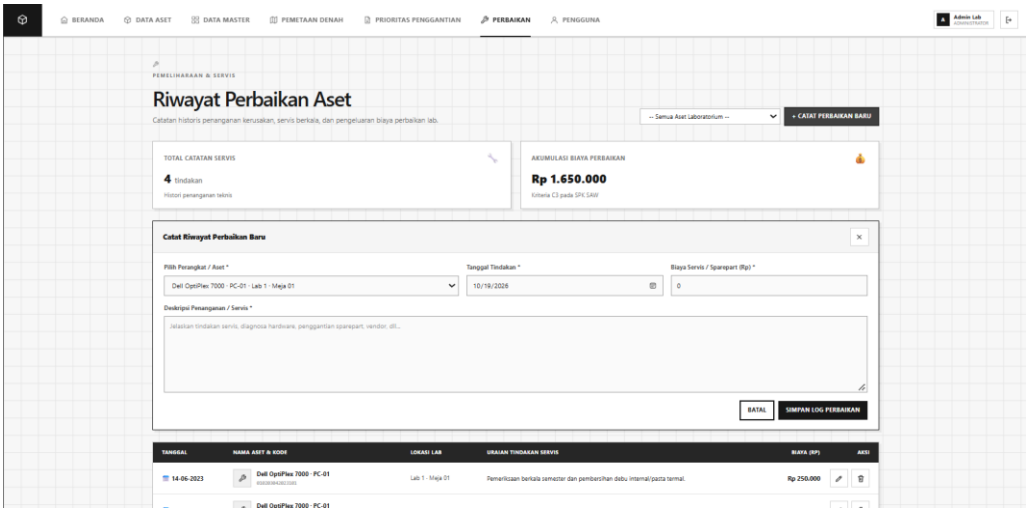



Gambar 4. 28 Rancangan Antarmuka Pemetaan Denah 2D Laboratorium

Sumber: Diolah oleh Penulis (2026)

*Editor* denah 2D menyediakan kanvas spasial interaktif yang menggambarkan tata letak fisik laboratorium beserta sarana pendukung (pintu, proyektor, meja dosen, AC). Admin memilih ruangan lab melalui *dropdown* dan mengatur posisi meja komputer menggunakan interaksi geser dan lepas (*drag-and-drop*). Panel kontrol bawah memuat tombol, 'Kunci Posisi', dan 'Simpan Konfigurasi Denah'. Penyimpanan mengirimkan data koordinat pos_x dan pos_y ke server disertai notifikasi konfirmasi.

9. Rancangan Antarmuka Perbaikan Aset

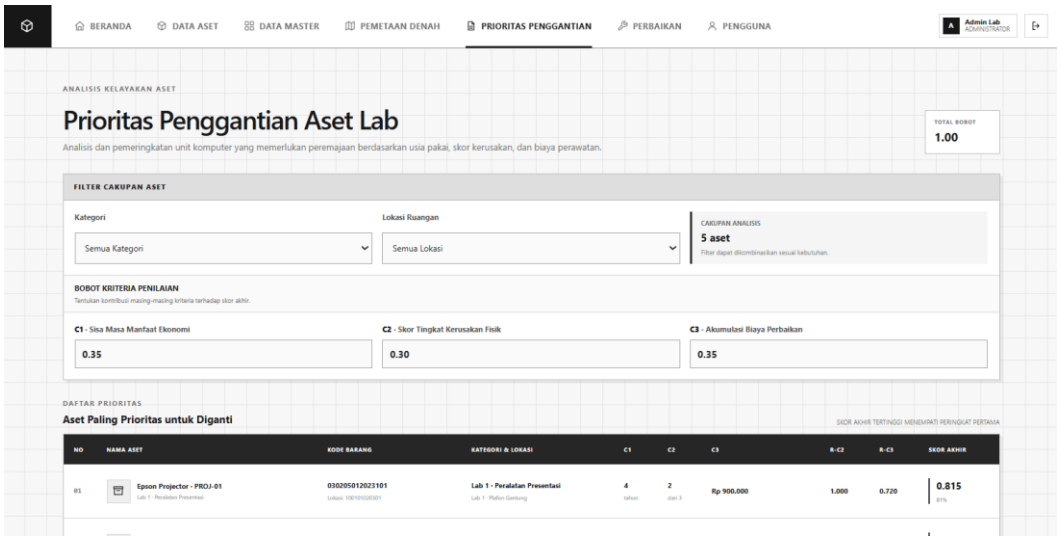



Gambar 4. 29 Rancangan Antarmuka Perbaikan Aset

Sumber: Diolah oleh Penulis (2026)

Pada antarmuka sistem, halaman modul perbaikan dirancang untuk menggabungkan tabel riwayat servis di area visual utama, yang kemudian diikuti oleh ringkasan kalkulasi akumulasi biaya pada bagian bawah, serta sebuah form modal interaktif untuk pencatatan data riwayat baru. Form modal ini mewajibkan pengguna untuk melengkapi parameter rincian, seperti pemilihan aset, tanggal tindakan, uraian spesifik kerusakan, total tagihan perbaikan, hingga identitas nama teknisi. Sebuah tombol aksi bertuliskan '+ Catat Perbaikan Baru' disediakan untuk memicu pemanggilan form tersebut. Sebagai hasil akhir pemrosesan, output menyajikan kronologi historis servis secara terstruktur yang secara otomatis langsung memperbarui nilai total pengeluaran biaya perawatan pada entitas aset terkait di dalam database.

10. Rancangan Antarmuka Prioritas Penggantian

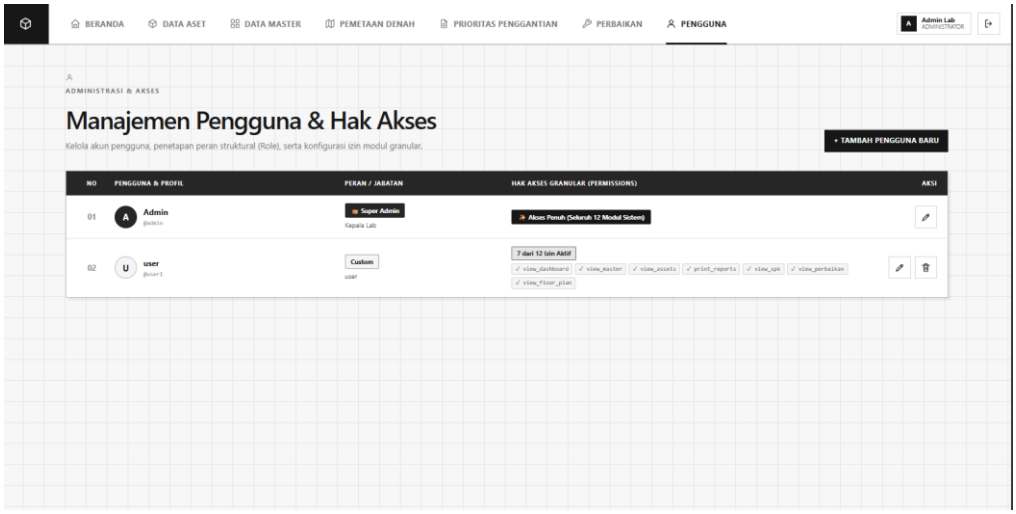



Gambar 4. 30 Rancangan Antarmuka Prioritas Penggantian Aset Metode SAW

Sumber: Diolah oleh Penulis (2026)

Halaman ini terbagi tiga zona: indikator validasi bobot dan filter lab di atas, form pengaturan bobot kriteria SAW (C1 sisa umur, C2 kerusakan fisik, C3 biaya perbaikan) di tengah, serta tabel hasil pemeringkatan di bawah. Pengguna menginputkan nilai desimal bobot yang harus berjumlah tepat 1.00 (100%). Tombol aksi mencakup 'Simpan Konfigurasi Bobot' dan 'Cetak Rekomendasi PDF'. Tabel menyajikan urutan ranking, nilai normalisasi matriks R, skor preferensi V, serta badge status prioritas.

11. Rancangan Antarmuka Manajemen Pengguna & Hak Akses




Gambar 4. 31 Rancangan Antarmuka Manajemen Pengguna & Hak Akses

Sumber: Diolah oleh Penulis (2026)

Antarmuka manajemen pengguna memuat tabel direktori akun terdaftar di atas dan form konfigurasi hak akses dinamis di bawahnya. *Form* membutuhkan isian username, password, nama lengkap, pilihan peran jabatan (*preset*), serta 12 checkbox izin granular. Tombol '+ Tambah Pengguna Baru' memicu penambahan akun, sedangkan tombol edit dan hapus tersedia di tiap baris. Output menampilkan daftar akun beserta badge hak akses aktif dan mekanisme proteksi agar Admin tidak menghapus akunnya sendiri.

12. Rancangan Stiker Label Aset

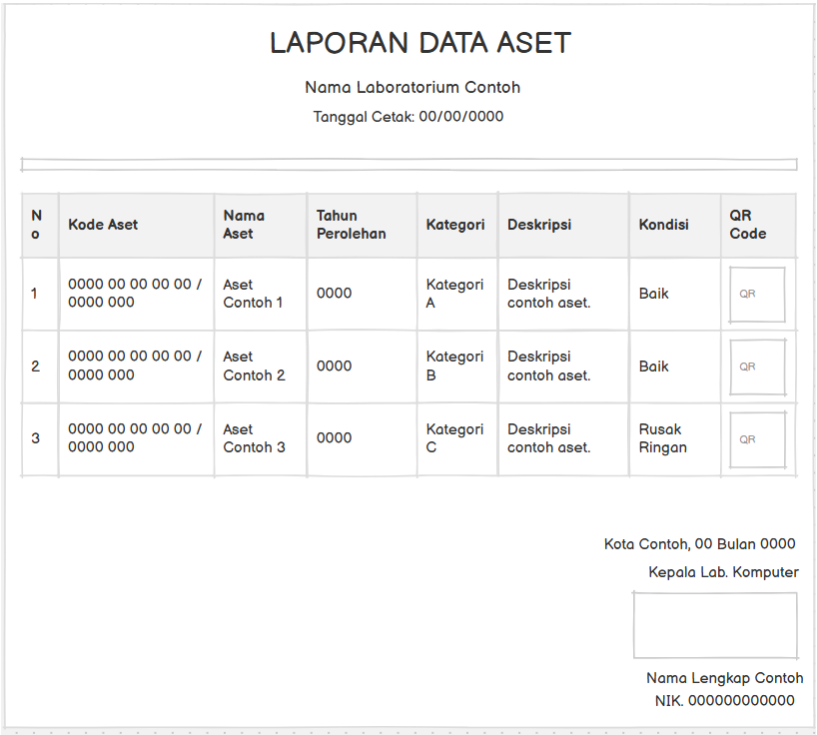



Gambar 4. 32 Rancangan Stiker Label Aset

Sumber: Diolah oleh Penulis (2026)

Rancangan antarmuka modul cetak label satuan dirancang secara presisi dengan dimensi ukuran fisik 13.5 cm x 5.5 cm. Tata letak visualnya terbagi atas bagian header yang memuat identitas resmi institusi universitas beserta fakultas, serta area body informasi utama yang dipisahkan secara tegas oleh elemen garis ganda. Proses penyusunan dokumen pelabelan ini beroperasi secara sepenuhnya otomatis tanpa memerlukan interaksi form masukan tambahan dari pengguna, karena sistem langsung mengekstraksi data relevan dari database merujuk pada parameter ID aset. Melalui implementasi fungsional skrip JavaScript window.print(), sistem akan otomatis memicu pemanggilan dialog cetak bawaan pada peramban seketika saat halaman dokumen selesai dimuat. Sebagai hasil akhirnya, keluaran operasional modul ini berupa cetakan stiker fisik yang mengintegrasikan logo UNIBBA/FTI, pemetaan citra QR Code beresolusi 120x120 piksel, beserta rincian teks komprehensif yang mencakup susunan kode unik 27 digit, nama spesifik barang, dan detail lokasi penempatan di laboratorium.

13. Rancangan Laporan Data Aset

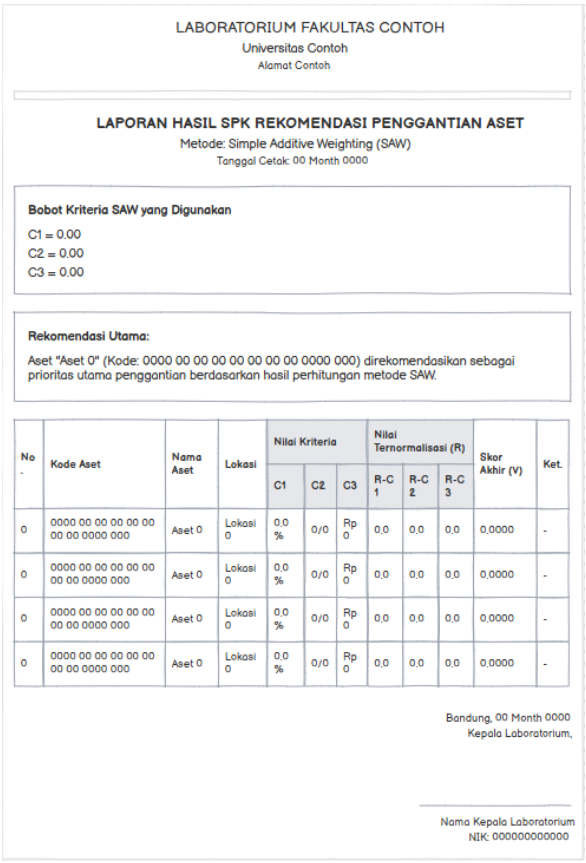



Gambar 4. 33 Rancangan Laporan Data Aset

Sumber: Diolah oleh Penulis (2026)

Rancangan luaran laporan rekapitulasi inventaris diatur secara spesifik menggunakan format kertas A4 dengan orientasi *portrait*. Struktur visual dokumen secara komprehensif memuat elemen kop laporan resmi pada bagian atas, yang diikuti penyajian tabel data yang terbagi ke dalam 9 kolom rincian (meliputi: Nomor, Kode Aset, Nama Barang, Tahun Perolehan, Kategori, Deskripsi, Kondisi, Thumbnail Foto, serta pemetaan Mini QR Code), dan diakhiri oleh blok tanda tangan pengesahan yang diposisikan di sudut kanan bawah. Seluruh entri informasi di dalam dokumen ditarik secara otomatis dari basis data berdasarkan parameter filter lokasi laboratorium yang diterapkan pengguna. Operasi pencetakan dieksekusi dengan memanfaatkan fungsionalitas skrip window.print() yang telah dikonfigurasi presisi dengan margin 1 cm. Sebagai hasil akhirnya, keluaran sistem ini berwujud cetakan dokumen fisik resmi yang terstruktur dan siap untuk disahkan secara langsung oleh Kepala Laboratorium terkait.

14. Rancangan Laporan Ranking Rekomendasi Penggantian Aset



LABORATORIUM FAKULTAS CONTOH
Universitas Contoh
Alamat Contoh

**LAPORAN HASIL SPK REKOMENDASI PENGGANTIAN ASET**
Metode: Simple Additive Weighting (SAW)
Tanggal Cetak: 00 Month 0000

**Bobot Kriteria SAW yang Digunakan**
C1 = 0.00
C2 = 0.00
C3 = 0.00

**Rekomendasi Utama:**
Aset "Aset 0" (Kode: 0000 00 00 00 00 00 00 00 0000 000) direkomendasikan sebagai prioritas utama penggantian berdasarkan hasil perhitungan metode SAW.

| No. | Kode Aset | Nama Aset | Lokasi | Nilai Kriteria | | | Nilai Ternormalisasi (R) | | | Skor Akhir (V) | Ket. |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | C1 | C2 | C3 | R-C1 | R-C2 | R-C3 | | |
| 0 | 0000 00 00 00 00 00 00 00 0000 000 | Aset 0 | Lokasi 0 | 0,0 % | 0/0 | Rp 0 | 0,0 | 0,0 | 0,0 | 0,0000 | - |
| 0 | 0000 00 00 00 00 00 00 00 0000 000 | Aset 0 | Lokasi 0 | 0,0 % | 0/0 | Rp 0 | 0,0 | 0,0 | 0,0 | 0,0000 | - |
| 0 | 0000 00 00 00 00 00 00 00 0000 000 | Aset 0 | Lokasi 0 | 0,0 % | 0/0 | Rp 0 | 0,0 | 0,0 | 0,0 | 0,0000 | - |
| 0 | 0000 00 00 00 00 00 00 00 0000 000 | Aset 0 | Lokasi 0 | 0,0 % | 0/0 | Rp 0 | 0,0 | 0,0 | 0,0 | 0,0000 | - |

Bandung, 00 Month 0000
Kepala Laboratorium,

Nama Kepala Laboratorium
NIK: 000000000000

Gambar 4. 34 Rancangan Laporan Rekomendasi Penggantian Aset

Sumber: Diolah oleh Penulis (2026)

Laporan rekomendasi peremajaan berformat portrait A4 memuat kop universitas/fakultas, ringkasan bobot kriteria SAW yang digunakan, tabel matriks hasil perangkingan (Rank, Kode, Nama Barang, Lokasi, Nilai Normalisasi R1-R3, Skor Preferensi V, dan Status Prioritas), serta blok tanda tangan pengesahan. Data dihitung dan ditarik secara otomatis dari sistem. Laporan ini menjadi dokumen pendukung pertanggungjawaban fisik untuk usulan pengadaan komputer baru kepada pimpinan institusi.

# BAB V

# IMPLEMENTASI DAN PENGUJIAN

## 5.1 Implementasi

Implementasi sistem merupakan tahap pengoperasionalkan rancang bangun arsitektur konseptual dan pemodelan basis data ke dalam bentuk perangkat lunak yang fungsional dan siap diujicobakan. Pada tahap ini, seluruh spesifikasi perancangan diterjemahkan ke dalam baris instruksi komputasi menggunakan bahasa pemrograman Python dengan kerangka kerja Flask dan sistem pengelolaan basis data relasional SQLite. guna membuktikan integritas struktur logika, algoritma kalkulasi penyusutan garis lurus (Straight Line Depreciation), serta proses komputasi metode *Simple Additive Weighting* (SAW) yang telah dirancang sebelumnya, tahap implementasi ini disajikan melalui cuplikan kode program inti (Listing Program) yang dikelompokkan secara terstruktur berdasarkan masing-masing modul utama aplikasi.

### 5.1.1 Listing Program

Listing program pada sub-bab ini menyajikan implementasi fungsi komputasi inti (*core computational logic*) yang dibangun pada *backend* Sistem Informasi Manajemen Aset Laboratorium Komputer. Kode program disusun berdasarkan pendekatan arsitektur dari bawah ke atas (*bottom-up architecture*), diawali dari inisialisasi lingkungan peladen (*server application factory*), modul data master, manajemen aset dan pengkodean hierarki, pembuatan label QR code dan perhitungan depresiasi, pencatatan log pemeliharaan, visualisasi denah 2D, kalkulasi algoritma *Simple Additive Weighting* (SAW), hingga mekanisme otorisasi hak akses berbasis peran (*Role-Based Access Control*).

a. Inisialisasi Peladen Flask, Registrasi Blueprint, dan Konfigurasi Database ORM

Fungsi *create_app()* pada berkas *app/__init__.py* menerapkan pola *Application Factory* untuk menginisialisasi instans aplikasi *Flask* serta menghubungkan basis data *SQLAlchemy ORM.* Skrip *run.py* bertindak sebagai

pemicu utama peladen yang secara otomatis mendeteksi ketersediaan porta jaringan sebelum aplikasi *dijalankan.*

```
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)


    from app import routes
    app.register_blueprint(routes.bp)


    @app.before_request
    def sync_user_session():
        user_id = session.get('user_id')
        if user_id:
            user = User.query.get(user_id)
            if user:
                session['permissions'] = user.get_permissions_list()
    return app
```

Proses inisialisasi diawali dengan pembentukan instans Flask yang membaca konfigurasi basis data SQLite dan mengikat ekstensi ORM ke dalam aplikasi. Sebelum setiap rute dieksekusi, fungsi *sync_user_session()* memverifikasi id pengguna pada sesi peramban untuk memperbarui hak akses terbaru langsung dari basis data. Selain itu, fungsi *has_permission()* didaftarkan ke dalam *globals* Jinja2 untuk mengontrol visibilitas tombol antarmuka HTML, sementara fungsi *find_free_port()* pada *run.py* memindai ketersediaan porta jaringan guna mencegah bentrokan porta saat peladen diaktifkan.

b. Operasional CRUD Data Master Referensi

Modul ini mengelola operasional data master yakni salah satunya Adalah data program studi yang menjadi referensi dasar pembentukan kode hierarki aset

*laboratorium.* Seluruh rute di dalam modul ini dilindungi dekorator otorisasi *@permission_required* untuk menjamin keamanan akses *data.*

```
# Berkas: app/routes/prodi.py


@bp.route('/prodi/add', methods=['GET', 'POST'])
@permission_required('edit_master')
def tambah_prodi():
    if request.method == 'POST':
        kode_prodi = request.form.get('kode_prodi', '').strip()

        if Prodi.query.filter_by(kode_prodi=kode_prodi).first():
            flash('Kode Prodi sudah terdaftar.', 'danger')
            return redirect(url_for('main.tambah_prodi'))

        new_prodi = Prodi(
            kode_aset=request.form['kode_aset'],
            kode_prodi=kode_prodi,
            nama_prodi=request.form['nama_prodi']
        )
        new_prodi.insert_data()

        return redirect(url_for('main.prodi'))

    return render_template('add_prodi.html')
```

Proses manipulasi data master diawali dengan pemeriksaan keunikan kode program studi pada rute penambahan guna mencegah terjadinya duplikasi data pada basis data. Penyimpanan dan pembaruan entitas memanfaatkan pola *Active Record* yang mengabstraksikan fungsi *add* dan *commit* basis data melalui model objek. Apabila pengguna melakukan perubahan atau penghapusan data, sistem

memvalidasi keberadaan entitas dengan metode *get_or_404()* sebelum mengeksekusi pembaruan atribut atau pembersihan baris tabel secara permanen.

c. Operasional CRUD Data Aset dan Pembentukan Kode Registrasi 27 Digit Hierarki

Fungsi *create_asset()* bertugas menerima input formulir inventarisasi dan membentuk kode registrasi unik 27 digit berbasis hierarki *kelembagaan.* Modul ini juga menangani penyimpanan relasi kode lokasi dan kode barang serta pengunggahan berkas foto fisik *aset.*

```
@bp.route('/assets/create', methods=['POST'])
@permission_required('edit_assets')
def create_asset():
    # Segmen Kode Lokasi
    c_yys = request.form['code_yayasan']
    c_rek = request.form['code_rektorat']
    c_fak = request.form['code_fakultas']
    c_prd = request.form['code_prodi']
    c_lok = request.form['code_lokasi']

    # Segmen Kode Barang
    c_gol = request.form['code_golongan']
    c_bid = request.form['code_bidang']
    c_kel = request.form['code_kelompok']
    c_sub = request.form['code_subkel']

    # Segmen Tahun dan Urutan
    c_thn = request.form['code_tahun']
    c_urut_lok = request.form['code_urutan_lok']
    c_no_urut = request.form['code_nomor_urut']

    # Penggabungan kode registrasi
    final_code = (
        f"{c_yys} {c_rek} {c_fak} {c_prd} {c_lok} "
```

```
    f"{c_gol} {c_bid} {c_kel} {c_sub} {c_thn} {c_urut_lok}{c_no_urut}"
)

# Simpan Kode Lokasi
new_kl = KodeLokasi(
    yayasan=c_yys,
    rektorat=c_rek,
    fakultas=c_fak,
    ruangan=c_lok
)
db.session.add(new_kl)
db.session.flush()

# Simpan Kode Barang
new_kb = KodeBarang(
    golongan=c_gol,
    bidang=c_bid,
    kelompok=c_kel,
    subkelompok=c_sub,
    tahun_pengadaan=c_thn,
    nomor_urut=c_no_urut
)
db.session.add(new_kb)
db.session.flush()

# Instansiasi dan Simpan Aset
new_asset = Asset(
    kode_aset=final_code,
    nama_aset=request.form['nama_aset'],
    kode_lokasi_id=new_kl.id,
    kode_barang_id=new_kb.id
)
```

```
    new_asset.insert_data()

    # Generate QR Code
    generate_asset_qr(new_asset)

    return redirect(url_for('main.assets'))
```

Formulasi kode registrasi 27 digit dilakukan dengan menggabungkan sebelas segmen variabel kode lokasi dan klasifikasi barang menggunakan pembatas spasi tunggal. Segmen kode lokasi dan kode barang disimpan terlebih dahulu ke dalam tabel normalisasi terpisah menggunakan perintah *flush()* untuk memperoleh *primary key* sebagai *foreign key* entitas aset utama. Setelah entitas aset berhasil tersimpan, sistem menangani pengunggahan berkas foto fisik ke folder statis peladen dan memicu pembuatan stiker QR code secara otomatis sebelum transaksi diakhiri dengan *commit*.

d. Pembuatan Stiker QR Code dan Perhitungan Nilai Buku Depresiasi Garis Lurus

Modul ini menangani pembuatan gambar stiker QR code berformat *PNG* serta perhitungan penyusutan nilai finansial aset secara *berkala.* Perhitungan nilai buku menggunakan metode depresiasi garis lurus berdasarkan harga perolehan, estimasi nilai residu, dan masa manfaat *barang.*

```
def generate_asset_qr(asset):
    qr_content = (
        f"Nama: {asset.nama_aset}\n"
        f"Kode: {asset.kode_aset}\n"
        f"Lokasi: {asset.location.nama_lab if asset.location else '-'}"
    )
    qr = qrcode.make(qr_content)
    qr_path = os.path.join(current_app.root_path, 'static', 'qrcodes',
f"asset_{asset.id}.png")
    qr.save(qr_path)
    asset.qr_code_path = f"qrcodes/asset_{asset.id}.png"
```

```
class Asset(db.Model, BaseModelMixin):
    def nilai_buku(self):
        if not self.harga_perolehan or not self.umur_ekonomis:
            return self.harga_perolehan or 0.0

        beban_tahunan = (self.harga_perolehan - (self.nilai_residu or 0.0)) / self.umur_ekonomis
        tahun_terpakai = datetime.utcnow().year - (self.tahun_pengadaan or datetime.utcnow().year)
        nilai_saat_ini = self.harga_perolehan - (beban_tahunan * tahun_terpakai)

        return max(nilai_saat_ini, self.nilai_residu or 0.0)
```

Fungsi *generate_asset_qr()* mengompilasi metadata aset beserta rekapitulasi riwayat perbaikan menjadi matriks biner QR code berformat citra PNG yang disimpan pada direktori statis peladen. Sementara itu, metode *nilai_buku()* menghitung beban penyusutan tahunan dengan membagi selisih harga perolehan dan nilai residu terhadap total umur ekonomis. Nilai penyusutan tersebut dikalikan dengan durasi tahun pemakaian untuk menghasilkan nilai buku saat ini, di mana pemanggilan fungsi *max()* menjamin nilai buku tidak akan pernah terdepresiasi di bawah batas estimasi nilai residu barang.

e. Pencatatan Log Perbaikan dan Rekalkulasi Akumulasi Biaya Perawatan Otomatis

Modul ini mengelola pencatatan transaksi perawatan teknis perangkat laboratorium ke dalam tabel log *perbaikan.* Fungsi internal *_recalculate_biaya()* secara otomatis menghitung ulang agregat pengeluaran servis untuk memperbarui kriteria biaya perawatan pada entitas *aset.*

```
def _recalculate_biaya(asset):
    total = db.session.query(
        db.func.sum(PerbaikanAset.biaya_perbaikan)
    ).filter_by(asset_id=asset.id).scalar() or 0.0
```

```
    asset.biaya_perawatan = float(total)
    generate_asset_qr(asset)
    db.session.commit()

@bp.route('/perbaikan/add', methods=['POST'])
@permission_required('manage_perbaikan')
def tambah_perbaikan():
    new_p = PerbaikanAset(
        asset_id=request.form['asset_id'],
        tanggal_perbaikan=request.form['tanggal'],
        biaya_perbaikan=request.form['biaya']
    )
    new_p.insert_data()

    _recalculate_biaya(Asset.query.get(request.form['asset_id']))

    return redirect(url_for('main.perbaikan'))
```

Setiap kali transaksi servis baru ditambahkan dan divalidasi, sistem menyimpan log perbaikan dan langsung memicu fungsi *_recalculate_biaya()*. Fungsi ini mengeksekusi perintah agregasi SQL *SUM* melalui SQLAlchemy ORM untuk menghitung total kumulatif biaya servis yang pernah dikeluarkan oleh aset terkait. Total biaya hasil akumulasi tersebut kemudian disematkan pada kolom *biaya_perawatan* sebagai variabel kriteria *C3* pada metode SAW, sekaligus memperbarui payload informasi pada stiker QR code fisik barang.

f. Pemetaan Spasial Denah 2D Laboratorium

Modul pemetaan denah mengelola posisi koordinat spasial dua dimensi dan indikator kondisi fisik dari seluruh komputer meja di *laboratorium.* Fungsi serialisasi menyusun data *JSON* aset beserta penanda warna status meja untuk ditampilkan pada kanvas *interaktif.*

```
# Berkas: app/routes/floor_plan.py

def serialize_asset(asset):
    status_color = 'red' if asset.skor_kerusakan >= 3 else ('amber' if asset.skor_kerusakan >= 2 else 'green')

    return {
        'id': asset.id,
        'nama_aset': asset.nama_aset,
        'status_color': status_color,
        'pos_x': round(asset.pos_x, 2) if asset.pos_x else None,
        'pos_y': round(asset.pos_y, 2) if asset.pos_y else None
    }

@bp.route('/api/floor-plan/save-layout', methods=['POST'])
def save_floor_plan_layout():
    data = request.get_json()

    for ws in data.get('layout_config', {}).get('workstations', []):
        if ws.get('asset_id'):
            asset = Asset.query.get(ws['asset_id'])
            asset.pos_x, asset.pos_y = float(ws.get('x', 0)), float(ws.get('y', 0))

    db.session.commit()
    return jsonify({'success': True})
```

Fungsi *serialize_asset()* mengklasifikasikan kondisi fisik perangkat berdasarkan skor kerusakan untuk menetapkan penanda warna hijau bagi kondisi normal, kuning sawo untuk perawatan, dan merah untuk rusak berat. Pada rute *save_floor_plan_layout()*, data struktur tata letak berformat JSON diterima dari peramban dan disimpan ke entitas lokasi. Selanjutnya, sistem melakukan perulangan pada setiap meja untuk menyinkronkan koordinat persentase *pos_x* dan

*pos_y* pada tabel aset, serta melepaskan koordinat bagi aset yang tidak diposisikan pada denah.

g. Metode Simple Additive Weighting

Fungsi *hitung_saw()* mengeksekusi algoritma *Simple Additive Weighting* untuk menghitung perangkingan prioritas penggantian komputer *laboratorium.* Analisis keputusan menggabungkan matriks normalisasi kriteria sisa umur, skor kerusakan, dan biaya perawatan terhadap bobot *preferensi.*

```
def hitung_saw(assets, bobot):
    data = [{
        'asset': a,
        'c1': round(a.sisa_umur_ekonomis() / a.umur_ekonomis, 6) if a.umur_ekonomis else 0.0,
        'c2': float(a.skor_kerusakan or 1),
        'c3': float(a.biaya_perawatan or 0)
    } for a in assets]

    min_c1 = min(d['c1'] for d in data)
    max_c2 = max(d['c2'] for d in data)
    max_c3 = max(d['c3'] for d in data)

    results = []
    for d in data:
        r_c1 = min_c1 / d['c1'] if d['c1'] > 0 else 1.0
        r_c2 = d['c2'] / max_c2 if max_c2 > 0 else 0.0
        r_c3 = d['c3'] / max_c3 if max_c3 > 0 else 0.0

        skor = (bobot.bobot_c1 * r_c1) + (bobot.bobot_c2 * r_c2) + (bobot.bobot_c3 * r_c3)

        results.append({
            'asset': d['asset'],
```

```
            'skor_akhir': round(skor, 4)
        })

    results.sort(key=lambda x: x['skor_akhir'], reverse=True)
    return results
```

Proses perhitungan diawali dengan pembentukan matriks keputusan dari rasio sisa umur ekonomis *C1*, skor kerusakan *C2*, dan akumulasi biaya servis *C3*. Pada proses normalisasi, kriteria biaya *C1* dibagi berdasarkan nilai minimum, di mana aset yang telah habis masa manfaatnya secara adaptif diberikan nilai normalisasi maksimum. Selanjutnya, nilai preferensi *V* dihitung dari penjumlahan perkalian matriks ternormalisasi dengan bobot kriteria, lalu seluruh alternatif diurutkan secara menurun sehingga komputer dengan skor tertinggi menempati peringkat utama rekomendasi penggantian.

h. Dekorator Keamanan dan Kontrol Hak Akses Dinamis RBAC

Modul kontrol keamanan menyediakan fungsi pembungkus dekorator untuk membatasi akses rute peladen berdasarkan otentikasi dan perizinan *pengguna.* Mekanisme *Role-Based Access Control* dinamis ini mencegah eksekusi rute oleh pihak yang tidak *berwenang.*

```
# Berkas: app/utils.py


def permission_required(perm):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                return redirect(url_for('main.login'))

            user_perms = session.get('permissions', [])
            user_role = session.get('role', '')
```

```
            if user_role in ['admin', 'Super Admin'] or 'all' in user_perms or perm in
user_perms:
                return f(*args, **kwargs)

            flash('Akses Ditolak: Anda tidak memiliki izin.', 'danger')
            return redirect(url_for('main.dashboard'))

        return decorated_function
    return decorator
```

Setiap dekorator mempertahankan metadata fungsi target melalui @wraps(f) dan memvalidasi keberadaan kunci id pengguna pada variabel sesi peramban. Pada dekorator permission_required(), sistem mengevaluasi izin secara bertingkat, memberikan akses penuh bagi peran *admin*, atau memverifikasi *string* izin spesifik di daftar sesi. Jika wewenang yang dipersyaratkan tidak terpenuhi, eksekusi rute diputus, dan peramban dialihkan ke dasbor utama beserta notifikasi peringatan.

*Source Code* lainnya berada pada laman git@github.com:AndhikaMS/Lab_Asset.git.

### 5.1.2 Implementasi Sistem

Implementasi sistem merupakan tahapan pengoperasionalkan program aplikasi yang telah selesai dibangun ke dalam lingkungan pemanfaatan nyata guna memverifikasi kinerja fungsionalitas dan akurasi perhitungannya. Pelaksanaan tahap implementasi aplikasi sistem informasi manajemen aset laboratorium berbasis metode *Simple Additive Weighting* (SAW) ini mencakup batasan keterangan mengenai tempat dan waktu implementasi sebagai berikut:

a. Tempat Implementasi

Implementasi dan ujicoba eksekusi sistem dilakukan secara lokal pada lingkungan workstation pelaksana rekayasa sistem sebelum diverifikasi langsung untuk studi kasus pengelolaan sarana dan prasarana di lingkungan Laboratorium Komputer Fakultas Teknologi Informasi. Pengujian dilakukan menggunakan komputer pelayan (*local server workstation*) yang terhubung melalui antarmuka

peramban (*web browser*) untuk mengevaluasi interaksi administrator terhadap pangkalan data SQLite dan pelaporan SPK.

b. Waktu Implementasi

Proses implementasi dan verifikasi kelayakan program ini diselesaikan dalam kurun waktu 6 (enam) bulan, yaitu sejak awal Februari hingga akhir Juli 2026. Keseluruhan rentang waktu tersebut dialokasikan secara bertahap yang meliputi kegiatan pemasangan modul dependensi sistem, inisialisasi tabel basis data referensi, import/pengisian data dummy inventaris laboratorium, verifikasi perhitungan logika *Straight Line Depreciation* serta SPK SAW, hingga eksekusi validasi fungsi operasional secara keseluruhan.

### 5.1.3 Spesifikasi Sistem

Guna menjamin stabilitas eksekusi peluncuran web server terdistribusi secara lokal (*localhost*) dan memastikan kelancaran pemrosesan komputasi algoritma matematis serta pembangkitan citra grafis stiker *QR Code*, diperlukan dukungan spesifikasi sistem komputasi yang memadai. Parameter kualifikasi spesifikasi minimum untuk mengoperasikan perangkat lunak ini dikelompokkan menjadi 2 (dua) sektor utama, yakni spesifikasi perangkat keras (*hardware*) dan perangkat lunak (*software*) sebagai berikut:

a. Spesifikasi Perangkat Keras (*Hardware*)

Perangkat keras yang digunakan untuk menjalankan server aplikasi lokal dan pengelola antarmuka sistem informasi manajemen aset laboratorium harus memegang spesifikasi minimum arsitektur fisik yang ditunjukkan pada Tabel 5.2 berikut:

Tabel 5. 1 Spesifikasi Minimum Perangkat Keras (Hardware)

| No | Komponen Perangkat Keras | Spesifikasi Minimum | Fungsi Keterangan di Dalam Sistem |
|---|---|---|---|
| 1 | Prosesor (CPU) | Intel Core i3 / AMD Ryzen 3 (atau prosesor *Dual-Core* berkecepatan 2.0 GHz ke atas) | Mengeksekusi instruksi komputasi alur loop normalisasi matriks SPK SAW dan proses *Multi-Threading* rute server. |
| 2 | Memori (RAM) | Minimal 4 GB DDR3 / DDR4 (Direkomendasikan 8 GB) | Menjamin kestabilan penahanan beban pemantulan memori saat |

| No | Komponen Perangkat Keras | Spesifikasi Minimum | Fungsi Keterangan di Dalam Sistem |
|---|---|---|---|
| | | | peramban (*web browser*) dan server WSGI beroperasi bersamaan. |
| 3 | Penyimpanan (*Storage*) | Minimal 500 MB ruang kosong (*Hard Disk Drive* / SSD) | Mengakomodasi berkas skema kode sumber, berkas database relasional SQLite (app.db), dan direktori ekspor file stiker *QR Code*. |
| 4 | Layar Resolusi (*Display*) | Monitor beresolusi minimal 1366 × 768 piksel | Menampilkan tata letak antarmuka dasbor interaktif, tabel rekapitulasi nilai SPK, dan kartu form aset secara utuh (*responsive*). |
| 5 | Perangkat Pendukung | *Printer Thermal* atau *Printer Inkjet/Laser* standar | Mencetak berkas laporan rekap SPK bertanda tangan Kepala Lab serta memproses cetakan stiker label *QR Code* 27 digit ke media fisik. |

Sumber: Diolah oleh Penulis (2026)

b. Spesifikasi Perangkat Lunak (*Software*)

Selain dukungan arsitektur perangkat keras, eksekusi kode sumber sistem ini membutuhkan integrasi lingkungan perangkat lunak, kerangka kerja aplikasi, serta pustaka pendukung eksternal yang dirinci secara khusus pada Tabel 5.3 berikut:

Tabel 5. 2 Spesifikasi Minimum Perangkat Lunak (Software)

| No | Komponen Perangkat Lunak | Spesifikasi Minimal / Versi | Peran dan Kegunaan Teknis |
|---|---|---|---|
| **1** | **Sistem Operasi** | Microsoft Windows 10 / Windows 11 (64-bit) | Lingkungan sistem operasi dasar pendukung eksekusi skrip peluncur otomatisasi berkas terminal run.bat dan *Command Prompt*. |
| **2** | **Bahasa Pemrograman** | Python (Versi minimal 3.8, direkomendasikan 3.10+) | Mesin penterjemah (*interpreter*) logika pemrograman utama yang mengoperasikan algoritma SAW, hitungan akuntansi, dan ORM. |
| **3** | **Kerangka Kerja Web** | Flask (Versi 2.3.x ke atas) | *Micro web-framework* berbasis Python yang mengatur *routing*, *Blueprints*, serta penjaga *HTTP Request/Response* server. |
| **4** | **Mesin Basis Data** | SQLite 3 | Pangkalan data relasional terintegrasi yang menyimpan seluruh rekam jejak tabel data master, aset inventaris, dan riwayat servis. |
| **5** | **Pustaka Pendukung (*Library*)** | Flask-SQLAlchemy, qrcode, & Pillow | • *Object-Relational Mapping* (ORM) penghubung objek kode ke database.<br>• Pustaka pembangkit gambar label stiker *QR Code* dua dimensi. |

| No | Komponen Perangkat Lunak | Spesifikasi Minimal / Versi | Peran dan Kegunaan Teknis |
|---|---|---|---|
| **6** | **Peramban Web (*Browser*)** | Microsoft Edge / Google Chrome / Mozilla Firefox | Klien interaktif antarmuka yang meluncur secara otomatis via multi-threading untuk menampilkan visualisasi HTML5 dan CSS sistem. |

Sumber: Diolah oleh Penulis (2026)

### 5.1.4 Instalasi Sistem

Instalasi sistem merupakan tahapan teknis untuk menyiapkan kode sumber (*source code*) dan lingkungan basis data agar aplikasi siap dioperasikan oleh pengguna. Proses ini terbagi menjadi dua tahapan utama, yakni instalasi aplikasi dan penyiapan basis data, dengan rincian sebagai berikut:

a. Instalasi Aplikasi

Proses instalasi ini dilakukan di sistem operasi Windows dengan mengandalkan fitur *virtual environment* dari Python. Tujuannya adalah untuk mengisolasi pustaka (*library*) yang digunakan oleh aplikasi agar tidak bentrok dengan sistem bawaan komputer. Berikut adalah langkah-langkah teknisnya:

1. Unduh dan instal Python minimal versi 3.9 (disarankan versi 3.10 atau 3.11) dari situs resminya. Pada saat awal instalasi, pastikan kotak opsi "Add python.exe to PATH" sudah dicentang. Langkah ini sangat penting agar perintah Python nantinya bisa langsung dijalankan lewat *Command Prompt* atau Terminal.
2. Pindahkan seluruh folder aplikasi Sistem Informasi Manajemen Aset Laboratorium ke dalam penyimpanan komputer Anda, misalnya di D:\AssetApp\.
3. Buka Terminal atau *Command Prompt* di dalam folder utama aplikasi tersebut, lalu jalankan perintah ***python -m venv venv***. Perintah ini akan menciptakan sebuah folder bernama venv yang berfungsi sebagai ruang isolasi mandiri untuk Python dan *pip*.
4. Aktifkan ruang virtual tersebut dengan mengetikkan perintah ***venv\Scripts\activate*** atau ***cmd /c "venv\Scripts\activate.bat && cmd"***. Jika berhasil, akan muncul tanda (venv) di bagian depan baris perintah pada terminal.

5. Setelah lingkungan virtual aktif, instal semua dependensi yang dibutuhkan dengan perintah ***pip install -r requirements.txt***. Perintah ini akan memasang lima pustaka utama, yaitu Flask (v3.0.0) untuk *framework* web, Flask-SQLAlchemy (v3.1.1) untuk *database* ORM, qrcode (v7.4.2) untuk membuat gambar QR Code, Pillow (v10.1.0) untuk mengolah gambar stiker, dan python-dotenv (v1.0.0) untuk mengatur variabel lingkungan.
6. Aplikasi dapat langsung dijalankan lewat terminal dengan perintah ***python run.py***. Sistem akan otomatis mencari *port* yang kosong (dimulai dari 5000), menyalakan *server* lokal, dan langsung membuka *browser* Microsoft Edge ke alamat (http://127.0.0.1:5000/). Untuk cara yang lebih praktis, pengelola cukup mengklik ganda pada *file* run.bat yang sudah memuat otomatisasi semua perintah di atas.

b. Instalasi Database

Sistem pangkalan data pada aplikasi ini dibangun menggunakan arsitektur relasional SQLite yang dijalin erat bersama mesin pemetaan SQLAlchemy ORM. Kesesuaian ini mengeliminasi kebutuhan instalasi server database mandiri karena berkas database diprovisikan secara nirkabel dan terintegrasi di dalam kode program. Langkah-langkah inisialisasi dan instalasi basis data meliputi:

1. Otomatisasi Pembangunan Skema Tabel (Auto-Migration): Pada saat aplikasi pertama kali dieksekusi melalui rute inisialisasi di dalam fungsi create_app(), sistem akan memeriksa eksistensi berkas basis data SQLite (app.db) pada direktori instance/. Apabila berkas belum tersedia, mesin ORM mengeksekusi perintah db.create_all(). Pemanggilan fungsi ini menerjemahkan pemetaan model Python pada app/models.py menjadi sepuluh tabel relasional (seperti *users*, *categories*, *locations*, *prodi*, *assets*, *perbaikan_aset*, serta *bobot_saw*) lengkap dengan aturan *Primary Key*, *Foreign Key*, dan tipe kolomnya secara presisi.
2. Penyemaian Data Awal (*Database Seeding*): Untuk mengisi pangkalan data yang baru saja tercipta dengan akun kredensial Administrator standar, bobot default Kriteria SPK SAW (w1=0.4, w2=0.35, w3=0.25$), serta data acuan ruangan laboratorium, teknisi dapat menjalankan berkas skrip

otomatisasi *seeder* yang telah disiapkan di dalam direktori scripts/ melalui perintah terminal:

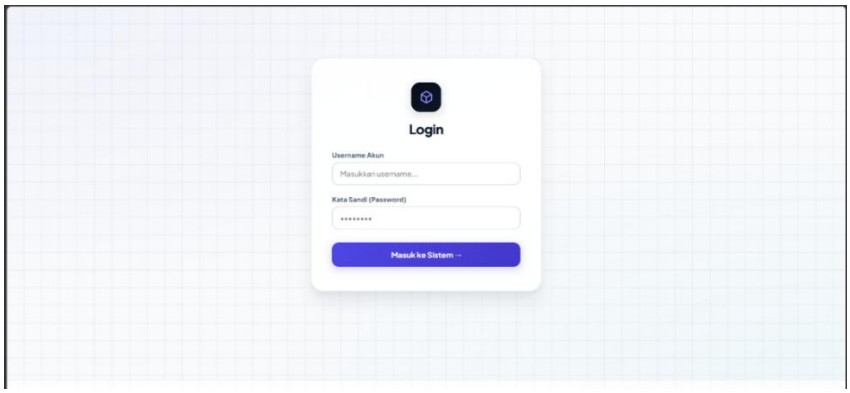
python scripts/seed_dummy_data.py

3. Verifikasi Integritas Relasi Basis Data: Setelah perintah seeding berhasil dikomit (*Commit Successful*), basis data dinyatakan sah dan siap berlanjut untuk menampung transaksi pendaftaran barang inventaris 27 digit serta pemrosesan rekomendasi peremajaan perangkat laboratorium.

### 5.1.5 Menjalankan Sistem

Menjalankan sistem merupakan tahap pembuktian operasional antarmuka perangkat lunak berbasis web yang telah dikembangkan pada lingkungan peladen lokal. Penjelasan fungsionalitas antarmuka disajikan secara runtut berdasarkan sepuluh node utama pada pohon navigasi aplikasi, yang dilengkapi dengan pembanding antarmuka dari sudut pandang aktor Admin dan User untuk mengonfirmasi keterterapan kontrol hak akses dinamis (*Dynamic Role-Based Access Control*).

a. Halaman Login



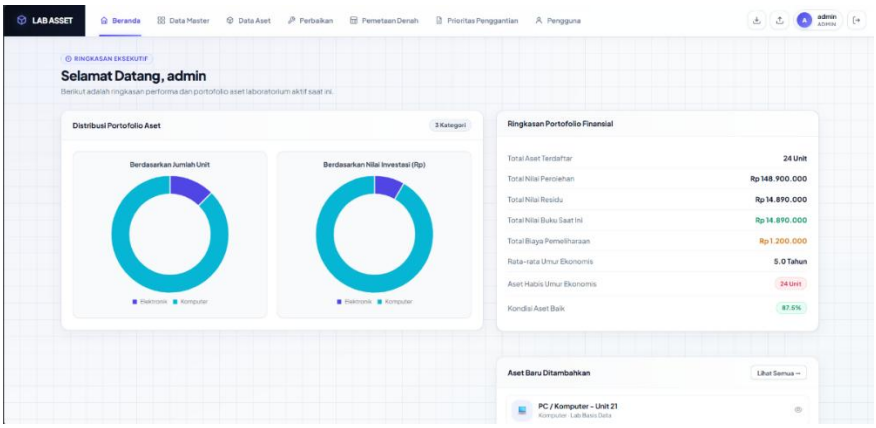

Gambar 5. 1 Halaman Login

Sumber: Diolah oleh Penulis (2026)

Pengguna memulai sesi dengan mengakses rute /login pada peramban, lalu mengisikan username dan password pada formulir terpusat sebelum mengklik tombol 'Masuk ke Sistem'. Di balik layar, pengendali Flask memproses permintaan POST dengan memverifikasi kata sandi melalui fungsi check_password_hash() pada basis data SQLite. Apabila otentikasi berhasil, peladen menginisialisasi sesi

pengguna (Flask-Login) dan mengarahkan antarmuka ke dasbor utama, sedangkan kredensial yang salah memicu pemunculan pesan kilat galat berwarna merah di atas formulir.

b. Halaman Dashboard untuk Admin

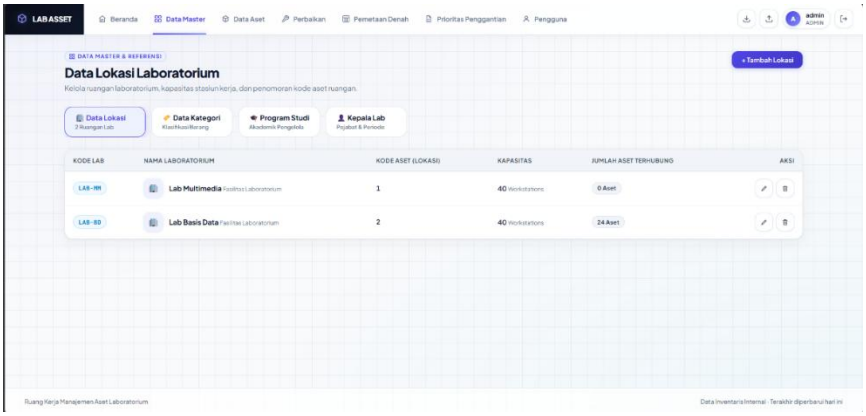



Gambar 5. 2 Halaman Dashboard untuk Admin

Sumber: Diolah oleh Penulis (2026)

Administrator mengakses dasbor utama pada rute /dashboard untuk memantau ringkasan portofolio laboratorium. Pengendali Flask secara real-time mengeksekusi agregasi SQL meliputi kalkulasi total investasi perolehan, estimasi depresiasi nilai buku, rekapitulasi biaya perbaikan, serta penarikan lima aset terbaru. Hasil pemrosesan diserialisasi ke pustaka Chart.js sehingga layar secara instan menampilkan dua grafik donat sebaran aset, kartu indikator finansial riil, dan daftar perangkat teranyar beserta laboratorium penempatannya.

c. Halaman Data Master



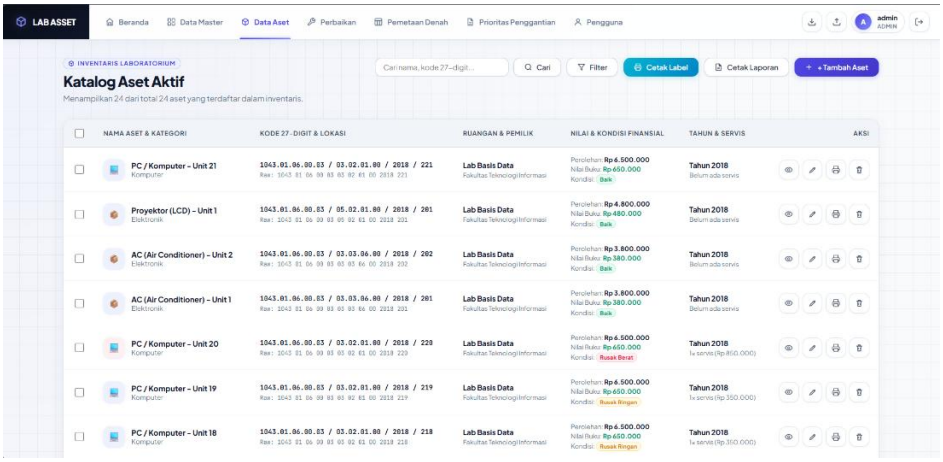

Gambar 5. 3 Halaman Data Master

Sumber: Diolah oleh Penulis (2026)

Admin mengelola data referensi kelembagaan dengan memilih tabulasi relevan seperti 'Ruangan Lab' dan mengisikan formulir modal penambahan entitas baru. Pengendali Flask pada berkas master.py memvalidasi keunikan kode lokasi, membungkus masukan ke dalam model ORM Location, lalu membukukannya ke basis data via db.session.commit(). Layar secara nyata memperlihatkan pembaruan tabel master dengan tombol sunting dan hapus, di mana sistem secara otomatis menolak penghapusan ruangan yang masih terikat dengan aset inventaris aktif.

d. Halaman Data Aset

Halaman Tabel Daftar Aset perspektif Admin merupakan pusat pengelolaan seluruh katalog inventaris laboratorium komputer. Antarmuka ini menyediakan fasilitas pencarian instan, filter lanjutan multi-kriteria, serta bilah aksi operasional barang.

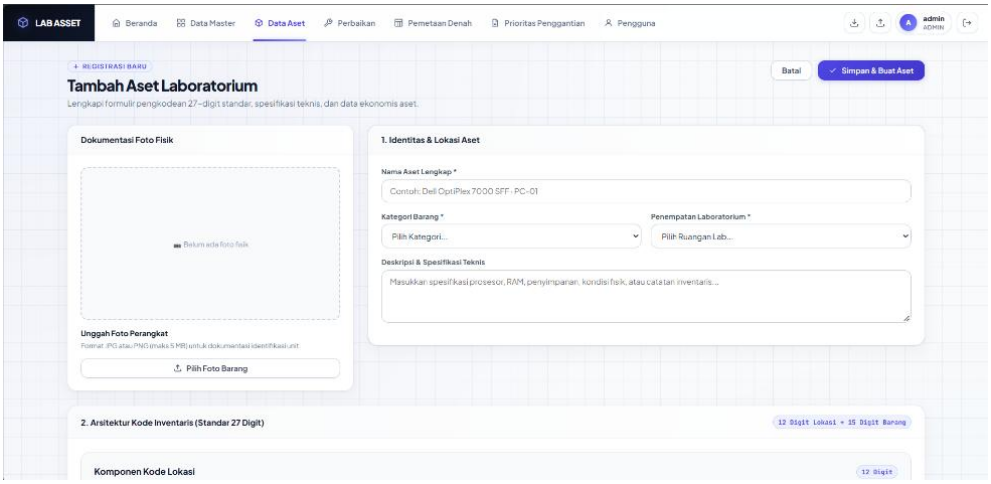



Gambar 5. 4 Halaman Data Aset

Sumber: Diolah oleh Penulis (2026)

Petugas laboratorium menelusuri katalog barang dengan memasukkan kata kunci nama aset atau memilih penapisan lokasi laboratorium dan kategori. Pengendali Flask pada rute /assets membangun kueri dinamis dengan klausa filter ilike() dan merekap paginasi data otomatis melalui .paginate(per_page=20). Layar menyajikan tabel data terstruktur yang menampilkan kode registrasi 27 digit berformat monospace, lencana kondisi fisik berkode warna, indikator jumlah rekaman aktif, serta barisan tombol aksi untuk mencetak label QR maupun laporan PDF.

e. Halaman Tambah Aset

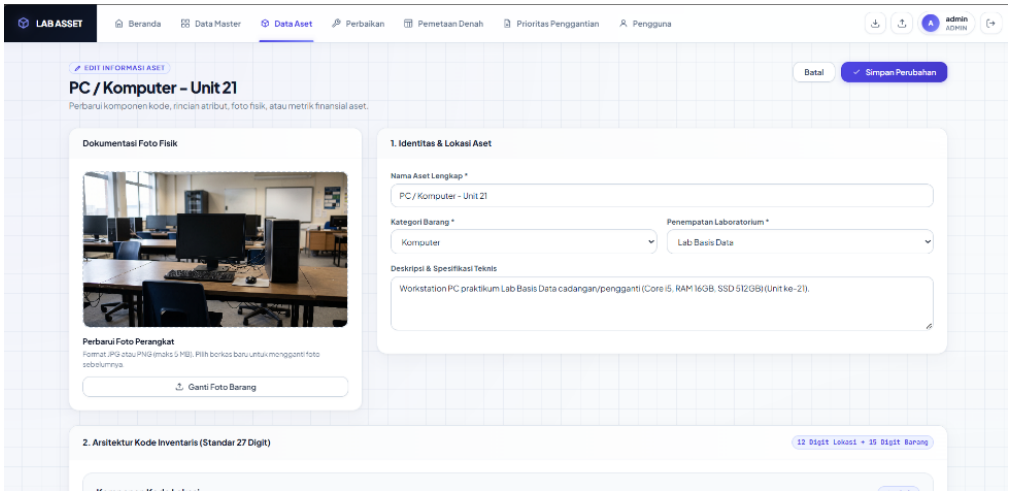



Gambar 5. 5 Halaman Tambah Aset

Sumber: Diolah oleh Penulis (2026)

Admin mendaftarkan komputer baru dengan mengisi rincian spesifikasi, segmen pembentuk kode 27 digit, parameter finansial, skor kerusakan awal, dan foto fisik. Pengendali Flask mengamankan berkas unggahan dengan secure_filename(), merangkai string kode registrasi, serta merekam objek Asset ke basis data. Pada siklus yang sama, fungsi generate_qr_code() mengeksekusi pustaka Python qrcode untuk memproduksi stiker matriks QR Code 2D di direktori statis, lalu membawa pengguna kembali ke katalog dengan notifikasi kilat hijau.

f. Halaman Edit Aset

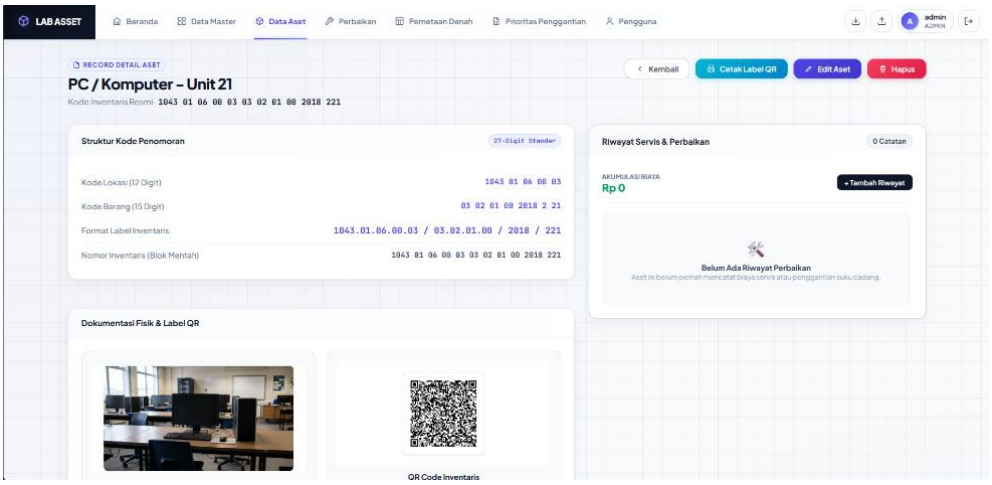



Gambar 5. 6 Halaman Edit Aset

Sumber: Diolah oleh Penulis (2026)

Melalui antarmuka sistem yang interaktif, administrator memiliki wewenang untuk memperbarui atribut spesifikasi teknis, kondisi fisik, maupun penempatan ruangan aset menggunakan formulir sunting yang telah memuat rincian data lama secara otomatis (*pre-filled*). Dari perspektif teknis, komponen pengendali (*controller*) Flask pada rute /assets/edit/<id> bertugas memproses pembaruan objek model aset, sekaligus memvalidasi kemungkinan adanya modifikasi pada komponen kode registrasi. Apabila sistem mendeteksi terjadinya perubahan struktur kode atau pemindahan lokasi, *backend* secara otomatis mengeksekusi ulang fungsi generate_qr_code() guna memperbarui citra *barcode* di peladen. Mekanisme pembaruan dinamis ini dirancang untuk memastikan representasi informasi fisik pada stiker *QR Code* selalu sinkron dan identik dengan rekaman data aset terbaru di dalam basis data.

g. Halaman Detail Aset

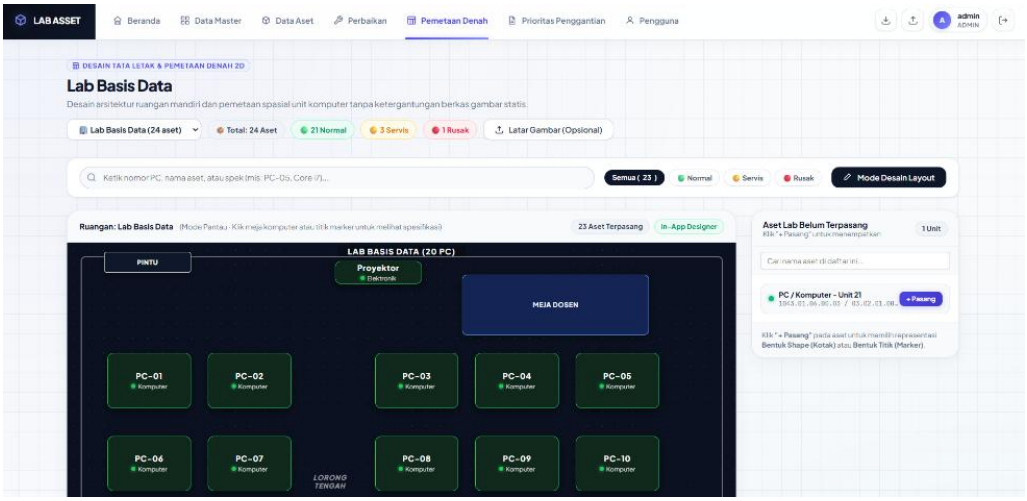



Gambar 5. 7 Halaman Detail Aset

Sumber: Diolah oleh Penulis (2026)

Pengguna memeriksa profil lengkap barang dengan mengklik nama aset pada katalog atau memindai barcode fisik QR Code. Pengendali Flask memuat objek model Asset, memanggil metode dinamis asset.nilai_buku() untuk menghitung depresiasi garis lurus, serta menarik kronologi pemeliharaan dari tabel Perbaikan. Layar secara nyata menampilkan foto dokumentasi berdampingan dengan gambar QR Code resolusi tinggi, kartu evaluasi penyusutan finansial, dan rekapitulasi total pengeluaran servis historis (kriteria C3).

h. Halaman Pemetaan Denah

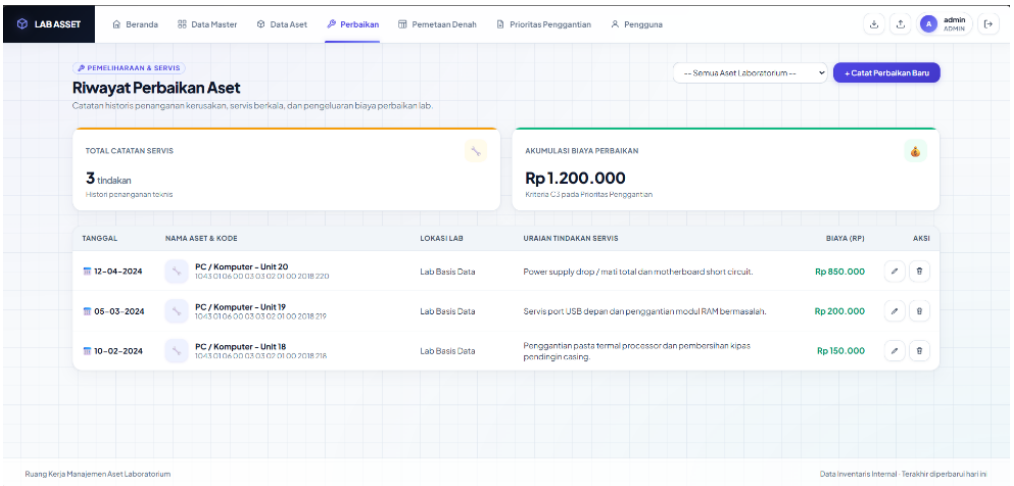



Gambar 5. 8 Halaman Pemetaan Denah

Sumber: Diolah oleh Penulis (2026)

Admin mengonfigurasi posisi fisik meja komputer dengan memilih laboratorium, mengaktifkan mode editor, lalu memindahkan posisi kotak PC-01 hingga PC-20 di atas kanvas denah secara seret dan lepas (*drag and drop*). Skrip JavaScript menghitung persentase koordinat spasial dan mengirirkannya via AJAX POST ke rute /api/floor-plan/save-layout. Pengendali Flask memperbarui kolom pos_x dan pos_y pada basis data, lalu mengembalikan respon sukses yang ditandai dengan munculnya pesan notifikasi konfirmasi di sudut layar.

i. Halaman Perbaikan Aset

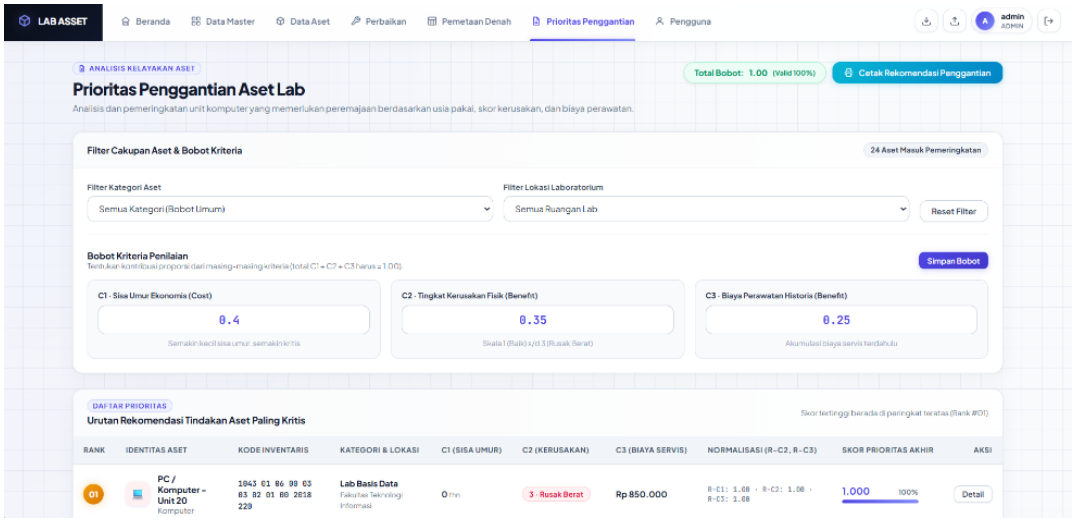



Gambar 5. 9 Halaman Perbaikan Aset

Sumber: Diolah oleh Penulis (2026)

Melalui antarmuka sistem, teknisi berwenang mencatat setiap riwayat transaksi pemeliharaan komputer dengan mengisi formulir *modal* yang memuat spesifikasi unit aset, tanggal tindakan, rincian uraian perbaikan, serta nominal biaya pengeluaran. Di sisi *backend*, komponen pengendali (*controller*) Flask pada rute /perbaikan/add bertugas memproses dan menyimpan rekaman data tersebut ke dalam tabel Perbaikan. Selanjutnya, sistem secara otomatis mengeksekusi fungsi internal _recalculate_biaya() guna memperbarui keseluruhan akumulasi biaya pemeliharaan aset secara permanen. Pada hasil akhirnya, antarmuka layar akan secara dinamis menampilkan penambahan *log* riwayat servis terbaru, pembaruan angka total pengeluaran kumulatif, sekaligus memicu penyesuaian langsung terhadap parameter kriteria biaya (C3) di dalam modul evaluasi komputasi SAW.

j. Halaman Prioritas Penggantian Aset

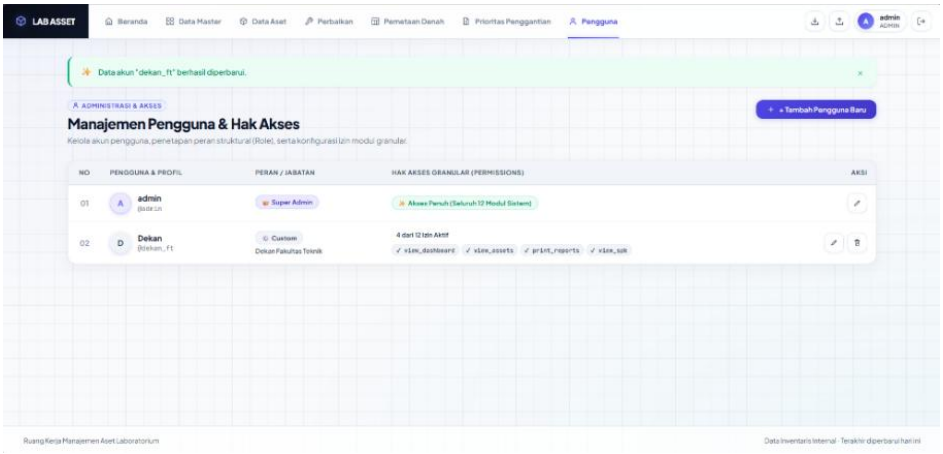



Gambar 5. 10 Halaman Prioritas Penggantian Aset

Sumber: Diolah oleh Penulis (2026)

Admin atau User meninjau analisis prioritas peremajaan fasilitas dan menyesuaikan proporsi bobot kriteria (C1 sisa umur, C2 kerusakan fisik, C3 biaya servis). Pengendali Flask pada berkas spk.py mengeksekusi algoritma SAW dengan menyusun matriks keputusan, menghitung normalisasi R, dan mengalikan bobot W untuk memperoleh nilai preferensi V. Layar menyajikan tabel perankingan terurut *descending* dari unit paling kritis serta menyediakan tombol pencetakan dokumen rekomendasi PDF.

k. Halaman Manajemen Pengguna

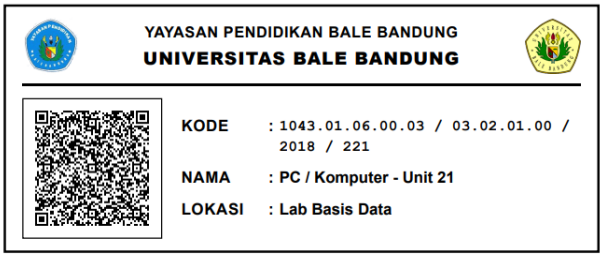



Gambar 5. 11 Halaman Manajemen Pengguna

Sumber: Diolah oleh Penulis (2026)

Melalui antarmuka manajemen sistem, administrator berwenang mengelola akun pengguna dengan melengkapi rincian identitas, menetapkan peran struktural, serta mengonfigurasi dua belas matriks izin akses granular pada formulir pendaftaran. Dari sisi *backend*, komponen pengendali (*controller*) Flask bertugas mengenkripsi kata sandi secara aman menggunakan fungsi generate_password_hash(). Sistem juga secara dinamis mengonversi daftar izin akses tersebut menjadi format *string* JSON untuk direkam pada kolom users.permissions di dalam basis data SQLite. Pada antarmuka layar, sistem memperlihatkan direktori akun terdaftar beserta lencana indikator total izin aktif. Lebih lanjut, sistem mengimplementasikan mekanisme proteksi keamanan mandiri yang secara otomatis menonaktifkan tombol hapus pada sesi akun yang sedang aktif digunakan.

l. Stiker Label Aset




Gambar 5. 12 Stiker Label Aset

Sumber: Diolah oleh Penulis (2026)

Untuk mencetak label fisik perangkat, petugas dapat mengklik tombol cetak pada antarmuka profil aset guna membuka rute /print/label/<id> yang berdimensi fisik 13.5 cm x 5.5 cm. Secara teknis, pengendali (*controller*) Flask bertugas merender berkas *template* print_label.html dengan memanggil fungsi asset.format_label(), sekaligus memicu eksekusi perintah JavaScript window.print(). Sebagai hasilnya, layar akan menampilkan format keluaran resmi yang memuat kop instansi FTI UNIBBA, matriks *QR Code* tajam berukuran 120 x 120 px di sisi kiri, serta rincian metadata kode registrasi 27 digit beserta nama laboratorium di sisi kanan.

m. Laporan Data Aset

**LAPORAN DATA ASET**

**Lab Basis Data**

Tanggal Cetak: 21 October 2026

| No | Kode Aset | Nama Aset | Tahun Perolehan | Kategori | Deskripsi | Kondisi | Foto Fisik | QR Code |
|---|---|---|---|---|---|---|---|---|
| 1 | 1043 01 06 00 03 03 03 06 00 2018 201 | AC (Air Conditioner) - Unit 1 | 2018 | Elektronik | Unit pendingin ruangan split wall 2 PK inverter (Unit ke-1). | Baik |  | |
| 2 | 1043 01 06 00 03 03 03 06 00 2018 202 | AC (Air Conditioner) - Unit 2 | 2018 | Elektronik | Unit pendingin ruangan split wall 2 PK inverter (Unit ke-2). | Baik |  | |
| 3 | 1043 01 06 00 03 05 02 01 00 2018 201 | Proyektor (LCD) - Unit 1 | 2018 | Elektronik | Proyektor LCD resolusi Full HD 3800 ANSI Lumens untuk presentasi dosen (Unit ke-1). | Baik |  | |
| 4 | 1043 01 06 00 03 03 02 01 00 2018 201 | PC / Komputer - Unit 1 | 2018 | Komputer | Workstation PC praktikum Lab Basis Data (Core i5, RAM 16GB, SSD 512GB - Sepaket Meja & Kursi) (Unit ke-1). | Baik |  | |
| 5 | 1043 01 06 00 03 03 02 01 00 2018 202 | PC / Komputer - Unit 2 | 2018 | Komputer | Workstation PC praktikum Lab Basis Data (Core i5, RAM 16GB, SSD 512GB - Sepaket Meja & Kursi) (Unit ke-2). | Baik |  | |

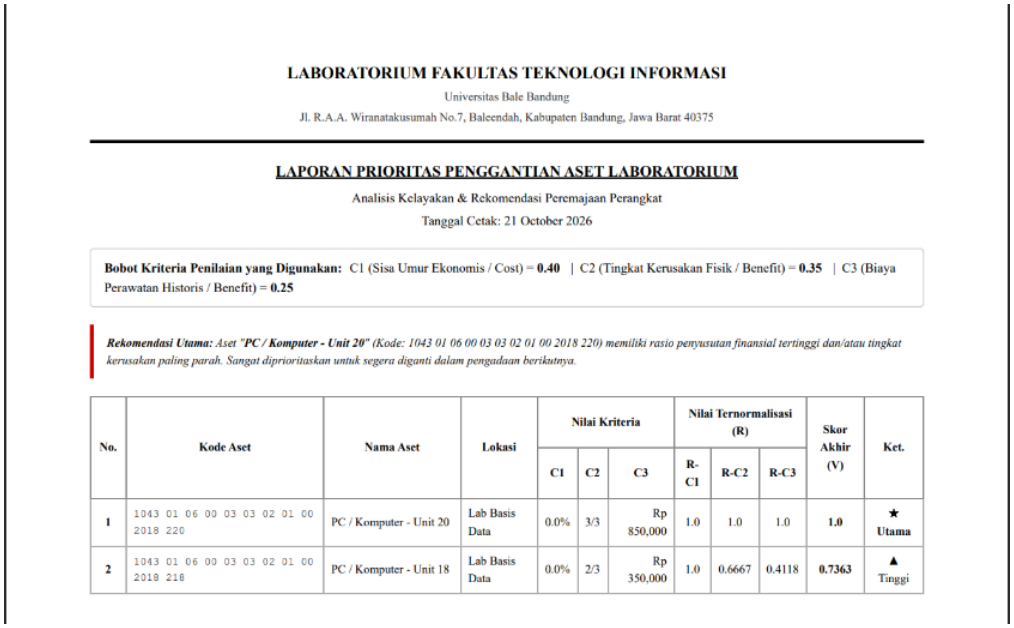
Gambar 5. 13 Laporan Data Aset

Sumber: Diolah oleh Penulis (2026)

Melalui antarmuka sistem, Administrator maupun *User* berwenang menerbitkan dokumen laporan resmi inventaris laboratorium melalui pemanggilan rute /print/inventory/<location_id>. Dari sisi *backend*, komponen pengendali Flask

secara dinamis mengekstraksi seluruh data aset yang terdaftar spesifik pada ruangan tersebut. Sistem otomatis mencatat tanggal pencetakan berdasarkan waktu *server*, sekaligus mengambil parameter nama dan NIK dari tabel basis data kepala_lab. Sebagai hasil akhir, dokumen keluaran menyajikan format administrasi perguruan tinggi baku yang memuat kepala laporan formal. Laporan ini turut menampilkan tabel inventaris sembilan kolom yang mencakup rincian kode registrasi 27 digit serta *thumbnail* foto atau matriks *QR Code*. Guna memastikan keabsahan administratif, sistem merender ruang lembar pengesahan resmi Kepala Laboratorium tepat di sudut kanan bawah dokumen.

n. Laporan Prioritas Penggantian Aset



**LABORATORIUM FAKULTAS TEKNOLOGI INFORMASI**

Universitas Bale Bandung

Jl. R.A.A. Wiranatakusumah No.7, Baleendah, Kabupaten Bandung, Jawa Barat 40375

**LAPORAN PRIORITAS PENGGANTIAN ASET LABORATORIUM**

Analisis Kelayakan & Rekomendasi Peremajaan Perangkat

Tanggal Cetak: 21 October 2026

**Bobot Kriteria Penilaian yang Digunakan:** C1 (Sisa Umur Ekonomis / Cost) = **0.40** | C2 (Tingkat Kerusakan Fisik / Benefit) = **0.35** | C3 (Biaya Perawatan Historis / Benefit) = **0.25**

***Rekomendasi Utama:*** *Aset* ***"PC / Komputer - Unit 20"*** *(Kode: 1043 01 06 00 03 03 02 01 00 2018 220) memiliki rasio penyusutan finansial tertinggi dan/atau tingkat kerusakan paling parah. Sangat diprioritaskan untuk segera diganti dalam pengadaan berikutnya.*

| No. | Kode Aset | Nama Aset | Lokasi | Nilai Kriteria | | | Nilai Ternormalisasi (R) | | | Skor Akhir (V) | Ket. |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | C1 | C2 | C3 | R-C1 | R-C2 | R-C3 | | |
| 1 | 1043 01 06 00 03 03 02 01 00 2018 220 | PC / Komputer - Unit 20 | Lab Basis Data | 0.0% | 3/3 | Rp 850,000 | 1.0 | 1.0 | 1.0 | **1.0** | ★ Utama |
| 2 | 1043 01 06 00 03 03 02 01 00 2018 218 | PC / Komputer - Unit 18 | Lab Basis Data | 0.0% | 2/3 | Rp 350,000 | 1.0 | 0.6667 | 0.4118 | **0.7363** | ▲ Tinggi |

Gambar 5. 14 Laporan Prioritas Penggantian Aset

Sumber: Diolah oleh Penulis (2026)

Kepala Lab menerbitkan dokumen rekomendasi peremajaan fasilitas dengan mengklik tombol cetak pada modul SAW menuju rute /spk/print. Pengendali Flask mengeksekusi komputasi algoritma SAW real-time, mengidentifikasi alternatif paling kritis, dan merender template spk_print.html dalam format A4 Landscape. Layar menampilkan kop surat resmi Fakultas Teknologi Informasi UNIBBA, tabel komputasi matematis berkepala ganda, sorotan unit komputer paling mendesak untuk diganti, serta kolom pengesahan pimpinan.

## 5.2 Pengujian

Tahap pengujian dan validasi pada Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA dilaksanakan secara komprehensif melalui tiga pilar evaluasi utama: Pengujian Fungsional (*Black-Box Testing*), Validasi Algoritma, serta Pengujian Penerimaan Pengguna (UAT). Struktur pengujian sistematis ini dirancang guna memvalidasi kelayakan perangkat lunak secara menyeluruh. Fokus evaluasinya mencakup pembuktian bebas cacat fungsional pada antarmuka (*client tier*), presisi logika komputasi metode *Straight Line* dan SAW di *backend*, hingga pengukuran indeks kepuasan pengguna akhir. Melalui integrasi ketiga instrumen tersebut, keabsahan sistem dapat dipertanggungjawabkan secara akademis guna menjamin solusi manajemen inventaris yang optimal dan bebas kesalahan komputasi di lingkungan nyata laboratorium.

### 5.2.1 Black-Box Testing

a. Skenario Kasus Uji (Test Case) via Teknik *Equivalence Partitioning*

Guna menghindari perulangan eksekusi uji coba yang tidak efisien atas kombinasi masukan data yang tak terbatas, konstruksi skenario pengujian pada penelitian ini dibangun mengandalkan metodologi *Equivalence Partitioning* (EP). Metodologi ini merupakan teknik pemetaan domain tes perangkat lunak yang membagi kemungkinan masukan data ke dalam kelas-kelas ekivalensi bersistem (equivalence classes), di mana masukan dalam kelas yang sama diperlakukan seragam oleh logika komputasi aplikasi. Secara prinsipil, teknik Equivalence Partitioning membelah partisi pengujian menjadi dua kategori yang saling bertolak belakang:

1. Partisi Kelas Valid (*Valid Equivalence Class*): Kombinasi masukan data yang sah, mematuhi parameter skema pangkalan data, dan berhak menerima izin pengesahan dari sistem untuk dievaluasi serta dipencarkan ke dalam tabel SQLite.
2. Partisi Kelas Tidak Valid (*Invalid Equivalence Class*): Kombinasi masukan data yang melanggar hukum validasi seperti duplikasi sandi kunci primer, formulir kosong, atau parameter numerik yang menyimpang yang wajib

dihadang oleh sistem dengan memunculkan notifikasi peringatan (*flash messaging/HTML5 form validation*) dan menolak eksekusi database.

Pemilihan sampel partisi data uji dari modul-modul interaktif utama aplikasi dirangkum pada tabel berikut

Tabel 5. 3 Matriks Uji Black-Box dengan Teknik Equivalence Partitioning

| No | Modul & Fitur Sistem | Partisi Masukan Valid (Valid Class) | Partisi Masukan Tidak Valid (Invalid Class) |
|---|---|---|---|
| 1 | Autentikasi Sistem (Formulir Login) | Memasukkan username dan password berstatus otorisasi akun terdaftar yang tercatat sah di dalam tabel users. | 1. Mengosongkan salah satu atau kedua kolom input login. 2. Memasukkan kombinasi kata sandi keliru ataupun akun asing yang tidak terdaftar di database. |
| 2 | Manipulasi Data Master (Contoh: Tambah Prodi) | Mendaftarkan sandi unik Kode Program Studi baru yang belum pernah menempati rekam jejak tabel prodi. | 1. Membiarkan field nama prodi dalam keadaan hampa. 2. Memaksakan input duplikasi atas Kode Prodi yang telah eksis sebelumnya (Primary Key duplication attempted). |
| 3 | Inventaris Aset & Stiker QR (Formulir Tambah Aset) | Mengisi lengkap seluruh referensi drop-down sandi institusi (untuk kodifikasi 27 digit), memilih skor kerusakan C2 (skala 1–3), dan menyemaikan harga beli serta umur ekonomis bernilai positif (> 0). | 1. Mengirimkan nilai minus/negatif (-) pada parameter form Harga Perolehan atau Umur Ekonomis. 2. Menyusupi karakter alfabet pada field bertipe numerik desimal. 3. Mengosongkan field wajib bertanda required. |
| 4 | Riwayat Pemeliharaan & Biaya (Formulir Tambah Servis) | Menginput tanggal servis, uraian tindakan perbaikan, nominal biaya perbaikan bernilai riil (≥ 0), dan nama teknisi pelaksana. | 1. Mengosongkan uraian kerusakan atau unit aset yang diservis. 2. Menginput nominal biaya perbaikan bernilai negatif (-). |
| 5 | Pemetaan Denah Visual 2D (Khusus Admin - Editor Layout) | Memilih laboratorium aktif, menata letak elemen/meja, menghubungkan unit PC ke workstation meja, dan menekan tombol "Simpan Denah". | 1. Menyimpan data koordinat yang korup atau data JSON kosong tanpa struktur. 2. Pengguna tanpa izin view_floor_plan mencoba menembus akses rute editor denah. |
| 6 | Konfigurasi Bobot SPK SAW (Formulir Kustomisasi Bobot) | Menyetel kombinasi rasio bobot kriteria (W1, W2, W3) yang memiliki nilai agregasi akumulasi tepat senilai mutlak 1,0 atau setara 100%. | Mengalokasikan kombinasi rasio bobot kriteria yang akumulasinya kurang atau melebihi batas 100% (Sebagai contoh: ∑W = 0,80 atau ∑W = 1,50). |
| 7 | Manajemen Pengguna & RBAC (Khusus Admin - Kelola Akun) | Mendaftarkan akun baru dengan username unik, memilih peran (Role), dan mencentang izin spesifik (dynamic permissions). | 1. Mendaftarkan username yang telah dipakai oleh pengguna lain. 2. Mengosongkan kata sandi pada pembuatan pengguna baru. 3. Akun non-admin |

| No | Modul & Fitur Sistem | Partisi Masukan Valid (Valid Class) | Partisi Masukan Tidak Valid (Invalid Class) |
|---|---|---|---|
| | | | (User/Dekan) mencoba mengakses rute /users. |

Sumber: Diolah oleh Penulis (2026)

b. Hasil Eksekusi Uji Fungsional (Black-Box Testing)

Setelah skenario kasus uji ditentukan berdasarkan partisi masukan data, pengujian diimplementasikan secara nyata melalui interaksi langsung terhadap form antarmuka aplikasi pada lingkungan peramban web Microsoft Edge. Evaluasi pengujian dilakukan dengan cara mencermati apakah *Hasil Pengamatan Eksekusi Sistem (Actual Output)* konsisten dan berselisih nol terhadap *Hasil yang Diharapkan (Expected Output)* dari perancangan awal.

Rincian rekapitulasi hasil eksekusi uji fungsional *Black-Box Testing* atas 12 (dua belas) skenario pengujian utama aplikasi distandarkan pada tabel berikut

Tabel 5. 4 Rekapitulasi Hasil Eksekusi Pengujian Fungsional

| No | Modul & Fitur | Skenario Input & Aksi | Hasil Diharapkan (Expected) | Hasil Pengamatan (Actual) | Kesimpulan |
|---|---|---|---|---|---|
| 1 | Autentikasi Sistem (Login - Partisi Valid) | Mengirimkan Username dan Password berotoritas Admin dari tabel users serta menekan tombol Login. | Sesi pengguna terbuka dan peramban mengarahkan rute navigasi menuju halaman Dashboard Utama dengan notifikasi berhasil. | Sistem membuatkan sesi pengakses dan beralih langsung menuju halaman Dashboard Utama sebagaimana diharapkan. | **Berhasil** |
| 2 | Autentikasi Sistem (Login - Partisi Tidak Valid) | Memasukkan kombinasi kata sandi yang salah atau mengosongkan kolom formulir login. | Sistem memblokir upaya masuk ke dasbor dan memantulkan pesan eror peringatan keamanan (Flash Warning). | Aplikasi menolak masuk, menahan operator di halaman login, dan menonjolkan pesan peringatan penolakan. | **Berhasil** |
| 3 | Manipulasi Data Master (CRUD Prodi - Partisi Valid) | Mendaftarkan parameter Kode Program Studi / Lokasi Ruangan baru yang valid melalui formulir tambah data. | Entitas acuan baru berhasil dikaitkan ke pangkalan data SQLite dan tersusun dalam direktori tabel data master. | Rekam jejak referensi baru tersimpan ke dalam tabel database dan langsung tampak di halaman daftar master. | **Berhasil** |
| 4 | Proteksi Anti-Duplikasi (Data Master - | Memaksakan pendaftaran sandi Kode Prodi yang telah terikat eksis | Transaksi komit pangkalan data dibatalkan dan sistem memancarkan | Sistem mencegah duplikasi Primary Key, menggagalkan penyimpanan, dan | **Berhasil** |

| | | | | | |
|---|---|---|---|---|---|
| | Partisi Tidak Valid) | sebelumnya pada basis data. | pesan penolakan: "Kode Prodi sudah ada!". | memaparkan pesan notifikasi duplikasi. | |
| 5 | Pendaftaran Aset & Generator QR (Form Tambah Aset - Partisi Valid) | Mengisi lengkap referensi drop-down sandi institusi, memilih skor kerusakan C2, dan mengisi nominal harga beli serta usia pakai. | Sistem menerbitkan rangkaian 27 Digit Kode Aset unik, mendaftarkan barang ke tabel assets, dan memicu ciptaan gambar stiker QR Code di direktori statis .png. | Kode sandi 27 digit menyatu dengan akurat, tabel terisi, dan foto stiker QR Code 2D terbit siap cetak pada menu Detail Aset. | **Berhasil** |
| 6 | Validasi Form Inventaris (Form Aset - Partisi Tidak Valid) | Menekan tombol Simpan ketika atribut wajib (required) seperti Nama Aset atau Harga Perolehan dibiarkan kosong hampa. | Web peramban melepaskan proteksi HTML5 Constraint Validation menolak penghantaran form dengan peringatan "Please fill out this field". | Formulir dibekukan oleh peramban secara otomatis dan meminta pengguna melengkapi field kosong sebelum dikomit. | **Berhasil** |
| 7 | Akuntansi Penyusutan Aset (Evaluasi Daftar Inventaris) | Menelusuri antarmuka halaman direktori tabel Daftar Aset laboratorium. | Sistem menjalankan method pemotongan nilai_buku() dan sisa_umur_ekonomis() secara aktual setiap tahun dari rumus Straight Line tanpa hitungan tangan manual. | Kolom Sisa Umur (dalam tahun) dan saldo Nilai Buku (dalam Rp) berangsur menyusut tepat waktu dan tertayang merata pada tabel. | **Berhasil** |
| 8 | Trigger Agregasi Kriteria C3 (Form Tambah Log Perbaikan) | Menyisipkan rekam medis perbaikan baru dengan mendaftarkan deskripsi servis serta nominal beban Biaya Perbaikan barang. | Sistem mencatat log reparasi dan menyalakan trigger _recalculate_biaya() yang merangkum akumulasi agregasi SQL SUM() ke dalam kolom biaya_perawatan aset induk. | Log servis baru tersurat dan total biaya perawatan di profil master aset terkait langsung meningkat secara seketika. | **Berhasil** |
| 9 | Pemetaan Denah Visual 2D (Khusus Admin - Sinkronisasi Meja) | Admin mengatur posisi meja PC pada kanvas denah 2D, menghubungkan unit PC-01 ke workstation, lalu menekan tombol "Simpan Denah". | API backend /api/floor-plan/save-layout menerima konfigurasi JSON, menyimpan struktur ke Location.layout_config, dan memperbarui koordinat pos_x | Konfigurasi denah tersimpan, koordinat posisi aset terupdate di database, dan indikator warna kondisi (Hijau/Kuning/Merah) tampil sesuai skor kerusakan. | **Berhasil** |

| | | | | | |
|---|---|---|---|---|---|
| | | | serta pos_y pada tabel assets. | | |
| 10 | Manajemen Otorisasi Pengguna (Khusus Admin - Pengelolaan Akun) | Admin mendaftarkan akun baru melalui modul /users/add, menentukan peran (Role), dan mengatur izin akses dinamis (Permissions). | Akun baru terenkripsi Bcrypt tersimpan ke tabel users, hak akses terserialisasi dalam format JSON, dan pengguna baru dapat login dengan wewenang terbatas sesuai izin. | Akun pengguna baru berhasil dibuat, terverifikasi ke tabel users, dan berhasil login sesuai pembatasan hak akses yang ditetapkan. | **Berhasil** |
| 11 | Proteksi Hak Akses Berbasis Peran (Otorisasi RBAC - Pelanggaran Akses) | Pengguna dengan peran User (Dekan) mencoba mengakses rute khusus Admin (misal: rute /users atau /perbaikan/add). | Dekorator @permission_required menolak akses, menerbitkan pesan peringatan "Akses Ditolak: Anda tidak memiliki hak akses...", dan mengarahkan kembali ke Dashboard. | Rute admin terlindungi secara ketat; pengguna non-admin dialihkan ke Dashboard dengan notifikasi penolakan akses berwarna merah. | **Berhasil** |
| 12 | Analisis Rekomendasi SPK SAW & Cetak PDF (Eksekusi Menu SPK & Laporan) | Mengakses halaman SPK SAW, menelaah filter Kategori Aset, mengevaluasi bobot (W1, W2, W3), lalu menekan tombol Cetak Laporan SPK. | Sistem menayangkan Matriks Normalisasi Rij, mengurutkan Nilai Preferensi Akhir Vi, serta menerbitkan dokumen Print Preview PDF bersertifikasi pengesahan Kepala Lab. | Perhitungan SAW tereksekusi akurat, alternatif kritis menduduki peringkat #1, dan dokumen PDF siap cetak memuat legalitas tanda tangan resmi. | **Berhasil** |

Sumber: Diolah oleh Penulis (2026)

### 5.2.2 Validasi Perhitungan Matematis

Bagian ini menjelaskan alur teknis dan tahapan pembabakan perhitungan numerik secara manual yang menjadi fondasi verifikasi atas algoritma komputasi di dalam sistem aplikasi. Pengujian tahapan perhitungan ini dibedah ke dalam dua alur proses utama, yaitu Tahapan Perhitungan Akuntansi (Depresiasi Garis Lurus / *Straight Line Method*) dan Tahapan Perhitungan Sistem Pendukung Keputusan (Metode *Simple Additive Weighting* / SAW).

Akibat terbatasnya dokumen historis aset di lapangan, parameter harga perolehan, biaya servis, Nilai Residu, dan skor kerusakan diestimasi menggunakan

data simulasi (*dummy data*) guna memvalidasi ketepatan formula Garis Lurus dan algoritma SAW pada sistem.

Tabel 5. 5 Data Sampel Alternatif Aset Laboratorium

| Kode Alt. | Nama Perangkat | Kode Registrasi (27 Digit) | Tahun Beli | Harga Perolehan (HP) | Nilai Residu (NR) | Umur Ekonomis (UE) | Skor Kerusakan (C2) | Biaya Servis (C3) |
|---|---|---|---|---|---|---|---|---|
| A1 | PC / Komputer - Unit 20 | 1043 01 06 00 03 03 02 01 00 2018 220 | 2018 | Rp 6.500.000 | Rp 650.000 | 5 Tahun | 3 (Rusak Berat) | Rp 850.000 |
| A2 | PC / Komputer - Unit 18 | 1043 01 06 00 03 03 02 01 00 2018 218 | 2018 | Rp 6.500.000 | Rp 650.000 | 5 Tahun | 2 (Rusak Ringan) | Rp 350.000 |
| A3 | PC / Komputer - Unit 1 | 1043 01 06 00 03 03 02 01 00 2018 201 | 2018 | Rp 6.500.000 | Rp 650.000 | 5 Tahun | 1 (Baik) | Rp 0 |

Sumber: Diolah oleh Penulis (2026)

A. Tahapan Perhitungan Akuntansi Penyusutan Aset (Straight Line)

Perhitungan akuntansi bertujuan memetakan depresiasi linier serta sisa umur pakai perangkat melalui 4 (empat) langkah sistematis:

Tahap 1: Identifikasi Parameter dan Variabel Input

Sebelum kalkulasi dilakukan, sistem memuat parameter dari tabel database:

1. Harga Perolehan (HP) = 6.500.000
2. Nilai Residu (NR) = Rp 650.000
3. Umur Ekonomis (UE) = 5 tahun
4. Tahun Pengadaan ($T_{beli}$) = 2018, Tahun Evaluasi ($T_{eval}$) = 2026

Tahap 2**:** Perhitungan Beban Penyusutan per Tahun ($D_{tahun}$)

1. Formula Garis Lurus: $D_{tahun}$ = (HP - NR) / UE
2. Perhitungan per alternatif (A1, A2, A3):

   $D_{tahun}$ = (Rp 6.500.000 - Rp 650.000) / 5 = Rp 5.850.000 / 5 = Rp 1.170.000 / tahun (berlaku sama untuk A1, A2, dan A3).

Tahap 3**:** Perhitungan Masa Pakai ($M_{pakai}$) dan Sisa Umur Ekonomis ($S_{umur}$)

1. Masa Pakai:

   Mpakai = Teval - Tbeli = 2026 - 2018 = 8 tahun.

2. Sisa Umur Ekonomis:

   $S_{umur}$ = max(UE - $M_{pakai}$, 0) = max(5 - 8, 0) = max(-3, 0) = 0 tahun (berlaku sama untuk A1, A2, dan A3 karena masa manfaat telah habis).

Tahap 4: Perhitungan Nilai Buku Terkini (NB)

1. Formula Nilai Buku:

   NB = max(HP - (D_tahun x M_pakai), NR)

   NB = max(Rp 6.500.000 - (Rp 1.170.000 x 8), Rp 650.000) = max(Rp 6.500.000 - Rp 9.360.000, Rp 650.000) = Rp 650.000 (berlaku sama untuk A1, A2, dan A3).

B. Tahapan Perhitungan Metode Simple Additive Weighting

Algoritma SAW dijalankan untuk menyusun rekomendasi prioritas peremajaan perangkat laboratorium yang paling kritis. Tahapan pemrosesan matriks SAW ini diuraikan melalui 6 (enam) langkah komputasi terstruktur sebagai berikut:

Tahap 1: Penentuan Kriteria Keputusan dan Sifat Kriteria

1. Kriteria C1 (Rasio Sisa Umur Ekonomis) [Cost]:
   Formula $x_{i1}$ = $S_{umur}$ / UE.
   Semakin kecil rasio umur tersisa, semakin usang perangkat sehingga prioritas peremajaan semakin tinggi.
2. Kriteria C2 (Skor Kerusakan Fisik) [Benefit]:
   Formula $x_{i2}$ = skor fisik (1 = Baik, 2 = Rusak Ringan, 3 = Rusak Berat).
   Semakin besar skor, semakin parah kerusakan.
3. Kriteria C3 (Akumulasi Biaya Perawatan) [Benefit]:
   Formula $x_{i3}$ = total biaya servis (Rp).
   Semakin besar biaya perawatan, semakin tidak efisien unit dipertahankan.

Tahap 2: Penentuan Vektor Bobot Preferensi Kriteria (W)

Vektor Bobot: W = [w1, w2, w3] = [0,40; 0,35; 0,25]

(Bobot C1 = 40%, C2 = 35%, C3 = 25%, total = 100%).

Tahap 3: Pembentukan Matriks Keputusan Mentah (X)

Data masukan per alternatif:

1. Alternatif A1 (PC Unit 20): C1 = 0/5 = 0,0000; C2 = 3; C3 = 850.000
2. Alternatif A2 (PC Unit 18): C1 = 0/5 = 0,0000; C2 = 2; C3 = 350.000

3. Alternatif A3 (PC Unit 1): C1 = 0/5 = 0,0000; C2 = 1; C3 = 0

Tabel 5. 6 Matriks Keputusan Mentah X (ukuran 3 x 3)

| Alternatif | C1 (Sisa Umur) | C2 (Kerusakan) | C3 (Biaya Servis) |
|---|---|---|---|
| A1 (PC Unit 20) | 0,0000 | 3 | Rp 850.000 |
| A2 (PC Unit 18) | 0,0000 | 2 | Rp 350.000 |
| A3 (PC Unit 1) | 0,0000 | 1 | Rp 0 |

Sumber: Diolah oleh Penulis (2026)

Tahap 4**:** Ekstraksi Nilai Ekstremum per Kolom Kriteria

1. Kolom C1 (Cost -> min): min($x_{i1}$) = min(0,0000; 0,0000; 0,0000) = 0,0000
2. Kolom C2 (Benefit -> max): max($x_{i2}$) = max(3; 2; 1) = 3 (pada A1)
3. Kolom C3 (Benefit -> max): max($x_{i3}$) = max(850.000; 350.000; 0) = 850.000 (pada A1)

Tahap 5: Normalisasi Matriks Keputusan (R)

Formula Normalisasi:

1. Cost: $r_{ij}$ = min / $x_{ij}$ (jika $x_{ij}$ = 0 karena sisa umur habis, ditetapkan $r_{i1}$ = 1,0000 sebagai prioritas tertinggi).
2. Benefit: $r_{ij}$ = $x_{ij}$ / max

Hasil Perhitungan Normalisasi:

1. Kolom C1 (Cost): $r_{11}$ = 1,0000; $r_{21}$ = 1,0000; $r_{31}$ = 1,0000
2. Kolom C2 (Benefit): $r_{12}$ = 3/3 = 1,0000; $r_{22}$ = 2/3 = 0,6667; $r_{32}$ = 1/3 = 0,3333
3. Kolom C3 (Benefit): $r_{13}$ = 850.000/850.000 = 1,0000; $r_{23}$ = 350.000/850.000 = 0,4118; $r_{33}$ = 0/850.000 = 0,0000

Tabel 5. 7 Matriks Normalisasi (R)

| Alternatif | r_i1 (C1 - Cost) | r_i2 (C2 - Benefit) | r_i3 (C3 - Benefit) |
|---|---|---|---|
| A1 (PC Unit 20) | 1,0000 | 1,0000 | 1,0000 |
| A2 (PC Unit 18) | 1,0000 | 0,6667 | 0,4118 |
| A3 (PC Unit 1) | 1,0000 | 0,3333 | 0,0000 |

Sumber: Diolah oleh Penulis (2026)

Tahap 6: Perhitungan Vektor Nilai Preferensi Akhir ($V_i$) dan Penentuan Peringkat

Formula Preferensi: Vi = (w1 x $r_{i1}$) + (w2 x $r_{i2}$) + (w3 x $r_{i3}$)

1. Alternatif A1 (PC Unit 20): $V_1$ = (0,40 x 1,0000) + (0,35 x 1,0000) + (0,25 x 1,0000) = 0,4000 + 0,3500 + 0,2500 = 1,0000 (Peringkat I - Prioritas Kritis)
2. Alternatif A2 (PC Unit 18): $V_2$ = (0,40 x 1,0000) + (0,35 x 0,6667) + (0,25 x 0,4118) = 0,4000 + 0,2333 + 0,1030 = 0,7363 (Peringkat II - Prioritas Menengah)
3. Alternatif A3 (PC Unit 1): $V_3$ = (0,40 x 1,0000) + (0,35 x 0,3333) + (0,25 x 0,0000) = 0,4000 + 0,1167 + 0,0000 = 0,5167 (Peringkat III - Prioritas Rendah)

Urutan Prioritas Rekomendasi: A1 (1,0000) > A2 (0,7363) > A3 (0,5167). Unit PC Unit 20 (A1) yang mengalami kerusakan berat dan biaya servis tertinggi menempati Peringkat 1 sebagai kandidat utama penggantian.

C. Evaluasi Validasi Hasil (Tahapan Manual vs Keluaran Sistem Aplikasi)

Komparasi langsung antara hasil perhitungan manual terhadap keluaran aktual sistem aplikasi disajikan pada tabel berikut.

Tabel 5. 8 Validasi Akuntansi Penyusutan

| Kode Alt. | Nama Perangkat Aset | Sisa Umur (Manual) | Sisa Umur (Aplikasi) | Nilai Buku (Manual) | Nilai Buku (Aplikasi) | Selisih Galat | Status Validasi |
|---|---|---|---|---|---|---|---|
| A1 | PC / Komputer - Unit 20 | 0 Tahun | 0 Tahun | Rp 650.000 | Rp 650.000 | 0,00% | VALID (100%) |
| A2 | PC / Komputer - Unit 18 | 0 Tahun | 0 Tahun | Rp 650.000 | Rp 650.000 | 0,00% | VALID (100%) |
| A3 | PC / Komputer - Unit 1 | 0 Tahun | 0 Tahun | Rp 650.000 | Rp 650.000 | 0,00% | VALID (100%) |

Sumber: Diolah oleh Penulis (2026)

Tabel 5. 9 Validasi Algoritma SAW

| Kode Alt. | Nama Perangkat Aset | Skor V_i (Manual) | Skor V_i (Aplikasi) | Peringkat Manual | Peringkat Aplikasi | Selisih Galat | Status Validasi |
|---|---|---|---|---|---|---|---|
| A1 | PC / Komputer - Unit 20 | 1,0000 | 1,0000 | Peringkat 1 (Kritis) | Peringkat 1 (Kritis) | 0,0000 | VALID (100%) |
| A2 | PC / Komputer - Unit 18 | 0,7363 | 0,7363 | Peringkat 2 (Menengah) | Peringkat 2 (Menengah) | 0,0000 | VALID (100%) |
| A3 | PC / Komputer - Unit 1 | 0,5167 | 0,5167 | Peringkat 3 (Rendah) | Peringkat 3 (Rendah) | 0,0000 | VALID (100%) |

Sumber: Diolah oleh Penulis (2026)

### 5.2.3 *User Acceptance Testing*

Pengujian kelayakan sistem ini mengandalkan metode *User Acceptance Testing* (UAT) untuk melihat sejauh mana aplikasi dapat diterima dan berfungsi dengan baik dari kacamata pengguna akhir (*end-user*). Proses pengujian ini melibatkan 3 (tiga) orang responden, yakni 1 (satu) Kepala Laboratorium Komputer FTI UNIBBA yang bertindak sebagai pengambil keputusan, serta 2 (dua) Asisten Laboratorium selaku operator harian yang menjalankan sistem secara langsung.

Untuk mengumpulkan data penilaian, digunakan instrumen berupa kuesioner terstruktur dengan skala Likert 5 tingkat. Kuesioner ini dirancang untuk menilai empat aspek utama kualitas aplikasi: Kemudahan (*Usability*), Fungsionalitas (*Functionality*), Kinerja (*Performance*), dan Kepuasan Pengguna (*Satisfaction*). Rincian data penilaian dari ketiga responden terhadap setiap butir pertanyaan dapat dilihat pada Tabel 5.1 berikut.

Tabel 5. 10 Distribusi Jawaban Responden Pengujian UAT

| **Aspek Pengujian** | **No.** | **Pernyataan Instrumen Evaluasi** | **R1** | **R2** | **R3** | **Skor Aktual** | **Skor Ideal** | **Persentase** |
|---|---|---|---|---|---|---|---|---|
| **Kemudahan (*Usability*)** | P1 | Tampilan antarmuka (*user interface*) dasbor aplikasi menarik, rapi, dan mudah dipahami oleh pengguna. | 3 | 4 | 4 | 11 | 15 | 73,33% |
| | P2 | Sistem navigasi menu aplikasi sangat jelas dan mempermudah operasional pemindahan halaman. | 4 | 5 | 4 | 13 | 15 | 86,67% |
| **Fungsionalitas (*Functionality*)** | P3 | Sistem berhasil mengonversi parameter data input secara otomatis menjadi 27 digit Kode Aset unik dan *QR Code*. | 5 | 5 | 4 | 14 | 15 | 93,33% |
| | P4 | Sistem mampu melakukan kalkulasi estimasi penyusutan aset (*Straight Line*) | 5 | 5 | 4 | 14 | 15 | 93,33% |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | dan pemeringkatan keputusan (SAW) secara otomatis. | | | | | | |
| **Kinerja (*Performance*)** | P5 | Kecepatan pemrosesan data, eksekusi kalkulasi algoritma, dan penayangan halaman cetak laporan berjalan dengan responsif. | 5 | 5 | 4 | 14 | 15 | 93,33% |
| | P6 | Penyimpanan, pembaruan, dan penghapusan data log perbaikan aset pada basis data SQLite berjalan dengan stabil tanpa galat. | 5 | 5 | 4 | 14 | 15 | 93,33% |
| **Kepuasan (*Satisfaction*)** | P7 | Aplikasi secara efektif membantu menyederhanakan alur kerja pengelolaan inventaris aset laboratorium dibandingkan metode manual. | 4 | 5 | 4 | 13 | 15 | 86,67% |
| | P8 | Hasil pemeringkatan kelayakan aset dari sistem mempermudah pengelola dalam merekomendasikan keputusan pengadaan aset baru secara objektif. | 3 | 5 | 4 | 12 | 15 | 80,00% |
| **Total Keseluruhan** | | | | | | 105 | 120 | 87,50% |

Sumber: Diolah oleh Penulis (2026)

(Keterangan Responden: R1 = Mahasiswa, R2 = Kepala Lab, R3 = Mahasiswa 2)

Berdasarkan sebaran jawaban pada Tabel 5.8 di atas, langkah selanjutnya adalah menghitung rekapitulasi persentase kelayakan untuk setiap parameter pengujian. Hasil rekapitulasi per aspek ini kemudian dirangkum pada Tabel 5.9 di bawah ini.

Tabel 5. 11 Rekapitulasi Persentase Hasil Kelayakan UAT per Aspek

| No. | Aspek Pengujian Kualitas | Total Skor Aktual | Total Skor Ideal | Persentase Kelayakan Aspek | Kategori Kelayakan |
|---|---|---|---|---|---|
| 1 | Kemudahan (*Usability*) | 24 | 30 | 80,00% | Layak / Baik |
| 2 | Fungsionalitas (*Functionality*) | 28 | 30 | 93,33% | Sangat Layak / Sangat Baik |
| 3 | Kinerja (*Performance*) | 28 | 30 | 93,33% | Sangat Layak / Sangat Baik |
| 4 | Kepuasan Pengguna (*Satisfaction*) | 25 | 30 | 83,33% | Sangat Layak / Sangat Baik |
| **Rata-rata Keseluruhan** | | **105** | **120** | | |

Sumber: Diolah oleh Penulis (2026)

Berikut adalah uraian analisis deskriptif dari masing-masing aspek yang telah diuji:

a. Pada Aspek Kemudahan (*Usability*), sistem meraih persentase 80,00% yang masuk dalam kategori "Baik". Angka ini membuktikan bahwa penataan menu pada *dashboard*, kejelasan alur navigasi, hingga tata letak tombol operasional sudah cukup ergonomis. Pengelola dapat menjalankan aplikasi ini dengan lancar tanpa butuh pelatihan teknis yang rumit.
b. Untuk Aspek Fungsionalitas (*Functionality*), sistem mencetak skor kelayakan tinggi sebesar 93,33% dengan kategori "Sangat Baik". Hal ini menunjukkan kepuasan penuh pengguna terhadap akurasi fitur-fitur inti aplikasi. Mulai dari perakitan 27 digit Kode Aset via Python Flask, pembuatan stiker *QR Code*, kelancaran *database* SQLite, hitungan penyusutan *Straight Line*, hingga perangkingan rekomendasi peremajaan via metode SAW semuanya dinilai mampu berjalan mulus tanpa adanya kesalahan logika komputasi.
c. Pada Aspek Kinerja (*Performance*), kelayakan sistem juga menyentuh angka 93,33% atau "Sangat Baik". Pencapaian ini menjadi bukti nyata bahwa aplikasi bekerja dengan sangat efisien. Baik saat memproses data masukan, merender dokumen laporan PDF, maupun ketika menyimpan riwayat servis perbaikan ke dalam *database*, sistem meresponsnya dengan cepat tanpa ada kendala keterlambatan (*delay*).

d. Sementara itu, Aspek Kepuasan Pengguna (*Satisfaction*) berhasil memperoleh nilai 83,33% dengan kategori "Sangat Baik". Hasil ini menggambarkan bahwa kehadiran aplikasi web ini benar-benar membawa dampak positif. Aplikasi dinilai sukses memangkas kerumitan birokrasi pencatatan inventaris fisik di UNIBBA, sekaligus mempermudah Kepala Laboratorium dalam menyusun rekomendasi pengadaan perangkat baru secara transparan, adil, dan objektif.

Secara keseluruhan, jika dihitung secara kumulatif dari semua aspek pengujian tersebut, nilai akhir *User Acceptance Testing* (UAT) untuk sistem ini mencapai 87,50%. Merujuk pada standar interval kelayakan, angka tersebut menempati rentang tertinggi (81% – 100%). Oleh karena itu, dapat ditarik kesimpulan meyakinkan bahwa Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA dinyatakan Sangat Layak dan Sangat Diterima untuk diimplementasikan sepenuhnya pada kegiatan operasional sehari-hari.

# BAB VI

# KESIMPULAN DAN SARAN

## 6.1 Kesimpulan

Setelah melewati seluruh rangkaian penelitian mulai dari proses analisis, perancangan, pengembangan aplikasi menggunakan Python Flask dan SQLite, hingga uji coba langsung di Laboratorium Komputer FTI UNIBBA terdapat beberapa kesimpulan utama yang dapat ditarik:

a. Digitalisasi dan Kodifikasi Aset: Aplikasi manajemen aset berbasis web ini sukses mengotomatisasi perakitan 27 digit Kode Aset yang unik. Dilengkapi dengan fitur pembuat stiker *QR Code* secara dinamis, sistem ini terbukti efektif dalam menekan risiko data hilang, mencegah pencatatan ganda, serta mempercepat pencarian profil komputer secara langsung di lapangan.
b. Pencatatan Riwayat Servis yang Terpusat: Sistem telah menghadirkan modul riwayat perawatan yang terhubung langsung dengan basis data utama. Fitur ini sangat andal untuk mendokumentasikan rincian kerusakan, memantau status perbaikan, sekaligus menghitung akumulasi total biaya servis setiap unit komputer secara otomatis dan *real-time*.
c. Objektivitas dalam Kebijakan Peremajaan: Aplikasi ini berhasil memadukan metode perhitungan akuntansi *Straight Line* (untuk menghitung depresiasi nilai dan sisa umur perangkat) dengan algoritma evaluasi *Simple Additive Weighting* (SAW). Hasilnya, sistem mampu menyajikan rekomendasi perangkat mana yang paling mendesak untuk diganti secara transparan, adil, dan berlandaskan hitungan matematis yang pasti.
d. Tingkat Penerimaan Pengguna yang Sangat Baik: Pengujian kelayakan (*User Acceptance Testing*) yang melibatkan Kepala Laboratorium, beserta dua mahasiswa menunjukkan hasil yang sangat memuaskan. Kuesioner evaluasi mencatat skor kelayakan sebesar 87,50% ("Sangat Layak" / "Sangat Baik"), yang mengonfirmasi bahwa aplikasi ini sudah benar-benar siap dan bisa diandalkan untuk kegiatan operasional harian.

## 6.2 Saran

Walaupun aplikasi ini sudah terbukti berjalan baik dan siap pakai, masih ada beberapa ruang pengembangan yang sangat direkomendasikan untuk menyempurnakan sistem ini di masa mendatang:

a. Migrasi ke Basis Data Skala Besar: Jika nantinya sistem ini ingin diadopsi secara luas untuk mengelola seluruh inventaris di tingkat fakultas atau bahkan universitas, disarankan untuk beralih dari pangkalan data SQLite lokal ke sistem berbasis *server-cloud* yang lebih tangguh (seperti MySQL atau PostgreSQL). Langkah ini penting untuk menjaga kestabilan aplikasi saat diakses oleh banyak pengguna secara bersamaan.
b. Sistem Notifikasi Perawatan Otomatis: Akan sangat membantu jika sistem dibekali fitur pengingat atau notifikasi, misalnya terintegrasi dengan WhatsApp atau *email*. Fitur ini nantinya akan otomatis mengirimkan pesan kepada asisten laboratorium saat sebuah komputer sudah mendekati jadwal perawatan rutin atau ketika kondisi fisiknya sudah memasuki status kritis.

# DAFTAR PUSTAKA


Abass, A. (2023). An Overview of Software Development Life Cycle (SDLC). *International Journal of Science and Research (IJSR)*, *12*(5), 1782–1784. https://doi.org/10.21275/sr23520153358

Adi Kurniyanti, V., & Murdiani, D. (2022). Perbandingan Model Waterfall Dengan Prototype Pada Pengembangan System Informasi Berbasis Website. *Jurnal Syntax Fusion*, *2*(08), 669–675. https://doi.org/10.54543/fusion.v2i08.210

Afandi, M. F., & Nasution, M. I. P. (2023). Sistem Informasi Manajemen Aset Bendung Serdang BWS Sumatera II Menggunakan Metode Straight Line dan Simple Additive Weighting. *Jurnal Teknologi dan Manajemen Informatika*, *9*(1), 56–67. https://doi.org/10.26905/JTMI.V9I1.10107

Akhirina, T. Y., Yulistyanti, D., Rusmardiana, A., & Pauziah, U. (2018). Pengujian Sistem Pendukung Keputusan Penjurusan SMA di Banten menggunakan Metode Black Box. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, *2*(3), 800–806. https://siap-ppdb.com/banten.

Akmal, F. M., & Suhada, J. (2025). Perancangan Basis Data: Konsep Entitas, Atribut, dan Relasi. *BIIKMA : Buletin Ilmiah Ilmu Komputer dan Multimedia*, (3), 473–478. https://jurnalmahasiswa.com/index.php/biikma

Aliyah, Hartono, N., & Muin, A. A. (2024). Penggunaan User Acceptance Testing (UAT) Pada Pengujian Sistem Informasi Pengelolaan Keuangan Dan Inventaris Barang. *Switch : Jurnal Sains dan Teknologi Informasi*, *3*(1), 84–100. https://doi.org/10.62951/switch.v3i1.330

Andrianto, D., Muhammad Jumasa, H., & Yunia Pasa, I. (2022). *Sistem Informasi Pengelolaan Inventaris Laboratorium Komputer Di SMK Muhammadiyah Purworejo*. *5*.

Arfani, A. R. Y., Kasih, P., & Pamungkas, D. P. (2021). Pengujian Aplikasi Presensi dengan Black box Testing dengan Metode Equivalence Partitioning dan Boundary Value Analysis. *Seminar Nasional Inovasi Teknologi*.

Augusto, D., Nathanael, R., & Wicaksono, S. R. (2026). Prosiding Seminar Nasional Universitas Ma Chung 2026 (Sistem Informasi dan Informatika)

RANCANG BANGUN SISTEM PENDATAAN DAN PENCATATAN INVENTARIS LABORATORIUM SMPK SANTA MARIA 2 MALANG 1. *Prosiding SENAM 2026: Seminar Nasional Sistem Informasi dan Teknik Informatika Universitas Ma Chung*, (1).

Christopher, M., Kizito, A. E., & Augustine, N. G. (2026). An Intelligent Role-Based Access Control Model Enhanced with Risk-Based Multi-Factor Authentication. *INTERNATIONAL JOURNAL OF RESEARCH AND INNOVATION IN APPLIED SCIENCE (IJRIAS)*, *9*(4). https://doi.org/10.51584/IJRIAS

Daniati, T., & Sapitri, D. (2025). PERANCANGAN DATABASE SISTEM INFORMASI MANAJEMEN PENJUALAN TOKO AFRIDA CAKE MENGGUNAKAN ERD. *Jurnal Sistem Informasi (TEKNOFILE)*, *3*(9), 656–662.

Dari, R. W., Sapriadi, S., & Syaputra, A. E. (2024). Penerapan Metode Simple Additive Weighting dalam Pengambilan Keputusan Penentuan Penerima Beasiswa Perguruan Tinggi. *Jurnal KomtekInfo*, 64–70. https://doi.org/10.35134/komtekinfo.v11i2.507

Dilla, J. S., & Jupron. (2026). SISTEM PENDUKUNG KEPUTUSAN KELAYAKAN PENERIMA BANTUAN SOSIAL MENGGUNAKAN METODE SAW BERBASIS WEB (STUDI KASUS: KECAMATAN PONDOK AREN). *Jurnal Manajemen Informatika, Sistem Informasi dan Teknologi Komputer (JUMISTIK)*, *5*(1), 863–872. https://doi.org/10.70247/jumistik.v5i1.314

Ekaputra, B. A., Pradana, F., & Pramono, D. (2020). *Pengembangan Sistem Management Dalam Melakukan Penugasan Berbasis Web*. *4*(9), 2791–2800. http://j-ptiik.ub.ac.id

Erik, Putra, F., & Yusran, Y. (2025). RANCANG BANGUN SISTEM INFORMASI INVENTARIS BERBASIS WEB PADA LABORATORIUM UNDHARI. *Jurnal Ilmiah Sistem Informasi*, *4*(3), 351–366. https://doi.org/10.51903/aybzsc29

Evan Adicandra, Yulindon Yulindon, Ratna Dewi, & Silfia Rifka. (2026). Implementasi Autentikasi dan Otorisasi pada Sistem Informasi Berbasis Web.

*JURNAL ILMIAH PENELITIAN MAHASISWA*, *4*(2), 289–298. https://doi.org/10.61722/jipm.v4i2.2273

Evan, D. J., Ocsa, P., & Saian, N. (2023). IMPLEMENTASI PYTHON FRAMEWORK FLASK PADA MODUL TRANSFER OUT TOKO DI PT XYZ. *JIPI (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika)*, *8*(4), 1121–1131. https://doi.org/10.29100/JIPI.V8I4.4020

Farinha, J. T., Raposo, H. D. N., de-Almeida-e-Pais, J. E., & Mendes, M. (2023). Physical Asset Life Cycle Evaluation Models—A Comparative Analysis towards Sustainability. *Sustainability 2023, Vol. 15, Page 15754*, *15*(22), 15754. https://doi.org/10.3390/SU152215754

Firliani, A. S., & Bakti, A. M. (2022). Penerapan Algoritma Reed Solomon Pada Sistem Informasi Absensi Pegawai Diskominfo Lahat. *SMATIKA JURNAL*, *12*(02), 156–164. https://doi.org/10.32664/smatika.v12i02.695

Fitrani, L. D., & Puspitaningrum, A. C. (2023). Utilization of Unified Modeling Language (UML) in the Design of Academic Information Systems based on the OOAD Method. *SISTEMASI: Jurnal Sistem Informasi*, *12*, 614–625. http://sistemasi.ftik.unisi.ac.id

Gaffney, K. P., Hipp, D. R., Prammer, M., Kennedy, D., Brasfield, L., & Patel, J. M. (2022). SQLite: Past, Present, and Future. *Proceedings of the VLDB Endowment*, *15*(12), 3535–3547. https://doi.org/10.14778/3554821.3554842

Irianto, Syahputra, A. K., & Sumantri. (2026). Perancangan Sistem Pendukung Keputusan Adaptif untuk Prioritas Penanganan Perkara Hukum. *Management of Information System Journal*, *4*(3), 350–359. https://doi.org/10.47065/mis.v2i3.3050

Juliartha, B., Putra, M., Fu'adi, A., Ariani, D., Yuniarti, F., Komputer, P., Jaringan, D., Komunitas, A., Pacitan, N., & Walanda, J. (2022). Analisa dan Rancangan Sistem Informasi Pariwisata Pacitan dengan UML dan ERD. *INFORMATION SYSTEM FOR EDUCATORS AND PROFESSIONALS*, *7*(1), 63–72.

Jung, H., & Kim, B. (2021). Identifying Research Topics and Trends in Asset Management for Sustainable Use: A Topic Modeling Approach. *Sustainability 2021, Vol. 13, Page 4792*, *13*(9), 4792. https://doi.org/10.3390/SU13094792

Kastella, T. B., & Ujianto, E. I. H. (2025). Penerapan Teknologi QR Code untuk Meningkatkan Efisiensi Absensi Karyawan Berbasis Android. *TIN: Terapan Informatika Nusantara*, *6*(7), 859–871. https://doi.org/10.47065/tin.v6i7.8735

Mairuhu, S., & Tinangon, J. J. (2014). ANALISIS PENERAPAN METODE PENYUSUTAN AKTIVA TETAP DAN IMPLIKASINYA TERHADAP LABA PERUSAHAAN PADA PERUM BULOG DIVRE SULUT DAN GORONTALO. *Jurnal EMBA*, *2*(4), 404–412.

Narulita, S., Nugroho, A., & Abdillah, M. Z. (2024). Diagram Unified Modelling Language (UML) untuk Perancangan Sistem Informasi Manajemen Penelitian dan Pengabdian Masyarakat (SIMLITABMAS). *Bridge : Jurnal Publikasi Sistem Informasi dan Telekomunikasi*, *2*(3), 244–256. https://doi.org/10.62951/BRIDGE.V2I3.174

Ningsih, W., & Nurfauziah, H. (2023). PERBANDINGAN MODEL WATERFALL DAN METODE PROTOTYPE UNTUK PENGEMBANGAN APLIKASI PADA SISTEM INFORMASI. *Jurnal Ilmiah Metadata*, *5*(1), 83–95.

Nurfadillah, N., Maulana, R., & Syahbuddin, S. (2026). Penerapan User Acceptance Testing (UAT) dalam Evaluasi Sistem Informasi Pencarian & Pemesanan Rumah Kost menggunakan Progressive Web App Berbasis Geospasial. *Switch : Jurnal Sains dan Teknologi Informasi*, *4*(3), 01–14. https://doi.org/10.62951/switch.v4i3.849

Nurzanah, A. S., & Yun, Y. (2023). Analisis Pemeliharaan Fasilitas Komputer di Laboratorium Rekayasa Perangkat Lunak (RPL) dengan Pendekatan Pemeliharaan Logic Tree Analysis pada SMK Sangkuriang 1 Cimahi. *Portofolio: Jurnal Ekonomi, Bisnis, Manajemen dan Akuntansi*, *20*.

Nyabuto, Mr. G. M. (2024). Architectural Review of Client-Server Models. *International Journal of Scientific Research and Engineering Trends*, *10*(1), 139–143. https://doi.org/10.61137/IJSRET.VOL.10.ISSUE1.126

Pargaonkar, S. (2023). A Comprehensive Research Analysis of Software Development Life Cycle (SDLC) Agile & Waterfall Model Advantages, Disadvantages, and Application Suitability in Software Quality Engineering. *International Journal of Scientific and Research Publications*, *13*(8), 120–124. https://doi.org/10.29322/ijsrp.13.08.2023.p14015

Pitrawati, & Sanjaya, A. (2021). Rekayasa Perangkat Lunak Perhitungan Harga Pokok Produksi Metode Full Costing Pada UMKM Mitra Cake Di Bandar Lampung. *Jurnal Informasi dan Komputer*, *9*(2), 154–162. https://doi.org/10.35959/JIK.V9I2.243

Purnama, Y., Sokibi, P., & Parman, S. (2024). RANCANG BANGUN SISTEM INFORMASI MANAJEMEN ASET MENGGUNAKAN METODE STRAIGHT LINE (Studi Kasus: SMK Samudra Nusantara Cirebon). *Jurnal Informatika dan Teknik Elektro Terapan*, *12*(1). https://doi.org/10.23960/jitet.v12i1.3599

Putra, D. J. (2025). Rancang Bangun Sistem Informasi Manajemen dan Monitoring Aset IT Laboratorium Komputer Berbasis Web. *J-SIGN (Journal of Informatics, Information System, and Artificial Intelligence)*, *3*(01), 1–15. https://doi.org/10.24815/j-sign.v3i01.43291

Ramachandrappa, N. C. (2024). SOLID Design Principles in Software Engineering. *International Journal of Computer Trends and Technology*, *72*(9), 18–23. https://doi.org/10.14445/22312803/ijctt-v72i9p104

Ramadhan, A. R., Naya, C., & AG, A. (2026). Implementasi Metode TOPSIS pada Sistem Pendukung Keputusan Penentuan Prioritas Penerima Bantuan Sosial Berbasis Aplikasi Desktop. *JURNAL FASILKOM*, *16*(1).

Ramadhani, F., Rianti, A., Alpin Gustio, D., Annisa, L., & Amal Bakti, S. (2026). Inspirasi Edukatif: Jurnal Pembelajaran Aktif ANALISIS ENTITAS, ATRIBUT, DAN RELASI PADA ENTITY RELATIONSHIP DIAGRAM (ERD): LITERATUR REVIEW. *Inspirasi Edukatif: Jurnal Pembelajaran Aktif*, *7*(1). https://ejurnals.com/ojs/index.php/

Rasiban, Septiansyah, A., Hasanah, S., Permatasari, V. N., & Yuliawati, A. (2024). SISTEM INFORMASI OTOMATISASI PELAPORAN DATA PENJUALAN TOKO BUKU NAZWA YANG MASUK DAN YANG KELUAR. *IKRAITH-INFORMATIKA*, *8*(1), 279–292. https://doi.org/10.37817/ikraith-informatika.v8i1

Ratnasari, Zulkifli, Triloka, J., Bintoro, P., Ramaputra, M. G., & Ardhy, F. (2022). Penerapan Model Waterfall Pada Pengembangan Aplikasi Pengajuan Bantuan Sosial Berbasis Web. *Syntax: Jurnal Informatika*, *11*(02), 34–45.

Risma, A., Saputra, H. K., Hadi, A., & Novaliendry, D. (2026). Penerapan User Acceptance Testing (UAT) pada Pengujian Sistem Informasi Manajemen Operasional Studio Foto Berbasis Client-Server. *Journal of Authentic Research*, *5*(2), 2396–2410. https://doi.org/10.36312/svscqg16

Rozakh, M., Nurcahyo, G. W., Sovia, R., & Rahman, Z. (2025). Penerapan Metode Simple Additive Weighting dan Fuzzy Logic dalam Menganalisa Mitigasi Risiko. *Jurnal KomtekInfo*, 31–37. https://doi.org/10.35134/komtekinfo.v12i1.621

Sadewa, G., Idrus, M., Ferawati, A., Tinggi, S., Yapi, I. E., Artikel, R., Kunci, K., Penyusutan, :, Tetap, A., & Keuangan, A. (2024). ANALISIS PENERAPAN METODE PENYUSUTAN AKTIVA TETAP BERDASARKAN STANDAR AKUNTANSI KEUANGAN PADA KALLA TOYOTA CABANG BONE. Dalam *Maret* (Vol. 2, Nomor 1). www.ojs.amiklps.ac.id

Saharuddin, & Prihatmono, M. W. (2022). *PENGENALAN DAN PELATIHAN DASAR BAHASA PEMROGRAMAN PYTHON PADA SISWA/I SMA NEGERI 3 MAKASSAR*. *6*.

Sahyudi, M., & Susanto, E. R. (2025). Analisis Implementasi Sistem Keamanan Basis Data Berbasis Role-Based Access Control (RBAC) pada Aplikasi Enterprise Resource Planning. *SATESI: Jurnal Sains Teknologi dan Sistem Informasi*, *5*(1), 105–116. https://doi.org/10.54259/satesi.v5i1.3997

Saputra, H. E., & Rasiban. (2024). Implementation of an Asset Management System Using the Straight-Line Method of Depreciation Based on Odoo 14 CE at PT Forecastle Indonesia. *International Journal Software Engineering and Computer Science (IJSECS)*, *4*(3), 916–927. https://doi.org/10.35870/ijsecs.v4i3.2975

Sharabiani, A. M. A., & Mousavi, S. M. (2023). A Web-Based Decision Support System for Project Evaluation with Sustainable Development Considerations Based on Two Developed Pythagorean Fuzzy Decision Methods. *Sustainability 2023, Vol. 15, Page 16477*, *15*(23), 16477. https://doi.org/10.3390/SU152316477

Sholeh, M., Gisfas, I., Cahiman, & Fauzi, M. A. (2021). Black Box Testing on ukmbantul.com Page with Boundary Value Analysis and Equivalence

Partitioning Methods. *Journal of Physics: Conference Series*, *1823*(1). https://doi.org/10.1088/1742-6596/1823/1/012029

Siahaan, T. L., Zaidiah, A., & Isnainiyah, I. N. (2021). Sistem Pendukung Keputusan Penghapusan Aset Menggunakan Metode Simple Additive Weighting Berbasis Web Pada PT. Andaru Arti Agung. *Seminar Nasional Mahasiswa Ilmu Komputer dan Aplikasinya (SENAMIKA) Jakarta-Indonesia*.

Solihin, I. P., & Adli, R. (2023). *SISTEM ABSENSI MAHASISWA MENGGUNAKAN QR CODE DENGAN METODE ZXING BERBASIS ANDROID*. *7*.

Sorongan, D. L., Kainde, Q. C., & Kumajas, S. C. (2025). Web-Based Decision Support System for Determining the Best Employee Using the Simple Additive Weighting Method. *Indonesian Journal of Innovation Studies*, *26*(4). https://doi.org/10.21070/ijins.v26i4.1835

Sudarman, L., Lelawatty, P., Sarimuddin, & Zunais, S. N. (2024). Optimalisasi Pengendalian Manajemen Aktiva Tetap dalam Meningkatan Efektivitas Penghitungan dan Penghapusan Aset Menggunakan Metode Straight Line dan Fuzzy Edas. *SemanTIK : Teknik Informasi*, *10*(2).

Sudiarti, Y. E., & Wahab, F. (2025). PERANCANGAN DATABASE SISTEM PENGOLAHAN NILAI MAHASISWA DI UNIVERSITAS ISLAM INDRAGIRI MENGGUNAKAN ERD. *Jurnal Sistem Informasi (TEKNOFILE)*, *3*, 97–104.

Suraya, S., & Sholeh, M. (2022). Designing and Implementing a Database for Thesis Data Management by Using the Python Flask Framework. *International Journal of Engineering, Science and Information Technology*, *2*(1), 9–14. https://doi.org/10.52088/IJESTY.V2I1.197

Suryanovi, S. (2023). URGENSI PENERAPAN NILAI RESIDU DALAM PENYUSUTAN ASET TETAP PEMERINTAH. *JURNAL PENDIDIKAN AKUNTANSI INDONESIA*, *21*, 81–97.

Sutiyono, & Salman Ramadan, F. (2025). RANCANG BANGUN SISTEM INFORMASI MENEJEMEN INVENTARIS LABOLATORIUM KOMPUTER BERBASIS WEB DENGAN INTEGRASI RADIO

FREQUENCI IDENTIFICATION (RFID). *Jurnal Sistem Informasi, J-SIKA*, *7*(1).

Taryono, C. L., Muis, S., & Irawan, A. (2024a). Perancangan Sistem Informasi Manajemen Aset Kampus Stmik Kuwera Berbasis Web Menggunakan Metode Simple Additive Weighting (SAW) Dan Straight Line. *Jurnal Sistem Informasi dan Teknologi (SINTEK)*, *4*(1), 1–12. https://doi.org/10.56995/SINTEK.V4I1.63

Taryono, C. L., Muis, S., & Irawan, A. (2024b). PERANCANGAN SISTEM INFORMASI MANAJEMEN ASET KAMPUS STMIK KUWERA BERBASIS WEB MENGGUNAKAN METODE SIMPLE ADDITIVE WEIGHTING (SAW) DAN STRAIGHT LINE. *JURNAL SISTEM INFORMASI DAN TEKNOLOGI (SINTEK)*, *6*(1). https://sintek.stmikku.ac.id/index.php/home

Triansyah, M. A., Wijayanto, B., & Pradana Yosua, P. B. (2026). Optimalisasi Manajemen Aset dan Pelaporan Kerusakan Laboratorium Komputer Melalui Sistem Monitoring Berbasis Web. *Citizen : Jurnal Ilmiah Multidisiplin Indonesia*, *6*(1), 290–302. https://doi.org/10.53866/jimi.v6i1.1204

Wicaksono, A. R. (2025). Analisis Kinerja Pengujian Black Box Terhadap Website Nestify Setelah Implementasi Equivalence Partitioning dan Boundary Value Analysis. *Jurnal Informatika dan Teknologi Pendidikan*, *5*(1), 77–98. https://doi.org/10.59395/jitp.v5i1.131

Wilyani, F., Arif, Q. N., & Aslimar, F. (2024). Pengenalan Dasar Pemrograman Python Dengan Google Colaboratory. *Jurnal Pelayanan dan Pengabdian Masyarakat Indonesia*, *3*(1), 08–14. https://doi.org/10.55606/jppmi.v3i1.1087

# LAMPIRAN

## Lampiran 1: Pedoman Wawancara

Narasumber: Sutiyono, S.T., M.Kom.

Tanggal: 20 Juni 2026

### I. Usulan Penelitian

Saya memohon izin kepada Kepala Laboratorium Komputer FTI UNIBBA untuk melanjutkan hasil Kerja Praktik yang telah dilakukan sebelumnya. Saya mengusulkan untuk pengembangan aplikasi manajemen aset adalah dengan menambahkan modul evaluasi kelayakan pakai aset. *(Kepala Laboratorium menyetujui).*

### II. Pertanyaan

**1. Bagaimana pengelolaan terkait evaluasi kelayakan aset di laboratorium ini? Apakah sebelumnya sudah ada aplikasi yang digunakan untuk keperluan tersebut?**

*Jawaban:* Sebetulnya aplikasi pencatatan aset dari pihak yayasan ini sudah ada, namun pada praktiknya sistem tersebut belum berjalan atau gagal diimplementasikan. Oleh karena itu, kita ingin mencoba membuat dan mengelolanya secara mandiri.

**2. Bagaimana cara Bapak memutuskan jika sebuah aset masih layak dipakai atau sudah tidak layak?**

*Jawaban:* Evaluasi kelayakan biasanya dilakukan dengan cara melihat umur aset tersebut. Yang Bapak tahu, kalau komputer itu umurnya kurang lebih 5 tahun. Tapi di sini, meskipun umurnya sudah mencapai 5 tahun dan secara hitungan sudah termasuk usang, jika masih bisa dipakai untuk praktikum, maka aset tersebut masih akan terus berjalan dan belum akan diganti.

**3. Lalu, bagaimana cara Bapak mengkategorikan sebuah aset agar dicatat dan dilaporkan sebagai aset yang benar-benar "tidak layak pakai"?**

*Jawaban:* Aset dicatat secara manual lalu baru dilaporkan sebagai aset tidak layak pakai jika aset tersebut sudah sepenuhnya mati (misalnya komputer sama sekali tidak bisa menyala atau tidak berjalan).

**4. Bagaimana prosedur pencatatan dan pelaporan mengenai kondisi aset-aset laboratorium tersebut dilakukan selama ini?**

*Jawaban:* Catatan kondisi aset direkap dan dilaporkan secara manual setiap satu tahun sekali. Terkait pencatatan dan pelaporan ini, salah satu masalah dari penggunaan *spreadsheet* yang sedang terjadi adalah kami selalu membuat catatan

yang baru dikarenakan sering kesulitan mencari file catatan sebelumnya. Istilahnya, daripada lama mencari data catatan sebelumnya untuk digabungkan, lebih baik membuat catatan baru saja dari awal.

**5. Pada aplikasi evaluasi kelayakan ini, nantinya sistem akan menghasilkan peringkat prioritas untuk menentukan aset mana yang paling mendesak untuk diganti. Dasar penilaian untuk menghasilkan peringkat tersebut menggunakan tiga kriteria. Saya mengusulkan ketiga kriteria tersebut meliputi: Sisa Umur Ekonomis, Tingkat Kerusakan Fisik, dan Biaya Perawatan Historis. Bagaimana tanggapan Bapak mengenai usulan kriteria tersebut?**

*Jawaban: (Bapak Kepala Laboratorium menyetujui usulan ketiga kriteria tersebut untuk digunakan sebagai acuan parameter di dalam sistem).*

**6. Untuk perhitungan evaluasinya nanti, bagaimana arahan Bapak terkait penentuan bobot dari ketiga kriteria tersebut? Mengingat fokus utama penelitian ini adalah pada penyusutan umur aset, saya memohon izin untuk menetapkan bobot kriteria Sisa Umur Ekonomis lebih besar dibandingkan dua kriteria lainnya. Bagaimana menurut Bapak?**

*Jawaban:* Boleh saja jika bobot kriteria sisa umur ekonomis ditetapkan lebih besar. Untuk dua kriteria lainnya, silakan Anda sesuaikan bobotnya berdasarkan referensi. Namun, arahan dari saya, pembobotan kriteria tersebut harus dibuat spesifik berdasarkan kategori asetnya. Sebagai contoh, standar bobot kriteria untuk kategori PC tentu akan memiliki rincian yang berbeda dengan kategori UPS. Jadi, silakan saja bobot umur ekonomis dibuat lebih tinggi dan disesuaikan nilainya, asalkan dibedakan per kategori perangkat.

| Pewawancara | Narasumber |
| --- | --- |
| Andhika Maulana Shiddiq | Sutiyono, S.T., M.Kom. |

**Lampiran 2: Term of Reference**

Pengelolaan aset laboratorium yang baik sangat penting untuk mendukung kelancaran kegiatan praktikum mahasiswa. Namun, perangkat komputer yang sudah usang sering kali terlambat terdeteksi, sehingga berisiko mengganggu perkuliahan dan memicu pemborosan anggaran. Untuk mengatasi kendala tersebut, penelitian di Laboratorium Komputer Fakultas Teknologi Informasi (FTI) Universitas Bale Bandung ini bertujuan merancang sebuah Aplikasi Manajemen Aset berbasis web. Aplikasi ini tidak sekadar mencatat inventaris barang, tetapi juga mampu menilai kelayakan pakai alat secara otomatis. Proses evaluasi kelayakan ini dilakukan dengan memadukan metode *Straight Line* untuk menghitung sisa umur ekonomis perangkat secara presisi, serta metode *Simple Additive Weighting* (SAW) untuk menyusun urutan prioritas peremajaan alat. Dengan demikian, pengelola dapat memperoleh rekomendasi objektif mengenai perangkat mana yang paling mendesak untuk segera diganti.

Agar pengembangan aplikasi ini tetap terarah dan berfokus pada tujuan utama penelitian, ditetapkan beberapa batasan masalah sebagai berikut:

1. Evaluasi kelayakan aset hanya difokuskan pada perangkat keras (*hardware*) yang sudah terdaftar secara resmi di Laboratorium Multimedia dan Laboratorium Basis Data FTI UNIBBA.
2. Pengembangan aplikasi mengandalkan *microframework* Flask dengan bahasa pemrograman Python, serta menggunakan SQLite sebagai media penyimpanan basis datanya.
3. Tahap implementasi dan pengujian sistem dibatasi hanya pada lingkungan server lokal (*localhost*), sehingga tidak mencakup peluncuran (*deployment*) ke *hosting* publik maupun konfigurasi jaringan untuk banyak pengguna dalam skala luas.

Bandung, 20 Juni 2026

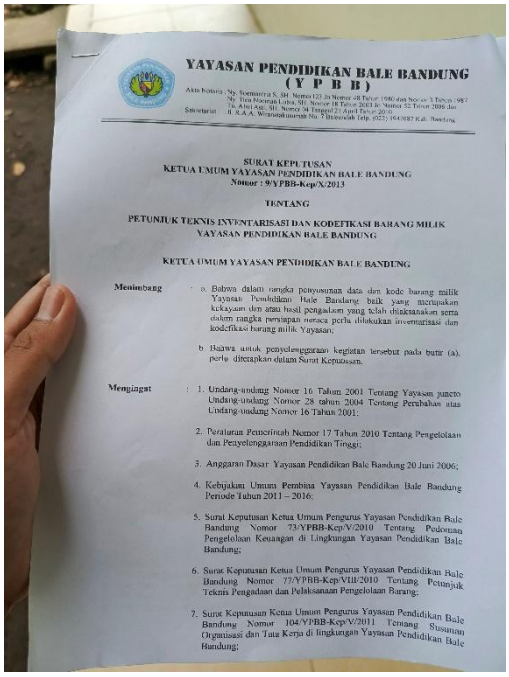
Disetujui oleh:

| Mahasiswa | Kepala Labkom FTI |
| --- | --- |
| Andhika Maulana Shiddiq | Sutiyono, S.T., M.Kom. |

## Lampiran 3: Dokumentasi







YAYASAN PENDIDIKAN BALE BANDUNG
( Y P B B )

SURAT KEPUTUSAN
KETUA UMUM YAYASAN PENDIDIKAN BALE BANDUNG
Nomor : 9/YPBB-Kep/X/2013

TENTANG

PETUNJUK TEKNIS INVENTARISASI DAN KODEFIKASI BARANG MILIK YAYASAN PENDIDIKAN BALE BANDUNG

KETUA UMUM YAYASAN PENDIDIKAN BALE BANDUNG

Menimbang : a. Bahwa dalam rangka penyusunan data dan kode barang milik Yayasan Pendidikan Bale Bandung baik yang merupakan kekayaan dan atau hasil pengadaan yang telah dilaksanakan serta dalam rangka persiapan neraca perlu dilakukan inventarisasi dan kodefikasi barang milik Yayasan;

b. Bahwa untuk penyelenggaraan kegiatan tersebut pada butir (a), perlu ditetapkan dalam Surat Keputusan.

Mengingat : 1. Undang-undang Nomor 16 Tahun 2001 Tentang Yayasan juncto Undang-undang Nomor 28 tahun 2004 Tentang Perubahan atas Undang-undang Nomor 16 Tahun 2001;

2. Peraturan Pemerintah Nomor 17 Tahun 2010 Tentang Pengelolaan dan Penyelenggaraan Pendidikan Tinggi;

3. Anggaran Dasar Yayasan Pendidikan Bale Bandung 20 Juni 2006;

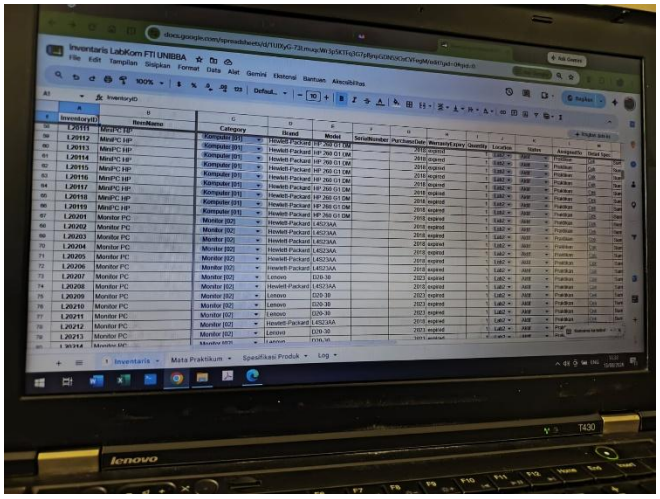
4. Kebijakan Umum Pembina Yayasan Pendidikan Bale Bandung Periode Tahun 2011 – 2016;

5. Surat Keputusan Ketua Umum Pengurus Yayasan Pendidikan Bale Bandung Nomor 73-YPBB-Kep/V/2010 Tentang Pedoman Pengelolaan Keuangan di Lingkungan Yayasan Pendidikan Bale Bandung;

6. Surat Keputusan Ketua Umum Pengurus Yayasan Pendidikan Bale Bandung Nomor 73/YPBB-Kep/VIII/2010 Tentang Petunjuk Teknis Pengadaan dan Pelaksanaan Pengelolaan Barang;

7. Surat Keputusan Ketua Umum Pengurus Yayasan Pendidikan Bale Bandung Nomor 104/YPBB-Kep/V/2011 Tentang Susunan Organisasi dan Tata Kerja di lingkungan Yayasan Pendidikan Bale Bandung;

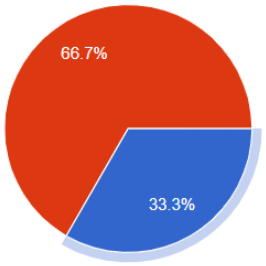


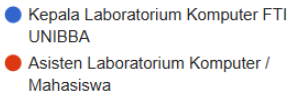


# Kuesioner Pengujian Aplikasi Manajemen Aset Laboratorium Komputer FTI UNIBBA (Metode User Acceptance Testing)

Yth. Bapak/Ibu/Rekan Pengelola Laboratorium Komputer FTI UNIBBA,
Kuesioner ini dirancang untuk mengukur tingkat penerimaan pengguna (*User Acceptance Testing* / UAT) terhadap Aplikasi Manajemen Aset berbasis web yang telah dikembangkan. Evaluasi ini menggunakan standar kualitas perangkat lunak yang berfokus pada aspek Kemudahan (*Usability*), Fungsionalitas (*Functionality*), Kinerja (*Performance*), dan Kepuasan Pengguna (*Satisfaction*).
Mohon kesediaan Anda untuk memberikan penilaian secara objektif setelah mencoba mengoperasikan aplikasi ini melalui skenario uji yang diberikan.
**Petunjuk Pengisian:** Pilih angka 1 sampai 5 pada pernyataan di bawah ini yang paling menggambarkan penilaian Anda:

- **1** = Sangat Tidak Setuju (STS)
- **2** = Tidak Setuju (TS)
- **3** = Cukup Setuju (CS)
- **4** = Setuju (S)
- **5** = Sangat Setuju (SS)




| Jabatan / Peran di Laboratorium | Tampilan | Sistem r | Sistem b | Sistem m | Kecepatan | Penyimp | Aplikasi ini | Hasil pe |
|---|---|---|---|---|---|---|---|---|
| Asisten Laboratorium Komputer / Mahasiswa | 3 | 4 | 5 | 5 | 5 | 5 | 4 | 3 |
| Kepala Laboratorium Komputer FTI UNIBBA | 4 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
| Asisten Laboratorium Komputer / Mahasiswa | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |

**Lampiran 4: Kode Program**

INISIALISASI PELADEN FLASK & DATABASE ORM (app/__init__.py)

```
from flask import Flask, session
from flask_sqlalchemy import SQLAlchemy
from config import Config

db = SQLAlchemy()

def create_app(config_class=Config):
    """Application factory inisialisasi Flask, SQLAlchemy ORM, dan Blueprint."""
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)

    # Registrasi blueprint modular
    from app.routes import bp as main_bp
    app.register_blueprint(main_bp)

    # Request hook: Sinkronisasi izin pengguna dinamis dari basis data ke sesi
    @app.before_request
    def refresh_user_permissions():
        if 'user_id' in session and session.get('role') != 'admin':
            from app.models import User
            user = User.query.get(session['user_id'])
            if user:
                session['permissions'] = user.get_permissions()
                session['nama_lengkap'] = user.nama_lengkap
                session['role'] = user.role
            else:
                session.clear()
    return app
```

OPERASIONAL CRUD DATA MASTER REFERENSI SISTEM
(app/routes/prodi.py, locations, categories, & kepala_lab.py)

```
from flask import render_template, request, redirect, url_for, flash
from app.models import db, Prodi, Location, Category, KepalaLab
from app.routes import bp
from app.utils import permission_required

# ── 1. MASTER PROGRAM STUDI (app/routes/prodi.py) ──────────────────

@bp.route('/prodi')
@permission_required('view_master')
def prodi():
    return render_template('prodi.html', prodi_list=Prodi.query.order_by(Prodi.kode_prodi).all())

@bp.route('/prodi/create', methods=['POST'])
@permission_required('edit_master')
def create_prodi():
    kode = request.form.get('kode_prodi', '').strip().upper()
    if Prodi.query.filter_by(kode_prodi=kode).first():
        flash('Gagal: Kode Prodi sudah terdaftar.', 'danger')
        return redirect(url_for('main.prodi'))
```

```
    Prodi(kode_prodi=kode, nama_prodi=request.form['nama_prodi'],
kode_aset=request.form['kode_aset']).insert_data()
    flash('Data Program Studi berhasil ditambahkan.', 'success')
    return redirect(url_for('main.prodi'))

@bp.route('/prodi/<int:id>/delete', methods=['POST'])
@permission_required('edit_master')
def delete_prodi(id):
    Prodi.query.get_or_404(id).delete_data()
    flash('Data Program Studi berhasil dihapus.', 'success')
    return redirect(url_for('main.prodi'))

# ── 2. MASTER LOKASI LABORATORIUM (app/routes/locations/) ──────────────

@bp.route('/locations')
@permission_required('view_master')
def locations():
    return render_template('locations.html',
locations=Location.query.order_by(Location.kode_lab).all())

@bp.route('/locations/add', methods=['POST'])
@permission_required('edit_master')
def create_location():
    kode = request.form['kode_lab'].strip().upper()
    if Location.query.filter_by(kode_lab=kode).first():
        flash('Gagal: Kode Lab sudah terdaftar.', 'danger')
        return redirect(url_for('main.locations'))
    Location(kode_lab=kode, nama_lab=request.form['nama_lab'],
kapasitas=int(request.form.get('kapasitas', 0)),
kode_aset=request.form['kode_aset']).insert_data()
    flash('Data Laboratorium berhasil ditambahkan.', 'success')
    return redirect(url_for('main.locations'))

# ── 3. MASTER KATEGORI ASET (app/routes/categories/) ──────────────────

@bp.route('/categories')
@permission_required('view_master')
def categories():
    return render_template('categories.html',
categories=Category.query.order_by(Category.nama_kategori).all())

@bp.route('/categories/add', methods=['POST'])
@permission_required('edit_master')
def create_category():
    nama = request.form['nama_kategori'].strip()
    if Category.query.filter_by(nama_kategori=nama).first():
        flash('Gagal: Nama kategori sudah ada.', 'danger')
        return redirect(url_for('main.categories'))
    Category(nama_kategori=nama).insert_data()
    flash('Kategori aset berhasil ditambahkan.', 'success')
    return redirect(url_for('main.categories'))

# ── 4. MASTER KEPALA LABORATORIUM (app/routes/kepala_lab.py) ──────────

@bp.route('/kepala-lab')
@permission_required('view_master')
def kepala_lab():
```

```
    return render_template('kepala_lab.html',
kepala_list=KepalaLab.query.order_by(KepalaLab.periode.desc()).all())

@bp.route('/kepala-lab/add', methods=['POST'])
@permission_required('edit_master')
def create_kepala_lab():
    KepalaLab(nik=request.form['nik'], nama=request.form['nama'],
periode=request.form['periode'], is_active=True).insert_data()
    flash('Data Kepala Laboratorium berhasil disimpan.', 'success')
    return redirect(url_for('main.kepala_lab'))
```

SIKLUS CRUD MANAJEMEN ASET & PERAKITAN KODE 27 DIGIT
(app/routes/assets/tampil_aset.py, tambah_aset.py, edit_aset.py, & hapus_aset.py)

```
import os
from flask import render_template, request, redirect, url_for, flash, current_app
from app.models import db, Asset, Prodi, KodeLokasi, KodeBarang
from app.routes import bp
from app.utils import permission_required, generate_asset_qr

# ── 1. PEMBACAAN & FILTER DAFTAR ASET (READ) (tampil_aset.py) ──────────────

@bp.route('/assets')
@permission_required('view_assets')
def assets():
    query = Asset.query
    if request.args.get('location'): query = query.filter_by(lokasi_id=request.args.get('location',
type=int))
    if request.args.get('category'): query = query.filter_by(kategori_id=request.args.get('category',
type=int))
    if request.args.get('kondisi'): query = query.filter_by(kondisi=request.args.get('kondisi'))
    return render_template('assets.html', assets=query.order_by(Asset.created_at.desc()).all())

# ── 2. REGISTRASI & PERAKITAN KODE 27 DIGIT (CREATE) (tambah_aset.py) ──────

@bp.route('/assets/create', methods=['POST'])
@permission_required('edit_assets')
def create_asset():
    skor = int(request.form.get('skor_kerusakan', 1))
    prodi = Prodi.query.get(request.form.get('prodi_id'))
    final_code = (f"{request.form['code_yayasan']} {request.form['code_rektorat']}
{request.form['code_fakultas']} "
                  f"{prodi.kode_aset if prodi else '00'} {request.form['code_lokasi']}
{request.form['code_golongan']} "
                  f"{request.form['code_bidang']} {request.form['code_kelompok']}
{request.form['code_subkel']} "
                  f"{request.form['code_tahun']}
{request.form['code_urutan_lok']}{request.form['code_nomor_urut']}")

    kl = KodeLokasi(yayasan=request.form['code_yayasan'],
rektorat=request.form['code_rektorat'], fakultas=request.form['code_fakultas'],
prodi_id=prodi.id if prodi else None, ruangan=request.form['code_lokasi'])
    db.session.add(kl); db.session.flush()

    kb = KodeBarang(golongan=request.form['code_golongan'],
bidang=request.form['code_bidang'], kelompok=request.form['code_kelompok'],
```

```
subkelompok=request.form['code_subkel'], tahun_pengadaan=request.form['code_tahun'],
nomor_urut=request.form['code_nomor_urut'])
    db.session.add(kb); db.session.flush()

    new_asset = Asset(
        kode_aset=final_code, nama_aset=request.form['nama_aset'],
        kategori_id=request.form['kategori_id'], lokasi_id=request.form['lokasi_id'],
prodi_id=prodi.id if prodi else None,
        kode_lokasi_id=kl.id, kode_barang_id=kb.id,
        kondisi='Baik' if skor == 1 else ('Rusak Ringan' if skor == 2 else 'Rusak Berat'),
        skor_kerusakan=skor, tahun_pengadaan=int(request.form['code_tahun']),
        harga_perolehan=float(request.form.get('harga_perolehan', 0)),
        umur_ekonomis=int(request.form.get('umur_ekonomis', 0)),
        nilai_residu=float(request.form.get('nilai_residu', 0))
    )
    db.session.add(new_asset); db.session.commit()
    generate_asset_qr(new_asset)
    flash('Aset baru berhasil disimpan beserta label QR.', 'success')
    return redirect(url_for('main.assets'))

# ── 3. PENGUBAHAN INFORMASI ASET (UPDATE) (edit_aset.py)
─────────────────────────────────────

@bp.route('/assets/edit/<int:id>', methods=['POST'])
@permission_required('edit_assets')
def edit_asset(id):
    asset = Asset.query.get_or_404(id)
    asset.nama_aset = request.form['nama_aset']
    asset.lokasi_id = int(request.form['lokasi_id'])
    asset.kategori_id = int(request.form['kategori_id'])
    asset.skor_kerusakan = int(request.form.get('skor_kerusakan', asset.skor_kerusakan))
    asset.kondisi = 'Baik' if asset.skor_kerusakan == 1 else ('Rusak Ringan' if
asset.skor_kerusakan == 2 else 'Rusak Berat')
    asset.harga_perolehan = float(request.form.get('harga_perolehan', asset.harga_perolehan))
    asset.umur_ekonomis = int(request.form.get('umur_ekonomis', asset.umur_ekonomis))
    asset.nilai_residu = float(request.form.get('nilai_residu', asset.nilai_residu))
    db.session.commit()
    generate_asset_qr(asset)
    flash('Perubahan data aset berhasil diperbarui.', 'success')
    return redirect(url_for('main.assets'))

# ── 4. PENGHAPUSAN ENTITAS ASET (DELETE) (hapus_aset.py)
─────────────────────────────────────

@bp.route('/assets/delete/<int:id>', methods=['POST'])
@permission_required('edit_assets')
def delete_asset(id):
    asset = Asset.query.get_or_404(id)
    if asset.qr_code_path:
        qr_file = os.path.join(current_app.root_path, 'static', asset.qr_code_path)
        if os.path.exists(qr_file):
            try: os.remove(qr_file)
            except OSError: pass
    db.session.delete(asset); db.session.commit()
    flash('Aset beserta berkas QR Code berhasil dihapus.', 'success')
    return redirect(url_for('main.assets'))
```

AKUNTANSI DEPRESIASI & GENERATOR QR CODE (app/models.py & utils.py)

```
import os, qrcode
from datetime import datetime
from flask import current_app
from app import db

class Asset(db.Model):
    __tablename__ = 'assets'
    id = db.Column(db.Integer, primary_key=True)
    kode_aset = db.Column(db.String(50), unique=True, nullable=False)
    nama_aset = db.Column(db.String(100), nullable=False)
    tahun_pengadaan = db.Column(db.Integer)
    harga_perolehan = db.Column(db.Float, default=0)
    umur_ekonomis = db.Column(db.Integer, default=0)  # Periode tahun
    nilai_residu = db.Column(db.Float, default=0)
    skor_kerusakan = db.Column(db.Integer, default=1)  # C2 SPK
    biaya_perawatan = db.Column(db.Float, default=0)   # C3 SPK
    qr_code_path = db.Column(db.String(255))

    def sisa_umur_ekonomis(self):
        """Menghitung sisa umur pakai aset untuk kriteria C1 SPK."""
        if not self.tahun_pengadaan or not self.umur_ekonomis:
            return 0
        sisa = self.umur_ekonomis - (datetime.utcnow().year - self.tahun_pengadaan)
        return max(sisa, 0)

    def nilai_buku(self):
        """Menghitung nilai buku aset via Straight Line Depreciation."""
        if not self.harga_perolehan or not self.umur_ekonomis:
            return self.harga_perolehan or 0
        depresiasi_tahunan = (self.harga_perolehan - (self.nilai_residu or 0)) / self.umur_ekonomis
        tahun_terpakai = datetime.utcnow().year - self.tahun_pengadaan
        total_depresiasi = depresiasi_tahunan * tahun_terpakai
        return max(self.harga_perolehan - total_depresiasi, self.nilai_residu or 0)

def generate_asset_qr(asset):
    """Membentuk stiker QR Code dinamis berbasis teks spesifikasi dan status biaya."""
    konten = f"Aset: {asset.nama_aset}\nKode: {asset.kode_aset}\nBiaya Servis: Rp
{asset.biaya_perawatan:,.0f}"
    qr = qrcode.QRCode(version=1, box_size=10, border=2)
    qr.add_data(konten); qr.make(fit=True)
    img = qr.make_image(fill_color="black", back_color="white")
    save_path = os.path.join(current_app.root_path, 'static', 'qrcodes', f'asset_{asset.id}.png')
    img.save(save_path)
    asset.qr_code_path = f'qrcodes/asset_{asset.id}.png'
    db.session.commit()
```

LOG SERVIS & REKALKULASI BIAYA C3 (app/routes/perbaikan.py)

```
from datetime import datetime
from flask import request, redirect, url_for, flash
from app.models import db, PerbaikanAset, Asset
from app.routes import bp
from app.utils import permission_required, generate_asset_qr
```

```
def _recalculate_biaya(asset):
    """Hitung ulang akumulasi biaya servis aset (Kriteria C3 SPK) via SQL SUM."""
    total = db.session.query(
        db.func.sum(PerbaikanAset.biaya_perbaikan)
    ).filter_by(asset_id=asset.id).scalar() or 0.0
    asset.biaya_perawatan = total
    generate_asset_qr(asset)
    db.session.commit()

@bp.route('/perbaikan/add', methods=['POST'])
@permission_required('manage_perbaikan')
def tambah_perbaikan():
    perbaikan = PerbaikanAset(
        asset_id=int(request.form['asset_id']),
        tanggal_perbaikan=datetime.strptime(request.form['tanggal_perbaikan'], '%Y-%m-
%d').date(),
        deskripsi=request.form['deskripsi'],
        biaya_perbaikan=float(request.form['biaya_perbaikan']),
        teknisi=request.form.get('teknisi'),
        status='Selesai'
    )
    db.session.add(perbaikan); db.session.commit()
    _recalculate_biaya(perbaikan.asset)
    flash('Log perbaikan tersimpan dan biaya C3 berhasil diperbarui.', 'success')
    return redirect(url_for('main.perbaikan'))
```

## PEMETAAN SPASIAL DENAH 2D LABORATORIUM (app/routes/floor_plan.py)

```
import json
from flask import request, jsonify
from app.models import db, Asset
from app.routes import bp
from app.utils import permission_required

def serialize_asset(asset):
    """Menyusun kamus data aset untuk visualisasi interaktif denah 2D."""
    kondisi_lower = (asset.kondisi or 'baik').lower()
    if 'rusak berat' in kondisi_lower or (asset.skor_kerusakan and asset.skor_kerusakan >= 3):
        color = '#ef4444'   # Merah: Rusak Berat
    elif 'rusak ringan' in kondisi_lower or (asset.skor_kerusakan and asset.skor_kerusakan == 2):
        color = '#f59e0b'   # Oranye: Rusak Ringan
    else:
        color = '#10b981'   # Hijau: Baik

    return {
        'id': asset.id, 'nama': asset.nama_aset, 'kode': asset.kode_aset,
        'kondisi': asset.kondisi or 'Baik', 'color': color,
        'pos_x': asset.pos_x, 'pos_y': asset.pos_y
    }

@bp.route('/api/floor-plan/save-coords', methods=['POST'])
@permission_required('manage_master')
```

```
def save_coords():
    """Menyimpan posisi koordinat spasial meja workstation lab."""
    data = request.get_json() or {}
    for item in data.get('assets', []):
        asset = Asset.query.get(item['id'])
        if asset:
            asset.pos_x = float(item['pos_x'])
            asset.pos_y = float(item['pos_y'])
    db.session.commit()
    return jsonify({'status': 'success', 'message': 'Koordinat denah berhasil diperbarui.'})
```

ALGORITMA SISTEM PENDUKUNG KEPUTUSAN SAW (app/routes/spk.py)

```
from flask import render_template, request
from app.models import db, Asset, BobotSAW
from app.routes import bp
from app.utils import permission_required

def hitung_saw(assets, bobot):
    """Normalisasi Matriks & Kalkulasi Nilai Preferensi SAW: V_i = sum(w_j * r_ij)."""
    if not assets:
        return []
    data = []
    for a in assets:
        c1 = round(a.sisa_umur_ekonomis() / a.umur_ekonomis, 6) if (a.umur_ekonomis and
a.tahun_pengadaan) else 0.0
        c2 = float(a.skor_kerusakan or 1)
        c3 = float(a.biaya_perawatan or 0)
        data.append({'asset': a, 'c1': c1, 'c2': c2, 'c3': c3})

    min_c1 = min(d['c1'] for d in data)
    max_c2 = max(d['c2'] for d in data)
    max_c3 = max(d['c3'] for d in data)

    results = []
    for d in data:
        # C1 (Cost): min / nilai; C2 & C3 (Benefit): nilai / max
        r_c1 = (min_c1 / d['c1']) if d['c1'] > 0 else 1.0
        r_c2 = (d['c2'] / max_c2) if max_c2 > 0 else 0.0
        r_c3 = (d['c3'] / max_c3) if max_c3 > 0 else 0.0
        skor = (bobot.bobot_c1 * r_c1) + (bobot.bobot_c2 * r_c2) + (bobot.bobot_c3 * r_c3)

        results.append({
            'asset': d['asset'], 'r_c1': round(r_c1, 4), 'r_c2': round(r_c2, 4),
            'r_c3': round(r_c3, 4), 'skor_akhir': round(skor, 4)
        })
    results.sort(key=lambda x: x['skor_akhir'], reverse=True)
    return results

@bp.route('/spk')
```

```
@permission_required('view_spk')
def kelayakan():
    kategori_id = request.args.get('kategori_id', type=int)
    bobot = BobotSAW.query.filter_by(kategori_id=kategori_id).first() or
BobotSAW.query.filter_by(kategori_id=None).first()
    assets = Asset.query.filter_by(kategori_id=kategori_id).all() if kategori_id else
Asset.query.all()
    hasil_saw = hitung_saw(assets, bobot)
    return render_template('spk.html', hasil_saw=hasil_saw, bobot=bobot)
```

DEKORATOR KEAMANAN & RBAC GRANULAR (app/utils.py)

```
from functools import wraps
from flask import session, redirect, url_for, flash


def login_required(f):
    """Memastikan sesi pengguna aktif sebelum mengeksekusi rute."""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            return redirect(url_for('main.login'))
        return f(*args, **kwargs)
    return decorated_function


def permission_required(perm):
    """Mengevaluasi hak akses izin granular (RBAC) pada rute modular."""
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                return redirect(url_for('main.login'))
            user_perms = session.get('permissions', [])
            user_role = session.get('role', '')
            if user_role in ['admin', 'Super Admin'] or 'all' in user_perms or perm in user_perms:
                return f(*args, **kwargs)
            flash('Akses Ditolak: Anda tidak memiliki wewenang untuk fitur tersebut.', 'danger')
            return redirect(url_for('main.dashboard'))
        return decorated_function
    return decorator
```

## RIWAYAT HIDUP



**Andhika Maulana Shiddiq** lahir di Bandung pada 20 April 2003. Ia merupakan putra sulung dari pasangan Rudi Kusdiana dan Rusmiati, serta memiliki seorang adik. Penulis mengawali jenjang pendidikannya di TK Al Barokah (2008), kemudian menyelesaikan pendidikan dasar di SDN Korpri 3 (2009–2015). Pendidikan tingkat menengah pertama ditempuhnya di SMPN 1 Baleendah (2015–2017), lalu dilanjutkan ke SMAIT Fithrah Insani Baleendah hingga lulus pada 2020. Untuk meraih gelar sarjana, penulis melanjutkan studi ke jenjang perguruan tinggi di Universitas Bale Bandung pada Program Studi Teknik Informatika, Fakultas Teknologi Informasi.